Microservices Patterns Handbook

Twelve patterns for building, migrating and operating microservices · resilience · data consistency · messaging · configuration

14 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — System Design Fundamentals**
    - Module Introduction
    - Part 1 — Foundations and performance
    - What system design is
    - Clients, servers and the request/response cycle
    - The building blocks of a web application
    - Latency, throughput and bandwidth
    - Percentiles, tail latency and why averages lie
    - Little's law
    - Latency numbers every engineer should know
    - Units, powers of two and data sizes
    - Back-of-the-envelope estimation
    - Availability and the nines
    - SLA, SLO, SLI and error budgets
    - MTBF, MTTR and measuring failure
    - Reliability, availability and durability
    - Faults, failures and fault tolerance
    - Redundancy, single points of failure and failover
    - Scalability
    - Vertical and horizontal scaling
    - Elasticity
    - Stateless and stateful services
    - Performance vs scalability
    - Concurrency vs parallelism
    - Bottlenecks: CPU, memory, disk and network
    - How to approach a system design interview
    - Part 1 glossary
    - Part 2 — Networking and communication
    - IP addresses, NAT, ports and sockets
    - DNS — the internet's phone book
    - Network layers — OSI and TCP/IP models
    - TCP, UDP and QUIC
    - HTTP — the language of the web
    - HTTPS and TLS
    - API styles — REST, RPC and gRPC, GraphQL
    - Real-time communication — polling, long polling, WebSockets, SSE, webhooks
    - API design basics
    - Proxies and API gateways
    - Load balancers
    - CDNs and anycast
    - Bandwidth vs latency, and network partitions
    - Part 2 glossary
    - Part 3 — Data storage
    - What a database is
    - Relational databases and SQL
    - Transactions and ACID
    - Isolation levels and anomalies
    - NoSQL database families
    - SQL vs NoSQL and BASE
    - Indexes
    - Normalization and denormalization
    - Replication
    - Partitioning and sharding
    - Consistent hashing and virtual nodes
    - Storage types: block, file and object
    - OLTP vs OLAP, row vs column storage, lakes and warehouses
    - Durability: WAL, fsync, backups and recovery
    - Connection pooling, ORMs and N+1 queries
    - Choosing a database
    - Part 3 glossary
    - Part 4 — Caching, async processing and consistency
    - Caching basics
    - Where caches live
    - Caching strategies
    - Eviction policies
    - Cache invalidation, stale data and stampedes
    - Message queues
    - Event streaming: topics, partitions, consumer groups and offsets
    - Delivery guarantees, idempotency and retries
    - Batch vs stream processing
    - Event-driven architecture, event sourcing and CQRS
    - Consistency models
    - CAP theorem and PACELC
    - Quorums
    - Consensus, leader election and distributed locks
    - Clocks and ordering
    - Distributed transactions: two-phase commit and sagas
    - Heartbeats, gossip and failure detection
    - Probabilistic data structures and Merkle trees
    - Unique ID generation
    - Geospatial indexing: geohash and quadtree
    - Part 4 glossary
    - Part 5 — Architecture, reliability, operations and security
    - Architectural styles: monolith, modular monolith, SOA, microservices and serverless
    - Synchronous vs asynchronous communication
    - Service discovery, sidecars and service mesh
    - Timeouts, retries, exponential backoff and jitter
    - Circuit breakers, bulkheads and fallbacks
    - Rate limiting and throttling
    - Load shedding and graceful degradation
    - Autoscaling
    - Virtual machines, containers and Docker
    - Kubernetes basics
    - Infrastructure as code
    - CI/CD
    - Deployment strategies and rollback
    - Observability: logs, metrics and traces
    - Dashboards, alerting, on-call and SLO-based alerts
    - Disaster recovery
    - Authentication and authorization
    - Sessions, cookies, tokens and JWT
    - OAuth 2.0, OpenID Connect, SSO and API keys
    - Encryption, hashing, passwords and secrets

- Network security: firewalls, WAF and DDoS
- Common web vulnerabilities: CORS, CSRF, XSS and SQL injection
- Least privilege and zero trust
- Cost awareness
- Part 5 glossary
- Part 6 — Putting it together
- A request's journey
- Worked example: designing a URL shortener
- Decision cheat sheet
- Top 60 interview questions
- Master A–Z glossary
- Part 6 glossary

2. **Strangler Fig Pattern**
- Part 0 — Concepts you need first
- Part 1 — Where the name comes from
- Part 2 — The problem in detail
- Part 3 — The solution
- Part 4 — Architecture
- Part 5 — The four implementation steps
- Part 6 — Worked example 1: routing a single function
- Part 7 — Worked example 2: a facade over a banking system
- Part 8 — Key insights: the proxy layer is everything
- Part 9 — Trade-offs and what actually goes wrong
- Part 10 — Common beginner misconceptions
- Part 11 — Check your understanding
- Part 12 — Interview soundbite (30 seconds)
- Part 13 — Implementation
- Related patterns

3. **API Gateway Pattern**
- Lesson 1 — Introduction to the API Gateway Pattern
- Lesson 2 — Advantages: the fifteen problems it solves
- Lesson 3 — Worked example: a library system
- Lesson 4 — Problems the gateway itself introduces
- Lesson 4 (cont.) — Performance implications
- Lesson 5 — System design example: Instagram
- Consolidation
- Implementation
- Related patterns

4. **Backends for Frontends (BFF) Pattern**
- Part 0 — Concepts you need first
- Part 1 — The core idea, in plain terms
- Part 2 — The problem in detail
- Part 3 — Architecture
- Part 4 — The benefits, explained
- Part 5 — Worked example: e-commerce product details
- Part 6 — Problems with BFF, and how to handle them
- Part 7 — System design example: Netflix
- Part 8 — BFF vs. API Gateway
- Part 9 — Common beginner misconceptions
- Part 10 — Check your understanding
- Part 11 — Interview soundbite (30 seconds)
- Part 12 — Implementation
- Related patterns

5. **Service Discovery Pattern**
- Part 0 — Concepts you need first
- Part 1 — The problem
- Part 2 — The solution
- Part 3 — Architecture
- Part 4 — Client-side vs. server-side discovery
- Part 5 — Inner workings
- Part 6 — Worked example (simplified Java)
- Part 7 — Performance implications and special considerations
- Part 8 — Security considerations
- Part 9 — Real-world implementations
- Part 10 — Common beginner misconceptions
- Part 11 — Check your understanding
- Part 12 — Interview soundbite (30 seconds)
- Part 13 — Implementation
- Related patterns

6. **Circuit Breaker Pattern**
- Part 0 — Concepts you need first
- Part 1 — The metaphor
- Part 2 — The problem
- Part 3 — The solution
- Part 4 — The three states
- Part 5 — Integration: it's additive, not invasive
- Part 6 — Configuration
- Part 7 — Worked example (Java)
- Part 8 — Performance implications
- Part 9 — Best practices and trade-offs
- Part 10 — Use cases
- Part 11 — Real-world examples
- Part 12 — System design example: distributed social media platform
- Part 13 — Common beginner misconceptions
- Part 14 — Check your understanding
- Part 15 — Interview soundbite (30 seconds)
- Part 16 — Implementation
- Related patterns

7. **Bulkhead Pattern**
- Part 0 — Concepts you need first
- Part 1 — Where the name comes from
- Part 2 — The problem: failure propagation
- Part 3 — The solution: isolation
- Part 4 — Architecture
- Part 5 — Inner workings
- Part 6 — Worked example: thread-pool bulkheads in Java

- Part 7 — Performance implications
- Part 8 — Use cases
- Part 9 — System design example: music streaming (Spotify-like)
- Part 10 — Bulkhead vs. Circuit Breaker
- Part 11 — Common beginner misconceptions
- Part 12 — Check your understanding
- Part 13 — Interview soundbite (30 seconds)
- Part 14 — Implementation
- Related patterns

8. **Retry Pattern**
   - Part 0 — Concepts you need first
   - Part 1 — The problem
   - Part 2 — The solution
   - Part 3 — Transient vs. permanent: the critical distinction
   - Part 4 — Architecture
   - Part 5 — Worked example (Java)
   - Part 6 — Performance implications and risks
   - Part 7 — Use cases
   - Part 8 — Real-world examples
   - Part 9 — System design examples
   - Part 10 — When NOT to use retry
   - Part 11 — Common beginner misconceptions
   - Part 12 — Check your understanding
   - Part 13 — Interview soundbite (30 seconds)
   - Part 14 — Implementation
   - Related patterns
9. **Sidecar Pattern**
   - Part 0 — Concepts you need first
   - Part 1 — The metaphor and the core idea
   - ⚠️ Part 2 — One correction to the course material
   - Part 3 — The problem: monolithic application management
   - Part 4 — The solution
   - Part 5 — Architecture
   - Part 6 — Worked example
   - Part 7 — Performance implications
   - Part 8 — Special considerations
   - Part 9 — Use cases
   - Part 10 — System design examples
   - Part 11 — Sidecar vs. shared library
   - Part 12 — Common beginner misconceptions
   - Part 13 — Check your understanding
   - Part 14 — Interview soundbite (30 seconds)
   - Part 15 — Implementation
   - Related patterns
10. **Saga Pattern**
   - Part 0 — Concepts you need first
   - Part 1 — The problem
   - Part 2 — The solution
   - Part 3 — The two types
   - Part 4 — Architecture
   - Part 5 — Worked example: online shopping (orchestration)
   - Part 6 — Issues and considerations
   - Part 7 — Use cases
   - Part 8 — System design examples
   - Part 9 — Saga vs. two-phase commit (2PC)
   - Part 10 — Common beginner misconceptions
   - Part 11 — Check your understanding
   - Part 12 — Interview soundbite (30 seconds)
   - Part 13 — Implementation
   - Related patterns
11. **Event-Driven Architecture (EDA) Pattern**
   - Part 0 — Concepts you need first
   - Part 1 — The problem
   - Part 2 — The solution
   - Part 3 — The benefits
   - Part 4 — The components
   - Part 5 — Three structural variants
   - Part 6 — Inner workings
   - Part 7 — Worked example: traffic management (Java)
   - Part 8 — Performance implications and considerations
   - Part 9 — Use cases
   - Part 10 — System design example: a ride-sharing app
   - Part 11 — Common beginner misconceptions
   - Part 12 — Check your understanding
   - Part 13 — Interview soundbite (30 seconds)
   - Part 14 — Implementation
   - Related patterns
12. **CQRS — Command Query Responsibility Segregation**
   - Part 0 — Concepts you need first
   - Part 1 — The problem
   - Part 2 — The solution
   - Part 3 — Architecture: three levels
   - Part 4 — Inner workings
   - Part 5 — Worked example: user profile management
   - Part 6 — Issues, considerations, and performance
   - Part 7 — Use cases
   - Part 8 — System design examples
   - Part 9 — Common beginner misconceptions
   - Part 10 — Check your understanding
   - Part 11 — Interview soundbite (30 seconds)
   - Part 12 — Implementation
   - Related patterns
13. **Configuration Externalization Pattern**
   - Part 0 — Concepts you need first
   - Part 1 — The framing
   - Part 2 — The problem
   - Part 3 — The solution
   - Part 4 — Architecture
   - Part 5 — Worked example: Spring Cloud Config
   - Part 6 — Considerations and implications
   - Part 7 — Real-world use cases

- Part 8 — Common beginner misconceptions
- Part 9 — Check your understanding
- Part 10 — Interview soundbite (30 seconds)
- Part 11 — Implementation
- Related patterns

14. **Chapter 14 — Interview Question Bank**
    - How to use this bank
    - 1. Fundamentals: monolith vs microservices
    - 2. Decomposition, DDD and migration
    - 3. Communication: REST, gRPC and messaging
    - 4. API Gateway, BFF, service discovery and service mesh
    - 5. Resilience and fault tolerance
    - 6. Data management: database per service, saga, outbox, CQRS, event sourcing
    - 7. Consistency, delivery guarantees and ordering
    - 8. Observability: logs, metrics and tracing
    - 9. Deployment: containers, Kubernetes, CI/CD and release strategies
    - 10. Configuration management
    - 11. Security: OAuth2, JWT, mTLS and zero trust
    - 12. Testing microservices
    - 13. Spring Cloud and Spring Boot specifics
    - 14. Scenario, troubleshooting and senior-level questions
    - 15. Rapid-fire "difference between X and Y" and short-answer round
    - Sources of the question lists

Chapter 1 — System Design Fundamentals

Condensed primer. This chapter is a ~21k-word primer that defines every system-design fundamental in a few precise sentences; the full ~100k-word treatment (worked examples, diagrams, interview Q&A) lives in System Design Patterns, Module 0, and every section below ends with a link to its full explanation there.

Module Introduction

Every other chapter assumes the basic vocabulary of system design: *latency*, *load balancer*, *cache*, *sharding*, *eventual consistency*. This primer defines all of it from zero, in six parts:

Part	Covers
1 — Foundations and performance	Clients/servers, latency, percentiles, estimation, availability, SLAs, scaling, bottlenecks, the interview method
2 — Networking and communication	IP, DNS, TCP/UDP/QUIC, HTTP, TLS, API styles, real-time push, proxies, load balancers, CDNs
3 — Data storage	Relational databases, ACID, isolation, NoSQL, indexes, replication, sharding, storage types, OLTP/OLAP, durability
4 — Caching, async and consistency	Caches, queues, streams, delivery guarantees, consistency models, CAP, quorums, consensus, clocks, sagas, sketches, IDs
5 — Architecture, operations and security	Architectural styles, resilience patterns, rate limiting, containers, Kubernetes, CI/CD, observability, DR, auth, security
6 — Putting it together	A request's journey, a worked URL shortener, a decision cheat sheet, 60 interview questions, an A–Z index

Read Parts 1–5 in order once; then use Part 6 as a reference. Terms are **bold** where they are defined.

→ Full explanation

Part 1 — Foundations and performance

The ground floor: what system design is, the standard building blocks, and the vocabulary for speed (latency, throughput, percentiles), size (units, estimation), staying up (availability, reliability, durability) and growing (scalability), ending with a method for the design interview.

Part 1 TL;DR: Describe speed with percentiles, not averages; estimate load from DAU (1 million/day ≈ 12/s); each extra nine of availability is 10× less downtime; series dependencies multiply availability down, redundancy multiplies it up; keep app servers stateless and scale out; find the bottleneck by measuring; in interviews go requirements → estimates → API → data model → design → deep dives → trade-offs.

→ Full explanation

What system design is

System design is deciding which parts a piece of software needs, what each does, how they talk and where data lives, so the whole meets its goals: number of users, response time, staying up when machines break, and cost. Coding asks "how does this function work?"; design asks "how do twenty programs on two hundred machines cooperate without losing data?". The working loop is: understand the goal → estimate the load → sketch the parts → find weak spots (what runs out first, what takes

everything down) → fix and justify, repeated as the system grows. Every choice is a **trade-off** (faster reads may mean slower writes; more copies mean safety but cost and disagreement), and in interviews the reasoning is graded, not one "right" answer.

→ Full explanation

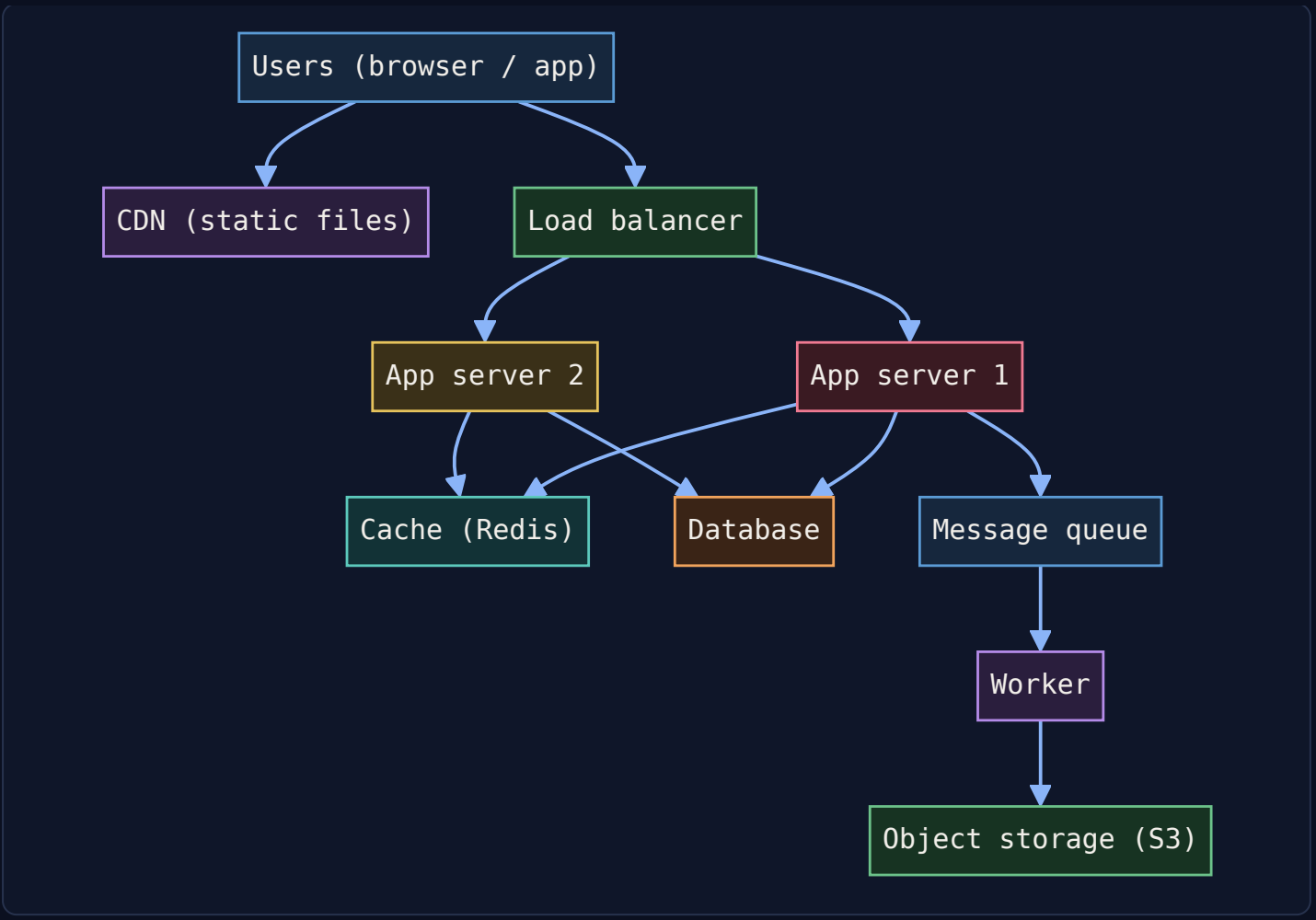
Clients, servers and the request/response cycle

A **client** is any program that asks for something (browser, app, or another server); a **server** is a program (and loosely its machine) that waits for requests and answers them — a *role*, so one logical server may be hundreds of machines and most microservices are both. They follow **request/response**: the client sends a request (`GET /profile`), the server does work and returns a response (`200 OK` + body), usually over **HTTP**. A **thin client** relies on the server; a **thick client** does much work locally. Normally the client **pulls**; features like chat need the server to **push** (WebSockets, SSE — Part 2). Remember the network between them is slow and unreliable compared with work inside one machine.

→ Full explanation

The building blocks of a web application

Part	Job
Client	Renders UI, calls the backend over HTTP
Load balancer	Spreads requests over servers, skips dead ones
App server	Runs business logic; run several identical copies
Database	Stores data durably and lets you query it; holds **state**, so hardest to scale
Cache	Fast in-memory copy of hot data (Redis, Memcached); RAM reads are well under 1 ms
Message queue + **workers**	Holds jobs for background programs so the request returns immediately
Object storage	Cheap storage for large files under keys (Amazon S3)
CDN	Servers worldwide keeping static files near users



Add each box only when you can name the problem it solves ("reads outnumber writes 100:1, so a cache"). Classic mistakes: images stored in the database (put them in object storage, keep the key), slow work (emails, encoding) inside the web request.

→ Full explanation

Latency, throughput and bandwidth

Latency is how long one thing takes (ms = thousandths, µs = millionths of a second); **throughput** is work done per unit time (requests/s, MB/s); **bandwidth** is a link's maximum capacity in bits per second (Mbps, Gbps).

Response time vs latency

Response time is what the client sees end to end; strictly, **latency** is only the delay before work starts (queueing, network hops) and **service time** is the work itself: response time = network delay + queueing delay + service time. In interviews "latency" usually means response time — say which you mean.

Latency comes from **propagation** (light in fibre ≈ 200 km/ms; London–New York round trip ≈ 70 ms), **transmission** (1 MB over 100 Mbps = 80 ms), **queueing** and **processing**. Latency and bandwidth are independent: more bandwidth does not speed up a small API call. Near full utilisation, queues grow and latency explodes while throughput flattens, so teams target ~50–70% peak utilisation. **Batching** raises throughput at a latency cost. 1 byte = 8 bits, so 100 Mbps ≈ 12.5 MB/s.

Low-latency checklist

Start with a **latency budget**: split the target (e.g. p99 < 200 ms) across hops (network, gateway, service + DB, slack) and work first on the hop that overspends. Levers, roughly by payoff: remove round trips (batch, parallelise, no N+1); reuse connections (pools, keep-alive, HTTP/2–3, TLS 1.3 resumption); move bytes closer (CDN, edge TLS, regional replicas); serve from memory (caches,

precomputed feeds and counts); do less while the user waits (queue side effects); shrink payloads (gzip/Brotli, Protobuf, pagination, only needed fields); make each query cheap (index, denormalise the hot read); keep headroom (50–70% utilisation, shed rather than queue); cut the tail (deadlines, hedging, bounded fan-out, fewer GC pauses and shared locks); measure per-hop percentiles with histograms and tracing. Rules: **fix the biggest term first** (a 70 ms ocean crossing dwarfs a 2 ms parse) and **every lever has a price** (staleness, money, load, delayed results) — say which you pay.

→ Full explanation

Percentiles, tail latency and why averages lie

A **percentile** is the value below which that share of measurements fall: **p50** (median), p90, p95, **p99** (1 in 100 slower), **p999**. **Tail latency** is the slow end (p99, p999, max).

Why averages lie

The **mean** blends fast and slow requests into a number no request had: nine 10 ms requests and one 1,000 ms give mean 109 ms, median 10 ms, p95/p99 1,000 ms. Real latency is skewed (GC pauses, cache misses, retries), so use percentiles.

Why the tail matters more than you think

Heavy users hit the tail most, and **fan-out** amplifies it: if a request waits on 100 servers each slow 1% of the time, 1 − 0.99^100 ≈ 63% of requests are slow ("The Tail at Scale", Dean and Barroso, 2013). Compute percentiles from **histograms** or sketches (HDRHistogram, t-digest); never average percentiles — merge histograms. Reduce tails with **hedged requests** (send a duplicate after ~p95), timeouts, less fan-out and spare capacity. Beware **coordinated omission**: a load tester that waits for slow replies under-reports the tail.

→ Full explanation

Little's law

For any stable system, **L = λ × W**: average items inside (L) = arrival rate (λ, equal to throughput when stable) × average time inside (W). 1,000 req/s × 0.2 s = 200 requests in flight, so you need ≥ 200 threads/connections; a pool of 20 connections with 5 ms queries caps at 20 / 0.005 = 4,000 queries/s; if latency doubles with fixed concurrency, max throughput halves. Convert ms to seconds, and it does not apply when the queue grows without bound.

→ Full explanation

Latency numbers every engineer should know

Orders of magnitude (rounded for current hardware; ratios matter, not exact values): CPU **L1 cache** 1 ns; **RAM** read 100 ns; compress 1 KB 2 µs; send 1 KB over 1 Gbps 10 µs; random 4 KB read from NVMe **SSD** 20–100 µs; read 1 MB sequentially from RAM 50–250 µs; round trip inside a data centre 0.5 ms; 1 MB from SSD 0.2–1 ms; **HDD** seek 10 ms; 1 MB from HDD 5–20 ms; London–New York round trip 70 ms; California–Europe 150 ms. Consequences: RAM is ~100–1,000× faster than SSD for random reads (caches live in RAM); a data-centre round trip costs thousands of memory reads (batch calls); cross-ocean latency only falls by moving data closer; sequential beats random I/O (why LSM-trees and logs append). A remote cache still pays the ~0.5 ms network hop.

→ Full explanation

Units, powers of two and data sizes

A **bit** is 0/1; a **byte** is 8 bits. KB, MB, GB, TB, PB each step ×1,000 in estimates (exactly 2^10 = 1,024; strictly KiB/MiB/GiB). Remember 2^10 ≈ 10^3, 2^20 ≈ 10^6, 2^30 ≈ 10^9, 2^40 ≈ 10^12, 2^32 ≈ 4.3 billion (IPv4 addresses; unsigned 32-bit limit, ~2 billion signed), 2^64 ≈ 1.8 × 10^19. Typical sizes: integer 4–8 B, timestamp 8 B, UUID 16 B, short post with metadata 0.5–1 KB, compressed photo 200 KB–2 MB, a

minute of HD video ~25–60 MB. A day ≈ 86,400 s ≈ **10^5 s**, a month ≈ 2.6 × 10^6 s, a year ≈ 3 × 10^7 s, so **1 million/day ≈ 12/s**. A 1 Gbps card moves at most 125 MB/s.

→ Full explanation

Back-of-the-envelope estimation

A **back-of-the-envelope estimate** is a quick rough calculation whose goal is the right **order of magnitude**, because that decides the architecture (100 QPS fits one server; 100,000 needs caching and sharding). Key terms: **DAU/MAU** (daily/monthly active users), **QPS/RPS** (queries/requests per second), average QPS = requests per day / ~10^5, **peak factor** (peak ÷ average, commonly 2–3×, 5–10× for spiky products), **read/write ratio**, storage/day = items × size, bandwidth = QPS × response size. Rough per-machine capacities: app server 1,000–10,000 req/s; relational DB 1,000–10,000 simple queries/s; Redis/Memcached 100,000+ ops/s; Kafka broker hundreds of MB/s. Don't forget peak, replication (×3 storage), metadata and indexes (+20–50%); round aggressively and say every assumption aloud.

Worked example: how many servers do we need?

Correct the naive "peak ÷ capacity" with measured cost, a utilisation ceiling, a concurrency check and failure headroom. Assume 30,000 req/s peak, 4 ms CPU and 50 ms end-to-end per request, 16-core machines, 20 KB responses, 3 AZs. One core: 1,000 ÷ 4 = 250 req/s; one server: 4,000 req/s; at a 60% ceiling 2,400 → 30,000 ÷ 2,400 = 12.5 → **13 servers**. Little's law: 30,000 × 0.05 s = 1,500 in flight, ~115 per server (size threads and DB pools to match). Survive losing one of 3 zones: ⌈13 ÷ 2⌉ = 7 per zone → **21**; +10% rolling-deploy surge → **~23**. Network: 600 MB/s ≈ 4.8 Gbps total, ~0.23 Gbps each — CPU is the limit. The naive 8 was ~3× low. **Repeat per tier** with its own bottleneck: cache by memory (working set ÷ RAM × replicas), database by IOPS or write throughput per primary (forcing sharding), streaming by network and disk bandwidth per broker.

→ Full explanation

Availability and the nines

Availability is the fraction of time (or of valid requests) the system works: uptime / (uptime + downtime).

The nines table

Availability	Per year	Per month (30 d)	Per day
99% (two nines)	3.65 days	7.2 h	14.4 min
99.9% (three)	8.76 h	43.2 min	1.44 min
99.95%	4.38 h	21.6 min	43 s
99.99% (four)	52.6 min	4.32 min	8.6 s
99.999% (five)	5.26 min	26 s	0.86 s

Each nine is 10× less downtime and usually far more than 10× the effort; at 99.99% recovery must be automatic. Parts in **series** multiply (99.99% × 99.9% × 99.9% ≈ 99.79%); independent parts in **parallel** combine as 1 − (failure probabilities multiplied) (two 99% servers → 99.99%), assuming failures are independent. Cloud single-service SLAs are usually 99.9–99.99%.

→ Full explanation

SLA, SLO, SLI and error budgets

An **SLI** (Service Level Indicator) is a measurement ("% of requests succeeding under 300 ms over 30 days"); an **SLO** (Objective) is the internal target for it ("99.9%"); an **SLA** (Agreement) is a customer

contract with credits or refunds if missed, normally looser than the SLO so the team is warned first. Choose SLIs users feel (failed checkouts, not CPU) and never set an SLO of 100%.

Error budgets

The **error budget** is 1 − SLO: at 99.9%, 0.1% of requests (100,000 of 100 million a month, ≈ 43 minutes of full outage) may fail. While budget remains, ship fast; when spent, slow releases and fix reliability. The idea comes from Google's **SRE** (Site Reliability Engineering, running production with software-engineering methods); Part 5 covers burn-rate alerts.

→ Full explanation

MTBF, MTTR and measuring failure

MTBF (mean time between failures — higher is better) and **MTTR** (mean time to repair/recover — lower is better); related: **MTTD** (mean time to detect) and **MTTF** (mean time to failure, for replaced parts like disks). Availability ≈ MTBF / (MTBF + MTTR): 1,000 h / 1,001 h ≈ 99.9%. At scale failures are constant, so cutting MTTR (monitoring, automatic failover, quick rollback) is usually the cheaper lever — the same failures with 10-second automatic recovery give ≈ 99.9997%. Detection time counts toward MTTR.

→ Full explanation

Reliability, availability and durability

Reliability: keeps doing the right thing correctly over time despite faults (measured by failure rate, MTBF). **Availability**: up and responding right now (% uptime). **Durability**: saved data stays saved (probability of losing an object per year). They are separate promises: Amazon S3 Standard is designed for 99.999999999% ("eleven nines") durability but 99.99% availability (SLA credits below 99.9%). A system can be available but wrong, or down but durable. Availability comes from redundancy and failover; durability from copies on separate disks/sites, writing to disk before acknowledging, and backups — replication is *not* backup, because a bad delete replicates too.

→ Full explanation

Faults, failures and fault tolerance

A **fault** is one component going wrong; a **failure** is the whole system no longer serving users; **fault tolerance** (resilience) keeps faults from becoming failures, perhaps in degraded form.

Fault type	Example	Defence
Hardware	Disk dies	Redundant machines, replicated data
Software	Bug, leak	Testing, gradual rollouts, restarts
Network	Packet loss, partition	Timeouts, retries, replicas in other zones
Human	Bad config, deleted table	Reviews, automation, backups, rollback
Dependency	Third-party API down	Timeouts, fallbacks, circuit breakers

Failure models are formal assumptions about how a faulty part behaves, each harder to tolerate than the last: **fail-stop** (halts and everyone reliably learns it; f+1 copies tolerate f), **crash (crash-stop)** (halts silently — looks like slowness, so consensus needs 2f+1 nodes), **crash-recovery** (restarts with only what it saved to disk — write durably before acknowledging), **omission** (drops some messages — acks and idempotent retries), **timing (performance)** (correct but too late or with drifting clocks — timeouts, leases, hedging), **Byzantine (arbitrary)** (wrong or malicious answers — 3f+1 replicas, signatures, checksums). Company data centres usually assume **crash-recovery plus omission plus timing faults**, plus checksums against corrupted data; blockchains design for Byzantine faults. State your model: it sets the replica count (Raft: 3 nodes survive 1 crash; Byzantine tolerance needs 4). The pattern is detect →

isolate → recover. Beware **correlated failures** (a shared power supply or bad deploy breaking "independent" copies).

→ Full explanation

Redundancy, single points of failure and failover

Redundancy is having spares; a **single point of failure (SPOF)** is any part whose loss alone stops everything (one DB, one LB, one DNS provider); **failover** switches work to a standby. Remove SPOFs by running ≥ 2 copies in separate **failure domains** (racks, **availability zones** — separate data centres in a region with own power and network — or regions), routing around dead copies, **health checks**, and automated failover. **Active-passive** (primary-standby) keeps an idle up-to-date standby; **active-active** serves from all copies (harder for writable state). Standbys are **cold** (must be started, minutes–hours), **warm** (running, partly current) or **hot** (in sync, seconds). **N+1/N+2** means one or two units beyond peak need. Risk: wrongly declaring a live primary dead gives **split brain** (two primaries accepting writes), prevented with quorums and fencing (Parts 3–4); test failover regularly.

→ Full explanation

Scalability

Scalability is handling more load by adding resources, without redesign or performance collapse, with cost growing roughly in proportion. Load is described by **load parameters** (QPS, concurrent users, read/write ratio, data size, followers per user); "scalable" only means something relative to one of them. Dimensions: **load** (more requests/s), **data** (more stored data), **geographic** (users worldwide needing low latency everywhere) and **organisational** scalability (more engineers without stepping on each other). Core techniques: scale out stateless servers, cache hot data, read replicas, sharding, queues for slow work, CDN for static content. Scale the actual bottleneck, and design so the next 10× is easy rather than 1,000× on day one.

→ Full explanation

Vertical and horizontal scaling

Vertical scaling (scale up) makes one machine bigger (cloud instances reach hundreds of cores and terabytes of RAM); **horizontal scaling (scale out)** adds machines and spreads work.

	Vertical	Horizontal
Simplicity	No code change	Needs LB, stateless design or sharding
Limit	Biggest machine	Practically unlimited
Fault tolerance	Still a SPOF	Losing a node loses a fraction
Cost	Big machines cost disproportionately	Commodity machines
Consistency	Easy, one copy	Harder

Scaling up first is often right (Stack Overflow in 2016 ran on 11 web servers and 4 SQL Server machines). What makes scale-out hard is **state**: stateless servers are trivial; data must be replicated or partitioned. Most systems do both.

→ Full explanation

Elasticity

Elasticity is adding *and removing* resources automatically and quickly as load changes, so you pay for roughly what you use (scalability = *can* grow; elasticity = grows and shrinks by itself). Mechanism: watch a metric (CPU, RPS, queue length), add servers above a threshold, remove below, LB follows — **autoscaling** (Part 5), also provided by **serverless** platforms that scale to zero. Limits: boot time

(seconds for containers, minutes for VMs), mostly stateless parts only, and **flapping** if scale-in is too eager.

→ Full explanation

Stateless and stateful services

State is data remembered between requests. A **stateless service** keeps no client-specific data in its memory, so any copy can serve any request and copies can be added, killed or replaced freely; a **stateful service** ties a client or data to a particular server. Make app servers stateless by putting sessions in a shared store (Redis) or a signed token (JWT), files in object storage, carts in a DB or cache. Stateful parts that remain (databases, game servers, WebSocket servers) need **sticky sessions** and a plan for server loss. Cost of statelessness: an extra hop (usually < 1 ms) — and the state store itself now needs scaling and redundancy.

→ Full explanation

Performance vs scalability

Performance is how fast the system is for one request at current load; **scalability** is how well performance holds as load grows. Diagnostic: one request to an idle system is slow → performance problem (profile it; more servers won't help); fast alone but slow under traffic → scalability problem. Some changes trade one for the other (a shared-cache hop costs a little per-request speed but enables scale-out).

→ Full explanation

Concurrency vs parallelism

Concurrency is many tasks in progress over the same period, interleaved whenever one waits (how work is *organised*); **parallelism** is tasks running at the same instant on multiple cores or machines (how it is *executed*). Web servers mostly wait on I/O, so concurrency lets one server juggle thousands of requests; parallelism speeds CPU-heavy work. Mechanisms: **processes** (isolated memory), **threads** (shared memory, need locks), **event loop / async I/O** (one thread never blocks: Node.js, NGINX, asyncio), **lightweight threads/coroutines** (goroutines, Java virtual threads). Hazards: **race condition** (result depends on timing, e.g. a lost withdrawal), **deadlock** (tasks wait forever on each other's locks), contention. **Amdahl's law**: speed-up is limited by the serial part (10% serial → at most 10×). A single-threaded Node.js server is concurrent but not parallel.

→ Full explanation

Bottlenecks: CPU, memory, disk and network

A **bottleneck** is the part limiting overall capacity; improving anything else doesn't help. Signs: **CPU** near 100%; **memory** swapping or OOM; **disk** high wait time or IOPS/MB/s at limit; **network** saturated link, loss, high RTT. Non-hardware bottlenecks are as common: one hot lock or row, a too-small connection pool, a rate-limited external API. Measure under load, find what is at its limit or where queues form, fix, re-measure (the bottleneck moves). **CPU-bound** work is limited by computation, **I/O-bound** by waiting; most web services are I/O-bound.

IOPS and disk performance

IOPS (input/output operations per second) limits small random access; **throughput** (MB/s) limits large sequential transfers — a disk can hit one without the other. Rough figures: HDD (7,200 rpm) 75–150 IOPS, 100–250 MB/s; SATA SSD tens of thousands, ~500 MB/s; NVMe SSD hundreds of thousands to ~1 million, several GB/s; AWS gp3 baseline 3,000 IOPS and 125 MB/s. HDD random reads are slow because the arm must move (5–10 ms). Cloud disks have provisioned limits that can throttle a DB while CPU looks idle.

→ Full explanation

How to approach a system design interview

A 45–60 minute open-ended design conversation; use a repeatable structure (rough timings for 45 min):

1. **Requirements (5 min)** — **functional** (what it does: features) and **non-functional** (how well: scale, latency, availability, durability, consistency, security, cost); agree what is out of scope.
2. **Estimates (3–5 min)** — QPS average and peak, read/write ratio, storage, bandwidth, and what they imply.
3. **API (5 min)** — main endpoints with inputs/outputs.
4. **Data model (5 min)** — entities, keys, access patterns, which database fits.
5. **High-level design (10 min)** — boxes and arrows; walk one read and one write end to end.
6. **Deep dives (10–15 min)** — the hardest parts given the numbers (ID generation, sharding, hot keys, failures).
7. **Trade-offs and wrap-up (3–5 min)** — weaknesses, SPOFs, monitoring, what changes at 10×.

Non-functional requirements and the levers that meet them

A non-functional requirement helps only if **measurable** ("p99 < 200 ms", not "fast") and tied to a **design lever**:

Requirement	Measure	Levers	Cost
Availability	Success-rate SLO, allowed downtime	Multi-zone redundancy, health checks, failover, degradation	Machines, copy consistency
Latency	p50/p95/p99 per endpoint	Caching, CDN, fewer round trips, async, headroom	Staleness, money
Scalability	Peak QPS, data growth	Stateless + LB, sharding, queues	Cross-shard work
Consistency	Who must see a write, how soon	Single leader + sync replicas, quorums, consensus, or eventual + conflict handling	Write latency, partition availability
Durability	No lost acknowledged write; RPO	WAL, replicate before ack, backups, cross-region	Latency, storage
Reliability	Error rate, MTTR	Timeouts, backoff retries, idempotency, breakers, bulkheads	Retry load, complexity
Security and privacy	Who may do what, sensitive data, compliance	AuthN/AuthZ at the edge, encryption, least privilege, audit logs	Check latency, ops
Maintainability	Time to diagnose, time to ship, deploys/day	See below	Up-front effort
Cost	Per request, user or GB	Right-size, autoscale, storage tiers, caching	Less headroom

Maintainability (most lifetime cost is operating and changing a system) splits into **operability** (easy to keep healthy: telemetry, SLO alerts, runbooks, automated deploys/rollbacks), **simplicity** (easy to understand: few component types, clear ownership, no **accidental complexity**) and **evolvability** (easy to change: versioned APIs, backward-compatible schemas, loose coupling, feature flags, online migrations, tests) — after Kleppmann's DDIA ch. 1 and ISO/IEC 25010.

Think aloud, justify every component with a requirement or number, give a complete simple design before depth, and never name a technology without the reason.

→ Full explanation

Part 1 glossary

Every Part 1 term is defined in bold in the sections above. A few extra one-liners:

Term	Meaning
Order of magnitude	Rough size as a power of ten
Fan-out	One request triggering many parallel downstream requests
Round trip time (RTT)	Time for a message to reach a destination and the reply to return
Transmission delay	Time to push a message's bits onto a link
NVMe	Fast SSD interface with very high IOPS
Thread	Independent flow of execution inside a process

→ Full explanation

Part 2 — Networking and communication

Part 1 was the boxes; this part is the arrows: how one computer finds another, how bytes travel, which **protocols** (agreed rules for how two programs talk) they speak, and which middlemen (proxies, load balancers, CDNs) sit in between.

> **Part 2 TL;DR:** IP finds the machine, the port finds the program; DNS maps names to IPs through cached root/TLD/authoritative servers; TCP is reliable and ordered but costs a handshake and head-of-line blocking, UDP is fire-and-forget, QUIC (HTTP/3) is reliable multi-stream encrypted transport over UDP; a fresh HTTPS connection costs ~2 RTTs before the request; REST for public APIs, gRPC inside, GraphQL for flexible clients; load balancers (L4 vs L7, algorithms, health checks, draining) make scale-out work; CDNs cache near users; round trips, not bandwidth, usually dominate.

→ Full explanation

IP addresses, NAT, ports and sockets

An **IP address** identifies a machine's network connection; routers forward each **packet** (a chunk of up to ~1,500 bytes on Ethernet) by destination IP. **IPv4** is 32 bits (~4.3 billion, exhausted in the 2010s); **IPv6** is 128 bits (~3.4 × 10^38). A **port** (0–65535) picks the program (22 SSH, 53 DNS, 80 HTTP, 443 HTTPS, 5432 PostgreSQL, 6379 Redis). A **socket** is IP + port; a TCP connection is identified by the **5-tuple** (protocol, source IP/port, destination IP/port), which is why one server port holds many connections. **Private IPs** (`10.0.0.0/8` , `172.16.0.0/12` , `192.168.0.0/16` — **CIDR** notation giving the fixed leading bits) live inside a network such as a cloud **VPC** (your isolated cloud network); **NAT** rewrites them to a shared **public IP** (a **NAT gateway** in the cloud). Consequences: servers can't call clients behind NAT (so push uses client-opened connections); a proxy to one backend has ~64,000 source ports (~28,000 with Linux defaults), so **ephemeral port exhaustion** is real; **carrier-grade NAT** makes IP-based rate limits hit many users at once; IPs change, so use DNS and service discovery.

→ Full explanation

DNS — the internet's phone book

DNS turns a **domain name** into IPs. A name is a hierarchy read right to left: root `.` → **top-level domain (TLD)** (`com`) → domain (`example`) → **subdomain** (`www`). Players: the OS **stub resolver**; the **recursive resolver** (ISP, company, 8.8.8.8, 1.1.1.1) that does the work and caches; **root servers** (13 named identities, ~2,000 anycast instances); **TLD servers**; the **authoritative server** holding the real records.

On a miss the resolver walks root → TLD → authoritative (the stub sends one **recursive query** — "give me the final answer"; the resolver then follows **iterative queries**, each server returning a **referral** to the next) and caches each answer for its **TTL**; busy resolvers usually answer from cache. Records: **A/AAAA** (name → IPv4/IPv6), **CNAME** (alias; not allowed at the bare **apex** domain, hence ALIAS/ANAME/CNAME flattening), **MX** (mail), **NS** (authoritative servers), **TXT** (verification, SPF), plus SOA, SRV, PTR, CAA; a **zone** is one server's set of records. Long TTLs mean fewer lookups but slow changes ("propagation" is just caches expiring); short TTLs (30–60 s) allow fast failover — lower the TTL before a migration. DNS also steers traffic: **round-robin DNS** (several A records, blind to health), **GeoDNS** (answer by location, helped by **EDNS Client Subnet**), **latency-based routing**. A cold lookup costs 20–120 ms; DNS outages (Dyn 2016, Facebook 2021) make healthy sites look down, so many use two providers.

→ Full explanation

Network layers — OSI and TCP/IP models

Networking is split into **layers**, each wrapping the data with its own header (**encapsulation**): HTTP request → TCP segment (ports) → IP packet (addresses) → Ethernet frame (**MAC address**, a network card's hardware address). The 7-layer **OSI model** is teaching vocabulary (7 application, 6 presentation, 5 session, 4 transport, 3 network, 2 data link, 1 physical); the 4-layer **TCP/IP model** (application, transport, internet, link) is what runs. "L3/L4/L7" decides what a device can see: L3 (routers, IP firewall rules) sees IPs; **L4** also sees ports and TCP/UDP state but not content; **L7** reads the HTTP request (path, headers, cookies) and can route by it, at the cost of CPU and decrypting TLS.

→ Full explanation

TCP, UDP and QUIC

These are **transport-layer** protocols. **TCP** is a connection-based, reliable, ordered byte stream: a **three-way handshake** (SYN, SYN-ACK, ACK) costs one **RTT** before data; **sequence numbers**, **ACKs** and **retransmission** give reliability; **flow control** (receive window) protects the receiver; **congestion control** (slow start; Reno, CUBIC — Linux default — and BBR) protects the network, so new connections start slow; either side closes with **FIN**, and the side that closes first holds **TIME_WAIT** (60 s on Linux). **Head-of-line (HOL) blocking**: one lost packet stalls all later bytes, including other requests sharing the connection. **UDP** sends independent **datagrams** with no handshake, ACKs, ordering or congestion control (8-byte header vs TCP's 20+) — used for DNS, video calls, games, where late data is useless. **QUIC** (IETF standard 2021, from Google) runs over UDP and combines transport + TLS 1.3 into 1 RTT (**0-RTT** on resume), has independent streams (loss stalls only its stream), **connection migration** via a **connection ID** (survives Wi-Fi → 4G), is always encrypted and lives in user space (more CPU; some firewalls block UDP, so clients fall back). Every new TCP + TLS connection costs 2–3 RTTs, so reuse connections.

→ Full explanation

HTTP — the language of the web

HTTP is a request/response protocol: method, path, headers, optional body → status code, headers, body. Methods: GET (read), HEAD, POST (create/action), PUT (replace), PATCH (partial update), DELETE, OPTIONS. **Safe** = changes nothing (GET, HEAD, OPTIONS); **idempotent** = repeating leaves the same state (GET, HEAD, PUT, DELETE, OPTIONS), so it can be retried after a timeout; POST needs an **idempotency key**. Status classes: 1xx (101 WebSocket upgrade), 2xx (200, 201 Created, 202 Accepted = queued, 204), 3xx (301, 302, 304 Not Modified), 4xx client error (400, 401 = not authenticated, 403 = authenticated but not allowed, 404, 409, 429), 5xx server error (500, 502 bad gateway answer, 503, 504 gateway timeout). Key headers: `Host`, `Content-Type`, `Authorization`, `Cache-Control`, `ETag` / `If-None-Match` (→ 304), `Accept-Encoding`, `X-Forwarded-For` (original client IP). HTTP is **stateless**; **cookies** (`Set-Cookie` with `HttpOnly; Secure; SameSite`) carry a session ID whose data lives in a shared store or signed token. **Keep-alive** reuses connections (default since HTTP/1.1).

	HTTP/1.1 (1997)	HTTP/2 (2015)	HTTP/3 (2022)
Transport	TCP	TCP	QUIC (UDP)
Format	Text	Binary frames, **HPACK** header compression	Binary, **QPACK**
Parallelism	One request at a time per connection (~6 connections)	**Multiplexing**: many interleaved streams on one connection	Streams on one connection
HOL blocking	HTTP and TCP level	TCP level	Per stream only

HTTP/3 is discovered via `Alt-Svc` or a DNS HTTPS record, falling back to HTTP/2. Don't use GET for state changes, and don't return 200 with an error body.

→ Full explanation

HTTPS and TLS

HTTPS is HTTP inside **TLS** (formerly **SSL**), giving **confidentiality**, **integrity** and **authentication** — **encryption in transit** (vs **at rest** for stored data). Current version TLS 1.3 (2018); 1.2 still common. A server proves identity with a **certificate** (domain, **public key**, expiry, signature by a **Certificate Authority**), checked along the **chain of trust** to a pre-installed **root CA**; Let's Encrypt issues free 90-day certificates renewed by ACME (planned 64 days in 2027, 45 in 2028), so automate renewal. **Asymmetric** crypto (key pair) is used only in the handshake to authenticate and agree a secret via **Diffie–Hellman** (each side sends a **key share** and both derive the same key without ever sending it); bulk data uses fast **symmetric** ciphers (AES-GCM, ChaCha20). TLS 1.3 takes **1 RTT** (1.2 took 2), so a fresh HTTPS-over-TCP request costs ~3 RTTs including the request; **session resumption** and **0-RTT** (replayable, so idempotent requests only) help returning clients. Fresh keys per session give **forward secrecy**. **SNI** puts the hostname in the ClientHello so one IP serves many certificates; **mTLS** has both sides present certificates (service-to-service).

→ Full explanation

API styles — REST, RPC and gRPC, GraphQL

An **API** is the set of operations a service offers to other programs. **REST** models **resources** (plural nouns: `/users/42/orders`) manipulated by HTTP methods, stateless and cacheable, usually JSON. **RPC** models remote functions (`chargeCard(order)`); **gRPC** (Google) runs on HTTP/2 with **Protocol Buffers** — a compact binary schema format whose numbered field tags allow backward-compatible evolution (add numbers, never reuse) — generates typed clients, supports unary, server, client and bidirectional **streaming**, and needs gRPC-Web or a gateway for browsers. **GraphQL** (Facebook, 2015) exposes one endpoint with a typed **schema**; clients query exactly the fields they want, writes are **mutations**, live updates **subscriptions**, each field filled by a **resolver**; risks are weak HTTP caching, expensive queries (limit depth/cost) and the **N+1 query problem** (fixed by DataLoader batching). REST suffers **over-fetching/under-fetching**.

RPC internals: stubs, marshalling, binding and call semantics

RPC (Birrell and Nelson, Xerox PARC, 1984; later ONC RPC, Java RMI, Thrift, gRPC) has five parts: an **interface definition (IDL)** both sides compile against (gRPC: `.proto`); a **client stub** with the remote procedure's signature that packs arguments and unpacks results; a **server stub (skeleton)** that unpacks, calls the real procedure and packs the result; **marshalling/unmarshalling** to and from bytes (fixing byte order, sizes, encoding; no pointers, structures copied by value); and the **RPC runtime** (connections, matching replies by **call ID**, timeouts, retransmission). **Binding** finds a server: static config, a directory (Grapevine; ONC's `rpcbind`), or DNS/service discovery with client-side balancing. A remote call is thousands of times slower and can **fail partially** — with no reply the caller **cannot know** whether it ran — so systems state **call semantics**: **maybe** (once or not, no retry), **at-least-once** (retry until reply; idempotent operations only), **at-most-once** (server dedups by call ID; caller may get "unknown outcome"), **exactly-once** (not achievable in general; approximated by retries plus durable dedup, i.e.

idempotency keys). gRPC has **no deadline by default** (always set one) and retries only when the server never saw the call or by an explicit per-method policy.

	REST	gRPC	GraphQL
Best for	Public APIs, CRUD, caching	Internal calls, streaming	Many client types, complex screens
Weak spot	Over/under-fetching	Browser support, debugging	Caching, query cost

→ Full explanation

Real-time communication — polling, long polling, WebSockets, SSE, webhooks

Plain HTTP only lets the client ask, so server push means a client-opened channel or, between servers, a callback. **Short polling** asks every few seconds (wasteful). **Long polling** holds the request open until news or timeout (~30 s), then re-asks. **WebSockets** upgrade an HTTP/1.1 request (`Upgrade: websocket` → `101 Switching Protocols`) into a two-way framed channel (`ws://` / `wss://`); send **heartbeats** (ping/pong every 20–60 s) so idle connections aren't cut (AWS ALB idle timeout 60 s by default) and reconnect with backoff. **Server-Sent Events (SSE)** stream one-way events in a never-ending `text/event-stream` response; the browser's `EventSource` reconnects with `Last-Event-ID` (used for LLM token streaming). **Webhooks** are server-to-server HTTP POSTs to a registered URL — sign them (HMAC), retry with backoff, and make receivers idempotent since events repeat and arrive out of order. Persistent connections make servers **stateful** (capacity = open connections; route messages to the right server via pub/sub; drain on deploy).

→ Full explanation

API design basics

Versioning: publish a new version only for breaking changes (URL `/v1/`, header, or Stripe-style dated versions); prefer **additive** changes and tell clients to ignore unknown fields. **Pagination**: never return unbounded lists. **Offset pagination** (`limit=20&offset=40`) allows page jumps but slows deep in the list and shifts under inserts; **cursor (keyset) pagination** (`after=<opaque last-key token>` → `WHERE id < 1234 ORDER BY id DESC LIMIT 20`) is constant-speed and stable — used by feeds. **Rate-limit headers** (`X-RateLimit-Limit/Remaining/Reset`; IETF standardising `RateLimit`) plus **429 Too Many Requests** with `Retry-After`. Also: consistent error bodies, idempotency keys on POST, filtering/sorting via query parameters, and an **OpenAPI** (formerly Swagger) specification.

→ Full explanation

Proxies and API gateways

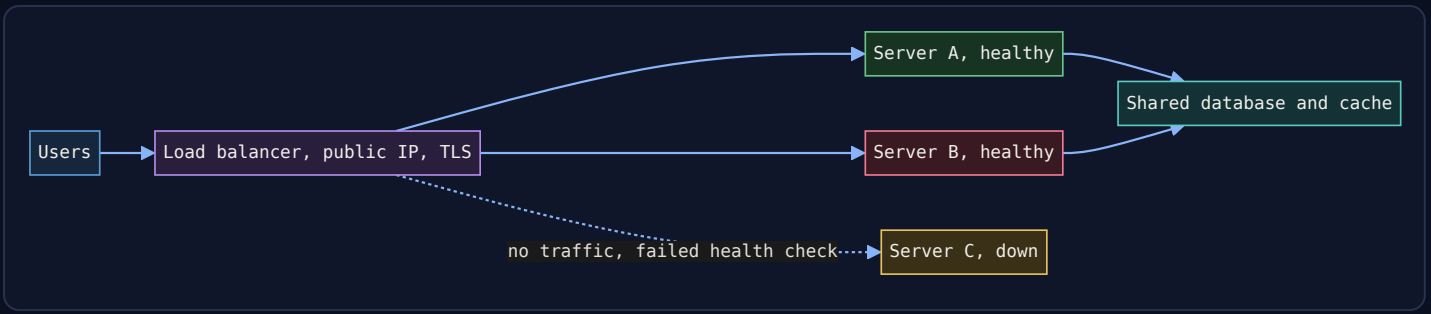
A **proxy** forwards requests on someone's behalf. A **forward proxy** acts for **clients** (corporate filtering, hiding client IPs, caching downloads); a **reverse proxy** acts for **servers** (load balancing, TLS termination, caching, compression, hiding internals — NGINX, HAProxy, Envoy). An **API gateway** is a reverse proxy specialised for APIs, the single front door that terminates TLS, authenticates (API key, JWT, OAuth token), rate-limits, routes by path/host/header, transforms or **aggregates** calls, and records logs/metrics/traces (AWS API Gateway, Kong, Apigee). **BFF (Backend for Frontend)** is one gateway per client type. Each proxy is an extra hop and potential SPOF (make it redundant); keep business logic out of the gateway; forward the real client IP via `X-Forwarded-For` (or the **PROXY protocol** at L4).

→ Full explanation

Load balancers

A **load balancer (LB)** spreads incoming traffic across a **pool** (target group) of servers, sending only to healthy ones; clients see one **virtual IP (VIP)**. It enables scale-out, survives server loss, allows

zero-downtime deploys, centralises TLS and logging and hides backends; servers behind it should be stateless.



Types

L4 balances TCP/UDP connections by IP/port without reading content: microseconds of overhead, any protocol, but per-connection (a long HTTP/2 or gRPC connection sticks to one backend) — AWS **NLB**. **L7** terminates the connection and parses HTTP: routes by path/host/header/cookie, balances per request, does TLS termination, compression, retries, redirects, WAF features — more CPU (~1 ms) — AWS **ALB**. Common layering: L4 in front of a fleet of L7 proxies. Forms: **hardware** appliances (F5 BIG-IP, NetScaler; fast, expensive), **software** (NGINX, HAProxy, Envoy, Traefik; Google **Maglev**, Meta **Katran** on commodity Linux), **managed cloud** (ALB, NLB, Gateway LB, Google Cloud LB, Azure), **DNS load balancing**, and **GSLB** (global server load balancing across regions via DNS steering and/or **anycast**). **Client-side load balancing** lets each client pick from a registry's healthy list (gRPC, Netflix Ribbon, Envoy sidecars), saving a hop.

Load-balancing algorithms

Static: **round robin** (in turn; blind to load), **weighted round robin** (by capacity; also for **canary releases**), **IP hash** (`hash(client IP) mod N`; sticky but remaps almost everyone when N changes), **consistent hashing** (ring with **virtual nodes**; only ~1/N keys move — for cache affinity; HAProxy/Envoy ring hash and Maglev), **random**. **Dynamic**: **least connections** (long or uneven requests, WebSockets), **least response time** (smoothed, e.g. **EWMA**; risks **herding**), **power of two choices (P2C)** — pick two at random, send to the less loaded; avoids herding across many LB instances (Envoy's least-request uses P2C; NGINX `random two least_conn`). Others: least bandwidth, resource-based, URL hash, locality-aware.

Health checks

Active checks probe each backend (TCP connect or `GET /health`), e.g. every 10 s, unhealthy after 2–3 failures, healthy after 2–5 successes. **Passive** checks (**outlier detection**) eject backends producing errors in real traffic (Envoy; NGINX `max_fails`). **Liveness** = process running; **readiness** = able to serve now. Avoid deep checks that fail the whole fleet on a shared-dependency blip; LBs **fail open** (send to all if all look unhealthy — AWS target groups, Envoy panic threshold).

Sticky sessions (session affinity)

Route one user to the same backend via an LB cookie (AWS `AWSALB`) or IP hash. Needed for unavoidable in-memory state (WebSockets); otherwise avoid — uneven load, lost sessions on scale-in. Prefer a shared session store or signed token.

SSL/TLS termination

The LB decrypts HTTPS and forwards plain HTTP inside the private network: one place for certificates, saves backend CPU, enables L7 routing/WAF — but internal traffic is unencrypted. **Re-encryption (bridging)** decrypts, inspects, re-encrypts; **passthrough** leaves decryption to backends (routing only by SNI).

Connection draining (deregistration delay)

On removal, stop sending *new* requests but let in-flight ones finish up to a timeout (AWS default 300 s, often 30–60 s); tell WebSocket clients to reconnect. Draining + health checks make **rolling deployments** invisible.

Making the load balancer itself highly available

Active–passive LB pair sharing a VIP via **VRRP** (`keepalived`); **active–active** nodes behind DNS, anycast or **ECMP** (routers spreading flows across LB machines announcing one IP) — how Maglev, Katran and cloud LBs work; managed ALB/NLB run nodes in every enabled AZ.

Direct server return (bonus detail)

In **DSR** an L4 LB forwards inbound packets but servers reply straight to clients, so the LB carries only the small inbound half (Maglev, Katran, LVS).

Mistakes: a single LB box, round robin for WebSockets, sticky sessions instead of a session store, L4 in front of gRPC (needs per-request balancing), forgetting `X-Forwarded-For`.

→ Full explanation

CDNs and anycast

A **CDN** is a worldwide network of caching **edge servers** grouped in **PoPs** (points of presence), serving content near users (Sydney fetches from Sydney, not Virginia). The **origin** holds the master copy; **cache hit/miss** and **hit ratio** (95% → origin sees 1 in 20) measure effectiveness; the **cache key** is host + path + selected parameters/headers. The origin controls caching with `Cache-Control` (`max-age`, `s-maxage` for shared caches, `no-store`, `private`) and validators (`ETag`, `Last-Modified` → 304). Benefits: lower latency (TLS terminates nearby), origin offload, absorbing spikes and **DDoS**. **Pull CDNs** fetch on first miss (most websites); **push CDNs** are uploaded in advance (large known files). Freshness: **purge** (seconds–minutes), **cache busting** with versioned names (`app.3f9a1c.js`, allows year-long TTLs), short TTLs. An **origin shield** is a middle tier collapsing edge misses into ~one origin request. Dynamic content still benefits (near TLS, warm connections, **edge computing** like Cloudflare Workers, Lambda@Edge). **Anycast** announces one IP from many sites via **BGP** (how networks advertise reachable IP ranges), routing users to the "nearest" site by routing policy — automatic proximity, DDoS spreading, fast failover (Cloudflare, 8.8.8.8, 1.1.1.1, DNS roots), versus **unicast** (one IP, one place). Never cache personalised responses under a shared key. Netflix runs its own CDN, Open Connect.

→ Full explanation

Bandwidth vs latency, and network partitions

Round trips usually dominate: 10 sequential trans-Atlantic round trips take ≥ 700 ms on any link speed — fix with fewer round trips (batching, HTTP/2, reuse) and moving data closer. Bulk transfers are bandwidth-bound: 10 TB over 1 Gbps ≈ 22 hours (hence AWS Snowball). Latency has a speed-of-light floor. The **bandwidth-delay product** (bandwidth × RTT) is the data in flight needed to fill a link (1 Gbps × 100 ms = 12.5 MB). A **network partition** splits healthy nodes into groups that can't talk; a node can't distinguish "crashed" from "unreachable", so systems use timeouts and heartbeats and then either stay consistent (minority side refuses) or stay available (reconcile later) — the CAP choice (Part 4); without quorums you risk split brain. The **fallacies of distributed computing** start with "the network is reliable", "latency is zero", "bandwidth is infinite". A timeout doesn't prove the remote operation failed — hence idempotent retries.

→ Full explanation

Part 2 glossary

Every Part 2 term is defined in bold above. Extra one-liners:

Term	Meaning
Router	Device forwarding packets toward their destination IP
Recursive vs iterative query	"Give me the final answer" vs "point me to who knows"
Session	Server-side record of a logged-in user, referenced by a cookie or token
Pool / target group	The set of backends behind a load balancer
Herding	Many balancers sending traffic to the same seemingly best server

→ Full explanation

Part 3 — Data storage

Code can be redeployed in minutes; data cannot be recreated. This part covers where data lives, how it is organised, how it survives crashes, and how it spreads over many machines once one is not enough.

> **Part 3 TL;DR:** Start with a relational database (ACID transactions, SQL, indexes) and know that most default to an isolation level weaker than serializable; NoSQL families each trade general querying for one access pattern; indexes turn scans into a few page reads but slow writes (B-tree for reads/ranges, LSM-tree for heavy writes); **copy for reads, split for writes** — replication for availability and read scaling, sharding by a good shard key for data size and write throughput, consistent hashing to move only ~1/N keys; put files in object storage; separate OLTP from OLAP; durability is layered: WAL + fsync, replicas, other regions, tested backups.

→ Full explanation

What a database is

A **database** (strictly a **database management system, DBMS**) stores data safely, finds it quickly and coordinates many concurrent users — things plain files can't do: indexed lookup among millions of rows in well under a millisecond, safe **concurrent** access, crash safety (a write fully happens or not at all) and a query language. Because app servers are stateless, the database holds the important **state**, making it the hardest part to scale and the focus of deep interview questions. Per request it parses, **plans** (the **query planner**/optimizer picks e.g. an index over a scan), executes (from the in-memory **buffer cache** or disk), **logs** writes durably first, and returns results. A **read** fetches without changing; a **write** inserts, updates or deletes; consumer apps often read 10–100× more than they write.

→ Full explanation

Relational databases and SQL

A **relational database** stores **tables** of **rows** (records) and **columns** (fields) under a **schema** declaring column **data types**; data that doesn't fit is rejected. You query it with **SQL**, which describes *what* you want (PostgreSQL, MySQL, Oracle, SQL Server, SQLite; cloud RDS, Aurora, Cloud SQL). A **primary key** uniquely identifies each row; a **foreign key** holds another table's primary key, and enforcing it is **referential integrity**; a **unique constraint** forbids duplicate values. A **join** combines matching rows: **inner join** (only matches), **left join** (all left rows, blanks where no match), **right join** (the mirror), **full outer join** (all rows from both). Schema-changing statements (`CREATE/ALTER/DROP TABLE`) are **DDL**. Strengths: ad hoc queries, transactions, enforced structure, mature tooling. Costs: schema changes on huge tables need online tools (`gh-ost` , `pt-online-schema-change`); scaling writes past one machine means sharding or **distributed SQL** (Spanner, CockroachDB, YugabyteDB, TiDB). One modern server (64 cores, 512 GB) handles tens of thousands of simple transactions/s — don't pick NoSQL "for scale" for a few hundred GB. Use surrogate IDs, not emails, as keys; no comma-separated lists in a column.

→ Full explanation

Transactions and ACID

A **transaction** groups reads and writes into one unit: `BEGIN … COMMIT` makes it permanent, `ROLLBACK` discards it. **ACID**: **Atomicity** — all or nothing, undone on failure or crash; **Consistency** — moves data between valid states per your declared rules (largely the application's job; not CAP's C); **Isolation** — concurrent transactions don't see each other's unfinished work (ideally as if serial; usually weaker levels for speed); **Durability** — once committed, it survives crashes (log flushed to disk before replying). Costs: locks or version tracking, and an fsync per commit (≈ 0.1–2 ms on SSD). A transaction across machines (**distributed transaction**, Part 4) is far slower. Never hold a transaction open across a slow external call.

→ Full explanation

Isolation levels and anomalies

Concurrent transactions touching the same data can produce **anomalies**: **dirty read** (reading uncommitted data), **dirty write** (overwriting uncommitted data; prevented almost everywhere), **non-repeatable read / read skew** (a re-read row changed), **phantom read** (a re-run query returns new rows), **lost update** (two read-modify-writes, one lost: 10 → 11 instead of 12), **write skew** (two transactions read the same rows, decide, and write *different* rows, together breaking a rule — e.g. both on-call doctors go off call).

Level	Dirty read	Non-repeatable	Phantom
Read uncommitted	Possible	Possible	Possible
Read committed	Prevented	Possible	Possible
Repeatable read	Prevented	Prevented	Possible (per standard)
Serializable	Prevented	Prevented	Prevented

Snapshot isolation (not in the original standard) reads a frozen snapshot via **MVCC** (multiple row versions, so readers and writers don't block); it still allows write skew. **Serializable** equals some one-at-a-time order: by actual serial execution (Redis, VoltDB), **two-phase locking**, or **serializable snapshot isolation (SSI)** (PostgreSQL). Defaults: PostgreSQL read committed (its repeatable read is snapshot isolation); MySQL InnoDB repeatable read; Oracle read committed (its "serializable" is snapshot). Without full serializability use atomic updates (`SET n = n + 1`), `SELECT … FOR UPDATE`, **optimistic concurrency control / compare-and-set** (version column, retry if 0 rows updated) or constraints. Last-ticket fix: `UPDATE … SET left = left - 1 WHERE id = 9 AND left > 0` and check one row changed.

→ Full explanation

NoSQL database families

NoSQL ("not only SQL", popular from ~2009 after Bigtable, Dynamo, Cassandra) is an umbrella for non-relational stores, each shaped around one access pattern and giving up joins, ad hoc queries or strong transactions.

Family	Model	Examples	Typical use
Key-value store	`put/get` by key, opaque value	Redis, Memcached, DynamoDB, etcd, Riak	Sessions, carts, counters, flags
Document store (document database)	JSON-like nested **documents** in **collections**, indexable fields	MongoDB (16 MB max doc), Couchbase, Firestore	Self-contained records read whole
Wide-column store	Rows grouped by **partition key**, sorted by **clustering key**, one table per query	Cassandra, ScyllaDB, HBase, Bigtable	Huge write volume read by key and time
Graph database	**Nodes** and **edges** with properties	Neo4j (Cypher), Neptune	Social graphs, fraud rings
Time-series database	Timestamped points, compression, retention	InfluxDB, TimescaleDB, Prometheus	Metrics, sensors
Search engine	**Inverted index** (word → documents), **tokenising**, stemming, **relevance ranking** (BM25)	Elasticsearch, OpenSearch (Lucene), Solr	Full-text search, as a secondary copy
Vector database	**Embeddings** (number vectors where similar items are close), **nearest-neighbour search** via **approximate nearest neighbour (ANN)** indexes such as HNSW	Pinecone, Milvus, pgvector	Semantic search, RAG

"Wide-column" is not "columnar" (analytics storage, below). Lines have blurred: MongoDB has multi-document transactions since 4.0, DynamoDB has transactions, PostgreSQL indexes JSON (`jsonb`).

→ Full explanation

SQL vs NoSQL and BASE

Choose from access patterns and correctness needs, not fashion. **BASE** describes many early NoSQL systems: **Basically Available** (keeps answering during failures, maybe with old data), **Soft state** (copies change while catching up), **Eventually consistent** (copies converge once writes stop). Lean SQL for money, inventory, bookings, complex queries and reporting, and when data fits one machine; document stores for varying nested shapes; wide-column/key-value for massive writes, key lookups and multi-region; graph for many-hop relationships. Using several types in one system is **polyglot persistence**. "NoSQL has no schema" is wrong: it is **schema-on-read** (in code) versus SQL's **schema-on-write**.

→ Full explanation

Indexes

An **index** is an extra structure, maintained on every write, that finds rows by value without a **full table scan**; check usage with `EXPLAIN` .

- **B-tree** — default: a shallow balanced tree of fixed-size **pages** (PostgreSQL 8 KB, InnoDB 16 KB) with high **branching factor** (~4 levels for billions of rows; a lookup walks root → **leaf**, one page per level); supports `=` and **range queries**/ordering; in-place updates (random writes).
- **LSM-tree** (Cassandra, ScyllaDB, HBase, RocksDB, LevelDB) — writes go to a log and an in-memory sorted **memtable**, flushed as immutable **SSTables** (sequential writes); background **compaction** merges files and drops old versions and deletion markers (**tombstones**); reads check memtable then SSTables, skipping files via **Bloom filters**. Faster writes, slower reads: the **read vs write amplification** trade-off.
- **Hash index** — O(1) exact lookups, no ranges (Bitcask/Riak; PostgreSQL `HASH`).
- **Composite index** `(user_id, created_at)` — helps queries on its **leftmost prefix** only; equality column first, range column after.
- **Covering index** — holds every needed column, enabling an **index-only scan** (`INCLUDE`).

- **Clustered** (rows stored in index order; InnoDB clusters by primary key) vs **secondary** indexes; also unique, **partial**, full-text, GIN/GiST, geospatial.

Costs: slower writes, space, memory; low-**selectivity** columns rarely benefit. Functions on columns (`LOWER(email)`) and leading wildcards (`LIKE '%shoe'`) defeat B-trees; build big indexes with `CREATE INDEX CONCURRENTLY`.

→ Full explanation

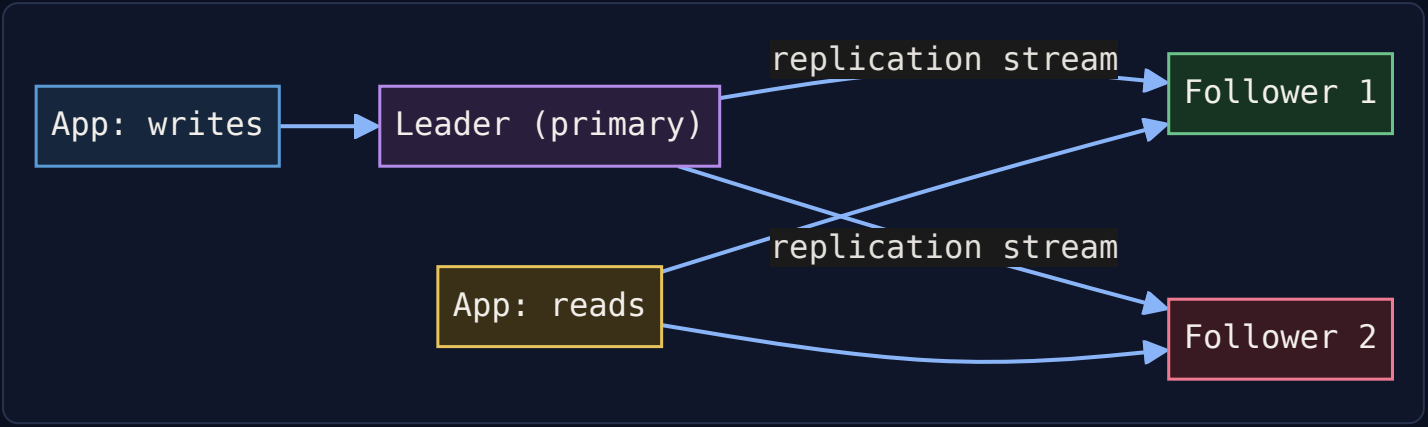
Normalization and denormalization

Normalization stores each fact once; **denormalization** duplicates facts to speed reads. Normal forms: **1NF** one value per cell; **2NF** non-key columns depend on the whole key; **3NF** they depend only on the key ("the key, the whole key, and nothing but the key", William Kent, 1983). Normalization prevents **update anomalies** (10,000 copies to rename). Denormalization techniques: copied columns, stored counts (`comment_count`, updated atomically), nested documents, **materialized views** (stored, refreshed query results). Start normalized; denormalize measured hot paths and decide how copies stay in sync; at scale and in join-less NoSQL (one Cassandra table per query) denormalization is the norm.

→ Full explanation

Replication

Replication keeps copies (**replicas**) of the same data on several machines for availability, read scaling and locality — but not write throughput: **copy for reads, split for writes**.



- **Leader-follower replication** (primary-replica): one **leader** takes writes and streams its log to **followers**; followers serving reads are **read replicas**. Default in PostgreSQL, MySQL, MongoDB, Redis.
- **Multi-leader replication**: several leaders (usually one per region) accept writes, so **write conflicts** must be resolved — **last write wins (LWW)** (loses data, clock-dependent), merge logic, or **CRDTs** (types whose concurrent updates always merge identically).
- **Leaderless replication** (Dynamo, Cassandra, Riak): write to W and read from R of N replicas, with **R + W > N** for overlap (a **quorum**, e.g. N=3, W=2, R=2); fix stale copies by **read repair**, **anti-entropy** (often Merkle trees) and **hinted handoff**.

Synchronous replication waits for a follower (no committed data lost, but slow followers stall writes); **asynchronous** replies immediately (fast, may lose the last writes on leader loss); **semi-synchronous** waits for one. **Replication lag** breaks **read-your-own-writes** (read own data from the leader briefly or track log position) and **monotonic reads** (pin users to a replica). **Failover** = detect (missed **heartbeats**, 10–30 s), choose the most up-to-date follower, reconfigure clients (Patroni, Orchestrator, managed DBs). **Split brain** is prevented by majority quorums, **fencing** (STONITH) and fencing tokens. Replicas are not backups. Examples: GitHub's 2018 43-second partition caused a cross-coast failover and a 24-hour degradation; Aurora keeps six copies across three AZs, writes need 4 of 6, reads 3 of 6.

→ Full explanation

Partitioning and sharding

Partitioning splits a dataset into pieces; **sharding** places the pieces (**shards**) on different machines, dividing both data and write load (each shard is still replicated). **Horizontal partitioning** splits rows; **vertical partitioning** splits columns (hot small vs cold large). The **shard key** decides placement:

Strategy	How	Strength	Weakness
Range	Shard owns a key range	Range queries hit few shards	Hot newest shard with timestamp keys
Hash	`hash(key)` picks shard	Even spread	Range queries scatter; `mod N` remaps on resize
Directory	Lookup service maps key → shard	Fully flexible	Extra lookup, must be HA
Geo	By user region	Latency, data residency (GDPR)	Uneven sizes

Routing happens in the app, a router (Vitess, `mongos`, Citus) or any node (Cassandra). **Federation** (functional partitioning) splits databases by feature, matching microservices. A good shard key has high **cardinality**, spreads load evenly, appears in the main query (single-shard; otherwise **scatter-gather**) and isn't monotonic under range sharding. A **hot spot** shard or **hot key** (celebrity, flash sale) needs **key salting**, caching, dedicated shards or write buffering. **Rebalancing** uses many fixed logical partitions moved whole (Elasticsearch, Couchbase, Riak), dynamic splitting (CockroachDB 512 MB ranges, MongoDB 128 MB chunks, HBase 10 GB regions) or consistent hashing. Costs: cross-shard joins and transactions, global IDs, **local vs global secondary indexes**, multiplied operations — so shard late, after indexes, caching, replicas and scaling up. Instagram and Notion (480 logical shards on 32, later 96, machines) map many logical shards to fewer servers.

→ Full explanation

Consistent hashing and virtual nodes

Consistent hashing assigns keys so adding or removing one of N servers moves only ~1/N of keys (plain `mod N` moves ~80% going from 4 to 5 servers). Servers and keys are hashed onto a **hash ring** (e.g. 0 to 2^32 − 1); each key belongs to the first server clockwise; replicas go to the next distinct servers. **Virtual nodes** give each server many ring positions (100–256; Cassandra moved its default from 256 to 16 in 4.0) for even load, spread-out takeover when a node leaves, and weighting by machine size. Alternatives: **rendezvous hashing** (highest score per key-server pair), **jump consistent hash** (fast, add/remove only at the end), **Maglev hashing**. Like all hashing it destroys key order. Used by Dynamo, Cassandra, Riak, Memcached clients (ketama), Discord and LBs.

→ Full explanation

Storage types: block, file and object

	Block	File	Object
Unit	Fixed-size blocks under a file system	Files in folders	Whole objects (**blob** + metadata) under a key in a **bucket**
Access	Attached disk (EBS, Persistent Disk)	Network protocol (**NFS**, **SMB**; EFS, NAS)	HTTP `PUT/GET/DELETE` (S3, GCS, Azure Blob, MinIO)
Shared	Usually one server	Many	Many
Partial edit	Yes	Yes	No, replace whole
Use	Databases, VM disks	Shared files	Media, backups, data lakes

Block storage is low-latency (local NVMe well under 1 ms; network block ~1–few ms) and measured in IOPS; object storage is cheapest per GB, practically unlimited and very durable but tens of ms to first byte. Upload pattern: the app issues a **pre-signed URL** (time-limited permission), the client uploads directly to object storage, the DB stores only key and metadata, downloads go through a CDN. S3: eleven nines durability, ≥ 3 AZs, objects up to 50 TB (since December 2025), strong read-after-write consistency since December 2020.

→ Full explanation

OLTP vs OLAP, row vs column storage, lakes and warehouses

OLTP is many small, fast transactional reads/writes from the application (ms latency; PostgreSQL, MySQL, DynamoDB); **OLAP** is few big analytical scans over many rows but few columns (seconds). Don't run OLAP on the OLTP database. A **row store** keeps each row together (good for OLTP); a **column store** keeps each column together, reading only needed columns, compressing well and processing in batches (ClickHouse, Druid, Pinot, Redshift, BigQuery, Snowflake, DuckDB; file formats **Parquet**, **ORC**). A **data warehouse** holds curated, structured, schema-on-write data, often as a **star schema** (a **fact table** of events plus **dimension tables**); a **data lake** holds raw data of any format cheaply in object storage, schema-on-read (risk: "data swamp"; queried by Spark, Trino/Presto, Athena); a **lakehouse** adds transactional table formats (Delta Lake, Iceberg, Hudi). Pipelines: **ETL** transforms before loading, **ELT** loads then transforms in the warehouse, **change data capture (CDC)** streams changes from the DB log (Debezium into Kafka).

→ Full explanation

Durability: WAL, fsync, backups and recovery

Durability is layered: WAL + fsync survive process crashes and power cuts; replication/RAID survive disk or machine loss; other zones/regions survive site loss; backups and point-in-time recovery survive bugs and humans; immutable separate-account backups survive ransomware. The **write-ahead log (WAL)** (redo log in InnoDB, commit log in Cassandra) appends each change sequentially and **fsyncs** it (forces it from the OS **page cache** to the device) before acknowledging; data pages are written later, a **checkpoint** marks how far they are current, and recovery replays the WAL. The same log feeds replication and CDC. fsync costs tens of µs to several ms, so databases use **group commit**; relaxed settings (`synchronous_commit = off`, `innodb_flush_log_at_trx_commit = 2`, Redis AOF fsync every second) can lose recent commits. **RAID** (1 mirror, 5/6 parity, 10 striped mirrors) only covers disk failure. Backups: **full**, **incremental**, **logical** (`pg_dump`), **physical** (`pg_basebackup`, XtraBackup), **snapshots** (copy-on-write). **Point-in-time recovery (PITR)** restores a base backup and replays archived WAL to any moment (RDS: up to 35 days). Follow **3-2-1** (3 copies, 2 media, 1 off-site) and test restores — GitLab (January 2017) found several backup mechanisms broken and lost ~6 hours of data.

→ Full explanation

Connection pooling, ORMs and N+1 queries

Opening a DB connection costs TCP + TLS + auth + session setup (several to tens of ms; PostgreSQL forks a process per connection, `max_connections` defaults to 100). A **connection pool** keeps open connections to borrow and return — in-app (HikariCP, driver pools) or as a proxy (**PgBouncer**, Pgpool-II, ProxySQL, RDS Proxy), where **transaction pooling** lends a real connection per transaction so dozens serve thousands of clients (vital for many instances or serverless). Size pools to a few times the DB's cores and remember instances × pool size (50 × 20 = 1,000). An **ORM** maps rows to objects (Hibernate, Django ORM, SQLAlchemy, ActiveRecord, Entity Framework, Prisma), parameterises queries and manages schema **migrations**, but hides the SQL. The **N+1 query problem**: 1 query for a list plus 1 per item for related data (51 round trips for 50 posts); fix with **eager loading** (`select_related` / `prefetch_related`, `includes`, `JOIN FETCH`) or batching one `WHERE id IN (...)` query (**DataLoader**), and detect it by logging query counts (Bullet).

→ Full explanation

Choosing a database

Justify from the **access pattern**, consistency and scale, in order: main queries; data size and growth; read/write ratio and peak writes; consistency needs (money vs likes); multi-region writes; what the team can operate (managed services often win).

Need	Choice	Examples
General data, transactions	Relational	PostgreSQL, MySQL, Aurora
Relational at global scale	Distributed SQL	Spanner, CockroachDB, YugabyteDB
Cache, sessions, counters	In-memory key-value	Redis, Memcached
Key access at huge scale	Managed key-value	DynamoDB, Bigtable
Nested records	Document	MongoDB, Firestore
Heavy writes by key and time	Wide-column	Cassandra, ScyllaDB, HBase
Many-hop relationships	Graph	Neo4j, Neptune
Metrics	Time-series	Prometheus, InfluxDB
Full-text, logs	Search engine	Elasticsearch, OpenSearch
Similarity	Vector	pgvector, Pinecone
Analytics	Columnar warehouse	Snowflake, BigQuery, ClickHouse
Files, media, backups	Object storage	S3, GCS
Small strongly consistent config	Coordination store	etcd, ZooKeeper

Default to PostgreSQL or MySQL and add specialist stores beside it as needs appear.

→ Full explanation

Part 3 glossary

Every Part 3 term is defined in bold above. Extra one-liners:

Term	Meaning
Commit / rollback	Make a transaction permanent / discard its changes
Concurrency	Several operations running at overlapping times
Selectivity	How few rows a filter value matches
Access pattern	How an application reads and writes its data
Hash function	Turns any input into a fixed-size, random-looking number

→ Full explanation

Part 4 — Caching, async processing and consistency

Three ideas keep a growing system fast and correct: **caching** (keep copies of popular data somewhere quicker), **asynchronous processing** (hand slow work to a queue and answer now) and **consistency in distributed systems** (decide what "the current value" means when data lives on many machines, and how machines agree).

> **Part 4 TL;DR:** Cache-aside with delete-on-write plus TTLs is the default; protect hot keys from stampedes (single flight, stale-while-revalidate, jitter); queues decouple and absorb spikes,

Kafka-style logs keep a replayable, per-partition-ordered history; assume at-least-once delivery and make consumers idempotent, with backoff, jitter and a DLQ; consistency models range from linearizable to eventual; CAP = during a partition choose consistency or availability, PACELC adds latency vs consistency otherwise; R + W > N quorums, Raft/Paxos majorities and fencing tokens prevent split brain; don't order events by wall clocks; sagas, not 2PC, across services.

→ Full explanation

Caching basics

A **cache** is a small fast store holding copies of data whose real home (the **source of truth** or origin) is slow or expensive. **Cache hit** = found; **cache miss** = go to the source and usually fill the cache; **hit ratio** = hits ÷ lookups; **TTL** = how long an entry may live; **stale data** = a copy that no longer matches the source; **warm/cold cache** = already holding / not yet holding the popular data. Wins: latency (in-process < 1 µs, Redis 0.2–1 ms, indexed DB query 1–10 ms), database offload (95% hits → DB sees 5% of reads) and cost. Average read time = hit ratio × cache time + miss ratio × (cache + DB time): with 1 ms cache, 20 ms DB, 90% hits ≈ 3 ms; 99% ≈ 1.2 ms and 10× fewer DB reads. Caches work because traffic is **skewed** (a small hot fraction gets most requests). Costs: staleness, RAM price, an extra moving part, and a cold restart sending everything to a DB sized for 5%. Never treat the cache as the source of truth; always set a TTL.

→ Full explanation

Where caches live

From the user inward — closer is faster but harder to invalidate:

Layer	What it is	Notes
Browser/client cache	Files and responses on the device, controlled by `Cache-Control: max-age`, `ETag` → 304	Fastest; can't be deleted remotely, so use TTLs and **cache busting**
CDN cache	Edge servers caching public content (Part 2)	Purge API, seconds
Reverse proxy cache	NGINX, Varnish storing whole responses	Skips the app entirely
In-process cache	A map inside the app (Caffeine, `lru_cache`)	Nanoseconds; one copy per server, lost on restart
Distributed cache	Shared cluster: **Memcached** (simple, multi-threaded, no persistence) or **Redis** (rich types, persistence, replication, cluster mode; **Valkey** is its 2024 fork)	Shared, survives deploys, < 1 ms hop
Database buffer cache	PostgreSQL **shared buffers**, InnoDB **buffer pool**	Free, automatic

Also DNS caches, CPU caches and the OS **page cache**. Use in-process for small rarely changing data (config, flags), Redis for user data. Facebook's Memcache paper (2013) describes billions of requests/s; Netflix runs EVCache.

→ Full explanation

Caching strategies

A **caching strategy** sets who reads/writes the cache relative to the database:

Strategy	Mechanism	Main risk / fit
Cache-aside (lazy loading)	App checks cache; on miss reads DB and fills; on write updates DB then **deletes** the key	Brief stale window; the default
Read-through	Cache itself loads from DB on miss (Caffeine loader, DAX)	Simpler app code
Write-through	Write goes to cache, which synchronously writes DB	Fresh reads, slower writes, caches unread data
Write-behind (write-back)	Write to cache only, flushed to DB later in batches	Fast; loses data if the node dies — counters only
Write-around	Writes skip the cache	For data rarely read soon after
Refresh-ahead	Reload hot entries before expiry	No miss waits; wasted refreshes

Delete rather than update on write because racing writers can leave the older value cached; keep a TTL as a safety net. **Negative caching** stores short-lived "not found" markers so lookups for missing keys don't all hit the DB.

→ Full explanation

Eviction policies

When full, a cache **evicts** by policy (distinct from TTL **expiration**): **LRU** (least recently used; hash map + doubly linked list, O(1); the default), **LFU** (least frequently used; good for steady favourites, needs count decay), **FIFO**, **random** (cheap, decent), TTL-based, **MRU**, and hybrids like **W-TinyLFU** (Caffeine). Redis `maxmemory-policy`: `allkeys-lru`, `allkeys-lfu`, `volatile-ttl`, `noeviction` (rejects writes — wrong for a cache); Redis approximates LRU by sampling keys. A big scan can flush an LRU cache; LFU or scan-resistant policies avoid that.

→ Full explanation

Cache invalidation, stale data and stampedes

Cache invalidation removes or updates copies when the source changes ("one of the two hard things", Phil Karlton). Options: TTL only; delete on write; **event-driven** invalidation from a CDC stream (works with many writers); **versioned keys** (`user:42:v7` → `v8`); CDN purge.

Stale data

Comes from TTLs, lost deletes, reader/writer races and filling the cache from a lagging replica; decide per feature how stale is acceptable (follower count yes, transfer-screen balance no).

The thundering herd / cache stampede

A **cache stampede** (thundering herd, dog-piling): a hot key expires and thousands of concurrent misses run the same expensive query, overloading the DB, which delays the refill.

Fixes for stampedes

Request coalescing / single flight (one rebuild per key; CDNs call it **request collapsing**); a **lock or lease** (`SET key:lock 1 NX PX 5000` ; Facebook's lease); **serve stale while revalidating** (`stale-while-revalidate`); **probabilistic early refresh**; **TTL jitter** (also prevents a **cache avalanche** of mass expiry); warm-up after restarts. Related: **cache penetration** (keys that exist nowhere — negative caching or a Bloom filter) and **hot key** (replicate it or cache in-process).

→ Full explanation

Message queues

A **message queue** holds messages until a consumer is ready. **Producer** sends; **broker** stores (RabbitMQ, SQS, Kafka); **consumer**/worker processes. It enables **asynchronous** processing (user gets a fast reply), **decoupling**, **load levelling** of spikes, reliability (a crashed worker's message is redelivered) and independent scaling. Lifecycle: broker persists the message; a consumer receives it and it is hidden (SQS **visibility timeout**, 30 s default); the consumer **acks** after work; no ack → redelivered (hence duplicates). **Point-to-point** queues give each message to one of many **competing consumers**; **pub/sub** gives every subscriber of a **topic** its own copy; they combine (SNS → SQS, RabbitMQ fanout). Monitor **queue depth** and **consumer lag**; put big payloads in object storage and send a reference (**claim check**); don't queue what the user must see immediately.

→ Full explanation

Event streaming: topics, partitions, consumer groups and offsets

An **event streaming platform** (log-based broker: Kafka, Kinesis, Pulsar, Redpanda) keeps messages in an append-only **log** for a retention period (Kafka default 7 days) whether read or not; consumers track their position and can replay. A **topic** is split into **partitions** (ordered logs, the unit of parallelism); a message **key** is hashed to a partition so a key's events stay in order; the **offset** is a message's position. A **consumer group** shares a topic's partitions (each partition to one member; extra consumers beyond partitions idle); each group records **committed offsets**; membership changes cause a **rebalance**; lag = newest − committed offset. Ordering holds only within a partition. Classic queues (RabbitMQ with **exchanges**; SQS standard = at-least-once, best-effort order; SQS FIFO = ordered with de-duplication) suit one-off jobs; streams suit pipelines, multiple independent readers, replay and CDC. Kafka now uses its Raft-based KRaft metadata quorum; Kafka 4.0 (2025) removed ZooKeeper.

→ Full explanation

Delivery guarantees, idempotency and retries

A **delivery guarantee**: **at-most-once** (ack before processing; may lose), **at-least-once** (ack after; may duplicate — the usual default), **exactly-once** — impossible as *delivery* over an unreliable network; systems offer **exactly-once / effectively-once processing** by dropping duplicates. **Idempotency** means doing it twice equals once ("set to 100" yes, "add 20" no). Implement with an **idempotency key** checked against a processed-keys table and recorded in the same transaction as the work (Stripe's `Idempotency-Key`), **upserts**, unique constraints or conditional updates. Kafka's exactly-once (since 0.11, 2017) combines an **idempotent producer** with transactions writing output and offsets atomically — side effects outside Kafka still need idempotency. SQS FIFO de-duplicates within 5 minutes.

Ordering

Global order means one lane, so order per key (partition or FIFO group); retries can reorder, so carry versions and ignore older ones.

Retries, dead-letter queues and backpressure

Retry transient failures with **exponential backoff** and **jitter**, capped. A message that keeps failing (**poison message**) moves after N attempts to a **dead-letter queue (DLQ)** for alerting, fixing and **redrive**. **Backpressure** is a slow consumer signalling "slow down": buffer, block/throttle producers (429/503), drop low-priority work (load shedding) or scale consumers; TCP flow control is a low-level form.

→ Full explanation

Batch vs stream processing

Batch processing runs a job over a bounded, already-collected dataset on a schedule (minutes–hours latency, easy to re-run); **stream processing** handles unbounded events continuously (ms–seconds, harder: state, late events). **MapReduce** (Google 2004, Hadoop): **map** emits key-value pairs, **shuffle** groups by key,

reduce combines; failed pieces re-run. **Spark** keeps intermediates in memory; warehouses run batch SQL. Stream engines (**Flink**, **Kafka Streams**, Spark Structured Streaming) keep state and group events into **windows** (tumbling, sliding), distinguishing **event time** from **processing time** and using **watermarks** to close windows. **Lambda architecture** runs batch + speed layers; **Kappa architecture** uses only a replayable stream.

→ Full explanation

Event-driven architecture, event sourcing and CQRS

Event-driven architecture (EDA): services publish **events** (past-tense facts, `OrderPlaced`, which listeners can't refuse) rather than sending **commands** (`ShipOrder`, which can be rejected); new consumers need no publisher change. The **dual write** problem (DB updated, event not published) is solved by the **transactional outbox**: insert the event into an outbox table in the same transaction; a relay or CDC (Debezium) publishes with retries; consumers are idempotent. **Event sourcing** stores the event sequence itself as the source of truth, rebuilding state by replay with periodic **snapshots** — full audit and time travel, but harder schema evolution and separate read models. **CQRS** separates the write model from read-optimised views (denormalized tables, search indexes) updated by events — reads scale independently, at the cost of lag (eventual consistency). EDA trades traceability for loose coupling.

→ Full explanation

Consistency models

A **consistency model** is the contract about what a read may return after a write (not ACID's C). From strongest:

- **Linearizability** (strong consistency): behaves like one copy; once any read sees a new value, no later read sees the old. Needed for locks, uniqueness, balances; costs latency and availability under partition.
- **Sequential consistency**: everyone sees one order consistent with each client's order, not necessarily real time.
- **Causal consistency**: causally related operations seen in order (reply after post); about the strongest model that stays available under partition.
- **Session guarantees**: **read-your-writes**, **monotonic reads**, **monotonic writes**, **writes-follow-reads** — cheap per-user promises.
- **Eventual consistency**: replicas converge if writes stop, with no time bound (DNS).

Many stores choose per request: DynamoDB strongly consistent reads at double the read cost; Cassandra levels ONE/QUORUM/ALL; Spanner externally consistent transactions.

→ Full explanation

CAP theorem and PACELC

The **CAP theorem** (Brewer 2000; proved by Gilbert and Lynch 2002): with a **network partition**, a replicated store must choose **C** (linearizability; refuse or wait — **CP**) or **A** (every non-crashed node answers, maybe stale — **AP**); **P** (partition tolerance) isn't optional, so the choice applies only during partitions. CP: ZooKeeper, etcd, Consul, Spanner, HBase, MongoDB with majority reads/writes. AP: Cassandra, Riak, Dynamo-style stores at low levels, CouchDB, DNS; many are tunable.

PACELC

Abadi (2010): if **P**artition, choose **A** or **C**; **E**lse choose **L**atency or **C**onsistency. PA/EL: Cassandra, DynamoDB defaults; PC/EC: Spanner, synchronous single-leader; PA/EC (MongoDB is often described so under some settings) and PC/EL also exist — labels vary with configuration.

Common mistakes (misreadings of CAP)

"Pick two of three" (CA isn't available to distributed systems); AP means stale only during partitions; CP fails only unsafe requests (minority side); CAP's C isn't ACID's C; choose per feature (cart AP, checkout CP).

→ Full explanation

Quorums

A **quorum** is the minimum number of replicas that must take part. With **N** replicas, **W** write acknowledgements and **R** read responses, **R + W > N** makes every read overlap the latest successful write (newest version wins; **read repair** and **anti-entropy** fix laggards). For N = 3: W2/R2 balanced and tolerates one down; W3/R1 fast reads; W1/R3 fast writes; W1/R1 may be stale. A **majority quorum** (> half) tolerates ⌊(N − 1)/2⌋ failures, so use odd sizes (4 nodes tolerate 1, like 3). A **sloppy quorum** with **hinted handoff** writes to stand-in nodes, raising availability but temporarily breaking the overlap guarantee.

Trade-offs and caveats

R + W > N alone isn't linearizability (concurrent writes, sloppy quorums, partial failures); larger W or R makes that operation slower and less available.

→ Full explanation

Consensus, leader election and distributed locks

Consensus gets machines to agree on a value or log despite crashes and lost messages — preventing split brain; it underpins **etcd** (Kubernetes state), **ZooKeeper**, **Consul**, CockroachDB, TiDB, Spanner. **Paxos** (Lamport, ~1989–90, published 1998) is proven but hard; **Raft** (Ongaro and Ousterhout 2014) is designed to be understandable (etcd, Consul, CockroachDB, KRaft). Raft: nodes are **follower/candidate/leader** in numbered **terms** (epochs); a follower missing **heartbeats** for its randomised **election timeout** (e.g. 150–300 ms) becomes a candidate, increments the term and asks for votes; a **majority** (voting only for candidates with up-to-date logs) elects a leader; entries are **committed** once a majority stores them; stale leaders are rejected by term. A 5-node cluster survives 2 failures (CP). Use a coordination service for **leader election** rather than writing your own.

Distributed locks and fencing tokens

A **distributed lock** lets one process across machines act at a time, with a **lease** (expiry). A process can pause (GC, VM migration) past its lease while another takes the lock; a **fencing token** — an increasing number issued with each grant, checked by storage, which rejects older tokens — stops the stale holder (ZooKeeper zxid, etcd revision). A Redis `SET NX PX` lock is fine for efficiency, not correctness.

→ Full explanation

Clocks and ordering

Machine clocks disagree (**clock skew**) and run fast or slow (**clock drift**); **NTP** corrects them to a few-tens of ms over the internet (< 1 ms in a good data centre) but can jump backwards. The **wall clock** gives time of day; the **monotonic clock** only moves forward (use it for timeouts). So **last-write-wins** by timestamp can silently drop a later write. **Logical clocks**: a **Lamport clock** (increment per event, send counter, on receive take max + 1) guarantees cause has a smaller number but can't detect concurrency; a **vector clock** (one counter per process, element-wise max) tells before/after/**concurrent**, which Dynamo-style **version vectors** use to keep conflicting "siblings" (size grows with participants). **Hybrid logical clocks (HLC)** (CockroachDB, YugabyteDB) mix wall time and a counter; **Google TrueTime** (Spanner) uses GPS and atomic clocks to give an uncertainty interval of a few ms, and Spanner waits it out before committing.

→ Full explanation

Distributed transactions: two-phase commit and sagas

A **distributed transaction** must commit all-or-nothing across several databases or services. **Two-phase commit (2PC)**: a **coordinator** asks **participants** to **prepare** (do the work durably, lock, vote yes/no), then tells all to **commit** or abort. True atomicity and isolation, but **blocking** (if the coordinator dies after yes votes, participants hold locks and wait), slow and availability-coupled; used inside Spanner (over Paxos-replicated groups) and via **XA**, rarely across microservices (**three-phase commit** is little used). A **saga** chains **local transactions**, with **compensating transactions** undoing earlier steps in reverse on failure (refund, release seat); coordinated by **choreography** (services react to events) or **orchestration** (a central orchestrator such as Temporal, Step Functions, Camunda). Sagas lack isolation (use `PENDING` states), compensations must be idempotent, and irreversible actions (emails) go last. Sagas are the microservices norm.

→ Full explanation

Heartbeats, gossip and failure detection

Failure detection guesses which nodes are alive, since dead, slow and unreachable look alike. A **heartbeat** is a periodic "I'm alive"; missing several makes a node **suspected dead**. Short timeouts react fast but cause **false positives** and **flapping**; long ones react slowly. Central monitoring (ZooKeeper sessions) must itself be HA. A **gossip protocol** has each node periodically exchange membership lists with heartbeat counters with a few random peers, spreading news to N nodes in ~log N rounds with no SPOF (Cassandra, original Dynamo, Consul's memberlist, Redis Cluster). The **phi accrual detector** (Cassandra, Akka) outputs a suspicion level from arrival history; **SWIM** asks other nodes to ping a suspect indirectly. A node declared dead may still be running — use fencing.

→ Full explanation

Probabilistic data structures and Merkle trees

Structures that trade a small controlled error for far less memory.

Bloom filter

Answers "is X in the set?" with "definitely not" or "probably yes" — false positives, never false negatives. An m-bit array plus k hash functions: adding sets k bits; any 0 bit on lookup means absent. ~10 bits per item with 7 hashes ≈ 1% false positives; no deletion (a **counting Bloom filter** allows it). Used per SSTable in LSM stores to skip disk reads, against cache penetration, by crawlers.

HyperLogLog

Estimates **cardinality** (distinct count) in fixed memory by tracking, across many buckets, the longest run of leading zeros in item hashes. Redis HLL: ≤ 12 KB, 0.81% standard error, billions of items, mergeable (`PFADD` , `PFCOUNT` , `PFMERGE`); approximate `COUNT(DISTINCT)` in warehouses.

Count-min sketch

Estimates item **frequency** with a d × w counter table and one hash per row; query takes the minimum, so it may overestimate, never underestimate. For heavy hitters, trending topics, hot-key detection, top-K.

Merkle tree

A tree of hashes up to a **root hash**; any data change alters the path to the root. Replicas compare roots and descend only into differing children, finding differences in ~log(blocks) comparisons — Cassandra/Dynamo anti-entropy, Git, blockchains, certificate transparency.

Summary table

Structure	Answers	Error	Memory
Bloom filter	Membership	False positives only	~10 bits/item for 1%
HyperLogLog	Distinct count	~0.81% (Redis)	12 KB
Count-min sketch	Frequency	Overestimates only	Fixed table
Merkle tree	Where datasets differ	Exact	One hash per block

→ Full explanation

Unique ID generation

IDs must be unique across machines without coordination, ideally time-sortable (index-friendly), compact (64-bit) and not leaking volume. **Auto-increment** is simple and ordered but centralised, collides after sharding (step by shard count, or a ticket server like Flickr's) and is guessable. **UUID** (128 bits, 36 chars): **UUIDv4** is 122 random bits (collision-negligible, but random order fragments B-trees); **UUIDv7** (RFC 9562, 2024) prefixes a 48-bit ms timestamp so it sorts by time — a good default (**ULID** is similar). **Snowflake ID** (Twitter 2010, 64 bits): 1 sign bit, 41 bits ms timestamp (~69 years), 10 bits machine ID (1,024 generators), 12 bits sequence (4,096/ms per machine, ~4 million/s); needs unique machine IDs and non-backward clocks (Discord, Instagram variant, Sonyflake). Also **segment/range allocation** (blocks of IDs handed out) and content hashing with collision checks.

→ Full explanation

Geospatial indexing: geohash and quadtree

B-trees sort one value, so "nearby" needs a **geospatial index**. **Geohash** repeatedly halves longitude/latitude, encoding bits as base-32 characters; a shared prefix means nearby (5 chars ≈ 5 × 5 km, 6 chars ≈ 1.2 × 0.6 km), so proximity becomes a prefix search — query the cell plus its 8 neighbours (Redis `GEOADD` / `GEOSEARCH`, Elasticsearch). A **quadtree** splits a map cell into four when it holds too many points, adapting to density (usually in memory). Others: **R-tree** (PostGIS), Google **S2** (sphere cells), Uber **H3** (hexagons with equidistant neighbours).

→ Full explanation

Part 4 glossary

Every Part 4 term is defined in bold above. Extra one-liners:

Term	Meaning
Skewed traffic	A small fraction of items receives most requests
Competing consumers	Several workers taking messages from one queue
Version vector	Per-replica counters used to detect concurrent writes
Lease	A lock or permission that expires automatically

→ Full explanation

Part 5 — Architecture, reliability, operations and security

How to arrange the building blocks into a system, keep it running when pieces break, ship changes safely, see what it is doing, and keep the wrong people out.

Part 5 TL;DR: Start with a modular monolith, split into microservices for team independence or independent scaling; prefer async messaging off the critical path; always set timeouts, retry only transient failures on idempotent calls with backoff + jitter at one layer, and wrap fragile dependencies in circuit breakers, bulkheads and fallbacks; rate limit (token bucket, 429) and shed load before collapse; autoscale with headroom and bounds; containers + Kubernetes reconcile desired state; ship through CI/CD with canaries, feature flags and fast rollback; observe with metrics, logs and traces and page on SLO burn rate; set RPO/RTO and test restores; AuthN ≠ AuthZ, keep tokens short-lived, hash passwords slowly, apply least privilege and zero trust.

→ Full explanation

Architectural styles: monolith, modular monolith, SOA, microservices and serverless

An **architectural style** answers how code is split into running programs and how they talk.

Style	What it is	Gains	Costs
Monolith	One codebase, one deployable, one DB; scaled by copies behind an LB	Simple build/test/debug, easy transactions	Slow builds and coupled releases as it grows; scale all of it
Modular monolith	One deployable with strictly bounded modules	Tidiness without the network (Shopify)	Discipline/tooling needed
SOA	2000s coarse services joined by an **enterprise service bus (ESB)**	Enterprise integration	Shared models, central bus
Microservices	Small services per business capability, **own database** (**database per service**: independent schemas and scaling, but no cross-service joins or transactions), independent deploys, "smart endpoints, dumb pipes"	Team independence, independent scaling and failure	**Distributed systems tax**: network failures/latency, no cross-service transactions (sagas), tracing, per-service ops
Serverless / FaaS	Functions run per event, scaled from zero, billed per invocation (Lambda, Cloud Functions, Workers)	No servers, idle costs nothing	**Cold starts** (~100 ms to seconds), 15-minute Lambda limit, lock-in, costly at steady load

Conway's law: systems mirror the organisation's communication structure. Avoid a **distributed monolith** (services deployed in lockstep, shared DB, long sync chains) and splitting by technical layer. Advice: monolith first; migrate with the **strangler fig pattern** (proxy in front, move features one by one). Prime Video (2023) cut one tool's cost > 90% by moving from Step Functions + Lambda to a single process — pick per workload.

→ Full explanation

Synchronous vs asynchronous communication

Synchronous: A calls B (REST, gRPC) and blocks until a reply or timeout. **Asynchronous**: A publishes to a queue/stream and returns; B processes later; results come by callback, webhook or status polling (HTTP **202 Accepted** + job ID). Sync chains multiply availability (five 99.9% services ≈ 99.5%, ~44 h/year down instead of ~9) and cascade slowness; async absorbs failures but gives eventual results, needs idempotent consumers and correlation IDs. Rule: synchronous with a timeout only if the user needs the result now (card authorisation); everything else (email, analytics, indexing) as events.

→ Full explanation

Service discovery, sidecars and service mesh

Service discovery answers "which addresses run service X now?". A **service registry** (Consul, etcd, ZooKeeper, Eureka, Kubernetes) records instances that **register**, send heartbeats and are removed when they stop. **Client-side discovery**: the caller queries the registry and load-balances (Eureka + Ribbon); **server-side discovery**: the caller hits a fixed router/LB that looks up the registry (AWS target groups, Kubernetes Services — the usual default); **DNS-based discovery** publishes instances as DNS names (`payments.default.svc.cluster.local`; caching can go stale). A **sidecar** is a helper process beside each instance handling networking, TLS, retries, logging; a **service mesh** is a fleet of sidecar proxies (the **data plane**, usually Envoy) configured by a **control plane** (Istio, Linkerd), giving mTLS, uniform retries/timeouts, traffic splitting and per-call telemetry without code changes — at the cost of an extra hop each side, CPU/memory per pod and operational complexity (overkill for ten services). Don't configure retries in both app and mesh.

→ Full explanation

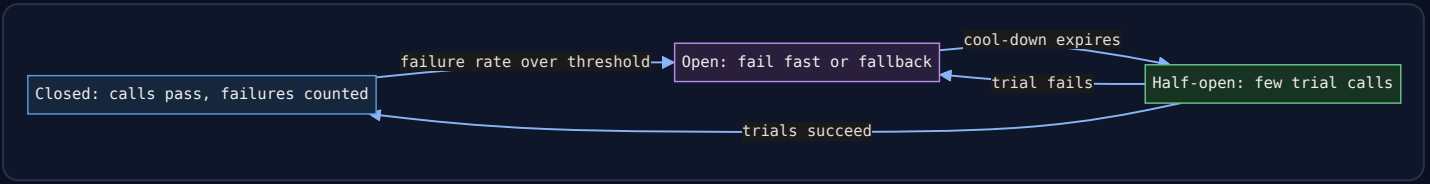
Timeouts, retries, exponential backoff and jitter

A **timeout** caps how long a caller waits; a **retry** repeats a request that hit a **transient fault**; **exponential backoff** doubles the wait each attempt up to a cap; **jitter** randomises it. Without timeouts, waiting callers exhaust threads and fail too — a **cascading failure**; careless retries multiply load on a struggling service — a **retry storm**. Set timeouts from measured latency (p99 300 ms → ~0.5–1 s) and propagate a **deadline** across the request (gRPC does). Retry only transient errors (timeouts, resets, 502/503/504, 429 after `Retry-After`), only idempotent operations (or with an idempotency key), 2–3 attempts, at one layer only (four layers × 3 attempts = 81 calls). **Full jitter**: wait random(0, min(cap, base × 2^n)). A **retry budget** caps retries at, say, 10% extra load. **Hedged requests** cut tail latency for a few percent extra load. AWS SDKs back off with jitter by default.

→ Full explanation

Circuit breakers, bulkheads and fallbacks

A **circuit breaker** wraps calls to a dependency: **Closed** (normal, counting failures in a rolling window) → **Open** when failures exceed a threshold (e.g. 50% of ≥ 20 calls; calls fail instantly or go to a fallback) → **Half-open** after a cool-down (e.g. 30 s; a few trial calls close it on success or re-open it). A **bulkhead** isolates resources (separate thread/connection pools per dependency, separate queues or clusters, **cell-based architecture** limiting **blast radius**) so one failure can't consume everything. A **fallback** returns cached, default or partial data. Costs: tuning thresholds, reserved idle capacity, stale answers (fine for recommendations, not balances). Netflix Hystrix (maintenance mode) → Resilience4j; meshes offer breakers and outlier ejection.



→ Full explanation

Rate limiting and throttling

Rate limiting caps requests per client per window and rejects the excess with **429 Too Many Requests** + `Retry-After` and `X-RateLimit-*` headers; **throttling** strictly means delaying/queueing instead. Purposes: protection, fairness, cost control, slowing brute force, business tiers. Limits are per key (user, API key, IP, user + endpoint).

Algorithm	How	Bursts	Memory	Accuracy
Token bucket	Bucket of B tokens refilled at R/s, one per request	Up to B	Tiny	Good (most used)
Leaky bucket	Queue drained at a fixed rate	Smoothed	Queue	Good
Fixed window	Counter per calendar window	2× at edges	Tiny	Rough
Sliding window log	Timestamp per request	No	Per request	Exact
Sliding window counter	current + previous × overlap fraction	Little	Two counters	Very good

Enforce at the edge/CDN/WAF (by IP), the API gateway (per key), inside services, and outbound to partners. **Distributed rate limiting** uses a shared store (Redis atomic Lua script) or local shares synced periodically; if the store fails, choose **fail open** (usual for user traffic) or **fail closed**. GitHub: 5,000 requests/hour authenticated, 60 unauthenticated; NGINX `limit_req` is a leaky bucket. Don't limit by IP alone or answer 500/503.

→ Full explanation

Load shedding and graceful degradation

Load shedding deliberately rejects some requests when overloaded so the rest succeed; **graceful degradation** turns off non-essential features (recommendations, reviews) to keep the core (search, checkout). Overload otherwise leads to **congestion collapse**: queues grow, requests time out after work is done, clients retry, **goodput** (useful completed work) falls to near zero. Steps: detect overload (CPU, in-flight limit, queue wait > ~100 ms); reject cheaply with 503 + `Retry-After`; shed by priority (bots and background first, checkout last); drop requests whose callers already timed out; keep **kill switches**. Backpressure is the cooperative cousin. Rejecting must be cheaper than serving.

→ Full explanation

Autoscaling

Autoscaling adds capacity (**scaling out**) and removes it (**scaling in**) automatically — elasticity in practice. Pick a metric (CPU, RPS per instance, queue depth, p95 latency) and a target (e.g. 60% CPU leaves headroom); the controller (Kubernetes HPA checks every 15 s) computes desired = current × metric/target (10 at 90% with a 60% target → 15); new instances boot (seconds for containers, ~1–5 min for VMs); scale in after a **cool-down** (5–10 min) with draining; set **min/max** bounds. Types: **reactive** (late), **scheduled** (known peaks), **predictive** (learned patterns; AWS, Netflix Scryer), horizontal vs vertical (VPA), **cluster autoscaling** (adds nodes). Watch for the DB connection limit, too-high targets, slow startup and unbounded bills. Scale queue workers on lag; KEDA scales on event sources.

→ Full explanation

Virtual machines, containers and Docker

A **virtual machine (VM)** is an emulated computer with its own OS kernel, created by a **hypervisor** (KVM, Xen, ESXi, AWS Nitro). A **container** is an isolated process sharing the host kernel, built from Linux **namespaces** (what it sees) and **cgroups** (what it may use). A **container image** is a read-only package of app + dependencies built in cached **layers**; **Docker** (2013) popularised the `Dockerfile`, runtime and **registries** (Docker Hub, ECR). Containers start in ms–seconds and weigh MB (dense, portable) but isolate less than VMs (separate kernel, tens of seconds, GB); usually containers run inside cloud VMs, and micro-VMs like Firecracker (Lambda, Fargate) boot in ~125 ms. Keep data out of the container filesystem (volumes/external storage), keep images small, don't run as root or bake in secrets.

→ Full explanation

Kubernetes basics

Kubernetes (K8s) is a **container orchestrator**: you declare desired state and it keeps reality matching (open-sourced by Google 2014 from Borg, now CNCF; managed as EKS, GKE, AKS). Vocabulary: **cluster** (machines + control plane); **node** (a machine running the **kubelet** agent); **control plane** (**API server**, **etcd** state store, **scheduler**, **controllers**); **pod** (smallest unit: one or more containers sharing an IP and volumes, replaced not repaired); **Deployment** (replicas + rolling updates for stateless apps); **Service** (stable name and virtual IP load-balancing over matching pods — built-in discovery); **Ingress/Gateway API** (external HTTP routing); **ConfigMap/Secret**; **Namespace**; **StatefulSet** (stable names and storage per pod); **liveness** (restart) and **readiness** (send traffic) **probes**. The **reconciliation loop** compares desired and actual state ("3, want 5" → create 2). It is complex; ECS, Cloud Run or a **PaaS** may suit small teams. Set resource requests/limits and readiness probes.

→ Full explanation

Infrastructure as code

Infrastructure as code (IaC) describes servers, networks, LBs, databases and permissions in version-controlled files applied by a tool, giving reproducible environments, code review and fast rebuilds. **Declarative** tools describe the end state (Terraform, CloudFormation, Kubernetes YAML, Pulumi): write, **plan** (diff), review, **apply**, recorded in a **state file**; **imperative** scripts list steps; **configuration management** (Ansible, Chef, Puppet) configures existing servers; **immutable infrastructure** replaces servers rather than patching them.

Trade-offs and common mistakes

Learning curve; state files must be stored and locked; manual console edits cause **drift** that the next apply may undo.

→ Full explanation

CI/CD

Continuous integration (CI): merge small changes at least daily, each triggering automated build and tests (minutes). **Continuous delivery (CD)**: every passing change is packaged and releasable at a button; **continuous deployment**: it goes to production automatically. A **pipeline**: push → build, lint, unit tests, security scans (< ~10 min) → one versioned artifact promoted unchanged through environments → staging and integration tests → production with a safe strategy and automatic rollback. Small frequent releases are safer; the **DORA metrics** (deployment frequency, lead time, change failure rate, time to restore) show high performers deploy more often with fewer failures. Avoid flaky tests, rebuilding per environment, long-lived branches, and non-backward-compatible schema changes — use **expand and contract**.

→ Full explanation

Deployment strategies and rollback

A **deployment strategy** replaces a running version without downtime while limiting **blast radius** (most outages are caused by changes).

Strategy	How	Downtime	Rollback	Blast radius
Recreate (big bang)	Stop all, start new	Yes	Slow	Everyone
Rolling deployment	Replace a few at a time behind health checks (Kubernetes default)	No	Medium	Grows per batch
Blue-green deployment	Switch all traffic from blue to a tested green environment	No	Instant	Everyone at switch; 2× capacity
Canary release (deployment)	1% → 5% → 25% … with automated metric comparison	No	Fast	Tiny at first
Feature flags	Ship dark, enable at runtime per audience; doubles as kill switch	No	Seconds	Chosen users

A **shadow (dark) launch** copies real traffic to the new version and discards responses (beware side effects). Flags separate **deploy** from **release**. **Rollback** to the last good version first, investigate later; when data or schema changed incompatibly, **roll forward**. Lessons: Knight Capital (2012) lost ~$440 million in ~45 minutes after a partial deploy reactivated dormant code; CrowdStrike (July 2024) pushed an update everywhere at once, crashing ~8.5 million Windows machines.

→ Full explanation

Observability: logs, metrics and traces

Monitoring watches known numbers; **observability** is the ability to explain unexpected behaviour from emitted data, via three pillars: **logs** (timestamped event records, ideally **structured** JSON), **metrics** (cheap numeric time series: **counters**, **gauges**, **histograms**), and **traces** (one request's path as a tree of timed **spans** sharing a **trace ID**). Instrument with **OpenTelemetry**, propagate a **correlation/trace ID** (W3C `traceparent`), ship to log stores (OpenSearch, Loki, Splunk), metrics DBs (Prometheus, Datadog, CloudWatch) and trace backends (Jaeger, Zipkin, Tempo), visualise in Grafana/Kibana, alert on metrics. Google's **Dapper** paper (2010) introduced large-scale tracing (inspiring Zipkin and Jaeger). **Sampling** keeps e.g. 1% of normal and all error/slow traces (**tail-based sampling**). Avoid high-**cardinality** metric labels (user IDs). Logs are rich but expensive, metrics cheap but aggregated, traces precise but sampled; don't log secrets.

Types: what to measure

Four golden signals (Google SRE): latency (successes and failures separately), traffic, errors, saturation. the **RED method** (Tom Wilkie) for services: rate, errors, duration. the **USE method** (Brendan Gregg) for resources: utilisation, saturation, errors. RED shows what users feel; USE shows why.

→ Full explanation

Dashboards, alerting, on-call and SLO-based alerts

A **dashboard** shows key metrics at a glance; an **alert** fires when a metric rule breaks; **on-call** engineers respond; a **page** is urgent, a **ticket** waits for working hours; a **runbook** says what to do; a **postmortem** analyses an incident **blamelessly**. Page only on user-facing symptoms that need human judgement, with runbook and dashboard links, to avoid **alert fatigue**. **SLO-based alerting** pages on **burn rate** — how fast the error budget is spent (1 = exactly the budget over 30 days). Google's SRE workbook scheme: 1 h window at 14.4× (2% of budget) → page; 6 h at 6× (5%) → page; 3 days at 1× (10%) → ticket; pair each long window with a short one. In incidents an **incident commander** coordinates, and **mitigation** (rollback, failover, shed load) comes before root cause.

→ Full explanation

Disaster recovery

Disaster recovery (DR) restores service and data after large failures (site loss, region outage, ransomware, deleted database); **business continuity** is the wider plan. **RPO** (recovery point objective) = how much data, in time, you may lose; **RTO** (recovery time objective) = how long you may be down. Lower values cost more. Backups follow **3-2-1**: full, incremental, snapshots, continuous WAL archiving (point-in-time recovery), **immutable / air-gapped** copies; rehearse restores. A cloud **region** contains several **availability zones** (separate data centres, ~1–2 ms apart): **multi-AZ** (synchronous replication practical; the baseline) vs **multi-region** (survives regional outages and serves users nearer home, but 50–150 ms+ apart, so usually async with RPO > 0, and costlier).

Types: DR strategies

Strategy	DR site runs	RTO	RPO
Backup and restore	Only backups	Hours–a day	Hours
Pilot light	Replicated data, servers off	Tens of minutes	Minutes
Warm standby	Scaled-down full copy	Minutes	Seconds–minutes
Active-active	Full copy serving traffic	Near zero	Near zero–seconds

Active-passive leaves idle capacity and a rarely exercised failover; active-active needs headroom and multi-writer conflict handling. **Chaos engineering** injects controlled failures (Netflix **Chaos Monkey**, 2012) to prove recovery works; **game days** rehearse it. Keep backups in another account/region and ensure DR doesn't depend on the failed region (Facebook's 2021 BGP/DNS outage locked engineers out of internal tools).

→ Full explanation

Authentication and authorization

Authentication (AuthN) proves who you are (something you know, have, or are; two or more = **MFA**); failure → **401 Unauthorized** (really "unauthenticated"). **Authorization (AuthZ)** decides what you may do; failure → **403 Forbidden**. Models: **ACL** (per-resource list of who may do what; fine-grained, hard at scale), **RBAC** (permissions via roles; simple, auditable, most common), **ABAC** (rules over user, resource and context attributes; flexible, harder to reason about), **ReBAC** (permissions follow graph relationships; Google Zanzibar). Always check authorization on the server for every request.

→ Full explanation

Sessions, cookies, tokens and JWT

Session-based authentication: after login the server stores a **session** (Redis/DB) and gives the browser a random **session ID** in a **cookie** — instantly revocable, but a store lookup per request. **Token-based authentication**: the client sends a token, usually `Authorization: Bearer <token>`; **opaque tokens** are looked up, **self-contained tokens** carry signed claims. A **JWT** ("jot") is `header.payload.signature`, Base64URL-encoded, signed with HS256 (shared secret, HMAC) or RS256 (RSA key pair), verified locally by any service via signature and `exp` — ideal for microservices, but **signed, not encrypted** (no secrets inside) and hard to revoke. Use short-lived **access tokens** (5–15 min) plus a carefully stored **refresh token** checked against a DB. Cookie flags: `HttpOnly` (no JS access, anti-XSS), `Secure` (HTTPS only), `SameSite=Lax/Strict` (anti-CSRF). Reject `alg: none`; prefer HttpOnly cookies over `localStorage` in browsers.

→ Full explanation

OAuth 2.0, OpenID Connect, SSO and API keys

OAuth 2.0 is **delegated authorization**: an app gets a scoped token to your data elsewhere without your password. Roles: **resource owner** (user), **client** (app), **authorization server** (issues tokens after consent), **resource server** (API checking token and **scopes**). **Authorization code flow with PKCE** (recommended): redirect to the authorization server with client ID, scopes, redirect URL and a hashed one-time secret → user logs in and consents → short-lived single-use **authorization code** returned → client exchanges code + PKCE secret (and client secret for server apps) server-to-server → **access token** (+ refresh token) → API calls with Bearer token. Other flows: **client credentials** (machine-to-machine), **device code** (TVs); implicit and password flows are discouraged. **OpenID Connect (OIDC)** adds authentication via a signed **ID token** (JWT) — "Sign in with Google". **SSO** reuses one identity-provider login across apps (Okta, Entra ID; OIDC or XML-based **SAML**). An **API key** identifies a calling program (header such as `X-API-Key`): simple for metering and rate limits, but a bearer secret without built-in expiry or user identity — never commit keys to Git. Don't treat OAuth access tokens as identity proof.

→ Full explanation

Encryption, hashing, passwords and secrets

Encryption is reversible with a key (**decrypt** to recover it): **symmetric encryption** (one key, AES; fast, bulk data) or **asymmetric** (public key encrypts/verifies, private key decrypts/signs; RSA, elliptic-curve; key exchange and signatures). **In transit** = TLS (mTLS between services); **at rest** = disks, DBs, backups, typically AES-256 with keys from a **KMS** (key management service). **Hashing** makes an irreversible fixed-size **fingerprint** (**hash/digest**) (SHA-256 = 256 bits) for matching, integrity and dedup. Passwords: never plain or encrypted; store a per-user random **salt** (defeats rainbow tables) with a deliberately slow hash — **bcrypt** (cost factor), **scrypt** or **Argon2id** (memory-hard; 2015 Password Hashing Competition winner's variant, OWASP's first choice) — tuned to tens–hundreds of ms; fast hashes allow billions of GPU guesses per second. **Envelope encryption**: data keys encrypt data and are themselves encrypted by a master key that never leaves the KMS. **Secrets management**: keep credentials out of code, images and logs, in Vault / AWS Secrets Manager / Google Secret Manager / KMS-backed Kubernetes Secrets, scoped per service and **rotated** automatically. Base64 is encoding, not encryption; never roll your own crypto.

→ Full explanation

Network security: firewalls, WAF and DDoS

A **firewall** filters by IP, port and protocol (L3/L4; cloud **security groups** and network ACLs). A **web application firewall (WAF)** inspects HTTP (L7) for SQL injection, XSS, bad bots and floods (AWS WAF, Cloudflare, ModSecurity with the OWASP Core Rule Set). **Network segmentation** keeps databases in **private subnets**, reachable only via LBs or a **bastion host**. A **DDoS** attack floods a service from many machines (often a **botnet**): **volumetric** (UDP floods, **amplification** via open DNS/memcached), **protocol** (**SYN flood**), **application layer** (expensive HTTP requests). Mitigate by absorbing at scale (anycast scrubbing: Cloudflare, AWS Shield, Cloud Armor, Akamai), edge filtering and SYN cookies, rate limiting and challenges, caching with hidden origin IPs, and capped autoscaling. Records: ~398 million HTTP requests/s (HTTP/2 "Rapid Reset", disclosed by Google October 2023); Mirai (2016) took down Dyn.

→ Full explanation

Common web vulnerabilities: CORS, CSRF, XSS and SQL injection

Same-origin policy stops a page from one **origin** (scheme + domain + port) reading another's responses; **CORS** lets a server opt in (`Access-Control-Allow-Origin`, `OPTIONS` **preflight**) — a browser relaxation, not server protection; never `*` with credentials. **CSRF** tricks a logged-in browser into sending a forged request with its cookies; fix with `SameSite` cookies, anti-CSRF tokens and no state changes on GET. **XSS** runs attacker script inside your site; fix with output escaping (React does by default), a **Content Security Policy (CSP)** and `HttpOnly` cookies. **SQL injection** splices input into SQL (`' OR '1'='1`); fix

with **parameterised queries**. The **OWASP Top 10** lists the most critical web risks; broken access control tops recent editions.

→ Full explanation

Least privilege and zero trust

Least privilege: each user, service and process gets only the permissions it needs, only for as long as needed. **Defence in depth**: independent layers (firewall, WAF, AuthN, AuthZ, encryption, monitoring). **Zero trust**: trust no request because it is "inside" (unlike the castle-and-moat **perimeter model** with a **VPN**) — authenticate user and device, authorise, encrypt (mTLS) and log every request, limiting **lateral movement**. Practice: narrow per-service **IAM** roles, no shared admin accounts, expiring just-in-time access, private subnets, device checks, audit logs. Google **BeyondCorp** (from 2014) based access on user and device identity rather than network location.

→ Full explanation

Cost awareness

Cost drivers: compute, storage (per GB-month, cheaper cold tiers), managed databases and **data transfer** — ingress usually free, **egress** to the internet ~$0.05–0.09/GB on AWS, cross-AZ ~$0.01/GB each way (why CDNs matter for video). Pricing: **on-demand**, **reserved instances / savings plans** (1–3-year commitments), **spot/preemptible** (up to ~90% off, reclaimable; batch jobs). Levers: autoscale and stop idle environments, right-size, cache, lifecycle old data to cold tiers, sample logs and traces, compare serverless vs always-on at real traffic. Every nine, region and lower RPO costs money; engineering time too. **FinOps** makes teams see and own their spend. In interviews name your design's dominant cost (video: egress; logging: storage) and one reduction.

→ Full explanation

Part 5 glossary

Every Part 5 term is defined in bold above. Extra one-liners:

Term	Meaning
Strangler fig pattern	Migrate by routing features one at a time from the old system to new services
Transient fault	A temporary failure likely to succeed on retry
Goodput	Useful completed work, excluding wasted or timed-out work
Lateral movement	An attacker moving from one compromised system to others

→ Full explanation

Part 6 — Putting it together

Parts 1–5 examined each piece alone; this part reassembles them: one request's full journey, a worked URL-shortener design, a decision cheat sheet, the 60 most common beginner interview questions with short answers, and an A–Z index of terms.

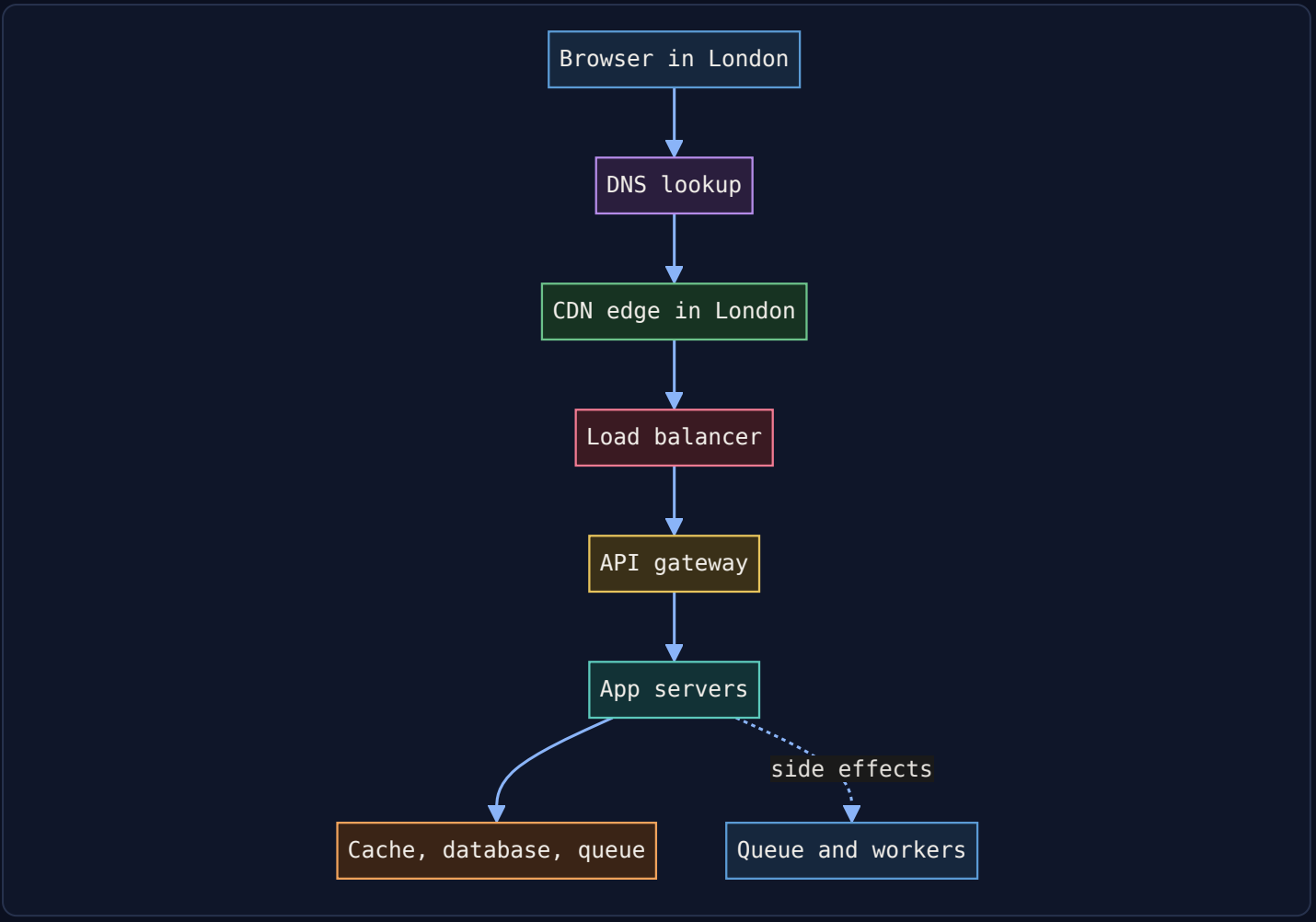
> **Part 6 TL;DR:** One click goes browser cache → DNS → TCP/TLS → CDN → load balancer → API gateway → stateless app server → cache → database and back, with side effects pushed through a queue; every hop costs time and can fail, so cache it, duplicate it or take it off the critical path. Every "reach for X" carries a "watch out for Y" — say both.

→ Full explanation

A request's journey

A **request's journey** is the full path one user action takes through a system — the classic "what happens when you type a URL and press Enter?" warm-up, showing where time goes and where things break.

The map



1. **Browser** splits the URL into **scheme** (`https`), **host** and **path**, and checks its **browser cache**.
2. **DNS** (0–50 ms): recursive resolver, root, TLD, authoritative; a CDN CNAME returns a nearby edge (GeoDNS/anycast).
3. **TCP** handshake (1 RTT; 5–10 ms to a London edge vs ~70 ms to New York).
4. **TLS** 1.3 handshake (1 RTT; **0-RTT resumption** for repeat visits; HTTP/3 merges 3 and 4).
5. **HTTP request** (`GET /feed`, cookie carrying the session — HTTP is stateless).
6. **CDN edge** serves static hits; personal `/feed` is forwarded to the origin (DDoS absorption, WAF here).
7. **Load balancer** (L7, TLS termination, health checks, algorithm; itself redundant).
8. **API gateway** authenticates (session/JWT), rate-limits (429), routes.
9. **Stateless app server** in containers on Kubernetes, autoscaled, with service discovery and timeouts.
10. **Cache** (cache-aside, ~0.5 ms; 90–99% hits).
11. On a miss, **database**: read replica, index, shard key, connection pool (2–20 ms).
12. **Fill the cache** with a TTL — readers see eventual consistency.
13. **Response** returns through gateway, LB and edge (compressed); the browser renders and fetches images from the CDN.
14. **Writes** (`POST` like) commit to the primary and publish an event; idempotent **workers** handle notifications and counters (at-least-once).
15. **Observability**: logs, metrics, trace ID; SLO alerts page on-call.

Where the time goes

Step	Typical	Shrink with
DNS	0–50 ms	Caching, long TTL
TCP + TLS	10–20 ms to nearby edge	CDN, HTTP/3, keep-alive
Edge → origin	5–80 ms	Origin in user's region
Gateway + app	5–20 ms	Fewer hops, parallel calls
Cache / DB	0.5 ms / 5–20 ms	Hit ratio, indexes
Rendering	50–200 ms	Smaller pages

On a first visit, network round trips often cost more than server work; repeat visits reuse DNS, connections and cached files.

What can go wrong at each hop

Hop	Failure	Protection
DNS	Provider outage	Two providers, sensible TTLs
CDN	Edge outage, stale content	Multi-CDN, invalidation
Load balancer	Node dies	Redundant pair or managed LB
App server	Crash, bad deploy	Health checks, canaries, rollback
Cache	Node dies, stampede	Replicas, request coalescing
Database	Primary dies	Replica failover, backups
Queue/worker	Crash mid-job	At-least-once, retries, DLQ
Any call	Slow dependency	Timeouts, circuit breaker, fallback

→ Full explanation

Worked example: designing a URL shortener

A **URL shortener** maps a short code (`sho.rt/aZ3kQ9b`) to a long URL and **redirects** (Bitly, TinyURL) — a classic 45-minute problem.

Step 1 — Requirements

Functional: create a short link; redirect; optional **custom alias**, expiry, click counts. Non-functional: 99.99% redirect availability, p99 < 50 ms, codes not guessable in sequence, links never lost, read-heavy (assume 100 reads per write). Out of scope: accounts, dashboards, previews.

Step 2 — Estimates (back of the envelope)

100 million links/month ÷ 2.6 million s ≈ 40 writes/s; × 100 = 4,000 reads/s, ~12,000 at 3× peak; 6 billion links in 5 years × ~500 B ≈ 3 TB; read bandwidth ~2 MB/s. **Base62** (a–z, A–Z, 0–9): $62^6 \approx 56.8$ billion, $62^7 \approx 3.5$ trillion → 7 characters (500× headroom). Cache: ~350 million redirects/day over ~50 million distinct links; the hottest 20% (10 million × 500 B) ≈ 5 GB fits one cache node (the **80/20 rule**).

Step 3 — API

`POST /api/v1/links` (`long_url` , optional `custom_alias` , `expires_at` ; `Idempotency-Key` header) → `201 Created` with the code; `GET /{code}` → redirect with `Location` . Versioned path, rate-limited creation (429), `404` for unknown and `410 Gone` for expired codes. **301 Moved Permanently** lets browsers cache the redirect

(less load, but lost click counts and no expiry); **302 Found** re-checks every time — choose 302 if analytics or expiry matter.

Step 4 — Data model

One table: `code` (primary key), `long_url`, `owner_id` (nullable), `created_at`, `expires_at`. Every read is one primary-key lookup, so PostgreSQL or a key-value store (DynamoDB, Cassandra) works. Don't increment a click counter in this row on each redirect.

Step 5 — Generating the short code

Approach	Pros	Cons
Hash the URL (7 base62 chars of MD5/SHA-256)	Same URL → same code	**Hash collisions** must be checked and retried
Random code	Simple, unguessable	Collision checks
Counter + base62	No collisions	Bottleneck, guessable
Key generation service (KGS)	Fast, no collisions	Another service

Recommended: **counter ranges** handed to each server by a coordination store (DB row, ZooKeeper/etcd), counted in memory (crashed ranges are simply skipped), **scrambled** with a reversible mixing function, then base62-encoded. Snowflake IDs need no coordination but give 11 base62 characters. Custom aliases are inserted directly; a duplicate key → **409 Conflict**.

Step 6 — High-level design

Client → CDN/DNS → load balancer → stateless link service → Redis, database (primary + replicas), click-event queue → analytics workers. **Redirect path**: Redis (cache-aside) → on miss a replica, cached with a ~24 h TTL → check expiry → publish click event asynchronously → 302. **Create path**: gateway auth and rate limit → validate URL (malware blocklist) → next number, scramble, encode → insert into primary → 201.

Step 7 — Deep dives

Caching: 90%+ hits turn 12,000 reads/s into ~1,200 DB reads; links rarely change; **request coalescing** prevents stampedes; the CDN can cache redirects without analytics. **Scaling**: shard by hash of `code` (every query knows it) with consistent hashing. **Availability**: two+ copies per tier across AZs; redirects keep working from replicas and cache while a primary fails over (graceful degradation). **Analytics**: clicks to Kafka, batched per minute into a column store (ClickHouse); de-duplicate by event ID. **Abuse**: rate limits, malware checks, disabling links.

Trade-offs we made

302 over 301 (more load, accurate analytics and expiry); counter ranges over hashing (no collision checks, small coordination store); eventual consistency for click counts; long cache TTLs (links effectively immutable). Don't skip estimates, ignore hash collisions, write on the redirect path, or open with 50 shards for 40 writes/s.

→ Full explanation

Decision cheat sheet

How to use it

Read the left column as "the interviewer just told me…", name the tool, then say why and what it costs — the "watch out for" earns the marks.

The table

If you need…	Reach for…	Watch out for…
Faster repeated reads	Cache (Redis), cache-aside	Staleness, invalidation, stampedes
Fast static files worldwide	CDN	Invalidation delay, cost per GB
Low latency far away	CDN, edge, multi-region	Cross-region consistency
More stateless capacity	Horizontal scaling behind an LB	Move session state out
Quick bump, small system	Vertical scaling	Ceiling, still one machine
Spread traffic	Load balancer (L7 HTTP, L4 TCP)	LB redundancy
Daily traffic swings	Autoscaling	Start-up time, right metric
One entry point for many services	API gateway	Bottleneck, "god" layer
Strong consistency, transactions	Relational DB	Scaling writes past one primary
Flexible nested records	Document store	Weaker joins/transactions
Key lookups at huge scale	Key-value store	Few query patterns
High write rate, time-ordered	Wide-column / LSM store	Query-driven tables, eventual consistency
Metrics over time	Time-series DB	Retention, downsampling
Friends of friends	Graph DB	Hard to shard
Full-text search	Search engine	It's a copy — keep in sync
Similarity on embeddings	Vector DB / pgvector	Approximate, memory hungry
Large files	Object storage	Not for small frequent updates
More DB read capacity	Read replicas	Lag, read-your-writes
More write capacity / data	Sharding with a good key	Hot spots, cross-shard queries, resharding
Change nodes without moving most data	Consistent hashing + vnodes	Some movement, balancing
Fast column queries	B-tree index	Slower writes, storage
Slow work off the request	Queue + workers	Duplicates, idempotency, DLQ
Many services react to one event	Pub/sub or stream (Kafka, SNS)	Ordering, schema changes
Replay, high-throughput logs	Kafka	Partitions cap parallelism
Safe payment retries	Idempotency key	Key storage and expiry
Absorb spikes	Queue buffer, backpressure	Growth hides overload
Stop abusive clients	Token-bucket rate limiting, 429	Fair limits, distributed counters
Failing dependency	Timeouts + circuit breaker + fallback	Thresholds, test the open state
Transient failures	Backoff + jitter retries	Retry storms; idempotent calls only
Contain failures	Bulkheads	Idle reserved capacity
Survive a data-centre loss	Multi-AZ, then multi-region	Cost, lag, split brain
Cross-service consistency	Saga with compensations	Interim states

If you need…	Reach for…	Watch out for…
Atomic commit across two DBs	2PC	Blocking coordinator, slow
One worker at a time	Lock with fencing tokens, leader election	Lease expiry mid-work
Agree on a value/leader	Raft via etcd/ZooKeeper	Majority needed, write latency
Uncoordinated unique IDs	UUIDv7 / Snowflake	UUIDv4 index locality, clocks
"Seen before?" cheaply	Bloom filter	False positives
Distinct counts	HyperLogLog	~1% error
Nearby places	Geohash / quadtree	Cell borders
Server → browser updates	SSE	One direction
Two-way real time	WebSockets	Stateful connections
Notify another company	Webhooks	Retries, signatures, duplicates
Fast internal calls	gRPC + Protobuf	Browser needs a proxy
Clients choose fields	GraphQL	Caching, query cost
Overnight analysis	Batch (Spark), warehouse	Hours-old results
Results in seconds	Stream processing (Flink)	State, correctness
Production visibility	Metrics, logs, traces, SLO alerts	Alert fatigue, log cost
Safe releases	Canary, blue-green, feature flags	Schema changes
One login, many apps	SSO with OIDC	Critical identity provider
Act for a user elsewhere	OAuth 2.0	Token storage, scopes
Stateless auth	JWT	Revocation
Store passwords	bcrypt/scrypt/Argon2	Never encrypt or fast-hash
Protect secrets	Secrets manager	Rotation, audit

→ Full explanation

Top 60 interview questions

One-line model answers; the section of the same name above has the detail.

Foundations and performance

1. **System design?** Choosing parts and how they fit to meet features, scale, speed and reliability — a trade-off exercise.
2. **Latency vs throughput?** Time per request vs requests per second; a cargo ship has huge throughput and high latency.
3. **Why p99, not average?** The mean hides slow requests; p99 shows the unlucky 1%, which fan-out makes common.
4. **Little's law?** L = λ × W: 1,000 req/s × 0.2 s = 200 in flight, sizing threads and pools.
5. **Estimate QPS?** DAU × actions ÷ ~10^5 s, × 2–3 for peak.
6. **Four nines?** 99.99%: ~52.6 min/year, ~4.4 min/month down.
7. **SLA vs SLO vs SLI?** Measurement, internal target, customer contract (looser, with penalties).
8. **Error budget?** 1 − SLO; spend on releases, stop and fix reliability when used up.
9. **Vertical vs horizontal?** Bigger machine (simple, capped, SPOF) vs more machines (needs LB, statelessness).

10. **Stateless service?** No per-user memory between requests, so any instance serves any request.
11. **SPOF?** A part whose loss stops everything; remove with redundancy, health checks, automatic failover.
12. **Reliability/availability/durability?** Correct over time / answering now / saved data survives (S3: eleven nines durability).
13. **Interview approach?** Requirements, estimates, API, data model, high-level design, deep dives, trade-offs.

Networking and communication

14. **How does DNS work?** Resolver → root → TLD → authoritative, each answer cached for its TTL.
15. **TCP vs UDP?** Reliable ordered connection with handshake vs connectionless datagrams for real-time.
16. **HTTP/1.1 → 2 → 3?** One request per connection → multiplexed streams over TCP → streams over QUIC without TCP HOL blocking.
17. **What HTTPS adds?** TLS encryption, integrity and server authentication via certificates.
18. **Idempotent methods?** GET, HEAD, PUT, DELETE, OPTIONS — safe to retry; POST needs an idempotency key.
19. **REST vs gRPC vs GraphQL?** Public CRUD and caching / internal typed streaming / clients picking fields.
20. **Real-time to browsers?** Polling, long polling, SSE (one-way), WebSockets (two-way).
21. **Load balancer?** Spreads traffic over healthy servers for scale and availability.
22. **L4 vs L7?** Connections by IP/port vs HTTP-aware per-request routing.
23. **LB algorithms?** Round robin, weighted, least connections, least response time, IP/consistent hash, random, P2C.
24. **Sticky sessions?** Same user → same server; avoided because they unbalance load and lose sessions.
25. **CDN?** Edge caches near users cut latency and origin load.
26. **Forward vs reverse proxy?** Acts for clients vs acts for servers.
27. **API gateway?** Front door doing auth, rate limits, routing, transformation.
28. **Offset or cursor pagination?** Offset for small admin lists; cursor for large or changing feeds.

Data storage

29. **SQL or NoSQL?** By access pattern and consistency; SQL by default.
30. **ACID?** Atomicity, consistency, isolation, durability.
31. **Index cost?** Fast lookups for slower writes, space and memory.
32. **Replication vs sharding?** Copies for reads and availability vs splitting for writes and size.
33. **Sync vs async replication?** No loss but slower writes vs fast but may lose recent writes.
34. **Replication lag?** Followers behind the leader; users may not see their own writes.
35. **Shard key?** High cardinality, even load, in the main query, not monotonic for ranges.
36. **Consistent hashing?** Ring placement so membership changes move ~1/N keys.
37. **Normalize or denormalize?** Normalize first; denormalize measured hot reads.
38. **Object storage?** For large files, backups, data lakes; keep keys in the DB.
39. **OLTP vs OLAP?** Small transactional queries on row stores vs big scans on column stores.
40. **N+1 problem?** One query per list item; fix with eager loading or batching.

Caching, async and consistency

41. **Cache-aside?** Read cache, on miss load DB and fill; on write update DB and delete key.
42. **Write-through vs write-back?** Synchronous DB write (fresh, slower) vs deferred (fast, can lose data).
43. **LRU?** Evict the least recently used entry.
44. **Cache stampede?** Many misses on one expired hot key; prevent with single flight, stale-while-revalidate, jitter.
45. **Why a queue?** Fast responses, decoupling, spike absorption, retries.
46. **Queue vs pub/sub?** One consumer per message vs every subscriber gets a copy.
47. **Delivery guarantees?** Maybe lost / maybe duplicated / effectively once via dedup.

48. **Idempotency?** Repeating has no extra effect; implement with idempotency keys stored transactionally.
49. **Dead-letter queue?** Where repeatedly failing messages go for inspection and redrive.
50. **Strong vs eventual?** Reads see the latest write vs converge eventually.
51. **CAP?** During a partition choose consistency or availability.
52. **Quorum?** R + W > N replicas so reads overlap writes.
53. **Why Raft?** Majority agreement on leader and log despite failures, preventing split brain.
54. **2PC or saga?** Saga across services; 2PC blocks and couples availability.

Architecture, reliability and security

55. **Monolith or microservices?** Modular monolith first; split for team or scaling needs.
56. **Circuit breaker?** Stops calling a failing dependency (closed, open, half-open).
57. **Jitter?** Desynchronises retries so they don't arrive in waves.
58. **Token bucket?** B tokens refilled at R/s, one per request; allows bursts.
59. **AuthN vs AuthZ?** Who you are (401) vs what you may do (403).
60. **RPO vs RTO?** Data you may lose vs time you may be down.

→ Full explanation

Master A–Z glossary

Every term below is defined in bold in Parts 1–6 above (search the page for it); use this list as a self-test index.

What it is

.

A

A / AAAA record; ABAC; ACID; Active-active; Active-passive; Active / passive health check; Anycast; API; API gateway; API key; At-least-once / at-most-once delivery; Authentication; Authoritative name server; Authorization; Autoscaling; Availability; Availability zone (AZ).

B

Back-of-the-envelope estimate; Backpressure; Bandwidth; BASE; Base62; Batch processing; bcrypt; Block storage; Bloom filter; Blue-green deployment; B-tree; Bulkhead; Burn rate.

C

Cache; Cache-aside; Cache hit / miss; Cache invalidation; Cache stampede; Canary deployment; CAP theorem; Causal consistency; CDN; Certificate (TLS); Chaos engineering; CI/CD; Circuit breaker; CNAME record; Cold start; Column-oriented storage; Compensating action; Concurrency / parallelism; Connection draining; Connection pool; Consensus; Consistency (ACID); Consistency (distributed); Consistent hashing; Consumer; Consumer group; Container; Cookie; CORS; Count-min sketch; Covering index; CQRS; CSRF.

D

Data lake; Data warehouse; DAU; DDoS; Dead-letter queue (DLQ); Denormalization; Dirty / non-repeatable / phantom read; Disaster recovery (DR); Distributed cache; Distributed lock; Distributed system; DNS; Document database; Durability.

E

Elasticity; Encryption in transit / at rest; Error budget; Event-driven architecture; Event sourcing; Event streaming; Eventual consistency; Exactly-once; Exponential backoff.

F

FaaS; Failover; Fallback; Fault / failure; Fault tolerance; Feature flag; Federation; Fencing token; File storage; Firewall; Foreign key; Forward proxy; fsync; Functional requirement.

G

GeoDNS; Geohash; Global server load balancing (GSLB); Golden signals; Gossip protocol; Graceful degradation; Graph database; GraphQL; gRPC.

H

Hashing; Head-of-line blocking; Health check; Heartbeat; Horizontal scaling; Horizontal / vertical partitioning; Hot key / hot spot; HTTP; HTTP/2 / HTTP/3; HTTPS; HyperLogLog.

I

Idempotency; Idempotency key; Index; Infrastructure as code (IaC); In-process cache; Inverted index; IOPS; IP address; IP hash; Isolation level.

J

Jitter; JWT.

K

Kafka; Keep-alive; Key-value store; Kubernetes.

L

L4 / L7; Lamport clock; Latency; Leader election; Leader-follower replication; Leaderless replication; Leaky bucket; Least connections; Least privilege; Linearizability; Little's law; Load balancer; Load shedding; Long polling; Lost update; LRU / LFU; LSM-tree.

M

MapReduce; Merkle tree; Message queue; Microservices; Modular monolith; Monolith; Monotonic reads; MTBF; MTTR; Multi-leader replication; Multi-region.

N

N+1 query; NAT; Network partition; Nines; Node; Non-functional requirement; Normalization; NoSQL; NS record; NTP.

O

OAuth 2.0; Object storage; Observability; Offset (messaging); OLAP; OLTP; On-call; OpenID Connect (OIDC); Origin; Origin shield; ORM; OSI model.

P

p50 / p95 / p99 / p999; PACELC; Pagination; Partition; Paxos; Peak factor; Percentile; PKCE; Point of presence (PoP); Port; Power of two choices; Primary key; Private / public IP; Protobuf; Pub/sub; Push / pull CDN.

Q

QPS; Quadtree; QUIC; Quorum.

R

Raft; Rate limiting; RBAC; Reactive / scheduled / predictive scaling; Read replica; Read-through / write-through / write-around; Read-your-writes; Recursive resolver; Redis; RED method; Redundancy; Region; Relational database; Reliability; Replication; Replication lag; Resharding; Response time; REST; Reverse proxy; Rollback; Rolling deployment; Root server; Round robin; Round trip time (RTT); RPC; RPO; RTO.

S

Saga; Salt; Saturation; Scalability; Secrets management; Serverless; Server-Sent Events (SSE); Service discovery; Service mesh; Session; Sharding; Shard key; Sidecar; SLA; SLI; Sliding window; SLO; Snowflake ID; SOA; Split brain; SQL injection; SSO; Stateless / stateful service; Sticky session; Stream processing; Strong consistency; Synchronous / asynchronous; Synchronous replication.

T

Tail latency; TCP; Three-way handshake; Throttling; Throughput; Thundering herd; Timeout; Time-series database; TLD; TLS; TLS termination; Token bucket; Topic; Trace; Transaction; TTL; Two-phase commit (2PC).

U

UDP; USE method; UUID.

V

Vector clock; Vector database; Vertical scaling; Virtual machine (VM); Virtual node.

W

WAF; WAL; Webhook; WebSocket; Wide-column store; Worker; Write-back / write-behind; Write skew.

X, Y, Z

XSS; Zero trust; ZooKeeper.

→ Full explanation

Part 6 glossary

Term	Meaning
Short code	The few characters (e.g. `aZ3kQ9b`) identifying a short link
Counter range	Block of numbers handed to one server for ID generation
Hash collision	Two different inputs producing the same hash
Request coalescing	One request refills a cache entry while others wait for it
80/20 rule	Rough rule that 20% of items get 80% of traffic

→ Full explanation

Strangler Fig Pattern

Category: Migration / modernization **One-liner:** Replace an old system gradually by putting a router in front of it and moving one capability at a time to the new system, instead of rewriting everything and switching over in one go.

Part 0 — Concepts you need first

What is a monolith?

A **monolith** is an application built and deployed as one unit: one **codebase** (all its source files), one **artifact** (the runnable package the build produces, e.g. a `.jar`), one **process** (a running program with its own memory). Changing one line in reporting means rebuilding and **redeploying** (putting a new version live) *everything*, including payments.

Monoliths are not evil — for a small application they're usually the right choice. Problems start when it grows large and many teams must change it at once. (Architectural styles compared: § Architectural styles.)

What is a legacy system?

Old software that is still running because it does something essential. Nobody keeps unimportant old software running. That creates the trap this chapter is about: the system is **too risky to keep** (fragile, expensive) and **too risky to replace** (load-bearing).

What is a "big-bang rewrite"?

Build a complete replacement, then switch everything over on one date. It fails often, predictably: while you spend eighteen months building, the old system keeps changing (fixes, features, regulation) — you're rebuilding a moving target. On switchover day you discover the hundred undocumented behaviours, with no way back.

What is a proxy or router (in this context)?

Software **between** a client and one or more servers; the client talks only to the proxy, which decides where the request goes. **The client cannot tell the difference** — and that property is the entire foundation of the pattern: if clients can't distinguish the middle layer from the real system, you can change what's behind it whenever you like.

What is a feature flag?

A configuration switch that changes behaviour **without deploying new code**:

```
if (flags.isEnabled("use-new-implementation")) {
    return newImplementation(x);
}
return oldImplementation(x);
```

You change behaviour by editing configuration — and change it *back* in seconds if something breaks. Flags are what make the migration reversible.

Part 1 — Where the name comes from

The **strangler fig** seeds in the branches of a host tree, grows roots down and branches up around it, and eventually the host dies and rots away, leaving the fig standing in its shape. Two properties carry over:

1. **The fig grows around the host while the host is still alive** — both coexist for a long time.

2. **The transition is gradual and continuous** — there is no single moment when the tree becomes a fig.

In software: the new system grows *around* the legacy system, which keeps serving real **production** traffic (live users, not test environments) the whole time it's being displaced.

Part 2 — The problem in detail

Why legacy systems are painful to live with

Pain	What it means in practice
Technical debt	Years of rushed patches by people who've left; nobody is confident what a change affects. ("Debt": a shortcut now that charges interest later as slower change.)
Obsolete technology	The **stack** (languages, frameworks, databases, servers) is outdated: hard to hire for, no vendor **patches** (bug/security fixes), **libraries** with bugs that will never be fixed
Integration issues	Modern tools don't connect easily → **silos of information**: data manually exported and duplicated
Scalability concerns	Built for its era's load; may assume one server; **not cloud-ready** — can't simply run as more copies on the **cloud** (rented on-demand servers: AWS, Azure, Google Cloud) when traffic grows
Costly maintenance	Every change takes longer; fewer people understand the code
Security vulnerabilities	Predates current practice: weak password storage, no protection against now-standard attacks, an **OS** that no longer gets patches

Why legacy systems are painful to replace

- **Critical operations** — disruption isn't "the site is slow", it's "we cannot take orders today".
- **Migration risks** — moving data risks **loss or corruption**; old **schemas** (table/column structure) hold years of irregularity: rule-violating records, reused fields, **encoding** inconsistencies (garbled accents/symbols).
- **Training and adoption** — staff who were fast on the old system are suddenly slow, and resist.
- **High costs** — building a replacement *and making it as reliable as the old one*. The old system's reliability came from a decade of production bug-fixing your new one hasn't had.
- **Unknown unknowns** — undocumented behaviours users depend on.

> This last one kills rewrites. You can't specify behaviour nobody wrote down. You discover it at switchover: "the report used to round differently and now our reconciliation breaks." A rewrite finds them all at once, on the worst possible day.

Part 3 — The solution

Don't replace the system. **Strangle it.** Build the new system alongside the old one, put a routing layer in front of both, migrate one capability at a time, and flip the router as each piece proves itself in production. When no traffic reaches legacy, delete it.

Why this beats a rewrite

- **Less risky** — every step is individually revertible; if the new checkout breaks, route checkout back to legacy in seconds.
- **Flexible prioritisation** — migrate by **urgency or business need**, not the dependency order a rewrite forces. Most painful module first.

- **No big-bang releases** — **continuous delivery** (always releasable, small frequent changes) instead of one catastrophic **cutover** date (when traffic switches from old to new).
- **Smooth transition** — users may not notice; each piece is tested before it carries real traffic.
- **Early feedback loops** — real users hit new code early, while problems are cheap.
- **Technology flexibility** — not locked into what was fashionable at kickoff, which matters when a migration takes two years.

Part 4 — Architecture

The Facade Interface — the gatekeeper

Every request hits the facade first, and it makes one decision: **has this functionality been migrated yet?** Yes → new system; no → legacy.



The critical property: **clients are unaware there are two systems.** They see one address, one **API** (the requests a system accepts and the responses it returns), one system. ("Facade" = the Facade design pattern: a simple unified front over something complicated — here, two entire systems.)

Layered, phased migration

Peel the legacy system layer by layer:

1. **Identify a piece of functionality that can be cleanly isolated** and redirected.
2. **Rebuild it** in the new system.
3. **Test it fully.**
4. **Flip the facade** for that functionality only; everything else still goes to legacy.
5. **Repeat.**

Step 1 is the hard one. "Cleanly isolated" means it doesn't share **mutable state** (data that changes at runtime) with everything else. Most legacy systems have one giant shared database, so finding a genuine **seam** (a boundary with few connections across it) is the real work. Look for capabilities that read a lot and write little, or own a table nothing else writes to.

Part 5 — The four implementation steps

1. **Identify entry points** — choose what to replace and build new services for it.
2. **Route traffic** — a router/proxy directs each request to old or new based on what's migrated.
3. **Incrementally build and replace** — slowly displace old functionality.
4. **Retire the old** — once everything is taken over, **decommission** (permanently shut down) the legacy system.

Part 6 — Worked example 1: routing a single function

Replace just `getUserDetails` in an old user-management system:

```python
class UserServiceRouter:
    def get_user_details(self, user_id):
        if self.use_new_service(user_id):
            return NewUserService().get_user_details(user_id)
        else:
            return OldUserService().get_user_details(user_id)

    def use_new_service(self, user_id):
        # Logic deciding which service to use.
        # For simplicity: use the new service for even user IDs.
        return user_id % 2 == 0
```

- `UserServiceRouter` is the facade — **callers use it, never the two services directly**, so the switch is invisible.
- `get_user_details` has the same **signature** (name, parameters, return type) as the underlying services: a **drop-in replacement**.
- `use_new_service` is the **decision function** — the entire migration policy lives in this one method. Even IDs ≈ 50% of users.

Why the even-ID trick is smarter than it looks

It's a **percentage rollout dial**. A **hash function** turns an input into a number in a scrambled but repeatable way (same input → same output, outputs spread evenly), so `hash(user_id) % 100` puts each user in a fixed bucket 0–99. You choose *how much* new:

```python
return hash(user_id) % 100 < 5     # 5% of users
return hash(user_id) % 100 < 50    # 50% of users
return True                        # everyone
```

It's **consistent per user**: user 42 always gets the same implementation. If users flipped randomly per request, any behavioural difference would look like an unreproducible intermittent bug. Widen the condition as confidence grows until the old path is **dead code** (can never run) — then delete it.

Part 7 — Worked example 2: a facade over a banking system

Legacy and new systems expose **identical signatures** — deliberately, because the facade can only substitute one for the other if they're interchangeable:

```java
public class LegacySystem {
    public void createAccount(String name) { /* old logic */ }
    public void deposit(int amount)        { /* old logic */ }
    public void withdraw(int amount)       { /* old logic */ }
    public int  checkBalance()             { /* old logic */ }
}
// NewSystem: the same four methods with /* new logic */
```

The facade — the class the outside world talks to — holds both and directs each request:

```java
public class BankingFacade {
    private LegacySystem legacySystem;
    private NewSystem newSystem;

    public BankingFacade(LegacySystem legacySystem, NewSystem newSystem) {
        this.legacySystem = legacySystem;
        this.newSystem = newSystem;
    }

    public void createAccount(String name) {
        if (newSystemReadyFor("createAccount")) {
            newSystem.createAccount(name);
        } else {
            legacySystem.createAccount(name);
        }
    }

    // deposit(...) and checkBalance() follow the same if/else shape

    private boolean newSystemReadyFor(String functionality) {
        // Check whether the new system is ready for this functionality
        return false;   // placeholder — starts false for everything
    }
}
```

How the migration actually proceeds

At the start `newSystemReadyFor` returns `false` for everything: **100% of traffic goes to legacy**. Deploy the facade first and verify it changes nothing *before* migrating anything — if something breaks then, you know it's the facade, not new code.

Then rebuild `createAccount`, test it, and make `newSystemReadyFor("createAccount")` return `true`: account creation goes to the new system while deposits, withdrawals and balance checks stay on legacy. Repeat per capability.

The key structural insight: the only thing that changes at each cutover is what `newSystemReadyFor` returns. Clients don't change; the two systems don't change. In reality that method reads a **feature flag**, so flipping a capability needs no deployment — and flipping it *back* takes seconds.

Part 8 — Key insights: the proxy layer is everything

The router is on the critical path for **every request**, so it must be robust. Three kinds of routing logic:

- **Feature flags** — turn new functionality on/off; the superpower is **instant rollback** (back to known-good behaviour) with no deploy or rebuild.
- **User segmentation** — **A/B testing** (two variants to different groups, compare results) or **phased rollouts** (growing share of users): internal staff → 1% → 10% → everyone, or a small market first.
- **API versioning** — route on version **headers** (request metadata, e.g. `Accept-Version: 2`) or **paths** (the URL part after the domain) — `/v1/orders` to legacy, `/v2/orders` to new.

Part 9 — Trade-offs and what actually goes wrong

- **Running two systems at once** — for what may be years: two **deployment pipelines** (automated build-test-release), two **on-call rotations**, two sets of **dashboards**.
- **The facade is a single point of failure** — everything flows through it, so it needs **high availability** (redundant copies) and a careful **latency budget** (max delay each component may add — see § Latency): whatever it adds, every request pays.
- **An extra hop** — one more network trip on every request, however small.

- **Data consistency — the hard part the pattern doesn't solve.** If old and new share or split data you need a strategy (shared database, sync, or **dual writes** — the app writing to two databases). See Part 13D.

  > Why: migrate "create account" but leave "check balance" on legacy, with separate databases, and accounts created in the new system are invisible to legacy's balance check. Teams usually keep **one shared database** during migration and split it later — a second strangler migration in its own right.

- **Duplicated logic** — while a capability is in flight, the same rule lives in two places; change both or they diverge.
- **Migration fatigue — the most common real-world failure.** The pattern makes it *safe* to stop halfway, so organisations do: the urgent 60% migrates, the pain fades, priorities shift, and both systems run forever — worse than either alone. **Set a decommission date up front and treat it as a real deliverable.**

Part 10 — Common beginner misconceptions

- **"The new system must be microservices."** (**Microservices**: many small, independently deployable services, each owning one capability.) No — Strangler Fig is *how* you migrate, not *what to*. You can strangle a monolith into a better monolith.
- **"You migrate the database first."** Usually the opposite: data is the riskiest part, so keep the shared database, move behaviour first, split data later.
- **"The facade is temporary."** Often it becomes the permanent API Gateway. Fine, if deliberate.
- **"Delete old code as soon as a capability migrates."** It's your rollback. Leave it unreachable but intact until the flag has been at 100% for a sensible period.
- **"Only for ancient systems."** It works whenever you replace something in use — a payment provider, a search engine, a cloud.

Part 11 — Check your understanding

1. Why is the strangler fig a better metaphor than "demolish and rebuild"?
2. What single method in the `BankingFacade` example controls the entire migration?
3. Why route by a *consistent* function of user ID rather than randomly per request?
4. Why deploy the facade before migrating anything?
5. What problem does the pattern explicitly *not* solve?
6. What's the most common organisational failure mode, and the countermeasure?

(Answers: 1 — both systems coexist and the transition is gradual and reversible; 2 — `newSystemReadyFor`; 3 — a user must get consistent behaviour, or differences look like intermittent bugs; 4 — to verify the facade itself is harmless before adding a second variable; 5 — data consistency between old and new; 6 — stalling halfway, prevented by committing to a decommission date.)

Part 12 — Interview soundbite (30 seconds)

> "Strangler Fig replaces a legacy system without a big-bang rewrite. You put a facade or proxy in front of the old system so clients only talk to that, then rebuild one capability at a time behind it. The facade routes each request to the new service or legacy depending on what's migrated — usually via feature flags or user segmentation, giving percentage rollouts and instant rollback without a deploy. Traffic to legacy drops to zero and you delete it. It's named after a tree that

grows around its host until the host rots away. It beats a rewrite because a rewrite chases a moving target and discovers all the undocumented legacy behaviour on switchover day, whereas this surfaces problems one capability at a time while you still have a way back. Trade-offs: running both systems for a long time, the facade as a critical single point of failure, and data consistency between old and new, which the pattern doesn't solve. The biggest practical risk is organisational — it's safe to stop halfway, so teams do; commit to a decommission date up front."

Part 13 — Implementation

A. Runnable demo

Full source: `demos/StranglerFigDemo.java`

```
cd demos && ./gradlew runCh02
```

(`./gradlew` is the **Gradle wrapper**: it downloads and runs Gradle, the Java build tool used for every demo in this book.)

The core is the flag check that makes rollout **consistent per user**:

```
boolean useNewSystem(String capability, String userId) {
    int percent = rolloutPercent.getOrDefault(capability, 0);
    if (percent <= 0)   return false;
    if (percent >= 100) return true;
    int bucket = Math.floorMod(userId.hashCode(), 100);   // stable per user
    return bucket < percent;
}
```

`Math.floorMod`, not `%`, because `hashCode()` can be negative and `%` would give a negative bucket.

B. What you should see

```
=== Step 1: facade deployed, nothing migrated ===
  LEGACY:created:alice
  >> all traffic still on LEGACY - facade proven harmless

=== Step 3: same user always gets the same system ===
  user-7 -> LEGACY:created:acct     (x3, never flips)

=== Step 4: widen the rollout ===
    [flags] createAccount -> 25%  ->  actual: 253/1000 users
    [flags] createAccount -> 50%  ->  actual: 493/1000 users
    [flags] createAccount -> 100% ->  actual: 1000/1000 users

=== Step 5: problem found - INSTANT rollback ===
  user-7 -> LEGACY:created:acct
  >> no deploy, no rebuild: config change only
```

Step 3 proves consistency — user-7 never flips. The percentages aren't exact (25% gave 253/1000, 50% gave 493) because **hash distribution is only approximately uniform** at these sample sizes. That's fine: you're **canarying** (sending a small slice of real traffic to the new version first), not metering.

C. Production — routing at the proxy

In reality the facade is usually infrastructure, not application code.

Nginx — route by path (a **reverse proxy** forwards client requests to servers; an `upstream` block names a backend group):

```
upstream legacy_system { server legacy.internal:8080; }
upstream new_system    { server orders-v2.internal:8080; }

server {
    location /api/orders  { proxy_pass http://new_system;    }   # migrated
    location /api/billing { proxy_pass http://legacy_system; }   # not yet
    location /            { proxy_pass http://legacy_system; }   # default: legacy
}
```

Istio — percentage rollout without touching application code (Istio is a **service mesh**, configured in YAML; see Sidecar):

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: orders
spec:
  hosts: [orders]
  http:
    # Internal staff always get the new system - your first canary group
    - match:
        - headers:
            x-user-group: { exact: internal }
      route:
        - destination: { host: orders, subset: v2 }
    # Everyone else: 5% to the new system
    - route:
        - destination: { host: orders, subset: v1 }
          weight: 95
        - destination: { host: orders, subset: v2 }
          weight: 5
```

Feature flags in application code — LaunchDarkly (hosted), Unleash (open source), or a table in your config store (Config Externalization):

```
if (flags.isEnabled("orders-v2", userContext)) {
    return newOrderService.create(order);
}
return legacyOrderService.create(order);
```

D. The part the pattern doesn't solve: data

Three strategies, in increasing difficulty:

1. **Shared database (start here).** Both systems use the same database and old schema — no sync problem. Split the data later as a separate migration.
2. **Dual writes.** The new system writes to both databases. Easy to describe, hard to get right: if the second write fails the copies diverge, and no **transaction** (all-or-nothing group of changes) spans both. (Same root problem as the Transactional Outbox.)
3. **Change Data Capture (CDC).** Watch every insert/update/delete in the legacy database's **transaction log** (its append-only record of committed changes) with **Debezium** and stream them into the new one. The new data is **eventually consistent** (may lag briefly, then catches up), and **the legacy system needs no modification**:

```
# Debezium connector: legacy Postgres -> Kafka -> new service
connector.class: io.debezium.connector.postgresql.PostgresConnector
database.hostname: legacy-db
table.include.list: public.orders,public.customers
plugin.name: pgoutput
```

(Kafka, the event log in the middle, is covered in Event-Driven.)

E. Implementation pitfalls

1. Random rather than consistent routing. `Math.random() < 0.05` sends the *same user* to different systems on consecutive requests, so any difference becomes an unreproducible bug. Always hash a stable identifier.

2. Deploying the facade and migrating a capability in one release. If it breaks you can't tell which change did it. Facade first, confirm no-op, then migrate.

3. Deleting legacy code the moment traffic hits zero. It's your rollback. Keep it until the flag has been at 100% for a full business cycle.

4. No comparison stage. Before flipping traffic, run **shadow mode**: send the request to both, return the legacy response, log differences. This finds undocumented behaviour *before* users do. The new system runs **asynchronously** (on another thread, so the user never waits) and records **metrics** (monitoring counters):

```
Result legacyResult = legacy.process(request);
executor.submit(() -> {                          // async, never blocks the user
    try {
        Result newResult = modern.process(request);
        if (!newResult.equals(legacyResult)) {
            metrics.increment("shadow.mismatch");
            log.warn("Divergence for {}: {} vs {}", request.id(), legacyResult, newResult);
        }
    } catch (Exception e) { metrics.increment("shadow.error"); }
});
return legacyResult;                             // user always gets the proven answer
```

5. Flags that outlive the migration. Each finished migration leaves a dead **conditional**. Make flag removal part of "done", or the codebase fills with permanent forks.

6. No decommission date. The organisational failure mode from Part 9. Put it in the plan.

Related patterns

- API Gateway — the facade is very often literally an API Gateway
- BFF — can bridge new frontends to legacy systems during migration
- Config Externalization — where the feature flags driving routing actually live

API Gateway Pattern

Category: Edge / routing **One-liner:** A single entry point that sits in front of all microservices, routing, aggregating, securing, and shaping requests so clients never talk to services directly.

Lesson 1 — Introduction to the API Gateway Pattern

The fast-food counter

You order a burger at the counter: you order, pay and collect there, and never walk into the kitchen or to a separate billing desk. The counter is the gateway — **the single point at which the customer interacts with all the restaurant's services**.

What the pattern actually is

In a **microservice architecture** (many small services, each running separately and owning one business capability), the API Gateway is **a single entry point for external consumers** to reach all of them. Three independent benefits follow:

1. **Easier to manage** — one place where request-handling policy lives.
2. **Secures your services** — services need not be individually exposed.
3. **A unified interface to the outside world** — clients see one coherent API, not the seams between your teams.

Why the pattern exists

The shift from **monoliths** (one big unit) to microservices brought scalability and flexibility, but its biggest new problem is communication: **how do clients and services communicate efficiently across many services, and how do you manage the growing complexity of their interactions?** The gateway fills that gap.

The Netflix example

Opening Netflix on a TV needs the movie list, your preferences, subtitles and more — each from a separate microservice.

- **Without a gateway:** the TV app calls each service itself, so it must know each one exists, where it lives, how to **authenticate** to it (prove identity), and its response shape.
- **With a gateway:** the app makes one call; the gateway talks to whichever services are needed.

Note *which* client benefits most: TV and mobile apps can't be force-updated, so any **service topology** (which services exist and where) baked into them is frozen in the field for years.

The master orchestrator

The gateway does three things in sequence:

1. Serves as the **single entry point** for all client requests
2. **Routes** them to the appropriate microservices
3. **Gathers the responses** and returns one **unified response**

Its value is **masking the underlying complexity**: the client doesn't know how many services were involved or how they talk.

The headline advantages

- **Simplified client** — one gateway instead of many services.
- **Security** — **authentication** (who is calling) and **authorization** (what they may do) at one point.
- **Performance** — **caching**, response compression, and **load balancing** across service copies (§ Caching, § Load balancers).
- **Cross-cutting concerns** (needs every service shares) — logging, analytics and similar, handled once.

Lesson 2 — Advantages: the fifteen problems it solves

Each is a distinct interview argument:

#	Problem without a gateway	Gateway's answer
1	**Complexity of interactions** — every service talking to every other; connections explode	Central hub: a mesh of connections collapses into one point
2	**Inconsistent service calls** — different interaction styles and **data formats** (JSON, XML, binary); clients implement several dialects	One consistent interface; **translates calls and formats** so the client sees uniformity the **backend** lacks
3	**Security and authentication** per service — duplicated, easily inconsistent	Authenticate and authorize **centrally**, before anything reaches a service
4	**Load balancing** — some services get far more traffic	Distributes traffic so **no single service is overwhelmed**
5	**Performance** — network calls between scattered services add delay	**Caches responses**, reducing repeated calls
6	**Cross-cutting concerns** (logging, monitoring, analytics) in every service	Handled once at the gateway
7	**Legacy systems** needing awkward access	Talks to legacy behind a **modern interface**; awkwardness stops at the boundary
8	**Version management** — clients need different service versions (acute for un-updatable mobile/TV)	Routes to the right version per client
9	**Aggregation** — data spread over several services means several calls, painful on mobile	Aggregates into a **single consolidated response**
10	**Orchestration** — calls must happen **in a particular order**	Sequences them and transforms data between steps
11	**Partial failures** in a **distributed system** (programs on many machines cooperating over a network)	Detects failures, **reroutes** or returns **fallback responses** (safe defaults)
12	**Request transformation** — services expect different request formats	Converts each request to what the target expects
13	**Client-specific adaptations** — mobile and web need different things	Adapts responses per client *(taken to its conclusion → BFF)*
14	**Centralized logging and monitoring** is hard across many services	**One point for observability** (understanding a running system from its logs, metrics, traces)
15	**Long-running operations** shouldn't block the client	Manages them **asynchronously**; clients **check progress or get a callback**

Lesson 3 — Worked example: a library system

Library users, staff and management use three services: **CatalogueService** (book searches), **UserService** (profiles), **InventoryService** (stock levels). Each is a Java **interface** (method signatures, no implementation) — e.g. `CatalogueService` exposes `searchByTitle(String)` and `searchByAuthor(String)`.

The gateway

A separate class is the single point of contact. It mirrors the services' methods, but **each method delegates to the appropriate service** rather than handling the request itself:

```java
public class ApiGateway {
    private UserService userService;
    private CatalogueService catalogueService;
    private InventoryService inventoryService;

    public ApiGateway(UserService userService,
                      CatalogueService catalogueService,
                      InventoryService inventoryService) {
        this.userService = userService;
        this.catalogueService = catalogueService;
        this.inventoryService = inventoryService;
    }

    public User getUserDetails(String userId) {
        return userService.getUserDetails(userId);
    }

    public Book searchByTitle(String title) {
        return catalogueService.searchByTitle(title);
    }

    // ...additional methods as needed...
}
```

Take the shape, not the detail: gateway methods *delegate*, never *implement*. Once a gateway method contains **business rules** (domain logic, e.g. "members may borrow 5 books"), you're on the road to a **distributed monolith** — services so tangled they must change and deploy together. Real gateways wrap each delegation with auth, **timeouts**, **retries** and caching, but the delegation shape stays.

Lesson 4 — Problems the gateway itself introduces

Problem	Mitigation
1. Single point of failure — if it goes down nothing communicates; its availability *is* your availability	**High availability** and **redundancy** (spare copies); monitor it diligently
2. Potential bottleneck — every request passes through it	Keep it fast; **scale horizontally** (more copies, not a bigger machine); optimise its code
3. Complexity in the gateway — routing, security, transformation all accumulate	Keep responsibilities clear; **split into multiple gateways** per client type or per functionality
4. Maintenance overhead — routes and rules drift from reality as services evolve	**Automate**; use **CI/CD** (automatic build, test and release of every change)
5. Security risk — the entry point is a **prime target**	Robust security practices, keep it patched, monitor for **vulnerabilities**

Problem 3 is the failure mode to name explicitly: the gateway is where business logic goes to hide. Once domain rules live there, every team must change it and it becomes the deployment bottleneck microservices were meant to eliminate. Keep it about **transport** (getting requests to the right place), not **domain** (the business problem).

Lesson 4 (cont.) — Performance implications

The costs the gateway imposes on every request:

Implication	Solution
1. Latency — an extra network **hop** on every request	Optimise; **caching and response compression**
2. Resource utilization — the gateway itself can consume a lot	Provision adequately; monitor; **scale as needed**
3. Content aggregation overhead — merging responses is resource-intensive	Optimise aggregation; use **asynchronous** calls so nothing blocks
4. Rate limiting (capping requests per client per time window) is hard **in distributed deployments** — several gateway **instances** must share each client's usage	Algorithms built for it: **token bucket** or **leaky bucket**, with a shared store
5. Load balancing overhead — avoid **hotspots** (one instance getting most traffic)	Smart LB algorithms; a **service mesh** for finer control *(Sidecar)*
6. Async operations at the gateway are complex	Optimise; give clients **clear status-check or callback** mechanisms

Token vs leaky bucket: a **token bucket** refills at a steady rate and each request spends a token — a full bucket permits a short burst; a **leaky bucket** queues requests and releases them at a fixed rate, smoothing bursts away. Details: § Rate limiting.

Verdict: with careful design, monitoring and optimisation the costs are manageable — and the alternative, every client knowing every service, is worse.

Lesson 5 — System design example: Instagram

The gateway's responsibilities here

Routing · **aggregation** into one response · **authentication & authorization** · **rate limiting** against abuse · **load balancing** across service instances · **logging and monitoring**.

The microservices behind it

- **User Management** — profiles, followers/following, settings
- **Photo Upload and Processing** — upload, filters, resizing per device
- **Feed** — the feed of photos from people you follow
- **Notification** — new followers, likes, comments
- **Search** — profiles, hashtags, locations
- **Comment** — comments on photos

Scenario A — uploading a photo *(sequential)*

1. User uploads via the app → request hits the gateway
2. Gateway **authenticates** the user and checks they're **authorized to upload** — two separate checks, both before any service is touched
3. Gateway routes to the Photo Upload and Processing Service, which processes the image and responds
4. Gateway returns a confirmation to the app

Scenario B — viewing the feed *(parallel — the interesting one)*

1. User opens the feed → request hits the gateway, which authenticates
2. Gateway makes **parallel requests** to the **Feed Service** (recent photos from people followed) and the **Notification Service** (new notifications)
3. Gateway **aggregates** them and returns a **single consolidated response**

Why the contrast matters

Upload is a single routed call. Feed is a **scatter-gather**: one client request fans out to parallel service calls and returns one merged response. That's where the gateway earns its latency cost: client-side, it would be several sequential **round trips** over a mobile network; via the gateway it's one round trip plus parallel calls inside the **data centre**, where inter-service latency is a fraction of latency to the phone.

Consolidation

API Gateway vs. BFF

One shared entry point optimised for all clients generically (risk: bottleneck / distributed monolith) vs. one backend per client type (risk: duplication and drift). Full comparison table: BFF Part 8. They're not mutually exclusive — a common setup is a thin shared gateway (**TLS** termination, coarse auth, rate limiting) with BFFs behind it.

When to use

- Multiple microservices **and** multiple client types
- Clients would otherwise make many round trips per screen
- You want cross-cutting concerns (auth, rate limiting, logging) in one place
- You expose services to untrusted external consumers

When to avoid

- Very few services — the gateway is pure overhead
- Purely internal service-to-service traffic — a **service mesh** fits better
- One client whose needs diverge sharply — consider BFF
- Ultra-low-latency paths where an extra hop is unacceptable

Interview soundbite (30 seconds)

> "An API Gateway is a single entry point in front of your microservices. Clients call it instead of services directly, and it handles routing, aggregation, authentication, rate limiting, caching and logging in one place. The big win is that clients don't need to know your service topology — which matters most for mobile and TV clients you can't force-update, because topology baked into them is frozen for years. The classic example is Instagram's feed: one request fans out in parallel to the feed and notification services and returns one response, beating several sequential round trips from the phone. Trade-offs: it's a single point of failure needing HA and redundancy, it adds a hop, rate limiting across distributed gateway instances needs token- or leaky-bucket algorithms, and it tends to accumulate business logic until it becomes a distributed monolith — so keep it about transport, not domain, and split it per client type or domain if it grows. If each client type needs a genuinely different API shape, that's Backends for Frontends."

Implementation

A. Runnable demo

Full source: `demos/ApiGatewayDemo.java`

```
cd demos && ./gradlew runCh03
```

The heart of it is scatter-gather with per-service fallbacks (`pool` is a **thread pool** of reusable workers; each `Future` is a placeholder for a result arriving later):

```
Future<String> fFeed    = pool.submit(() -> feed.fetch(userId));
Future<String> fNotifs  = pool.submit(() -> notifications.fetch(userId));
Future<String> fProfile = pool.submit(() -> profile.fetch(userId));

result.put("feed",          get(fFeed,    "feed-unavailable"));
result.put("notifications", get(fNotifs,  "no-notifications"));
result.put("profile",       get(fProfile, "profile-unavailable"));

private String get(Future<String> f, String fallback) {
    try { return f.get(1, TimeUnit.SECONDS); }
    catch (Exception e) { return fallback; }        // partial failure contained
}
```

All three calls are **submitted before any result is read** — that's what makes them parallel. Reading `fFeed` before submitting the others would **serialise** them.

B. What you should see

```
=== 1. Without a gateway: client calls each service itself ===
  3 round trips from the client: 288ms
  >> on mobile add ~150ms latency PER trip: ~738ms

=== 2. Gateway, sequential aggregation ===
  1 client round trip, 3 sequential server calls: 279ms

=== 3. Gateway, PARALLEL scatter-gather ===
  1 client round trip, 3 parallel server calls: 129ms
  >> ~the slowest service (120ms), not the sum

=== 4. Auth enforced once, at the edge ===
  rejected: 401 Unauthorized  (no service was touched)

=== 5. Partial failure -> graceful degradation ===
  {feed=feed-data, notifications=no-notifications, profile=profile-data}

=== 6. Rate limiting, also at the edge ===
  429 Too Many Requests after 5 requests
```

Compare 2 and 3: 279ms → 129ms — parallel aggregation costs the *slowest* service, not the sum. **Section 4**: an unauthenticated request (HTTP **401**) touched **zero** backend services — the security argument as an observable fact. (**HTTP status codes** are the three-digit codes on every web response: 401 = not authenticated, 429 = too many requests.)

C. Production — Spring Cloud Gateway

Spring Cloud Gateway is a ready-made gateway on **Spring**, the dominant Java framework, added via a **Maven** dependency (Maven: Java build tool that fetches libraries). In the YAML, `lb://` means "resolve via **service discovery**" (Service Discovery), **predicates** decide which requests a route matches, and **filters** modify or guard them.

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
```

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: orders
          uri: lb://order-service          # lb:// = resolve via service discovery
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=2
            - name: CircuitBreaker
              args:
                name: orderCB
                fallbackUri: forward:/fallback/orders
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 100    # steady-state req/s
                redis-rate-limiter.burstCapacity: 200    # burst allowance
            - name: Retry
              args:
                retries: 2
                statuses: BAD_GATEWAY,SERVICE_UNAVAILABLE
                methods: GET                             # only idempotent methods

        - id: products
          uri: lb://product-service
          predicates:
            - Path=/api/products/**
            - Header=X-API-Version, v2                   # route by version
```

Global cross-cutting filter — auth on every route (`Mono` is a **reactive** type: one value delivered later without blocking a thread):

```java
@Component
public class AuthFilter implements GlobalFilter, Ordered {
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String token = exchange.getRequest().getHeaders().getFirst("Authorization");
        if (!tokenService.isValid(token)) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();      // never reaches a service
        }
        return chain.filter(exchange);
    }
    @Override public int getOrder() { return -100; }          // run early
}
```

Kubernetes-native alternative — an **Ingress** or **Gateway API** resource (Kubernetes objects describing how outside traffic enters the cluster; § Kubernetes basics) does routing, TLS and rate limiting with no gateway application to write or run. Prefer it when needs are routing-shaped rather than aggregation-shaped.

D. Implementation pitfalls

1. Sequential calls that should be parallel. The most common gateway performance bug: calling `.get()` on each future as you create it is sequential code that looks **concurrent**.

2. No per-call timeout in aggregation. One slow service makes the whole response slow. Bound each future (`f.get(1, TimeUnit.SECONDS)`) so the slowest dependency can't dictate your latency.

3. Business logic in the gateway. The distributed-monolith trap (Lesson 4). Keep it to transport: routing, auth, rate limiting, aggregation, observability.

4. In-memory rate limiting behind multiple gateway instances. Three **replicas** each allowing 100 req/s means the real limit is 300. Use a **shared store** — **Redis**, a fast in-memory key-value database — hence `redis-rate-limiter` above.

5. Retrying non-idempotent requests. `methods: GET` is deliberate: a gateway retrying a **POST** can create duplicate orders. An operation is **idempotent** if repeating it has the same effect as doing it once — see Retry.

6. Running the gateway as a single instance. It's a single point of failure by design: run at least two replicas behind a **load balancer**, with resource limits and monitoring like any critical service.

7. Losing trace context. The gateway is the ideal place to generate a **correlation ID** (a unique ID passed to every service a request touches, so logs can be joined up) and propagate it **downstream**. Skip it and you can't trace a request across services.

Related patterns

- **BFF** — one gateway per client type instead of one shared gateway
- **Circuit Breaker** — the gateway is a natural place to configure breaker rules per backend route
- **Service Discovery** — how the gateway finds service instances; the gateway is the classic server-side discovery router
- **Strangler Fig** — the migration facade is often literally an API Gateway
- **Sidecar** — gateway handles north-south (external, in and out of the system) traffic; a mesh of sidecars handles east-west (internal, service-to-service)

Backends for Frontends (BFF) Pattern

Category: Edge / routing **One-liner:** Instead of one shared backend serving every kind of client, give each client type its own backend tailored to exactly what that client needs.

Origin: Coined by **Sam Newman**, to solve building backends for structurally different clients — mobile apps, web apps, and other client-side **frameworks** (foundations you build an app on, such as React or SwiftUI).

Part 0 — Concepts you need first

What is a "frontend" and a "backend"?

The **frontend** is what the user sees — mobile app, website, smart-TV interface. The **backend** is the server-side software it calls. A **client** here means a frontend application; "client types" are categories: mobile, web, TV, watch.

What is over-fetching?

Over-fetching is a client downloading more data than it needs. A mobile product list shows thumbnail, name and price, but the API returns the full product — a 2,000-word description, fifteen high-res image **URLs**, full specs, seller history. Why it hurts:

- **Bandwidth** — on a **metered** plan (charged per MB), it's the user's money
- **Latency** — larger **payloads** (the data in a request/response) take longer on slow links
- **Battery and CPU** — the phone must **parse JSON** (the common text data format) it then discards
- **Memory** — constrained devices hold data they never use

What is an API?

The contract for how software asks other software for something (defined in Strangler Fig). "One uniform set of APIs for all frontends" means every client gets the same **endpoints** (URLs a server answers, like `GET /products/42`) returning the same shapes.

What is aggregation?

One component calls several sources and returns a single merged result — product details, reviews and stock from three services in one response, so the client makes one request instead of three.

Why round trips matter so much on mobile

A **round trip** is one request out and its response back: **5–20 ms** on an office network, **100–300 ms or worse** on mobile. Six round trips on desktop are barely noticeable; on mobile they can be two seconds of nothing. This one fact drives most of the chapter.

Part 1 — The core idea, in plain terms

A company has three platforms: **a detailed website** (extensive information), **a mobile app** (quick essentials on the go), **a smartwatch app** (core functions only). The website can display a thousand words; the watch about eight.

Traditionally one backend serves all three, and the backend complexity appropriate for the website **overwhelms** the simpler needs of mobile and watch. The website wins by default — usually the biggest, oldest client — and everyone else gets its leftovers. BFF gives each its own backend (the course's image: each section of a diverse restaurant gets its own kitchen).

Netflix does exactly this — browsers, mobile apps and smart TVs each get their own backend service, because each device has different capabilities and interfaces.

Part 2 — The problem in detail

One-size-fits-all

A traditional backend is a single entity serving all frontends, exposing **one uniform set of APIs regardless of each client's requirements**:

	Desktop	Mobile
Processing power	High	Limited
Display	Large	Small
Network	Usually fast, unmetered	Possibly slow, possibly metered

Should mobile receive the same data as desktop, and process it all on-device? **Not quite.**

Performance inefficiency and over-fetching

Over-fetching causes slow loads and bandwidth waste. A product-list screen: **shared API response 250 KB**; **mobile displays ~4 KB** (name, price, thumbnail). The user paid for 250 KB and used 4 — seconds of waiting on a slow connection for data that's thrown away.

Complexity in code management

One codebase serving many frontends becomes **bloated** — harder to maintain, troubleshoot and update — and endpoints fill with **conditionals**:

```
if (clientType.equals("mobile")) {
    // trim the description
} else if (clientType.equals("tv")) {
    // use high-res images
} else if (clientType.equals("watch")) {
    // just the title
}
```

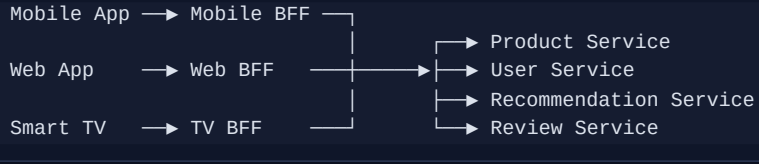
Every client's needs tangled into one method nobody can change safely.

Lack of flexibility and scalability

Two symptoms that signal you need BFF:

1. **Sifting through a monolithic codebase to change one small feature for one frontend**
2. **Scaling the entire backend because one frontend has high traffic** — a TV advert spikes mobile traffic, and you pay to scale capacity serving web and internal tools that aren't busy, because the parts can't scale separately.

Part 3 — Architecture



Three layers: **(1) Frontend applications** (desktop, mobile, web) → **(2) BFFs**, each paired with one frontend → **(3) Microservices** the BFFs call for data.

The important structural point: **the microservices are shared; only the BFF layer is duplicated.** There is still exactly one Product Service holding product truth — the **source of truth** (the authoritative copy). BFFs don't own data; they reshape it.

What a BFF actually does

- **Tailored data delivery** — acts as a **filter** so its frontend gets *just* what it needs.
- **Streamlining communication** — calls several microservices, consolidates, and presents one shape (an "interpreter" between many services and one client).
- **Handling client-specific logic** — absorbs computation the client would otherwise do.

> *Why server-side logic matters especially on mobile:* you can't force app updates. Display logic in the app means a release, app-store review, and waiting for users to update — some never will. In the BFF, you fix and **deploy** it in minutes.

Part 4 — The benefits, explained

The course lists seventeen, grouped here:

Theme	Benefit	Why
Fit & UX	Diverse frontend needs	Acknowledges that mobile, web and TV genuinely differ
	Enhanced user experience	Each device served to its capabilities
	Localization / internationalization	A BFF can serve region-specific language and features
Performance	Performance optimization	Tune data and interactions per device (slow networks, weak CPUs)
	Reduced bandwidth	Sends only what that frontend needs
	Independent scalability	A mobile spike scales only the Mobile BFF
Frontend simplicity	Simplified frontend logic	One tailored backend instead of many services
	Consistency	A **stable API while underlying microservices change** — backend teams can reorganise without breaking apps
	Versioning control	Each frontend version gets appropriate logic — acute on mobile, where old versions persist
Team & delivery	Agility	Teams ship their BFF independently; mobile doesn't queue behind web
	Easier maintenance/upgrades	Change the mobile backend without touching web
	Cross-functional teams	One team owns **a frontend and its backend end to end**
Capability	Security	Per-client auth mechanisms (e.g. cookies vs tokens)
	Error handling & resilience	Per-frontend error handling — web may degrade gracefully where mobile can't
	A/B testing	Test on one client type **without affecting others**
	Feature flagging	Toggle features per frontend at the backend
	Handling legacy systems	A **bridge** from new frontends to legacy during migration (*Strangler Fig*)

Part 5 — Worked example: e-commerce product details

One shared **Product Service**, a **Web BFF** (extensive detail), a **Mobile BFF** (concise, to save data and load faster).

1. Product Service — the real backend

```
@RestController
@RequestMapping("/products")
public class ProductServiceController {

    @Autowired
    private ProductRepository productRepository;

    @GetMapping("/{productId}")
    public Product getProductById(@PathVariable String productId) {
        return productRepository.findById(productId)
                .orElseThrow(() -> new ProductNotFoundException(productId));
    }
}
```

Reading it (Spring; `@` words are **annotations** — labels telling the framework how to treat code):

- `@RestController` — a web handler returning data, not **HTML** pages
- `@Autowired` — Spring **injects** the `ProductRepository` (the class that reads/writes this data in the database)
- `@RequestMapping("/products")` + `@GetMapping("/{productId}")` — handles `GET /products/123`; `@PathVariable` pulls `123` into the parameter
- `orElseThrow` — no product → a clear error instead of returning `null`

This service owns the **full** `Product` and knows nothing about mobile or web.

2. Web BFF — the detailed view

```
@RestController
@RequestMapping("/web/products")
public class WebProductController {

    @Autowired
    private ProductServiceClient productServiceClient;

    @GetMapping("/{productId}")
    public ProductDetails getProductDetails(@PathVariable String productId) {
        Product product = productServiceClient.getProductById(productId);
        return convertToWebProductDetails(product);
    }

    private ProductDetails convertToWebProductDetails(Product product) {
        // Convert to a detailed view suitable for web:
        // richer descriptions, full-size images, etc.
    }
}
```

The prefix `/web/products` gives each BFF its own namespace, so clients can't call the wrong one. `ProductServiceClient` is a **Feign client** (you declare an interface; the library generates the HTTP-calling code, so `getProductById(id)` looks like a normal method call) or any HTTP client.

3. Mobile BFF — the concise view

Identical, except `@RequestMapping("/mobile/products")`, return type `SimpleProductDetails`, and a `convertToMobileProductDetails` that strips descriptions and shrinks images.

The structural point — read this part twice

The two BFFs are **nearly identical**: both call the **same** `productServiceClient.getProductById(productId)` and receive the **same** full `Product`. They differ **only in the transformation step** and therefore the return type (Web: `ProductDetails`, rich; Mobile: `SimpleProductDetails`, stripped).

A BFF is a projection layer, not a second source of truth. (A **projection** is a reshaped view of data that lives elsewhere.) One Product Service holds one truth; each BFF projects a different view of it. A BFF *storing* data or deciding what a product *is* has stopped being a BFF.

Part 6 — Problems with BFF, and how to handle them

Problem	Solution
1. Code duplication across BFFs — auth handling, error formats, logging	**Shared libraries**, or **internal microservices** the BFFs consume
2. Increased maintenance — three BFFs = three **deployment pipelines**, three sets of **dependencies** to patch	**DevOps** practices (dev + ops working together, heavy automation): automated testing, deployment, monitoring
3. Coordination overhead between frontends and their BFFs in large teams	**Clear contracts** and **consumer-driven contract testing**

Consumer-driven contract testing: the consumer (the app) writes its expectations of the BFF as an executable test that runs in the BFF's build. If the BFF breaks something the app relies on, the BFF's build fails — before production, not after.

Performance implications

Implication	Solution
Network latency — an extra hop	Optimise network paths; efficient **protocols** (e.g. HTTP/2, gRPC); place **BFFs geographically close to their frontends**
Resource utilization — each BFF needs its own resources	Optimise; **serverless** (provider runs and scales servers, pay per call) where apt; scale on demand
Caching challenges — same data cached in three shapes	Caching at both BFF and frontend; good use of `Cache-Control` **headers** (how long a response may be reused)
Load on backend services — three BFFs fetching the same product = three calls	Scalable, monitored backends; **rate limiting** if needed

> ⚠️ **The duplication trap is the one that actually bites.** Three BFFs slowly drifting apart on auth or error semantics is how "tailored" becomes "inconsistent" — users hit a bug on mobile that doesn't exist on web. Shared libraries are not optional at scale.

Part 7 — System design example: Netflix

Netflix serves phones, tablets, smart TVs, laptops and consoles. **Scenario: viewing movie details** — title, description, genre, rating, cast, recommendations, reviews.

BFF	Device priority	Responsibilities
Mobile BFF (phones, tablets)	Quick loading, less data	**Authentication** (logged in, valid subscription) · **routing** to Movie Details and Recommendation services · **aggregation** · **transformation**: smaller images, shorter descriptions, **a subset of recommendations**
TV BFF	Rich visuals, detail, high-quality images	**Authentication & authorization** — not just login but **whether the device is authorized to play HD content** · routing & aggregation tuned to TV capabilities and networks · high-res images, full cast, elaborate descriptions, full recommendations
Web BFF (browsers)	Balance of performance and richness	Standard login check · routing & aggregation for browsers · content for a wide range of browsers and screen sizes

Benefits realised: tailored UX per device, streamlined data and faster loads, **maintainability** — frontend teams iterating fast with their own BFF. **Challenges realised:** maintaining every BFF, and keeping behaviour consistent across them.

> **The TV BFF's HD-entitlement check is the clearest single argument for BFF over one shared gateway.** The rule is meaningless for phones and browsers. In a shared gateway it would sit behind a device-type conditional, polluting a component every team touches; in the TV BFF it's simply that BFF's business.

Part 8 — BFF vs. API Gateway

	API Gateway	BFF
How many	One shared entry point	One per client type
Optimised for	All clients generically	One specific client
Client-specific logic	Pollutes it	Belongs there
Main risk	Bottleneck / distributed monolith	Duplication and drift
Owned by	Usually a platform team (builds shared infrastructure)	Usually the frontend team

They are not mutually exclusive. A very common production setup is a thin shared gateway doing **TLS termination** (decrypting HTTPS at the edge), coarse authentication and rate limiting, with BFFs behind it: the gateway does what's identical for everyone, the BFFs do what differs.

Think of BFF as API Gateway specialised per client — and note the ownership difference: being owned by the frontend team is a large part of why BFF delivers agility.

Part 9 — Common beginner misconceptions

- **"Each BFF needs its own database."** No. BFFs are generally **stateless** (keep no data between requests); one with a database has usually become a microservice by accident.
- **"BFF means one backend per screen."** No — per **client type**. All mobile screens share the Mobile BFF.
- **"You need BFF as soon as you have a mobile app."** Only when needs genuinely diverge. If mobile and web want nearly the same data, a shared gateway is simpler; adopt BFF when you start adding device-conditionals to shared code.
- **"GraphQL makes BFF obsolete." GraphQL** is an API query language where one endpoint accepts queries naming exactly the fields wanted — a genuine alternative for *shaping* (it solves over-fetching). It doesn't solve client-specific *logic* (the HD check), independent scaling, or ownership. Some teams use one GraphQL layer per client type.

- **"BFFs make the system faster."** Not inherently — they add a hop. They make it faster *for the client* by cutting round trips and payload size, which dominates on mobile.

Part 10 — Check your understanding

1. What is over-fetching, and why does it matter more on mobile than desktop?
2. In the worked example, what is the **only** thing that differs between Web BFF and Mobile BFF?
3. Why is it wrong for a BFF to own a database?
4. What's the strongest argument for the TV BFF over a shared gateway?
5. Name the main cost of BFF and its standard mitigation.
6. When should you *not* use BFF?

(Answers: 1 — downloading more data than needed; mobile has metered data, slow networks, limited CPU/memory; 2 — the transformation step and its return type, both call the same service for the same object; 3 — BFFs are projection layers, not sources of truth — one service owns each piece of data; 4 — the HD-entitlement authorization check is meaningless to other clients and would pollute shared code; 5 — code duplication across BFFs, mitigated by shared libraries and internal services; 6 — when clients' needs are nearly identical, or the team is too small to maintain several backends.)

Part 11 — Interview soundbite (30 seconds)

"BFF, from Sam Newman, gives each client type its own backend instead of sharing one. It solves over-fetching and one-size-fits-all APIs — a mobile app on a metered connection shouldn't receive a desktop payload or need six round trips on a slow network for one screen. Each BFF calls the same microservices but aggregates and transforms for its client: small images and short descriptions for mobile, high-res and full cast for TV. Crucially the BFF is a projection layer, not a source of truth — the microservices still own the data. Netflix does this across phones, TVs and browsers, and their TV BFF adds an HD-entitlement check — client-specific logic that would pollute a shared gateway. It also lets frontend teams own and ship their backend independently and scale per client. Costs: code duplication, more codebases, an extra hop, and keeping behaviour consistent — so shared libraries and consumer-driven contract testing matter. It's essentially API Gateway specialised per client, and the two are often combined."

Part 12 — Implementation

A. Runnable demo

Full source: `demos/BffDemo.java`

```
cd demos && ./gradlew runCh04
```

Three BFFs, one **upstream** service (the one a component calls for data), three projections:

```
// Mobile: trim the copy, shrink the image
new MobileProductDetails(
    p.id(), p.name(), p.price(),
    truncate(p.longDescription(), 120),
    p.hiResImages().get(0).replace("4k", "thumb"));

// Watch: two fields, nothing else
new WatchProductDetails(truncate(p.name(), 20), p.price());
```

B. What you should see

```
=== 1. The over-fetching problem ===
  shared API payload:   3,525 bytes
  >> a watch showing name + price downloads ALL of that

=== 2. Each BFF projects the same object differently ===
  Web BFF:      3,525 bytes  (100% of shared)
  Mobile BFF:     251 bytes  (  7% of shared)
  Watch BFF:       31 bytes  (  0% of shared)
  >> mobile saves ~3274 bytes PER PRODUCT

=== 3. All BFFs call the SAME upstream service ===
  ProductService invocations so far: 4
  >> BFFs are projection layers - one source of truth remains

=== 4. Client-specific logic belongs in its own BFF ===
  certified TV:     Full HD payload ... with 3 4K images
  uncertified TV:   SD-only payload ...
```

Mobile receives 7% of the shared payload — across a 50-item list, the difference between instant and visibly stalling on a train. Section 3 is the conceptual check: every BFF called the *same* `ProductService`; **they add and own no data**.

C. Production — a BFF is just a small service

No special framework — an ordinary application whose job is aggregation and shaping. `CompletableFuture.supplyAsync` runs each call on a background thread (parallel), `.exceptionally(...)` supplies a default on failure, `join()` waits:

```java
@RestController
@RequestMapping("/mobile/v1")
public class MobileBffController {

    private final ProductClient products;
    private final ReviewClient reviews;
    private final InventoryClient inventory;

    @GetMapping("/products/{id}")
    public MobileProductView get(@PathVariable String id) {
        // Fan out in parallel - the BFF exists to save the client round trips
        var pf = CompletableFuture.supplyAsync(() -> products.byId(id));
        var rf = CompletableFuture.supplyAsync(() -> reviews.summaryFor(id))
                    .exceptionally(e -> ReviewSummary.empty());     // degrade, don't fail
        var inf = CompletableFuture.supplyAsync(() -> inventory.stockFor(id))
                    .exceptionally(e -> Stock.unknown());

        CompletableFuture.allOf(pf, rf, inf).join();
        return MobileProductView.from(pf.join(), rf.join(), inf.join());
    }
}
```

GraphQL as an alternative for the *shaping* half — clients request exactly their fields, so one endpoint serves all without over-fetching:

```
query MobileProduct($id: ID!) {
  product(id: $id) { id name price thumbnail }     # mobile asks for four fields
}
```

It does **not** solve client-specific logic, independent scaling, or ownership; a common setup is **one GraphQL BFF per client type** — shaping from GraphQL, isolation from BFF.

Shared code across BFFs — the duplication mitigation (**tracing** = recording a request's path across services):

```
bff-common/          <- auth, error format, tracing, resilience config
  ├─ mobile-bff/
  ├─ web-bff/
  └─ tv-bff/
```

D. Implementation pitfalls

1. Giving a BFF its own database. The most common mistake. BFFs are **stateless projection layers**; **persistent state** means you've created a microservice with duplicated data ownership.

2. Sequential upstream calls. Three sequential server-side calls just move the latency instead of removing it. Fan out with `CompletableFuture.allOf`.

3. No fallback on secondary data. If reviews are down the page should still render. Note `.exceptionally(...)` on reviews and inventory but *not* the product — the primary resource legitimately fails the request; supporting data must not.

4. Copy-pasting auth into every BFF. How three BFFs drift into three subtly different security behaviours. Extract to `bff-common` from the start.

5. One BFF per screen. Per **client type**. Twenty BFFs is twenty pipelines for no benefit.

6. Business rules migrating into the BFF. "Is this order refundable?" belongs in the order service so every client gets the same answer. Only **presentation** decisions belong in a BFF.

7. Versioning by breaking changes. (A **breaking change** — e.g. removing or renaming a field — makes existing clients fail.) Old mobile versions live for years: version the path (`/mobile/v1`, `/mobile/v2`) and keep old versions running — a major reason BFF exists.

Related patterns

- **API Gateway** — the shared-entry-point alternative, and frequent companion
- **Strangler Fig** — a BFF can bridge new frontends to legacy systems mid-migration
- **Service Discovery** — how BFFs locate the services they aggregate
- **Circuit Breaker** — a BFF calling several services needs protection when one fails

Service Discovery Pattern

Category: Communication **One-liner:** A registry that lets services find each other by *name* at runtime, instead of hardcoding network addresses that change constantly.

(A **registry** is a central list of what exists and where; **runtime** = while the program runs; **hardcoding** = writing a value into source code so it can't change without editing and redeploying.)

Part 0 — Concepts you need first

- **IP address and port.** An **IP address** identifies a machine (`10.0.3.47`); a **port** identifies a program on it (`8080`). Together — `10.0.3.47:8080` — they identify one running service instance. To call a service you must know them — and in modern deployments **those numbers change all the time**. That is the whole problem.
- **Instance.** One running copy of a service. Five copies sharing load = five **instances**, each with its own IP. Adding copies rather than enlarging one is **horizontal scaling**.
- **Container.** An app packaged with everything it needs, starting anywhere in seconds; **Kubernetes** runs containers across a **cluster** of machines (§ Kubernetes basics). Containers are **disposable** — started, stopped, moved and replaced constantly, and **each new one typically gets a new IP**. An address looked up ten minutes ago may point at nothing.
- **Auto-scaling.** Automatically adding instances as load rises and removing them as it falls (e.g. **AWS** auto-scaling groups). It saves money — and means instance counts and addresses change all day with nobody touching anything.
- **Load balancing.** Spreading requests across instances so none is overwhelmed (§ Load balancers).
- **DNS.** The **Domain Name System** turns names into IP addresses (`example.com` → an IP). It's the internet's original name lookup, and some discovery systems (notably Kubernetes) extend exactly this idea inside the cluster.

Part 1 — The problem

In a **monolith**, component A calling B is a **method call in the same process** — no network, nothing to discover. In **microservices**, services are **loosely coupled** (each knows little of the others' internals), deployed independently, possibly on different continents; every call is a network call to an address that must be known. Three forces make hardcoded addresses unworkable:

1. **Dynamic infrastructure. Virtual machines** (software-simulated computers sharing a physical server), containers and Kubernetes give services **no fixed IPs or hosts**. An AWS auto-scaling group launches and terminates instances by load, **each time with new IPs**.
2. **Undetermined number of instances.** Which instance should a client call? Hardcode one and all traffic goes there — no load balancing, and if it dies everything fails. Maintaining a list by hand is error-prone and untenable at scale.
3. **Coupling.** If every client must know every service's address you have **tight coupling**: a location change means updating **configuration** in **every client that calls it**. Deployment details leak into clients and the independence you adopted microservices for disappears.

Concrete scenario: an Order Service must call Inventory (availability) then Payment. How does it find them? What happens when Inventory moves server, or extra Payment instances spin up under load — how does Order learn they exist?

Part 2 — The solution

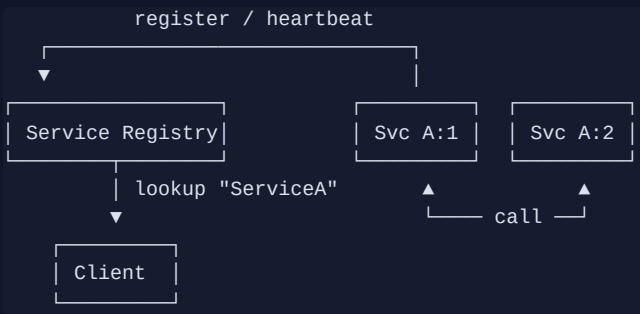
Introduce a **central registry and lookup mechanism**.

- **Dynamic registration** — each instance **registers its host and port with a Service Registry when it starts** and deregisters on shutdown, so the registry always lists active instances.
- **Lookup by logical name** — instances register under a name (`OrderService` , `UserService`); clients ask the registry for `"OrderService"` and get back the current location(s).

> **This indirection is the entire trick.** Addresses can change freely because clients only ever know the *name* — a phone book: you look someone up by name, and it doesn't matter that their number changed.

- **Load balancing and scaling** — discovery combines with a load-balancing strategy; scaling out is just **registering more instances, which clients discover automatically**.
- **Decoupled configuration** — clients hold no endpoint configuration; they supply a name and **routing happens at runtime**, which is what makes continuous deployment and auto-scaling possible without disrupting clients.

Part 3 — Architecture



The Service Registry

A database of all services and their current locations — the **centralized source of truth**. Three responsibilities:

1. **Registration** — on startup a service registers name, IP, port, and optional **metadata** (version, **health endpoint** — a URL answering "am I working?").
2. **Deregistration** — on graceful shutdown, or when the registry detects the instance is gone.
3. **Health checking** — the registry periodically **pings** services or their health endpoints; no response within a timeframe → failed and removed.

> *Why health checking is essential:* deregistration only covers graceful shutdown. An instance that **crashes**, loses power or is **partitioned** off the network (cut off by a network failure) never says goodbye. Without health checks the registry hands out dead addresses forever.

The payoff: a service moves or scales → the registry updates → others still find it; a service dies → removed → callers apply **fallbacks**.

Part 4 — Client-side vs. server-side discovery

Both use a registry; they differ in **who performs the lookup and load balancing** — the single most likely interview question here.

Client-side discovery — "smart client"

1. **Registration** — instances register at startup, perhaps with a **heartbeat** (periodic "still alive" message), and unregister on shutdown.
2. **Lookup** — the client **queries the registry directly** for active instances.
3. **Client-side load balancing** — the client (or a library) picks one, often **round-robin** (take turns) or **consistent hashing** (same key → same instance; adding/removing instances moves few keys — § Consistent hashing).
4. **Invocation** — the client calls **the chosen instance directly**; later requests repeat the lookup or use a **cached** result.

Classic stack: Netflix **Eureka** (open-source registry) + a client load-balancer library — **Ribbon** or **Spring Cloud LoadBalancer**.

Advantages: no extra hop after discovery; **intelligent, application-specific load balancing** (e.g. by request content) because the client sees all instances; **fewer moving parts** (no routing layer besides the registry); slightly lower latency (no intermediate **proxy**).

Disadvantages: every client needs the discovery library; **painful in polyglot environments** (many languages → a compatible client for each, a real barrier); **two network calls per request** (registry, then service — though caching usually removes the first); **couples clients to the discovery mechanism**, so changing the registry means updating many clients.

Standard mitigation: clients **cache** registry responses — no lookup per call, *and* the system keeps working if the registry is briefly down.

Server-side discovery — "smart router, dumb client"

1. **Registration** — identical.
2. **Request to load balancer** — the client calls a **known, fixed endpoint**: load balancer, gateway, DNS name, or **virtual IP** (a stable IP not tied to one machine). **The client never queries the registry.**
3. **Discovery & routing** — the load balancer queries the registry for healthy instances,
4. **picks one** with its own algorithm,
5. **forwards** the request. The client just called one endpoint.

Advantages: much simpler clients (no discovery code or libraries); **language-agnostic** — any client that can make a normal HTTP request works, the decisive advantage in a polyglot estate; **one network call per request**; **abstraction** — locations can change without affecting clients; **often provided out of the box** by Kubernetes and cloud load balancers.

Disadvantages: an extra hop; the **load balancer must be highly available and scalable** (critical to every request); **operational overhead** if your platform doesn't provide one; **the client loses routing control** and must trust the router.

Choosing between them

	Client-side	Server-side
Who queries the registry	The client	The load balancer / gateway
Network hops	Fewer (direct call)	One more (through the router)
Client complexity	Higher — needs a library	Very low
Polyglot friendliness	Poor	Excellent
Routing control	Fine-grained, app-specific	Whatever the router does
Typical stack	Eureka + Ribbon (Spring Cloud)	Kubernetes Services, cloud LBs

Practical guidance:

- In **Spring Cloud / Netflix OSS** (Java microservice libraries / Netflix's open-source tooling), client-side Eureka + Ribbon is straightforward.
- On **Kubernetes**, built-in DNS discovery means you lean server-side — just call `http://my-service`.

- **Not mutually exclusive** — client-side for internal calls plus an API gateway (server-side) for external traffic.

In modern systems — Kubernetes, **service meshes** (a proxy beside every service handling inter-service traffic) — **server-side discovery is the norm**, because the platform hides discovery behind DNS or a gateway.

Part 5 — Inner workings

Self-registration vs. third-party registration

	Self-registration	Third-party registration
Who registers	The service itself, on start/stop	A **service manager** that monitors services and updates the registry
Upside	Simple; service has full control	Burden removed from services
Downside	Service must keep its status current	Another component to manage

Health checks — two models

- **Self-reporting (heartbeat)** — services periodically tell the registry they're alive; no signal within the window → removed.
- **Active monitoring** — the registry pings services; no response → offline.

> Under load: heartbeats put work on the services and scale with instance count; active monitoring puts work on the registry and scales with *its* capacity to poll everyone.

Part 6 — Worked example (simplified Java)

The registry — a phone book

```java
import java.util.concurrent.ConcurrentHashMap;

public class ServiceRegistry {
    private ConcurrentHashMap<String, String> services = new ConcurrentHashMap<>();

    public void register(String serviceName, String serviceAddress) {
        services.put(serviceName, serviceAddress);
    }

    public String discover(String serviceName) {
        return services.get(serviceName);
    }
}
```

Keys are names, values addresses. **Why `ConcurrentHashMap`?** Many **threads** register and look up simultaneously; a plain `HashMap` corrupts under concurrent modification, `ConcurrentHashMap` is built for it.

A service registering itself, and a client discovering

```
public class Service {
    private String name;
    private String address;

    public void register(ServiceRegistry registry) {
        registry.register(name, address);      // self-registration at startup
    }
}

public class ClientService {
    private ServiceRegistry registry;

    public void callService(String serviceName) {
        String serviceAddress = registry.discover(serviceName);

        if (serviceAddress == null) {
            throw new RuntimeException("Service not found: " + serviceName);
        }

        // Make a request to the service
    }
}
```

In a real system registration is automatic — via a service manager or the deployment process.

What this toy version is missing

- **Health checks** to keep registrations current
- **A distributed registry** for traffic and **redundancy**
- **Failure handling in** `callService` — retry, or fall back to a backup service
- **A *list* of instances, not one address** — `Map<String, String>` stores one address per name, so this design **cannot load balance at all**. A real registry stores `Map<String, List<String>>` and returns all healthy instances.

That last gap is the difference between a demo and something usable.

Part 7 — Performance implications and special considerations

1. Latency overhead

Every lookup is an extra step, usually small: client-side discovery **caches** the instance list and refreshes every ~30 s or on a **cache miss**; a lookup is a millisecond-scale local call. To minimise it:

- Cache discovery results briefly rather than querying per request
- Keep the registry fast (in-memory **indexes**)
- Use **asynchronous** updates — pushes or **long-polling** (server holds the request open until there's news)
- In high-performance cases, bake a local endpoint cache into each instance at startup

2. Scalability of the registry

Thousands of registrations plus frequent lookups can make it a **bottleneck**.

- **Clustering** — several registry **nodes** (e.g. three Eureka or **Consul** — HashiCorp's registry/config tool — instances) sharing load; give clients multiple registry URLs
- **Sharding** by service domain — uncommon; one cluster usually suffices for a large organisation
- **Efficient heartbeats** — thousands every 30 s is real load; Eureka lets you tune the interval and has **self-preservation mode** to avoid overwhelming the network during an outage or partition
- **Lightweight protocols** — Consul uses **gossip** (nodes swap news with a few random peers, spreading it like a rumour); **Zookeeper** (Apache's coordination service) uses an event-driven model over

persistent connections

3. Fault tolerance and high availability

If the registry is down, services may not find each other — especially with client-side discovery. (§ Fault tolerance.)

- **Run multiple registry instances** (two or three in production), synced via **consensus** (nodes agree on one value despite failures — Consul, Zookeeper) or **peer-to-peer** copying (Eureka)
- **Client-side caching of last-known-good data** — clients keep using the last good instance list if the registry is unreachable; Spring's Eureka client does this by default. A registry outage then only blocks *new* registrations: **running services keep working** as long as topology hasn't changed drastically. *(The same trick protects config servers — Config Externalization.)*
- **Failover** — give clients several registry addresses so they try the next when one fails
- **Monitor the registry itself**, not just what it tracks
- **Persistence** — Consul, **etcd** (Kubernetes' distributed key-value store) and Zookeeper persist to disk and recover after restart; Eureka keeps state in memory and relies on clients re-registering. Persistence helps if every node reboots, but may be overkill if services re-register on startup

Consistency vs. availability — the CAP trade-off

During a network partition a distributed system must choose **consistency** (never serve wrong data, even if that means serving nothing) or **availability** (always answer, possibly stale) — § CAP.

	Zookeeper / Consul	Eureka
Choice	**CP** — consistency	**AP** — availability
Behaviour	Avoids serving possibly-stale data; may become unavailable until consensus is restored	Continues serving whatever it has, even if stale
On partition	May refuse to answer	**Self-preservation** — won't purge entries, so some info stays available

Know and tune your registry's behaviour — e.g. turn self-preservation **off** in small setups for quicker removal of dead instances.

> *Which is right?* For discovery, **availability usually wins**: a slightly stale list is survivable (try an address, it fails, retry another); a registry that refuses to answer means nothing can talk to anything. That's why Eureka chose AP.

4. Load balancing strategies

- **Round robin** — next instance in turn; even if instances have similar capacity
- **Random** — roughly balances over many requests; trivially simple
- **Least connections / load** — send to the instance with fewest active requests; balances *actual* load but needs tracking, so usually in server-side LBs or meshes
- **Hashing (sticky sessions)** — same client → same instance, for caching or **session affinity** (e.g. hash(user ID) mod instance count)
- **Geographic / AZ affinity** — prefer an instance in the client's **availability zone** (separate data centre within a cloud region) for latency; discovery metadata carries the zone

Ribbon defaults to round-robin; an AWS **ELB** does round-robin or least-connections by configuration.

5. Failover handling

With a discovered pool, a failed call can go to another instance:

- **Client retry** — on timeout or error, pick another instance (from the registry or cache) and retry; Ribbon and Spring Cloud LoadBalancer support this (Retry)
- **Circuit breakers** — **Hystrix** (Netflix, older) or **Resilience4j** (its modern Java replacement) stop calling a repeatedly failing target for a cooldown; with discovery this can be **per-instance**: stop

sending to failing instance X while Y and Z continue (*Circuit Breaker*)

- **Health-based removal** — the registry promptly marks failing instances unhealthy so lookups stop returning them — proactive, better than reacting per request
- **Short timeouts** — **fail fast** and try another instance, viable precisely because you have a pool

6. Consistency of data

The registry lags reality: a crashed instance may be handed out until the next heartbeat period. Mitigations:

- **Shorter heartbeat intervals** — faster detection, more chatter
- **Client pre-flight health check** before critical requests — overhead, rarely done
- **Tolerate the failed attempt and retry another instance** — usually the right answer
- Strongly consistent systems like Zookeeper notify watchers almost immediately on disconnect, but under high **churn** (instances constantly starting and stopping) an AP registry like Eureka may give better overall availability

7. Data volume and registry size

Hundreds of services and thousands of instances make a large dataset: registries **index by service name**; provision enough RAM/CPU. **Tagging and filtering** (e.g. only instances tagged `version: v2`) helps during **rolling deployments** (replacing instances a few at a time).

8. Transactional consistency — subtle but important

If a service registers **before it's ready**, clients call it while it's still warming up — loading caches, opening **connection pools** (reusable database/network connections), **JIT-compiling** hot code.

> **Best practice:** register only once fully initialised, and **deregister immediately when shutting down, before actually going offline.** A very common source of confusing intermittent errors right after deployments.

Part 8 — Security considerations

The registry knows where everything is and steers traffic; compromise it and an attacker can misroute traffic or map your topology. Treat it as **security-sensitive critical infrastructure**.

Threat	Mitigation
Service spoofing — a malicious instance registers as a legitimate service and **intercepts or manipulates data**	Authenticate and authorize registration: Consul **ACLs and tokens** (access-control lists checked against a secret token); Eureka with **HTTP basic auth or mutual TLS** (**mTLS**: both sides present certificates proving identity)
Unauthorized registry access — free querying gives an attacker **a map of all your services**	Internal network only (never expose Eureka publicly); authenticate API and **UI**; Consul ACLs for reads; Kubernetes DNS is cluster-isolated and **RBAC** (role-based access control) limits who reads what
Communication security — registry traffic may cross untrusted networks	**TLS** (the encryption behind HTTPS) on all registry traffic, especially multi-datacenter; with server-side discovery, secure client→gateway and gateway→service too
Least privilege — not every service needs to see every other	Segment the registry or use ACL policies; a "frontend" service sees `accounts` and `orders`, not internals
Audit and monitoring	**Audit trails**; alert on registrations from unknown IPs or spikes in failed auth
DDoS and abuse (floods of traffic from many machines)	**Capacity planning** and **rate limiting** on discovery operations
Server-side components accept client traffic *and* talk to the registry	Harden the gateway / load balancer accordingly

---

Part 9 — Real-world implementations

- **Netflix — Eureka.** Hundreds of services register with Eureka and look each other up, letting Netflix scale and move services across data centres without clients knowing. Proven at streaming scale.
- **Kubernetes — the most widespread use today.** A **Service** is a stable virtual IP and DNS name for a set of **pods** (Kubernetes' smallest unit: one or more containers). As pods come and go Kubernetes updates the endpoints list, and **kube-proxy** (a network component on every node) load-balances Service traffic to current pods. Any pod calls `http://service-x`. Under the hood: an **etcd**-backed registry plus cluster DNS (**CoreDNS**).

  > This is why "we're on Kubernetes" is usually a complete answer to discovery — building a second layer on top is usually a mistake.

- **Airbnb — SmartStack.** Pre-Kubernetes: **HAProxy** (open-source load balancer) on every host plus **ZooKeeper** as registry. Services registered via a local **agent**, and the local HAProxy reconfigured itself — every app got a local load balancer for other services. Built because manually tracking service pools had become untenable.
- **Consul (Stripe, PagerDuty).** DNS-based discovery plus an HTTP API; multi-datacenter support makes it popular where Kubernetes isn't the **orchestrator** or a platform-agnostic solution is needed.
- **Service meshes — Lyft's Envoy. Envoy sidecars** (a high-performance proxy deployed next to each service) **receive updates from a central control plane** about every service's location. Service A calls its local Envoy, which knows all instances of B and routes intelligently. **Discovery leaves application code entirely and moves into infrastructure** *(Sidecar)*.

Use cases

- **Microservices architectures** — inter-service connections become unmanageable at scale without it
- **Cloud-based systems** — services are placed dynamically by demand and cost and re-register wherever they land

Design example 1 — e-commerce

User, Inventory, Shopping Cart, Order. Without discovery, Inventory moving server forces User and Cart to update their connections; with it, Inventory re-registers and the others just query the registry.

Design example 2 — streaming platform

Profiles, video streaming, payments, recommendations. Streaming scales hard at peak, adding and removing instances; without discovery, callers hit instances that no longer exist; with it, instances register/deregister as they come and go and callers always get a valid one.

Part 10 — Common beginner misconceptions

- **"Service discovery replaces load balancing."** They're complementary: discovery says *what instances exist*; load balancing picks *which one*. Client-side puts both in the client, server-side both in the router.
- **"The registry is a single point of failure, so the pattern is bad."** It is a **SPOF**, which is why you cluster it *and* cache on clients. An outage should degrade you (stale data), not stop you.
- **"You need to install Consul or Eureka."** Not on Kubernetes — you already have DNS discovery. Another registry usually creates two competing sources of truth.
- **"Registering at startup is enough."** It must be paired with health checks and prompt deregistration, or dead entries pile up.
- **"DNS alone solves this."** DNS caching and **TTL** (how long a cached answer may be reused) were designed for rarely-changing addresses; clients and language **runtimes** (e.g. the JVM) cache DNS aggressively, sometimes forever. That's why Kubernetes pairs DNS with a stable virtual IP rather than returning raw pod addresses.

Part 11 — Check your understanding

1. Why doesn't a monolith need service discovery?
2. What is the "indirection" at the heart of the pattern?
3. Give the decisive advantage of server-side discovery in a polyglot organisation.
4. Why does Eureka choose availability over consistency?
5. Why must a service register only *after* it's fully initialised?
6. What's the most important gap in the toy Java registry example?

(Answers: 1 — components call each other in-process, there's no network address to find; 2 — clients know logical names, not addresses, so addresses can change freely; 3 — it's language-agnostic, since any client that can make an HTTP request works, with no per-language discovery library; 4 — a stale instance list is survivable via retry, but a registry that won't answer stops everything; 5 — otherwise clients discover and call it while it's still warming up, causing intermittent errors after deploys; 6 — it maps one name to one address, so it can't return multiple instances and therefore can't load balance.)

Part 12 — Interview soundbite (30 seconds)

"Service discovery solves the fact that in a dynamic environment services have no fixed addresses — containers and auto-scaling groups get new IPs constantly, so hardcoding endpoints couples every client to every service's deployment. Instances register with a registry at startup under a logical name and deregister on shutdown, with health checks or heartbeats removing dead ones; clients look up by name. Two flavours. Client-side: the client queries the registry and load-balances itself — Eureka plus Ribbon — no extra hop and smart routing, but a discovery library in every language. Server-side: the client calls a fixed load balancer or gateway that does the lookup — simpler and

language-agnostic, but an extra hop and the LB becomes critical. Kubernetes does server-side for you via CoreDNS and an etcd-backed registry, so on Kubernetes you usually shouldn't add another layer. Key considerations: run the registry as an HA cluster, cache on clients so a registry outage degrades rather than kills you, and the CAP trade-off — Consul and Zookeeper are CP, Eureka is AP and serves stale data rather than going down, usually the right call here. And secure it: anyone who can write to the registry can spoof a service and intercept traffic."

Part 13 — Implementation

A. Runnable demo

Full source: `demos/ServiceDiscoveryDemo.java`

```
cd demos && ./gradlew runCh05
```

The most important line — the fix for the toy example's fatal flaw:

```
// A LIST of instances per name - Map<String,String> could never load balance
private final Map<String, Map<String, Registration>> services = new ConcurrentHashMap<>();
```

Eviction by heartbeat TTL — catches crashed instances that never deregistered:

```
void evictExpired() {
    long now = System.currentTimeMillis();
    services.forEach((name, m) -> m.entrySet().removeIf(e ->
        now - e.getValue().lastHeartbeat > ttlMs));
}
```

And the caching that makes a registry outage survivable:

```
List<Instance> instancesOf(String serviceName) {
    try {
        List<Instance> fresh = registry.discover(serviceName);
        cache.put(serviceName, fresh);
        return fresh;
    } catch (IllegalStateException e) {
        return cache.getOrDefault(serviceName, List.of());   // last known good
    }
}
```

B. What you should see

```
=== 2. Client-side discovery + round robin ===
  call 1 -> 10.0.0.2:8080
  call 2 -> 10.0.0.3:8080
  call 3 -> 10.0.0.1:8080      <- cycles evenly

=== 4. An instance dies (stops heartbeating) ===
    [registry] EVICTED 10.0.0.3:8080 (no heartbeat)
  >> crashed instance removed WITHOUT graceful deregistration

=== 5. The stale-entry window ===
  a client discovering NOW still gets: [..., 10.0.0.4:8080]
  >> unavoidable. The fix is not prevention but RETRY ON ANOTHER INSTANCE

=== 6. Registry outage - cache keeps the system alive ===
    [client] registry DOWN - using cached list (3 instances)
  still routing to 10.0.0.2:8080
  >> running services survive; only NEW registrations are blocked

=== 7. Filtering by metadata (canary routing) ===
  all instances: [10.0.0.2, 10.0.0.1, 10.0.0.4, 10.0.0.5]
  v2.0 only:     [10.0.0.5:8080]
```

Sections 5 and 6 teach the most. The stale-entry window is unavoidable — there's always a gap between death and detection — so the answer is **retry against another instance**, not better detection. Section 6: the registry is down and traffic still flows, thanks to client-side caching.

C. Production — you probably already have this

Kubernetes (do this by default). Discovery is built in:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector:
    app: order-service          # matches pods by label
  ports:
    - port: 80
      targetPort: 8080
```

Any pod calls `http://order-service`; CoreDNS resolves it and kube-proxy balances across healthy pods. **Readiness probes are the health check** — the piece people skip. (A **probe** is a periodic check Kubernetes runs against a container; `/actuator/health` is Spring Boot Actuator's health URL.)

```yaml
readinessProbe:                 # "am I ready to receive traffic?"
  httpGet: { path: /actuator/health/readiness, port: 8080 }
  initialDelaySeconds: 10
  periodSeconds: 5
livenessProbe:                  # "am I alive, or should I be restarted?"
  httpGet: { path: /actuator/health/liveness, port: 8080 }
```

> **Readiness enforces the Part 7.8 rule**: an unready pod is removed from the Service endpoints, so no traffic reaches it while warming up.

Spring Cloud + Eureka (client-side, when not on Kubernetes). A **lease** is a registration that expires unless renewed by heartbeats; `RestTemplate` is Spring's classic HTTP client; `@Bean` has Spring create and manage the object:

```yaml
eureka:
  client:
    serviceUrl.defaultZone: http://eureka-1:8761/eureka,http://eureka-2:8761/eureka  # failover
    registryFetchIntervalSeconds: 30      # cache refresh
  instance:
    leaseRenewalIntervalInSeconds: 10     # heartbeat frequency
    leaseExpirationDurationInSeconds: 30  # TTL before eviction
```

```java
@LoadBalanced                              // makes RestTemplate resolve service names
@Bean RestTemplate restTemplate() { return new RestTemplate(); }

// Call by LOGICAL NAME, not address
restTemplate.getForObject("http://order-service/orders/{id}", Order.class, id);
```

Graceful shutdown — deregister *before* going offline:

```yaml
server.shutdown: graceful
spring.lifecycle.timeout-per-shutdown-phase: 30s
eureka.client.shouldUnregisterOnShutdown: true
```

D. Implementation pitfalls

1. One address per service name. The toy example's flaw: name → single address is a naming service, not discovery — no load balancing, no failover.

2. Registering before you're ready. Register only after caches are warm and pools open, or clients get random-looking errors right after deploys. Gate traffic with a **readiness** probe, not a liveness probe.

3. No client-side caching. The registry becomes a hard dependency of every request — a SPOF that stops everything. With caching, an outage is an inconvenience.

4. Heartbeat TTL too long or too short. 90-second TTL = up to 90 seconds of traffic to a dead instance; too short and you evict healthy instances during a **GC pause** (the JVM briefly halting to reclaim memory) or network blip. 10s heartbeat / 30s expiry is a reasonable start.

5. Not deregistering on shutdown. Every deploy leaves a window of **connection-refused** errors (nothing listening on that port) from already-stopped instances.

6. Adding a registry when you're on Kubernetes. Eureka inside Kubernetes gives two sources of truth that disagree during rollouts. Use the platform.

7. Treating discovery as a substitute for retry. Discovery tells you what *was* alive. Always pair it with retry-on-another-instance *(Retry)*.

8. Leaving the registry unauthenticated. Anyone who can write to it can register a malicious instance under a real service name and intercept traffic. Enable ACLs or mTLS.

Related patterns

- **API Gateway** — typically the server-side discovery router
- **Circuit Breaker** — per-instance breakers over a discovered pool
- **Retry** — retry against a *different* discovered instance
- **Sidecar** — service mesh sidecars (Envoy) absorb discovery entirely
- **Config Externalization** — services need discovery to find the config server too

Circuit Breaker Pattern

Category: Resilience **One-liner:** Wrap calls to a **remote dependency** (another service, database or API your code calls over the network) in a component that stops calling it once it's clearly failing — failing instantly instead of hanging, and probing periodically to see whether it recovered.

Origin: Popularised by **Michael Nygard** in his book *Release It!*

Part 0 — Concepts you need first

What is a thread, and why do we care?

A **thread** is one line of execution inside a program. A **web server** (the program accepting HTTP requests) keeps a **thread pool** — a fixed set of, say, 200 threads — and assigns one per request, returning it to the pool when done.

The critical property: a thread waiting on a network call does nothing else. It's blocked, occupying a slot. With 200 threads all blocked on a slow service, the server accepts no requests at all — for *anything*. It looks dead though its own code is perfect. That one fact is why the Circuit Breaker exists.

What is a timeout?

The maximum wait before giving up. **Timeouts are necessary but not sufficient**: they stop you waiting *forever*, but every call to a dead service still burns its full timeout. 100 requests/s against a dead service with a 30 s timeout = 3,000 threads tied up. Timeouts bound each wait; the breaker prevents the wait altogether. (Timeouts, retries, backoff and jitter in general: § Timeouts and retries.)

What is a "cascading failure"?

One component's failure causing the failure of those depending on it, like dominoes. **The components that fail second were never broken** — they died waiting on something that was.

What is a fallback?

A substitute response when the real one is unavailable — earlier cached data, a sensible default, a degraded feature. It turns a hard failure into a mild one.

What does "fail fast" mean?

Returning an error *immediately* rather than after a long wait. Often better for users: an instant "this feature is unavailable right now" beats two seconds of waiting followed by an error.

Part 1 — The metaphor

A home **electrical circuit breaker trips** on a dangerous overload, cutting the circuit to prevent fire. You later flip it back; if the fault persists it trips again. Two features carry over:

1. **It cuts the connection to prevent wider damage** — lose one circuit, not the house.
2. **Restoration is a deliberate test** — flip it back and see.

A software breaker trips to stop calls to a failing service, so the system **fails fast and degrades gracefully** instead of grinding to a halt.

Part 2 — The problem

Distributed applications constantly call remote things — **microservices**, databases, external **APIs**. Any call can fail or be slow, and if the application keeps calling an unresponsive service, **requests pile**

up, threads hang, and the failure cascades into a wider outage.

Cascading failures, step by step

Service A depends on B. B goes down or — worse — becomes very slow.

1. Calls from A to B **hang until they time out**
2. During each wait A's threads and memory are **tied up doing nothing**
3. This happens across many threads at once
4. A **runs out of threads or memory**
5. **A now fails**

> **A is broken although nothing is wrong with A's own code** — its logic and database are fine. It died of waiting. Preventing that is the entire purpose of the pattern. And it propagates: whatever calls A now waits on A, so one slow service at the bottom can take down everything above it.

> **Slow is worse than down.** If B refuses connections instantly, A gets an immediate error and moves on. If B accepts and takes 30 s, A's threads are held hostage.

Unresponsive services and wasted work

Long timeouts on slow services delay users and block threads. **Blindly retrying a known-failing operation** wastes CPU and **overloads the failing service further** — dozens of components retrying against an already-struggling database only make it worse.

> The vicious circle: slow because overloaded → clients time out and retry → retries add load → slower → more retries. Client traffic alone can stop a service from ever recovering.

When a fault won't resolve quickly (network outage, crashed service), **continuously retrying is pointless** and prevents graceful handling. **What's needed:** detect failure quickly and stop sending traffic *before* it drags down the whole system.

Part 3 — The solution

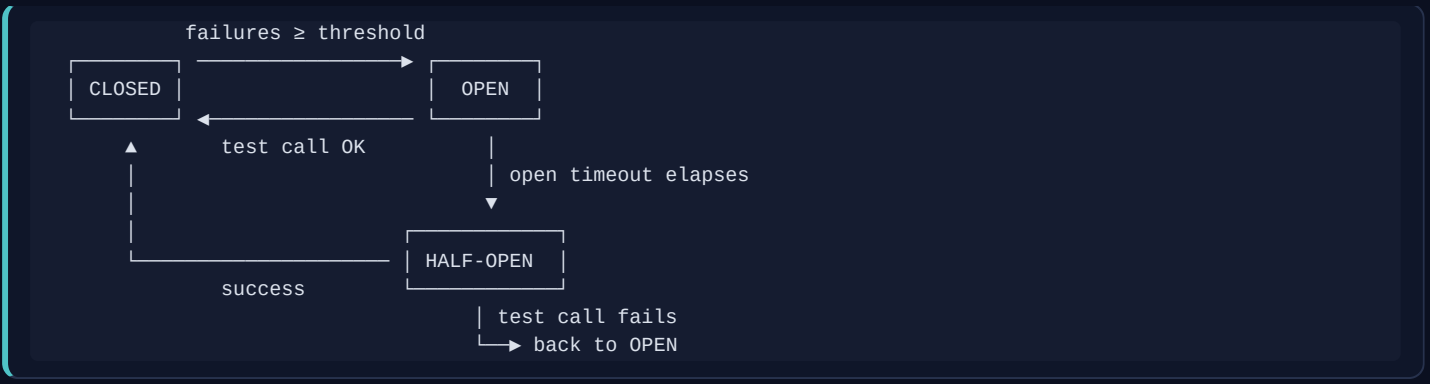
Wrap calls to an external service in a **Circuit Breaker** that monitors their outcomes and stops making them once failures cross a threshold. When it trips, two things happen — the second is easy to overlook:

1. **Your application stops waiting.** Calls are rejected immediately; threads stay free for work that can succeed.
2. **The failing service stops receiving load** — giving it **a chance to actually recover**, breaking the vicious circle.

The pattern prefers **fast failure and graceful degradation** over long waits and resource exhaustion.

Part 4 — The three states

The core of the pattern, and what interviewers ask about.



> *Naming warning:* **CLOSED means traffic flows**, **OPEN means traffic is blocked** — an electrical circuit: a closed loop carries current, an open one is broken.

CLOSED — normal operation

Traffic flows; the breaker **monitors recent failures**. If they exceed a **failure threshold** within a time window, it goes Open. **Successful calls generally reset the failure counter**, so a **transient** error (a glitch that clears itself) doesn't trip it — you want to distinguish "one blip" from "genuinely broken".

OPEN — tripped

Calls are **blocked immediately** — error or fallback **without attempting the real operation** — for a configured **timeout period** (the open interval). Purpose: avoid useless load, and give the service time to recover.

HALF-OPEN — trial mode

After the open timeout, **a limited number of test requests** go through ("Are you OK now?"):

- **Test succeeds** → likely recovered; reset to **Closed** (usually clearing counters)
- **Test fails** → **immediately back to Open**; the timeout restarts

Only a few requests pass in half-open — deliberately, to avoid blasting a recovering service. Often one success closes the circuit; some implementations require several consecutive successes.

> Half-open is the cleverest part: without it you'd need a human to reset the breaker, or a blind reopening of full traffic that instantly re-kills a service just getting back up.

The per-call logic

- **Closed** → attempt the call, record the outcome
- **Open** → skip the call, return failure or fallback immediately
- **Half-Open** → allow the call (perhaps limiting **concurrency** — calls in progress at once), then adjust state by the result

The breaker is a **safety valve**: a failing component can't drag down the rest.

Part 5 — Integration: it's additive, not invasive

A breaker **requires no system overhaul** — it's a protective layer around existing calls, leaving service logic untouched; every request goes *through* it. **Rollout advice:** introduce it gradually, starting with **critical services whose failure would significantly impact the system**, then expand as benefits show.

Part 6 — Configuration

Parameter	What it controls
Failure Threshold	Failed requests before switching Closed → Open
Timeout Period	How long it stays Open before Half-Open — the recovery time given to the service
Number of Requests in Half-Open	How many trial requests test recovery

They must **balance** protecting the service, giving it recovery time, and keeping it available. The right values depend on the nature of the service and system, its expected load, and how critical it is.

These parameters shouldn't be static

They **can and should be adjusted dynamically** with system state. Under high demand you might **increase the failure threshold** (tolerate more failures before tripping) and **reduce the timeout period** (attempt recovery sooner, to serve more requests at peak).

Part 7 — Worked example (Java)

A basic implementation with no libraries, illustrating the core algorithm:

```java
public class CircuitBreaker {
    private enum State { CLOSED, OPEN, HALF_OPEN }

    private State state = State.CLOSED;
    private int failureCount = 0;
    private final int failureThreshold;
    private final long openTimeout;      // how long to stay open before half-open
    private long lastFailureTime = 0;

    public CircuitBreaker(int failureThreshold, long openTimeout) {
        this.failureThreshold = failureThreshold;
        this.openTimeout = openTimeout;
    }

    public synchronized <T> T call(Callable<T> remoteCall) throws Exception {

        // 1. If OPEN, has the timeout passed?
        if (state == State.OPEN) {
            long timeSinceFailure = System.currentTimeMillis() - lastFailureTime;
            if (timeSinceFailure < openTimeout) {
                // Still within the open window — fail fast, don't call the service
                throw new IllegalStateException(
                    "Circuit is OPEN. Failing fast without calling remote service.");
            } else {
                // Timeout has passed — allow one trial call
                state = State.HALF_OPEN;
            }
        }

        // 2. Attempt the call (allowed when CLOSED or HALF_OPEN)
        try {
            T result = remoteCall.call();
            // Success: reset the counter and close the circuit
            failureCount = 0;
            state = State.CLOSED;
            return result;
        } catch (Exception e) {
            // Failure: count it and record when it happened
            failureCount++;
            lastFailureTime = System.currentTimeMillis();
            if (failureCount >= failureThreshold) {
                state = State.OPEN;   // trip
            }
            throw e;   // propagate (or return a fallback here)
        }
    }
}
```

Walking through the mechanics

- **The state check comes first.** Open and within the timeout → throw immediately; **the remote service is never contacted** — no network call, no blocked thread, microseconds instead of seconds.
- `Callable<T>` **makes it generic** — Java's interface for "an operation that returns a value and may throw", so the breaker wraps *any* operation: an HTTP call, a database **query**, anything.
- `synchronized` **handles concurrency** — without **synchronization** (threads taking turns on shared data) two threads could both read `failureCount` as 2 and lose a count. (Also its main performance weakness — below.)
- **Success resets everything** — `state = CLOSED` unconditionally: from Half-Open that's recovery; from Closed it clears accumulated failures.
- **Failure records the time** — `lastFailureTime` is what the Open check measures against.

Using it

```
CircuitBreaker cb = new CircuitBreaker(3, 10000);  // trip after 3 failures, stay open 10s

try {
    String data = cb.call(() -> unreliableService.getData());
    // use data...
} catch (Exception e) {
    useCachedData();   // fallback
}
```

1. `getData()` fails three times → circuit opens
2. Any call in the next 10 s throws **immediately**, without calling `getData()`
3. After 10 s the next call goes through (Half-Open)
4. Success → Closed, normal operation; failure → re-opens for another 10 s

What this example simplifies

- **A naive counter, not a rolling window.** Three failures trip it *whenever* they happen — even spread over an hour of perfect traffic. Real implementations use a **rolling time window** ("more than 50% of calls in the last 10 s failed").
- `synchronized` **serialises every call.** Every request takes the same **lock**, so the breaker becomes a **bottleneck** under load. Production libraries use **atomic operations and non-blocking synchronization** (indivisible steps needing no lock).
- **All exceptions count as failures.** A **validation error** (rejected input) would trip the breaker though the service is healthy. Real breakers distinguish exceptions that mean the *service* is unwell.
- **No fallback built in.** Most frameworks let you register a fallback function directly.

Part 8 — Performance implications

The breaker adds a negligible per-call overhead (check state, update counters). Without it a failing call ties up a thread for **seconds** until timeout; with it, the failure is handled in **milliseconds**. For extremely sensitive paths there are **asynchronous** calls or optimised breaker logic, but the benefit far outweighs the cost.

Tuning the failure threshold

- **Too low** → trips on every minor glitch: needless interruptions and **flapping** (rapid on/off cycling)
- **Too high** → waits too long, failing to protect until many errors have occurred

Tripping after **1** failure is too sensitive; after **100** too late. Pick a threshold that **separates normal transient failures from a real problem**. In high-traffic systems prefer a **percentage-based threshold** over an absolute count, so spikes don't trip it.

Tuning the open timeout

- **Too short** → goes Half-Open **before the service has recovered**, fails again: noisy open → half-open → open **flapping**
- **Too long** → refuses calls after the dependency is healthy again, degrading UX

It should roughly match **how long the service might need to recover**.

Finding the right values

Monitor **how often the breaker opens, how long it stays open, and whether it trips too often**; feed **metrics** (failure counts, open events, half-open trial outcomes) into **dashboards** and adjust.

Part 9 — Best practices and trade-offs

Fallbacks. When open, return cached data, a default, or a simpler alternative rather than just an error — this is what makes the system **degrade gracefully instead of failing completely**. Many frameworks take a fallback function.

Combine with retries and timeouts. The breaker doesn't replace other **resilience** patterns: you still need **timeouts**, and may **retry** a few times on certain failures before counting one failure — retry handles a transient glitch, the breaker stops you after repeated failures that indicate a real outage. How the two interact (and their ordering): Retry Part 4 and pitfall 6. **Bulkheads** and **rate limiting** complement both.

Monitoring and alerts. A tripped breaker usually means a **downstream** service (one you call) is unwell: log/emit an event on every open and close, and alert **on-call** engineers or trigger automated recovery.

> The hidden danger: the system quietly protects itself, users see a degraded feature, and **nobody realises anything is broken** — the breaker suppresses exactly the error flood that would normally raise the alarm.

Thread-safety. Counters and state transitions **must be thread-safe** — two threads mustn't both decide they're "first" to open the circuit. Use atomic counters, synchronized sections, or a library (high-performance ones use non-blocking atomics).

Avoiding misuse. Use breakers on **potentially unreliable or high-latency** calls (network calls); there's **no benefit** on in-memory operations unlikely to fail. Be cautious where failures are very local or self-correcting — **an aggressive breaker constantly tripping on ephemeral (very short-lived) errors does more harm than good.** Tune it to your reliability needs.

Part 10 — Use cases

Microservices architecture

E-commerce with user management, product catalog, cart and order processing; ordering depends on the catalog (availability) and user management (credentials). If the **catalog starts failing**, without a breaker the failures propagate into ordering — degraded UX or total outage. With one, ordering detects it quickly, stops calling, and falls back to a **cached product list** or fails gracefully with a clear message.

External API integration

External APIs are **outside your control**: downtime, latency spikes, rate limits. A weather app pulling from several weather APIs puts a breaker on **each**; a failing or slow one is isolated, and the app **switches to another API** or degrades until it recovers.

Database access

- **Reads:** on rising error rate or latency the breaker trips and **redirects reads to a read replica** (a synced read-only copy) **or a cache**, keeping the service available.
- **Writes:** possible **with more caution** — the breaker could temporarily **buffer writes** (queue them to apply later), but **data consistency is at risk** and the buffer can itself become a bottleneck.

Part 11 — Real-world examples

Netflix and Hystrix. Netflix implemented the pattern in its **open-source** library **Hystrix**: a slow or failing **backend** endpoint is isolated, stopping the cascade and returning fallbacks.

> The canonical illustration: **if the recommendations service is down, Netflix still streams video** — it omits that row or shows a default. A non-essential dependency failing must not take down the essential function.

E-commerce product pages. If the **reviews service** fails, its breaker trips and the page **skips reviews** ("Reviews are unavailable at the moment") while details and pricing load normally — **graceful degradation**. Without it the page might hang or crash fetching reviews. Same for payment gateways and shipping calculators: fall back quickly ("Sorry, we can't process payments right now") rather than leave users waiting.

Cloud and service meshes. Service meshes provide breakers **at the infrastructure level** (Sidecar), and an **API Gateway** can apply breaker rules per backend route — rejecting an erroring backend briefly, protecting it and freeing the gateway for other routes.

Enterprise integration. Any external integration — **SOAP** or **REST** API (SOAP: XML messages with a strict contract; REST: plain HTTP methods on URLs), database, messaging system — benefits. **Middleware** calling a slow **legacy** SOAP service uses a breaker to **avoid tying up all its worker threads**. In **reactive programming** and **stream processing**, breakers support **backpressure** (a slow receiver signalling the sender to slow down): if a **downstream** consumer is slow or failing, the **upstream** producer trips and stops feeding it until it recovers.

Libraries: Hystrix (Java, now in maintenance mode), **Resilience4j** (Java), **Polly** (.NET), plus built-in support in cloud platforms and service meshes.

Part 12 — System design example: distributed social media platform

User Management, **Post Management**, **Feed Generation** and more, each on multiple **nodes** for **high availability** and **load balancing**. Breakers apply:

- **In front of User Management** — Post Management and Feed Generation detect when it struggles and **reduce load** on it by degrading: **cached user information** or **simplified post feeds**.
- **Between Post Management and its database** — on DB problems, serve **cached posts** or **temporarily stop accepting new posts**.
- **In front of the external email service** for notifications — a failing email provider can't affect the rest of the system.

Distributed breaker state

Option: a **shared breaker per service stored in a distributed cache** (e.g. Redis, readable by all nodes), so a trip on one node trips it on all — **balanced against the complexity and latency** of consulting that cache on every call.

> Per-node breakers: each node burns its own failures learning the same lesson. Shared state: one node's discovery protects everyone — but every call now depends on a cache lookup, and the cache is one more thing that can fail.

Monitoring

Monitor all breaker states and alert on trips for quick remediation.

> **The goal is not to eliminate failures** — they're inevitable — but to **manage them**, preventing them from cascading into system-wide outages.

Part 13 — Common beginner misconceptions

- **"Open means working."** The most common mix-up: Open = blocked, Closed = flowing.
- **"The circuit breaker retries for you."** No — it *stops* calls. Retry is a separate pattern; they're used together but do opposite things (retry tries harder, the breaker gives up sooner).
- **"It fixes the broken service."** It protects the *caller* and gives the service breathing room; someone still fixes the real problem — hence alerting.
- **"One breaker per application."** One **per dependency**. Shared, a failure in reviews would block your calls to payments.
- **"If I have timeouts I don't need this."** Timeouts bound each wait; the breaker stops the waiting once the service is known bad. You need both.
- **"It makes my system slower."** The state check costs microseconds; against a 30 s timeout per call to a dead service, it makes failure handling thousands of times faster.

Part 14 — Check your understanding

1. Why does CLOSED mean traffic flows?
2. Why is a slow service more dangerous than a completely dead one?
3. What two separate benefits does an open circuit provide?
4. Why does Half-Open allow only a *few* requests through?
5. What's the hidden risk of a circuit breaker working well?
6. Why does the example's `synchronized` become a problem at scale?

(Answers: 1 — it's an electrical metaphor; a closed circuit is a complete loop that current flows through; 2 — a dead service refuses instantly so callers move on, while a slow one holds caller threads hostage until timeout; 3 — the caller stops wasting resources waiting, and the failing service stops receiving load so it can recover; 4 — to avoid overwhelming a service that has only just started recovering; 5 — it suppresses the errors that would normally alert you, so problems go unnoticed without explicit alerting on breaker state; 6 — every call acquires the same lock, so the breaker itself becomes a bottleneck under high concurrency.)

Part 15 — Interview soundbite (30 seconds)

"A circuit breaker stops you repeatedly calling a dependency that's already failing. The problem is cascading failure: if B hangs, A's threads block waiting on it, and A runs out of threads and dies even though its own code is fine. Slow is worse than down, because a dead service refuses instantly while a slow one holds your threads hostage. The breaker has three states — Closed, where traffic flows and failures are counted; Open, once failures cross a threshold, where calls are rejected instantly with a fallback; and Half-Open, after a timeout, where a few trial requests probe recovery. Success closes it, failure re-opens it, and only a few requests pass so you don't re-kill a service just getting back up. It does two things: your service stops wasting resources, and the failing service stops receiving load so it can recover — breaking the vicious circle where retry traffic prevents recovery. Netflix's Hystrix is the classic: if recommendations is down they still stream video and drop that row. Tuning matters — too sensitive and it flaps, too lenient and it doesn't protect — so use a percentage threshold over a rolling window, not a raw count. And alert on breaker state, because a breaker working well hides the errors that would tell you something's broken."

Part 16 — Implementation

A. Runnable demo — watch the state machine work

(A **state machine** is always in exactly one of a fixed set of states, moving between them on defined events — exactly what the breaker is.)

Full source: `demos/chapters/06-circuit-breaker/CircuitBreakerDemo.java`

```
cd demos && ./gradlew runCh06
```

The demo's breaker is the Part 7 class with two changes: `call(action, fallback)` takes a **fallback** that is returned both when the circuit rejects a call and when a call fails, and **a failed half-open probe re-opens immediately** (`state == HALF_OPEN || failureCount >= threshold`). It also logs each transition. A `FlakyService` (thread-safe `AtomicInteger` call counter, `volatile` health flag visible to all threads) is healthy, then broken, then healed; `main` drives four phases with a `"payments"` breaker of 3 failures / 2 s open:

```
synchronized <T> T call(Callable<T> action, Callable<T> fallback) throws Exception {
    if (state == State.OPEN) {
        if (System.currentTimeMillis() - lastFailureTime < openTimeoutMs) {
            log("REJECTED (circuit OPEN) -> fallback");
            return fallback.call();          // fail fast, no remote call
        }
        transitionTo(State.HALF_OPEN);       // timeout elapsed: allow a probe
    }
    try {
        T result = action.call();
        failureCount = 0;
        transitionTo(State.CLOSED);
        return result;
    } catch (Exception e) {
        failureCount++;
        lastFailureTime = System.currentTimeMillis();
        if (state == State.HALF_OPEN || failureCount >= failureThreshold) {
            transitionTo(State.OPEN);
        }
        return fallback.call();
    }
}
```

B. What you should see

```
--- Phase 1: service healthy ---
  result = real-data          (x3, circuit stays CLOSED)

--- Phase 2: service breaks ---
    [payments] call failed (1/3): service unavailable
    [payments] call failed (2/3): service unavailable
    [payments] call failed (3/3): service unavailable
    [payments] state CLOSED -> OPEN
    [payments] REJECTED (circuit OPEN) -> fallback
  >> service was actually called 6 times so far

--- Phase 3: still broken, circuit stays open ---
  >> extra real calls during OPEN: 0

--- Phase 4: wait for timeout, service recovers ---
    [payments] state OPEN -> HALF_OPEN
    [payments] probe SUCCEEDED
    [payments] state HALF_OPEN -> CLOSED
  result = real-data
```

The line that proves the pattern is `extra real calls during OPEN: 0` **.** Three requests came in; the failing service received none — the caller didn't wait, and the sick service was left alone.

C. Production — Resilience4j

Nobody hand-writes breakers in production. **Resilience4j** is the current Java standard (Hystrix is in maintenance mode). This is the **one place the book shows its setup**; the Bulkhead and Retry chapters reuse it and only show their own config block.

Dependency (Maven): Spring Boot is the framework for building Java services; **AOP** (aspect-oriented programming) is what lets an annotation like `@CircuitBreaker` wrap a method with extra behaviour, so it must be on the classpath:

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.2.0</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Configuration (`application.yml`) — where the real learning is (`400` = HTTP "bad request"):

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        # Use a percentage over a rolling window, not a raw count
        slidingWindowType: COUNT_BASED
        slidingWindowSize: 20              # consider the last 20 calls
        minimumNumberOfCalls: 10           # don't judge until we have 10 samples
        failureRateThreshold: 50           # open if >50% of them failed
        waitDurationInOpenState: 10s       # how long to stay OPEN
        permittedNumberOfCallsInHalfOpenState: 3
        automaticTransitionFromOpenToHalfOpenEnabled: true
        # Treat slow calls as failures too - crucial, see pitfall 3
        slowCallRateThreshold: 50
        slowCallDurationThreshold: 2s
        recordExceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
        ignoreExceptions:
          - com.example.ValidationException   # never trip on a 400
```

Usage:

```
@Service
public class PaymentClient {

    @CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
    public PaymentResult charge(Order order) {
        return restClient.post()
                .uri("/payments")
                .body(order)
                .retrieve()
                .body(PaymentResult.class);
    }

    // Signature must match, plus a trailing Throwable
    private PaymentResult paymentFallback(Order order, Throwable t) {
        log.warn("Payment circuit open for order {}: {}", order.getId(), t.toString());
        return PaymentResult.deferred(order.getId());   // queue for later
    }
}
```

Monitoring — expose breaker state so you can alert on it:

```
management:
  endpoints.web.exposure.include: health,circuitbreakers,prometheus
  health.circuitbreakers.enabled: true
```

Then alert on the **Prometheus** (open-source metrics collection and alerting) metric `resilience4j_circuitbreaker_state`.

D. Implementation pitfalls

1. Raw failure count instead of a rate over a window. Three failures scattered over an hour shouldn't trip anything. Use `slidingWindowSize` + `failureRateThreshold`, and `minimumNumberOfCalls` so a single early failure (100% rate) can't open the circuit.

2. Counting business errors as failures. A 400 or validation exception means the *request* was bad, not the *service*. Put them in `ignoreExceptions` or a client bug takes down a healthy dependency.

3. Forgetting `slowCallDurationThreshold`. A service returning `200 OK` after 30 s never "fails", so a failure-only breaker never opens — while your threads are consumed as badly as if it were down. Slow-call detection catches the most dangerous case.

4. No timeout on the underlying call. The breaker only reacts to calls that *return*. Set connect and read timeouts on the HTTP client, or a hung **socket** blocks forever and the breaker never counts a failure.

5. One breaker shared across dependencies. Name breakers per downstream service; shared, a reviews failure blocks payments.

6. Fallbacks that call other services. The fallback runs when things are already broken: keep it local and fast — cache, default, or queue. A networked fallback is a second failure waiting to happen.

7. Not alerting on state. The one people skip: a breaker doing its job hides errors. Alert on the open transition or you'll hear from a customer.

Related patterns

- **Bulkhead** — isolates resource pools; the breaker limits *how long* you try, the bulkhead limits *how much* damage one failure can do
- **Retry** — handles transient faults; the breaker stops retries becoming an attack on a downed service
- **Service Discovery** — per-instance breakers across a discovered pool
- **API Gateway** — a natural place to configure breaker rules per backend route
- **Sidecar** — service meshes provide breakers at the infrastructure level, with no application code

Bulkhead Pattern

Category: Resilience **One-liner:** Partition a system into isolated compartments with their own dedicated resources, so a failure or overload in one cannot consume the resources everything else needs.

Part 0 — Concepts you need first

What is a thread pool?

A fixed collection of worker **threads** kept ready: work arrives, a free thread takes it, then returns to the pool (introduced in Circuit Breaker Part 0). Pools are fixed-size because threads are expensive — each needs ~1 MB of **stack** (memory for its method calls and locals), and with thousands the **operating system** spends more time switching between them than working.

The consequence that drives this chapter: the pool is a *shared, finite resource*. If something exhausts it, **every** piece of work in that program stops — not just the work that caused the problem.

What does "resource exhaustion" mean?

Using up all of a limited resource so nothing else gets any:

- **Threads** — all busy, so no new work can start
- **Memory** — allocated, so new allocations fail
- **Database connections** — **connection pools** (reused open connections) are also fixed-size and exhaustible
- **Network bandwidth / file handles / CPU** — a **file handle** is the OS's ticket for one open file or connection, and there's a limit

What is a "noisy neighbour"?

A component consuming disproportionate shared resources and degrading everything sharing them. The term comes from cloud hosting (one tenant's heavy usage slowing others on the same machine); in your app, one slow **dependency** can be a noisy neighbour to every other feature.

Why does one slow dependency block unrelated work?

The crux. Your service has **one pool of 200 threads** and calls Inventory (usually busy) and Shipping (usually quiet). Inventory slows to 30 s per call instead of 50 ms:

1. Requests needing Inventory each take a thread
2. Each blocks for 30 s
3. At any moderate rate, all 200 threads end up blocked on Inventory
4. A request needing only Shipping arrives — **no thread is available**

Shipping is perfectly healthy. Its requests still fail, because Inventory ate all the threads. That's what this pattern prevents. (It's the same thread-starvation mechanism behind cascading failure.)

Part 1 — Where the name comes from

A ship's hull is divided into **watertight compartments** — bulkheads. A breach floods **one compartment**, the ship stays afloat, and there's time for rescue. Without them one breach floods everything: a **cascading failure in a physical system**.

In **distributed systems**, a bulkhead **isolates parts of the system so that if one part fails, it doesn't cause the rest to fail.**

Part 2 — The problem: failure propagation

Services fail — bugs, resource exhaustion, network issues — and a failure **ripples** to everything depending on it.

The e-commerce walkthrough

A **microservices** e-commerce platform with user management, inventory, payments:

1. **Inventory** fails due to a database **outage**
2. **User service**, which shows product availability, **also starts failing**
3. **Payment service**, which checks inventory before charging, **likewise fails**
4. Soon **the entire platform is unavailable** — from one service's failure. A classic cascading failure.

Part 3 — The solution: isolation

The essence is **isolation**: partition components into units that operate **independently**, so if one fails the others continue. Bulkheads are **failure containment** — they limit the **scope** of a failure, so unaffected parts keep serving and **availability** is preserved as far as possible.

Applied: with inventory in its own bulkhead, user and payment services keep offering limited functionality — browsing products, viewing past orders — while inventory is down.

The half people miss

By stopping the inventory failure from **consuming system-wide resources** (CPU, memory), the pattern keeps those resources **available to the remaining services**.

> Isolating *resources* matters as much as isolating *code*. Two services can be perfectly separate in the **source tree** and still kill each other through a shared thread pool. The bulkhead is about the resources, not the modules.

Not a magic solution

A ship with a big enough breach may sink anyway; a total network outage may still take a system down. Bulkheads **buy time and maintain some level of service** — significant mitigation, not prevention.

Part 4 — Architecture

The architecture revolves around **isolation** and **resources**.

What is a bulkhead, concretely?

A **partition**: a separate **service**, a group of related **processes**, a set of **threads** (a thread pool — the most common implementation), or a dedicated **CPU or memory area**. The requirement: each operates **independently**.

What are the resources?

Anything the bulkheads need — CPU time, memory, bandwidth, database connections. Each bulkhead gets a **dedicated share**, so one failure can't consume everything.

Step 1 — Divide the system into bulkheads

Isolate services **prone to failure**, services doing **resource-intensive tasks**, or group by **function** or by **type of users served**. In e-commerce: separate bulkheads for user management, inventory, payments. Aim: if one fails **or becomes a resource hog**, the others aren't directly affected.

Step 2 — Allocate resources

Proportional to needs and importance: more for **payment processing** (financial transactions) than **user management** (profiles), based on expected load, performance requirements, and the consequences of failure. This is the critical mechanism: a failing bulkhead that over-consumes still **can't take the other bulkheads' allocations**.

Step 3 — Manage interactions between bulkheads

- **Keep interactions to a minimum** — the more one bulkhead depends on others, the likelier failure spreads.
- **When necessary, isolate them** — e.g. via a **message queue** or an **API gateway**, so an inventory failure doesn't directly hit user management.

> Why a queue helps: with a direct call the caller's thread waits while the callee struggles. With a queue the caller drops a message and moves on; the failure becomes "messages accumulate" instead of "caller's threads are all blocked". (See Event-Driven Architecture.)

Part 5 — Inner workings

Isolation in practice

Each bulkhead might run in its **own process or container**, have its **own database and cache**, and use its **own thread pool**. But:

> Isolation is **not just technical** — it needs **organisational commitment**. Changes to one bulkhead shouldn't require changes to others; interdependencies must be minimised and managed.

Perfect technical isolation can be destroyed in six months by teams adding convenient direct dependencies. It must be defended, not just built.

Resource allocation — fairness and efficiency

Aim for **fairness** (each bulkhead gets what it needs) and **efficiency** (nothing wasted): **resource quotas or limits** per bulkhead, fair scheduling, and **dynamic adjustment** — e.g. temporarily lending an idle bulkhead's resources to one under a surge.

Failure detection and mitigation

The system must **detect, isolate and mitigate** a bulkhead failure: **continuous monitoring** of each bulkhead's health; **alerts** plus **automatic recovery**; **throttling** (deliberately slowing) or **restarting** a bulkhead that over-consumes; **rerouting traffic** or **bringing up new instances** if it stays unresponsive.

It's not a silver bullet

It can't prevent failures or guarantee 100% **uptime**. Complement it with **redundancy, replication, load balancing, and caching** (§ Fault tolerance).

Part 6 — Worked example: thread-pool bulkheads in Java

The scenario

An **Order Service** calls **Inventory** (often high traffic) and **Shipping** (usually less load). **Without bulkheads**, an Inventory spike can clog Order Service and stop it completing Shipping work.

Creating separate thread pools

Java's `ExecutorService` (the standard interface for running tasks on a thread pool):

```
ExecutorService inventoryExecutor = Executors.newFixedThreadPool(100);
ExecutorService shippingExecutor  = Executors.newFixedThreadPool(50);
```

`newFixedThreadPool(100)` = exactly 100 threads; extra tasks queue until one frees. Sizes follow expected load — Inventory gets 100, Shipping 50. **These two pools are the bulkheads**: threads in one can't be consumed by work for the other.

Using them

```
public void createOrder(Order order) {
    inventoryExecutor.submit(() -> {
        inventoryService.checkAndUpdateInventory(order);
    });

    shippingExecutor.submit(() -> {
        shippingService.scheduleShipping(order);
    });
}
```

The bulkhead in action

1. Requests start slowing the **Inventory Service**
2. Threads in `inventoryExecutor` get stuck — potentially all 100
3. **The 50 threads in `shippingExecutor` are unaffected** — separate pool
4. `createOrder` **keeps scheduling shipping** while inventory checks are delayed

Versus the single-pool version: all 200 threads blocked on Inventory and shipping failing for no reason of its own. **The only change is drawing from two buckets instead of one.**

What the example deliberately leaves out

- **Error handling** — no response, or an **exception**?
- **Timeouts** — a stuck thread needs a bound *(pair with Circuit Breaker)*
- **Pool sizing** — how to choose the numbers (Part 14D)
- `submit()` returns a `Future` (holder of the task's result or error) that's ignored here, so **failures are silently swallowed** (pitfall 3).

Part 7 — Performance implications

Benefits

- **Efficient resource utilization** — work continues while one service is sluggish; no whole-system freeze from one faulty component.
- **Load balancing** — separate pools distribute load, tunable per task or service.
- **Response time** — a delay in one part doesn't hold up others, which are served swiftly.

Considerations

Determining bulkhead sizes — a real challenge, often **trial and error**. Starting point: size each pool to the **concurrency** the dependency can actually handle. If Inventory degrades past 100 concurrent requests, a larger pool just queues work inside *it* instead of inside you — worse, because you lose visibility.

Monitoring and adjusting — as load evolves, revisit sizes; tools like **VisualVM** (shows a running JVM's threads, memory, CPU) reveal thread usage and **bottlenecks**.

Error handling — requests can still fail or stall when a service is down; handle with a **timeout** or a Circuit Breaker.

Scalability concerns:

> The Bulkhead pattern improves **resilience** but **does not directly address scalability**. Under significant load growth, creating more threads causes **excessive context switching** (the CPU repeatedly saving one thread's state and loading another's — pure overhead) and *decreases* performance. For scale, use horizontal scaling or **asynchronous** processing.

⚠️ **The point beginners get wrong most often.** Bulkheads bound damage; they don't add capacity. Bigger pools as a load fix make things slower.

Part 8 — Use cases

- **E-commerce** — search, cart and ordering are interconnected but independent. An overwhelmed **search service** (traffic spike or bug) would otherwise slow the whole platform — **even cart and ordering**.
- **Online gaming** — each game a separate service; **player load in one game shouldn't affect others**.
- **Financial systems** — account management, transaction processing and reporting can function independently.

> **A delay generating reports shouldn't slow transaction processing** — the cleanest illustration: reporting is heavy and slow, transactions are **latency**-critical, and there's no reason the former should interfere with the latter.

Part 9 — System design example: music streaming (Spotify-like)

Services: streaming, user management, playlist management, recommendations.

Bulkheading services (coarse-grained)

- **User management isolated from streaming** — a surge of new sign-ups doesn't affect playback; **users keep listening**.
- **Playlist management isolated from recommendations** — slow, analysis-heavy recommendations don't stop users managing playlists.

Bulkheading at a lower level (fine-grained)

Within one service: put **fetching a song** (from the song database) and **streaming the song** to the user in **separate thread pools**, so a slow song database **doesn't interrupt streaming of songs already fetched**.

> The most useful takeaway: bulkheads apply *between* services **and** *within* one. The fine-grained version often saves the experience — music that keeps playing while search is broken.

Part 10 — Bulkhead vs. Circuit Breaker

Complementary, not alternatives:

	Bulkhead	Circuit Breaker
Mechanism	Partitions resources	Stops calls after repeated failures
Protects against	Resource exhaustion spreading	Wasted calls to a known-dead service
When it acts	Always — it's structural	On detecting failures — it's reactive
Analogy	Watertight compartments	A fuse that trips
Limits	*How much* damage one failure can do	*How long* you keep trying

Use both. The bulkhead caps the **blast radius** (how much of the system one failure can damage); the breaker stops you feeding the fire. Inventory can only ever consume its own 100 threads, and a breaker on Inventory stops those threads being wasted once it's clearly down.

Part 11 — Common beginner misconceptions

- **"Bulkhead means microservices."** You can bulkhead inside a **monolith** by giving different **workloads** different pools — often the highest-value place to start.
- **"Bigger pools are safer."** Past a point the opposite: threads cost memory and context switches. Match the pool to what the dependency can handle.
- **"It's the same as rate limiting." Rate limiting** caps how many requests you *accept* per time window; bulkheading caps how many resources a category of work can *consume*.
- **"Isolation is a one-time build."** It erodes as teams add cross-dependencies; it needs ongoing defence — an organisational commitment.
- **"Bulkheads help with load."** They help with *failure*. For load, scale horizontally.

Part 12 — Check your understanding

1. Why can a slow Inventory Service cause Shipping requests to fail, when Shipping is healthy?
2. What are the two things a bulkhead isolates, and which do people forget?
3. Why does a message queue between bulkheads help contain failure?
4. Why doesn't the Bulkhead pattern help with scalability?
5. Give an example of fine-grained bulkheading inside a single service.
6. How does Bulkhead differ from Circuit Breaker?

(Answers: 1 — they share one thread pool, and Inventory's blocked threads consume all of it; 2 — code and resources, and people forget resources; 3 — the caller drops a message and moves on instead of holding a thread waiting for a struggling callee; 4 — more threads cause excessive context switching and degrade performance, so capacity comes from horizontal scaling; 5 — separate thread pools for fetching songs versus streaming already-fetched songs; 6 — bulkhead limits how much damage a failure can do and is always active, breaker limits how long you keep trying and activates on detected failures.)

Part 13 — Interview soundbite (30 seconds)

"Bulkhead comes from ship design — watertight compartments so one breach doesn't sink the ship. In software you partition resources so a failing dependency can't consume everything. Classic case: an Order Service calls Inventory and Shipping from one shared thread pool; Inventory gets slow, every thread ends up blocked on it, and Shipping calls fail though Shipping is healthy. The fix is separate pools sized per dependency, so Inventory's problems are bounded to its own hundred threads. It applies between services or inside one — e.g. separating song fetching from streaming so a slow database doesn't interrupt playback. It pairs with circuit breakers: bulkhead caps how

much damage a failure can do, the breaker caps how long you keep trying. Gotchas: pool sizing is trial and error and needs monitoring — start from Little's Law — and it improves resilience, not scalability: more threads for more load just means context switching, so capacity comes from horizontal scaling."

Part 14 — Implementation

A. Runnable demo — see thread starvation, then see it fixed

(**Thread starvation**: work can't run because every thread is taken.)

Full source: `demos/chapters/07-bulkhead/BulkheadDemo.java`

```
cd demos && ./gradlew runCh07
```

The program runs the *same* workload twice. Inventory is slow (`Thread.sleep(2000)`), Shipping fast (50 ms) and healthy; 20 inventory tasks are submitted, then 5 shipping tasks, and it counts shipping completions (with a thread-safe `AtomicInteger`) within 3 s. The only difference between the runs is where the tasks go:

```
// WITHOUT bulkheads: one shared pool of 10
ExecutorService shared = Executors.newFixedThreadPool(POOL);        // POOL = 10
shared.submit(() -> { slowInventory(); return null; });              // x20
shared.submit(() -> { fastShipping(); shippingDone.incrementAndGet(); return null; });  // x5

// WITH bulkheads: separate pools
ExecutorService inventoryPool = Executors.newFixedThreadPool(POOL);
ExecutorService shippingPool  = Executors.newFixedThreadPool(5);
```

B. What you should see

```
=== WITHOUT bulkheads (shared pool of 10) ===
  shipping completed in 3s: 0/5   (finished=false, 3001ms)

=== WITH bulkheads (separate pools) ===
  shipping completed in 3s: 5/5   (3000ms)
```

The whole pattern in two lines. Same workload, same total resources. In the first run twenty slow inventory tasks occupy all ten threads, and five trivial shipping tasks get **zero**. In the second, shipping has its own pool and finishes immediately — while inventory is just as slow. Nothing was made faster; something was made *isolated*.

C. Production — Resilience4j Bulkhead

(Library setup — Maven dependency, Spring AOP — is in Circuit Breaker 16C.) Two flavours, and choosing correctly matters:

Type	Mechanism	Use when
`SemaphoreBulkhead`	Caps **concurrent calls** on the caller's own thread (a **semaphore** is a counter of permits)	Default choice — cheap, no extra threads
`ThreadPoolBulkhead`	Runs calls on a **separate thread pool**	You need true thread isolation, or async execution

```yaml
resilience4j:
  bulkhead:                          # semaphore-based
    instances:
      inventoryService:
        maxConcurrentCalls: 25       # at most 25 in flight
        maxWaitDuration: 100ms       # then reject rather than queue forever
      shippingService:
        maxConcurrentCalls: 10
        maxWaitDuration: 100ms

  thread-pool-bulkhead:              # thread-pool-based
    instances:
      reportGeneration:
        maxThreadPoolSize: 4
        coreThreadPoolSize: 2
        queueCapacity: 20
```

Usage (to **shed load** = reject excess work quickly instead of queuing it; `sku` = stock-keeping unit, a product ID):

```java
@Bulkhead(name = "inventoryService", fallbackMethod = "inventoryFallback")
public Inventory check(String sku) {
    return inventoryClient.get(sku);
}

private Inventory inventoryFallback(String sku, BulkheadFullException e) {
    // Bulkhead full — shed load rather than queue
    return Inventory.unknown(sku);
}
```

Kubernetes-level bulkheads — the same idea for whole services. A **pod**'s `requests` is what it's guaranteed, `limits` the most it may use (`500m` = half a CPU core; `Mi` / `Gi` = mebibytes/gibibytes):

```yaml
resources:
  requests: { cpu: "500m", memory: "512Mi" }
  limits:   { cpu: "1000m", memory: "1Gi" }   # this pod cannot starve its neighbours
```

Database connection pools are bulkheads too — often the most valuable one (**HikariCP** is Spring Boot's default pool):

```yaml
spring.datasource.hikari:
  maximum-pool-size: 20
  connection-timeout: 3000     # fail fast when the pool is exhausted
```

D. How to size the pools

Start from **Little's Law** (items in progress = arrival rate × time each spends — § Little's law):

```
concurrency = throughput × latency
```

A dependency serving **50 requests/second** at **200ms** average latency has `50 × 0.2 = 10` calls in flight at **steady state**. Size the bulkhead a bit above — say 15 — to absorb bursts.

Key insight: never size it larger than the dependency can actually handle. If Inventory degrades beyond 100 concurrent requests, a bulkhead of 500 just moves the queue from your process into theirs, where you can't see or shed it. Then measure and adjust; a short `maxWaitDuration` (~100ms) means you **shed load** rather than accumulate it.

E. Implementation pitfalls

1. Pools sized by guesswork. Start from Little's Law, then tune with real metrics. "200 because that's the default" protects nothing.

2. Unbounded queues. `Executors.newFixedThreadPool` uses an **unbounded** `LinkedBlockingQueue`; under sustained overload it accumulates tasks until `OutOfMemoryError` — you traded thread exhaustion for memory

exhaustion. Use a bounded queue and an explicit **rejection policy**:

```
new ThreadPoolExecutor(10, 10, 0L, TimeUnit.MILLISECONDS,
    new ArrayBlockingQueue<>(50),                    // bounded
    new ThreadPoolExecutor.AbortPolicy());           // reject when full
```

3. Ignoring the `Future`. `submit()` swallows exceptions into a `Future` nobody reads. Use `execute()` with an `UncaughtExceptionHandler` (called when a thread dies from an unhandled error), or check the result.

4. Forgetting the database connection pool. Teams bulkhead HTTP calls and leave one shared 20-connection Hikari pool where one slow query starves every **endpoint**. The connection pool is usually the *scarcest* resource you own.

5. Bulkheading without timeouts. A bounded pool of threads blocked forever is still dead. The bulkhead limits *how many* can be stuck; only a timeout limits *how long*.

6. Too many small pools. ~1 MB of stack per idle thread: twenty pools of fifty threads ≈ a thousand threads and ~1 GB before serving a request. Bulkhead by *dependency*, not endpoint.

7. Treating it as a scaling tool. Repeated because it's the most common error (Part 7): past the CPU's capacity, more threads mean *less* throughput.

Related patterns

- **Circuit Breaker** — the natural companion; stop calling what has already failed
- **Retry** — retries inside a bounded bulkhead can't exhaust the whole system
- **Sidecar** — service meshes can enforce bulkheads at the infrastructure level
- **Event-Driven Architecture** — message queues between components are a natural isolation boundary

Retry Pattern

Category: Resilience **One-liner:** Automatically re-attempt operations that failed for temporary reasons, spacing attempts out with backoff and randomness, so brief glitches don't become user-visible errors.

> This chapter is the book's home for **idempotency keys**, **jitter / the thundering herd**, and the **"retry at one layer only"** rule; other chapters link here.

Part 0 — Concepts you need first

What is a "transient" failure?

Transient = temporary; it clears without anyone fixing anything. **Permanent** = fails identically however often you try. The single most important idea here: **retrying a transient failure helps; retrying a permanent one is pure waste.**

Transient (retry helps)	Permanent (retry is useless)
Network dropped a packet	The password is wrong
Server briefly out of connections	The request body is malformed
Momentary CPU spike	You lack permission
Load balancer restarted an instance	The record doesn't exist

(A **packet** is one chunk of network data; the **request body** is the data sent with a request, e.g. a JSON order.)

What is "backoff"?

Waiting between attempts instead of retrying instantly. **Exponential backoff** makes each wait longer, typically doubling: 1s, 2s, 4s, 8s. Why wait? Retrying instantly adds **load** to a struggling service at the worst moment and gives it no time to recover.

What is "jitter"?

Deliberate randomness in the wait: instead of every client waiting exactly 4 s, each waits a random time around 4 s. It solves the thundering herd.

What is the "thundering herd" problem?

A service has a two-second blip; **ten thousand clients** fail and all start a 4 s backoff at almost the same instant. Four seconds later **all retry simultaneously**, the just-recovered service is hit by a synchronised wall of traffic and falls over again; they back off 8 s and it repeats, harder. The clients have accidentally become a **denial-of-service attack**. **Jitter breaks the synchronisation** by spreading retries across a window. (The same effect on caches: § Cache stampedes.)

What does "idempotent" mean?

An operation is **idempotent** if performing it many times has the same effect as once: `set balance to 100` is; `add 100 to balance` is not (five times adds 500). It matters because a retry might be the *second* execution of something that already succeeded. (§ Delivery guarantees and idempotency.)

Part 1 — The problem

In a distributed microservices architecture **failures are inevitable**: unreliable networks, transient server **outages**, overloaded load balancers, bugs, operator mistakes. You can't build a system that never fails, so you design for **resilience** — a small hiccup mustn't snowball into an outage.

Transient failures specifically

Short-lived errors that resolve themselves given a little time — a network glitch, momentary loss of connectivity, temporary unavailability.

The classic example: a database connection is refused at the peak of a spike in **concurrent** users; **a few seconds later, after some connections close, a retry goes straight through.** Nothing was fixed; the condition passed. Other common cases: network **timeouts**, transient **HTTP 5xx** errors (500–599, "the server failed"; 4xx means "the request was wrong"), temporary resource exhaustion.

Why this needs solving

Without a strategy, services **propagate errors to end-users or upstream systems** (callers waiting on you) **unnecessarily** — a tiny blip triggers a user-facing error or manual intervention, though **the system might have recovered milliseconds later**.

Part 2 — The solution

If an operation fails for a temporary reason, **wait briefly and try again**, assuming the issue may have cleared. Distributed systems often fail **partially** (some requests fail, others succeed) or briefly, so the same request often simply succeeds second time — and behind a load balancer the retry **may hit a less-busy instance** entirely. Retries **mask sporadic failures**, raising the reliability users perceive.

Why it works

Retries assume the failure was transient or **non-deterministic** (not guaranteed to recur). Network blips clear, **threads** free up, services restart — so a later attempt has a good chance where the first failed. This sharply cuts the error rate seen higher up, and a service can complete the **downstream** call **without any manual fix**.

Part 3 — Transient vs. permanent: the critical distinction

Retry only errors **likely to be temporary**.

Worth retrying: network timeouts · HTTP 5xx · 502 Bad Gateway from a load balancer · database **deadlocks** (two transactions each waiting for the other's lock; the database kills one) · connection refused · **throttling / rate-limit** errors (the service deliberately refusing because you send too much)

Not worth retrying: HTTP 400 (validation failure) · HTTP 401 (**authentication** failure) · **business logic** errors (e.g. insufficient funds) · configuration errors · fatal **exceptions**

> **No amount of retrying fixes a bad request or invalid credentials** — those are permanent *for that request*, and retrying just repeats the failure and adds load.

So a good retry mechanism **checks the error or exception type**, retrying only transient conditions and letting permanent ones **fail fast**.

Retry vs. fail-fast

The alternative is **fail-fast**, typically via a Circuit Breaker:

	Retry	Fail-fast (Circuit Breaker)
Behaviour	Keeps trying, hoping for eventual success	Stops trying after detecting a pattern of failures
Favours	Eventual success	Conserving resources
Cost	Extra wait time and work	Gives up on things that might have worked
Right for	Transient faults	Long-lasting or permanent faults

For a service down for hours, blind retrying wastes time and resources; breakers or **graceful degradation** (defaults or **cached** data) fit better. **Retries for hiccups; fall back or fail fast for lasting errors** — complements, not competitors.

Part 4 — Architecture

The pattern is a **wrapper around an operation** that intercepts failures and repeats per a policy.

Retry policy and parameters

Defines **when and how**: **maximum attempts**, **which exceptions/error codes are retryable**, and the **delay strategy** — from *"3 times, 2 s apart"* to *"5 times, exponentially increasing delays with randomness, never on certain error types"*.

Retry loop (the invoker)

Calls the operation, catches failures, checks the policy, waits, calls again — until success or max attempts. If it never succeeds it **propagates the final failure** to its caller, often after logging it.

Backoff strategy

Avoid a **tight loop** (repeating as fast as the CPU allows):

Strategy	Behaviour	When
Constant delay	Always wait the same, e.g. 5s	Simple cases; predictable recovery time
Exponential backoff	Each wait longer than the last: 1s, 2s, 4s	The general default
Capped exponential backoff	As above with a maximum	Prevents absurdly long waits

Exponential backoff **eases load on struggling services** and raises the odds the issue has cleared by the next attempt. The cap matters: five doublings from 1 s is a 32 s wait; eight is over four minutes.

Jitter (randomization)

Clients that fail together **retry in unison** after the same interval — the thundering herd, **a second failure spike**. Jitter randomises each delay (e.g. somewhere between 2 and 4 s instead of exactly 4), **spreading bursts** so downstream load stays even. Variants — **full jitter**, **equal jitter**, **decorrelated jitter** — share one idea: retries **de-synchronized across clients**.

> Adding randomness on purpose seems odd, but it measurably reduces **contention** (many clients competing for one resource) and speeds recovery.

Circuit breaker integration

Retries handle transient failures; breakers handle persistent ones by cutting off calls after a threshold. A breaker **prevents a retry loop from endlessly hitting a downed service**: after, say, 50 straight failures it opens and calls fail immediately; after a **cooldown**, **half-open** lets a test call

through and success closes it. **The balance:** allow ~3 quick retries per request, but if many requests in a row fail — a bigger problem — open the circuit for a while. (Ordering the two in Resilience4j: pitfall 6.)

Idempotency and side effects

If an operation has **side effects** — **charging a card, sending an email, incrementing a counter** — a retry can perform it **multiple times** unless you design against it.

Retried operations should ideally be **idempotent**. Many **APIs** already are — deleting a resource, a `GET` (HTTP read). For non-idempotent actions (creating an order, taking a payment) use **idempotency keys**: a unique ID the client attaches to an operation and reuses on every retry, so the server can recognise a duplicate and not process it twice (implementation: Part 14D).

> The failure mode to picture: the payment succeeds on the server, but the response is lost. The client sees a timeout — indistinguishable from "it never happened" — and retries. **Without an idempotency key, the customer is charged twice.** The client cannot tell "lost request" from "lost response", which is why the **server** must **deduplicate**.

Placement in the architecture

- **Client side of a service-to-service call** — A's client library or **HTTP client** retries; B stays **stateless** and unaware, seeing only repeated requests.
- **API gateway or service mesh** — infrastructure retries automatically.

⚠️ **The key rule: never duplicate retries at multiple layers.**

> If every hop retries, **one failure multiplies into a storm**: three layers × three attempts = **27** requests from one original. **Centralize retries in one layer** — the edge service, or the immediate caller — disabling them in lower-level clients, or coordinate with a **token bucket** limiting overall retry rate (tokens refill at a fixed rate and each retry spends one, so retries can never exceed it).

Part 5 — Worked example (Java)

```java
public class RetryUtil {

    public static <T> T executeWithRetry(Callable<T> task, int maxAttempts,
            long baseDelayMillis, double backoffFactor, boolean useJitter) throws Exception {

        int attempt = 1;
        Exception lastException = null;
        Random random = new Random();

        while (attempt <= maxAttempts) {
            try {
                return task.call();                 // success — return immediately
            } catch (Exception e) {
                lastException = e;                  // remember it for the final throw

                if (attempt == maxAttempts) {
                    break;                          // no retries left
                }

                // Work out how long to wait
                long delay = baseDelayMillis;
                if (backoffFactor > 1.0) {
                    // exponential: base * factor^(attempt-1)
                    delay = (long) (baseDelayMillis * Math.pow(backoffFactor, attempt - 1));
                }
                if (useJitter) {
                    // "full jitter": random value between 0 and the computed delay
                    delay = (long) (random.nextDouble() * delay);
                }
                // (a maximum cap could be applied here)

                try {
                    Thread.sleep(delay);
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();   // restore the interrupt flag
                    break;                                // stop retrying
                }

                attempt++;
            }
        }
        throw lastException;    // all attempts exhausted
    }
}
```

The parameters

Parameter	Meaning
`maxAttempts`	Total tries **including the initial attempt**. `3` means 1 try + 2 retries
`baseDelayMillis`	Fixed delay if constant; the initial delay if exponential
`backoffFactor`	The multiplier. `2.0` with base 100ms gives 100ms, 200ms, 400ms. Set `1.0` for a constant interval
`useJitter`	Applies full jitter — a random value between 0 and the computed delay

Reading the logic

- `Callable<T>` (an operation that returns a value and may throw) makes it reusable for any network call or query.
- **Success returns immediately** — no further attempts or delay.
- **The exception is saved, not swallowed** — `lastException` is rethrown at the end so the caller learns *why*.

- **The attempt check precedes the delay** — no sleeping before giving up.
- `Math.pow(backoffFactor, attempt - 1)` — attempt 1 waits `base × 1`, attempt 2 `base × factor`, and so on.
- **The `InterruptedException` handling is correct and worth copying.** When a thread is **interrupted** (asked to stop), catching the exception clears the flag; `Thread.currentThread().interrupt()` restores it so code higher up still knows. Forgetting this is a classic Java bug that stops applications shutting down.

Using it

```
String result = RetryUtil.executeWithRetry(
    () -> fetchData(url),
    3,        // up to 3 attempts
    1000,     // 1 second base delay
    2.0,      // exponential backoff, doubling
    true      // use jitter
);
```

Up to 2 retries — about 1 s before the first, about 2 s before the second, randomised — throwing after roughly 3 s of total waiting. **Often one retry is all it takes** to ride out a glitch.

Design choices, and one to change

- **Generic `Callable<T>`** — reusable across operation types.
- **Configurable backoff and jitter** — constant (`1.0`, `false`), exponential (`2.0`, `false`), or exponential + jitter (`2.0`, `true`).
- **`maxAttempts` prevents infinite loops** that would endlessly hammer a failing service.
- **Change before production:** it **catches broad `Exception`**, so *any* failure is retried — including validation errors. Catch **only specific transient exceptions** (`IOException`, timeouts) and let others bubble up; **your error filtering belongs here**.

Jitter refinement: full jitter (`random × delay`) can yield a near-zero wait, partly defeating backoff. Many implementations randomise between **50% and 100%** of the delay ("equal jitter") to keep a minimum gap.

Worth adding: a log line and **metric** per retry, and a check that aborts retrying if a higher-level circuit is already open.

Part 6 — Performance implications and risks

Increased load on dependencies

Every retry is **another request** — negligible when failures are rare. **But** if a service is failing *because it's overloaded*, retries worsen it; many clients retrying a struggling service create a **retry storm that amplifies load exponentially**. Hence **backoff**, **limited attempts**, and breakers or throttling layered on top.

Increased latency for end users

Retries lengthen response time. Sometimes acceptable — **a slightly slower success beats an immediate error** — but overdone, and chained across services, they degrade the experience. Countermeasure: **an overall timeout for the operation** — e.g. succeed **within 5 s including all retries** or not at all — to keep a responsive **SLA** (service-level agreement: promised speed or uptime). Otherwise retries silently blow your **latency budget** (the maximum time a request may take).

Wasted work and resource consumption

A retry that ultimately fails wasted CPU, **bandwidth** and memory; if the issue isn't transient, *all* retries are waste. Detect non-transient failures quickly — "permission denied" or "invalid request" won't improve — with a **blacklist of non-retryable errors**. Also: **if the first attempt already took**

nearly the full client timeout, another attempt will likely time out too; propagate the error instead of tying up threads.

Cascading effects in complex systems

A → B → C, each with its own policy: C fails, B retries, *and* A retries its call to B — multiplying load on B and C. Decide **which layer retries** and let others fail fast; commonly the edge/client-facing service retries and intermediate services propagate errors. Monitor for excessive retries — they often signal a downstream **incident**.

Retry storms and thundering herd

When a popular service glitches, many services retry at once. Backoff and jitter avoid synchronisation — **jitter specifically tackles the thundering herd**. Also: **retry budgets** (a cap on retries per second) and **adaptive algorithms** that lengthen backoff while errors continue.

Idempotency revisited

Duplicate side effects cost performance too — the same email sent twice uses twice the resource. **Never retry non-idempotent operations blindly** without deduplication keys or **check-pointing** (recording progress so a repeat skips completed steps).

The mitigation checklist

- **Exponential backoff** — don't hammer
- **Jitter** — avoid synchronised retries
- **Limit attempts**, and total time spent retrying
- **Circuit breakers** / fail-fast where failure is likely persistent
- **Distinguish error types** — retry only transient ones
- **Monitor and tune** — frequent retries signal a struggling dependency or a bad policy
- **Ensure idempotency**, or server-side deduplication for critical actions
- **Test under failure** — simulate a dependency down or slow and verify retries improve resilience rather than overwhelm the system

---

Part 7 — Use cases

- **Web services** — the internet is unreliable and the **cost asymmetry** favours retrying: a failed request is costly (a user-facing error or expensive fallback), a retry is cheap (one more request).
- **Database operations** — network issues, concurrency conflicts, temporary unavailability. Clearest case: you save a record while **another user is updating it**; retry after a short delay lets the conflicting **transaction** finish and yours succeed.
- **Microservice architectures** — retrying gives a failing dependency **a chance to recover**, so a hiccup doesn't become a full outage.

Part 8 — Real-world examples

Cloud SDKs (AWS, Google Cloud). (An **SDK** is a platform's official client library.) AWS SDKs automatically retry transient 5xx and throttling errors with **exponential backoff and jitter**; Google Cloud libraries retry idempotent requests (a timed-out storage upload retried behind the scenes). **Developers benefit without even realising it.**

Netflix. Internal libraries retry calls to services or **data stores** when a request fails or a **host** is unresponsive — **retrying on another instance**, perhaps after a brief backoff, rather than erroring. With circuit breakers, sporadic failures don't disrupt viewing.

Payment processing (Stripe, PayPal). Financial APIs hit transient failures talking to banking networks, so they retry behind the scenes — and **encourage idempotency keys** on requests, so a client can safely

retry with the same key and the server **won't duplicate the charge**. The API **explicitly assumes clients will retry**: a glitch between **merchant** and **gateway** (the payment provider's service) doesn't lose the order or double-charge.

Message queues and event processing. In **RabbitMQ** or **Kafka** (message brokers), retry means **requeueing** or **dead-letter queues** (parking messages that keep failing). A **consumer** that hits a temporary error doesn't **acknowledge** the message, so it returns to the queue for a later attempt — retry in an **asynchronous** context. E.g. reserving inventory fails on a **lock timeout** (waited too long for another transaction's row lock); retried a minute later, the lock may be free. Many e-commerce platforms rely on this in **event-driven architectures** for **eventually consistent** outcomes (Event-Driven).

Part 9 — System design examples

E-commerce checkout. The user clicks Buy, but the **payment gateway** (the third-party service charging cards) **is temporarily unavailable**. Rather than reporting failure, retry and handle the glitch gracefully — and this is exactly where idempotency keys are non-negotiable (the double-charge scenario).

IoT data collection. IoT (Internet of Things) devices send readings periodically, but the **database is temporarily unavailable** (network glitch, scheduled maintenance). Rather than discard data, retry to **ensure nothing is lost**.

Part 10 — When NOT to use retry

- The error is clearly **non-transient** (configuration error, fatal exception) — retrying only delays the inevitable
- The service is **known to be down for an extended period** (planned outage) — use a breaker or fallback
- **Strict latency requirements** — **real-time** systems may prefer failing fast to violating their SLA
- The operation is **extremely expensive**, or has **side effects that can't safely repeat**

Alternatives: **manual compensation** (undoing effects), **eventual reconciliation** (comparing records later and fixing mismatches) for async processes, or **alerting a human**.

Part 11 — Common beginner misconceptions

- **"More retries means more reliability."** Only up to a point; after a few attempts you mostly add latency and load. If three with backoff failed, the fourth probably will too.
- **"Retry and circuit breaker are alternatives."** Complements doing opposite things: retry handles the blip, the breaker the outage.
- **"Jitter is a micro-optimisation."** At ten thousand clients it's the difference between graceful recovery and a self-inflicted outage.
- **"My operation is a GET, so idempotency doesn't matter."** Check the whole path: a GET that triggers a counter increment or **audit write** isn't as idempotent as it looks.
- **"Retries are free if they usually succeed."** They're free when failures are rare — and most expensive exactly when the system is struggling, which is when they fire most.
- **"I'll retry at every layer to be safe."** The most damaging mistake here: layered retries multiply into an exponential storm.

Part 12 — Check your understanding

1. Why is retrying an HTTP 401 pointless?
2. Explain the thundering herd, and how jitter fixes it.

3. Why can't a client distinguish "lost request" from "lost response," and what follows from that?
4. Why must you set an overall operation timeout alongside a retry policy?
5. If A retries B and B retries C, how many requests can one failure become at 3 attempts each?
6. What's the one line in the example code you should change before production use?

(Answers: 1 — authentication failure is permanent for that request; no number of attempts fixes wrong credentials; 2 — many clients fail simultaneously, back off by the same amount, and retry in unison, re-overloading the service; jitter randomises delays so retries spread out; 3 — both look like a timeout, so the server must deduplicate using an idempotency key rather than the client guessing; 4 — otherwise cumulative retry delays silently exceed your latency budget; 5 — nine, since each of A's 3 attempts triggers 3 of B's; 6 — catching broad `Exception`, which retries permanent errors too.)

Part 13 — Interview soundbite (30 seconds)

"Retry handles transient failures — network blips, timeouts, throttling, connection refused during a spike — things that fix themselves in seconds. You wrap the call in a policy: max attempts, which errors are retryable, and a delay strategy. Two details matter most: exponential backoff, so you're not hammering a service that may be failing precisely because it's overloaded; and jitter, because otherwise every client that failed together retries together and the thundering herd causes a second outage. You need idempotency: the client can't distinguish a lost request from a lost response, so retrying a payment without an idempotency key double-charges — which is why Stripe's API is built around them and assumes clients retry. Distinguish transient from permanent: retrying a 400 or 401 is waste. The biggest architectural mistake is retrying at every layer — if A retries B and B retries C, one failure becomes nine requests — so pick one layer and let the rest fail fast. Pair it with circuit breakers — retry for hiccups, the breaker for outages — and set an overall operation timeout so retries can't blow the latency budget."

Part 14 — Implementation

A. Runnable demo — backoff, jitter, and the thundering herd

Full source: **demos/chapters/08-retry/RetryDemo.java**

```
cd demos && ./gradlew runCh08
```

The demo's `executeWithRetry` is Part 5's utility with the three production fixes applied — a separate catch for permanent errors, a **cap**, and **equal jitter** (50–100% of the delay):

```
} catch (PermanentException e) {
    System.out.println("      permanent error, NOT retrying: " + e.getMessage());
    throw e;                                  // fail fast
} catch (Exception e) {
    last = e;
    if (attempt == maxAttempts) break;

    long delay = (long) (baseDelayMs * Math.pow(factor, attempt - 1));
    delay = Math.min(delay, 5000);            // cap
    if (useJitter) {
        // "equal jitter": 50-100% of the computed delay
        delay = delay / 2 + (long) (random.nextDouble() * delay / 2);
    }
    ...
```

`main` runs: a `FlakyService` that throws `TransientException("timeout")` twice then succeeds (5 attempts, base 100 ms, factor 2); a `PermanentException("HTTP 401")`; the same backoff schedule against a permanent

`503` with and without jitter; and a herd simulation that computes 1,000 clients' delay (base 1,000 ms, optionally jittered) and reports the busiest 100 ms bucket.

B. What you should see

```
=== 1. Transient failure, recovers on 3rd attempt ===
      attempt 1 failed (timeout), waiting 100ms
      attempt 2 failed (timeout), waiting 200ms
      succeeded on attempt 3
      result: OK after 3 calls

=== 2. Permanent failure — should NOT retry ===
      permanent error, NOT retrying: HTTP 401
      gave up immediately (correct)

=== 3. Backoff schedule, no jitter ===
      attempt 1 failed (503), waiting 100ms
      attempt 2 failed (503), waiting 200ms
      attempt 3 failed (503), waiting 400ms
      attempt 4 failed (503), waiting 800ms      <- doubling

=== 4. Same schedule WITH jitter ===
      attempt 1 failed (503), waiting 74ms
      attempt 2 failed (503), waiting 133ms
      attempt 3 failed (503), waiting 291ms      <- randomised
      attempt 4 failed (503), waiting 512ms

=== 5. Thundering herd: 1000 clients retry at once ===
      jitter=false -> busiest 100ms bucket holds 1000 of 1000 clients
      jitter=true  -> busiest 100ms bucket holds ~205 of 1000 clients
```

Section 5 is the one to internalise. Without jitter **all 1000 clients retry in the same 100ms window** — a synchronised wall hitting a service that just recovered. With jitter the worst bucket holds about a fifth. Same number of retries, spread over time instead of stacked.

C. Production — Resilience4j Retry

(Library setup: Circuit Breaker 16C.)

```yaml
resilience4j:
  retry:
    instances:
      inventoryService:
        maxAttempts: 3                  # 1 initial + 2 retries
        waitDuration: 200ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
        enableRandomizedWait: true      # <- jitter
        randomizedWaitFactor: 0.5
        retryExceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
          - org.springframework.web.client.HttpServerErrorException   # 5xx
        ignoreExceptions:
          - org.springframework.web.client.HttpClientErrorException   # 4xx — never retry
```

```java
@Retry(name = "inventoryService", fallbackMethod = "inventoryFallback")
@CircuitBreaker(name = "inventoryService")     // order matters — see pitfall 6
public Inventory check(String sku) {
    return inventoryClient.get(sku);
}
```

Spring's own `@Retryable` is an alternative without Resilience4j:

```java
@Retryable(
    retryFor = { IOException.class },
    noRetryFor = { ValidationException.class },
    maxAttempts = 3,
    backoff = @Backoff(delay = 200, multiplier = 2, random = true))
public Inventory check(String sku) { ... }

@Recover
public Inventory recover(IOException e, String sku) { return Inventory.unknown(sku); }
```

D. Implementing idempotency keys

The most important production concern, usually missing from tutorials — and the book's reference implementation (Saga steps and event consumers apply the same idea with a saga ID or event ID). The client sends a stable key — usually a **UUID** (random, practically unique 128-bit ID) in an HTTP **header**; the server guarantees one execution per key.

Client:

```java
String idempotencyKey = UUID.randomUUID().toString();   // generated ONCE, before the first attempt

executeWithRetry(() ->
    http.post("/payments")
        .header("Idempotency-Key", idempotencyKey)      // same key on EVERY retry
        .body(payment)
        .execute(),
    3, 200, 2.0, true);
```

> **Critical detail:** generate the key **outside** the retry loop. Generated inside, each attempt looks brand-new and the customer is charged three times — the very bug the mechanism exists to prevent.

Server:

```java
@PostMapping("/payments")
public ResponseEntity<PaymentResult> pay(
        @RequestHeader("Idempotency-Key") String key,
        @RequestBody Payment payment) {

    // Atomic "insert if absent" — the DB unique constraint does the work
    Optional<PaymentResult> existing = idempotencyStore.find(key);
    if (existing.isPresent()) {
        return ResponseEntity.ok(existing.get());     // replay the original response
    }

    PaymentResult result = paymentService.charge(payment);
    idempotencyStore.save(key, result, Duration.ofHours(24));
    return ResponseEntity.ok(result);
}
```

Store keys in **Redis** with a **TTL** (expiry), or a table with a **unique index** on the key (the database rejects a second row with the same value). The **unique constraint** is what makes it safe under concurrent duplicate requests.

E. Implementation pitfalls

1. Retrying non-idempotent operations without a key. The headline risk: double charges, duplicate orders, duplicate emails.

2. Generating the idempotency key inside the retry loop. Silently defeats the mechanism (above).

3. Retrying 4xx errors. A 400 or 401 fails identically every time. Always configure `ignoreExceptions`.

4. No cap on exponential backoff. Base 1s × 2 over 10 attempts ends at a **17-minute** wait. Always set a maximum.

5. No overall deadline (one limit for the whole operation, all attempts included). A 5s timeout × 3 attempts plus backoff is potentially 20 seconds. Budget the whole operation:

```
resilience4j.timelimiter.instances.inventoryService.timeoutDuration: 3s
```

6. Wrong ordering of retry and circuit breaker. Resilience4j's default is **Retry(CircuitBreaker(call))** — the breaker is *inside*, so it sees each attempt. Usually what you want. Inverted, the breaker counts a whole exhausted retry sequence as one failure and opens far too slowly.

7. Retrying at every layer. HTTP client 3× × service 3× × gateway 3× = **27** requests to an already-struggling service. Pick one layer and explicitly disable the rest — most HTTP clients retry by default.

8. Retrying immediately with no delay. A tight retry loop against a struggling service is indistinguishable from an attack.

9. Not logging retries. Invisible retries make a dependency degrading from 0.1% to 15% failure look healthy until it tips over. Emit a metric per retry.

Related patterns

- **Circuit Breaker** — the essential companion; stops retries becoming an attack on a downed service
- **Bulkhead** — bounds the resources retries can consume
- **Service Discovery** — retry against a *different* discovered instance
- **Saga** — compensation for when retrying isn't safe or sufficient
- **Event-Driven Architecture** — requeueing and dead-letter queues are retry in an async context

Sidecar Pattern

Category: Cross-cutting concerns **One-liner:** Attach a helper process alongside each service instance to handle concerns like logging, metrics, security, and networking — so the service itself contains only business logic.

Part 0 — Concepts you need first

What is a "cross-cutting concern"?

Functionality **many parts of a system** need that isn't any part's core purpose. An Order Service exists for orders — its **business logic** — but it also must log, emit **metrics** (request counts, response times), **authenticate** callers, **encrypt** traffic, retry failed calls and report **traces** (a request's path through every service). **Every service needs those**; none is about orders.

Both obvious solutions are bad:

- **Copy the code into every service** → duplication; every change touches everything
- **A shared library** every service imports → better, but see the polyglot problem below

What is a process?

A running program with its own memory. Two processes are genuinely separate — one crashing doesn't corrupt the other.

What is a container?

An application packaged with everything it needs (libraries, **configuration**), running isolated from other containers on the same machine. **Docker** is the common tool for building them.

What is a Kubernetes Pod?

The smallest deployable unit in **Kubernetes** (§ Kubernetes basics). Crucially, **a Pod can contain more than one container**, and containers in one Pod:

- Are always **scheduled onto the same machine** together
- Share a **network namespace** (one isolated set of addresses and ports) — they talk over `localhost`
- Can share **storage volumes** (mounted directories)
- Are **created and destroyed together**

This is what makes the Sidecar pattern practical. Without co-scheduling and shared networking, a "helper alongside the service" would just be another remote service.

What is a proxy?

Software that relays traffic between two parties. A **transparent** proxy does so without the application knowing — exactly how production sidecars work.

What is a service mesh?

An infrastructure layer handling all service-to-service communication: **a sidecar proxy next to every service**, plus a central **control plane** that pushes settings to all of them. **Istio** (using **Envoy**, a high-performance open-source proxy) is the best-known example. (§ Service mesh.)

The polyglot problem — why shared libraries aren't enough

Polyglot = many programming languages. You write an excellent library for retries, metrics and **mTLS** (mutual TLS: both sides present certificates proving identity, traffic encrypted). Your services are in Java, Go, Python and Node — so you maintain **four versions**. They drift: a bug fixed in Java lingers in Python for months, and a fifth language needs a fifth implementation before that team can ship.

A sidecar solves this because it's a separate process speaking network protocols — it doesn't care what language the service beside it uses.

Part 1 — The metaphor and the core idea

A **motorcycle sidecar** is attached to the motorcycle and shares the journey, yet remains a distinct unit (the course also offers Batman and Robin, or Holmes and Watson — an aide handling specific tasks autonomously within one larger entity).

The pattern applies **modularity and separation of concerns** (each piece responsible for one job): deploy components as **separate processes running alongside the main application**, each sidecar dedicated to one function. That **encapsulates the logic and complexity** of those functions in the sidecar, making each component easier to develop, manage and scale independently — and makes it possible to **use different technologies for different tasks within the same application**.

⚠️ Part 2 — One correction to the course material

The course says the sidecar and main application "run in the same process" and are "deployed together in the same container." **That is not how it works in practice:**

- The sidecar runs as a **separate process, in its own container**
- Both containers are **co-located in the same Kubernetes Pod** (or equivalent scheduling unit)
- They **share the Pod's network namespace** (hence `localhost`) and can share volumes
- They **share a lifecycle** — created, scheduled and destroyed together

Same-process or same-container would defeat the isolation that gives the pattern its value: a sidecar crash would take down the app, and you couldn't use a different language for each. The course's other framing — **same lifespan, shares resources, communicates locally** — is correct.

Part 3 — The problem: monolithic application management

Problem	What it means
Lack of isolation	In a **monolith** all components are intertwined, so **one failing component can bring down the whole application** — all eggs in one basket; no **fault isolation**
Scalability — moving the mountain	**You can't scale one component; you scale the entire application**, wasting resources
Complexity — a tangled web	As features grow, the codebase becomes hard to understand, manage or modify; maintenance slows development
Technology constraint — one size does not fit all	You're **stuck with the stack chosen at the start**; adopting new technology is a mammoth task

---

Part 4 — The solution

Pair each service with a **sidecar** that takes over some of its responsibilities; the two run side by side and communicate directly, forming a self-contained module.

- **Flexibility** — change one function by **updating the relevant sidecar, leaving the rest untouched** — no full **redeploy**.
- **Enhanced scalability** — each sidecar scales independently: surge in logins → scale the **authentication sidecar**; heavy processing → the **data-processing sidecar**.
- **Fault isolation** — a failure in one sidecar doesn't affect the others.

- **Reduced complexity** — self-contained modules are easier to understand and maintain.
- **Technology freedom** — **different technologies for different sidecars**, rather than lock-in.

Part 5 — Architecture

Co-located deployment

The application and its sidecar are deployed **together**, sharing resources and communicating efficiently — **operating as a single unit while keeping their individuality**.

Multiple sidecars

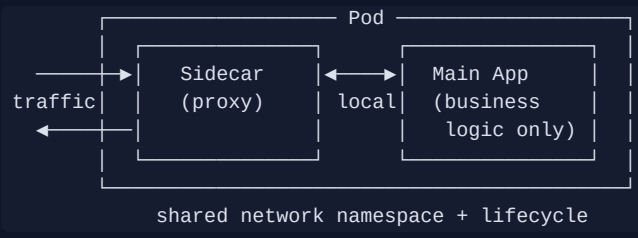
Not restricted to one: **several sidecars, each for a specific function** (logging, metrics, proxying…), modularising the application.

The network navigator — the sidecar as proxy

The most important role, and the one that makes service meshes work: the sidecar **intercepts all incoming and outgoing network traffic** of the application, acting as a **proxy**, so it can do:

- **Load balancing** (§ Load balancers)
- **Circuit breaking** *(Circuit Breaker)*
- **Security** — including mutual TLS
- **Service discovery** *(Service Discovery)*

...leaving the application **free to focus on its primary service**.



Why "intercepts" is the key word. The application doesn't call the sidecar or know it exists; it makes a normal HTTP call and traffic is transparently routed through the sidecar. That's how you add mTLS, retries and **tracing without changing a line of application code** — even code you can't easily modify.

Shared lifespan

Sidecar and app are **created, deployed and destroyed together**, so the app **always has its sidecar beside it**. That guarantee matters: if the sidecar could be scheduled elsewhere or restart independently, the app would have to handle its absence — reintroducing the complexity you removed.

Part 6 — Worked example

The main application — a Spring Boot REST service

(**Spring Boot**: Java framework for building web services; a **REST** service answers plain HTTP requests on URLs such as `/api`.)

```java
@SpringBootApplication
@RestController
public class MainApplication {

    public static void main(String[] args) {
        SpringApplication.run(MainApplication.class, args);
    }

    @GetMapping("/api")
    public String getData() {
        return "Hello, Sidecar!";
    }
}
```

The sidecar — a logging helper

```java
@SpringBootApplication
@RestController
public class LoggingSidecar {

    public static void main(String[] args) {
        SpringApplication.run(LoggingSidecar.class, args);
    }

    @GetMapping("/log")
    public String logRequest() {
        System.out.println("Request received at /api endpoint");
        return "Logged!";
    }
}
```

It prints a log line whenever `/log` is hit. It's Java here, but that's incidental — in practice it usually wouldn't be.

Wiring them together

```java
@GetMapping("/api")
public String getData() {
    ResponseEntity<String> response =
        restTemplate.getForEntity("http://localhost:8081/log", String.class);
    System.out.println("Response from sidecar: " + response.getBody());
    return "Hello, Sidecar!";
}
```

`RestTemplate` is Spring's HTTP client. Each `/api` hit calls the sidecar's `/log`, which logs, and the app continues. **Note** `localhost:8081` — the shared network namespace: no service discovery needed; the sidecar is always at a known local port.

> ⚠️ **Pedagogically useful but architecturally backwards.** The app *explicitly calling* the sidecar defeats the central benefit — it now knows about the sidecar, contains code **coupling** it to the sidecar's details, and breaks if the sidecar changes. Real sidecars are **transparent**, intercepting traffic unknown to the app (as Istio/Envoy do below). Read this as "how two co-located processes communicate", not the intended usage.

The deployment factor

The example skips deployment. In production: **Docker** packages the app and sidecar; **Kubernetes orchestrates** them (starts, places, restarts, scales). Kubernetes has **built-in support for sidecar containers**, making it the natural platform.

Part 7 — Performance implications

"Does a sidecar slow my application down?" A real trade-off.

Resource cost

Each sidecar is an **additional process** — more CPU, memory and network **bandwidth**. Small individually, but **multiplied by every instance**: with hundreds or thousands of instances, **cumulative usage becomes significant**.

> Concretely: 100 MB per sidecar × 1,000 pods = 100 GB of RAM spent on infrastructure rather than application work — the most common objection to adopting a service mesh.

Latency

An **additional hop** adds **latency**. It's **usually negligible** — a **loopback** connection (traffic never leaving the machine) inside the Pod, not a real network traversal — but in **high-performance, low-latency systems it can matter**; measure rather than assume.

The verdict

The slight performance cost often **pales next to the benefits** in functionality, maintainability and operations — but weigh it for your situation.

Part 8 — Special considerations

1. **Sidecar and main application versioning.** You version **both**. What if a new sidecar is **incompatible with an older app version**, or vice versa? How do you **roll out** updates across a **fleet** (all running instances)? Answer before adopting, not after.
2. **Shared resources.** Sidecars share network and disk with the app and must **not hog them** — use **container-level resource limits and quotas** (hard caps on CPU and memory). Without limits, a misbehaving sidecar — say one **buffering** logs in memory during a log-collector **outage** — can starve the very application it supports.
3. **Security.** Sidecars often need **permissions similar to the app's**, which **increases the attack surface**. An mTLS sidecar holds **private keys** (whoever has them can impersonate the service); a log sidecar can read application output. Compromising the sidecar can be as valuable as compromising the app.
4. **Complexity.** Developers and operators now manage **multiple parts instead of one application**; debugging a request means reasoning about the app *and* the proxy in front of it. Not a one-size-fits-all solution.

Part 9 — Use cases

- **Monitoring and metrics collection** — embedding monitoring code in every app is complex; a sidecar collects metrics and keeps app code clean.
- **Logging** — a sidecar **collects logs and forwards them to a central logging system**; changing the log destination touches no application.
- **Security and authentication** — delegated to a sidecar, security is **more consistent and easier to update**: patch one sidecar image instead of auditing dozens of services.
- **Networking features** — **service discovery, connection pooling** (reusing open connections), **circuit breaking, intelligent routing**.

> The unifying test: **is this concern identical across all your services, and unrelated to what any of them does?** If yes, it's a sidecar candidate.

Part 10 — System design examples

Microservices architecture

Each microservice can have a sidecar providing **ancillary** (supporting) services — logging, monitoring, service discovery — while the service focuses on its main task.

Kubernetes and Istio — the canonical example

Istio deploys an **Envoy sidecar proxy alongside each service** to **handle all network communication**, **enforce policies**, and **collect telemetry** (metrics, logs and traces sent automatically). The application contains **zero** networking, retry, mTLS or **observability** logic; all of it lives in infrastructure, configured centrally and applied uniformly — the pattern at its purest.

Part 11 — Sidecar vs. shared library

The real decision most teams face:

	Shared library	Sidecar
Languages	One per language, maintained separately	One, language-agnostic
Upgrade	Rebuild and redeploy every service	Update the sidecar image
Runtime cost	Negligible — same process	An extra process per instance
Latency	None	A local hop
Failure isolation	A library bug crashes the app	Sidecar can fail separately
Best when	One language, few services, latency-critical	Polyglot, many services, frequent policy changes

Rule of thumb: one language and a handful of services → a library is simpler and cheaper. Several languages, many services, or a need to change cross-cutting behaviour without redeploying everything → the sidecar earns its overhead.

Part 12 — Common beginner misconceptions

- **"The sidecar and app are the same container."** Separate containers in the same Pod (Part 2).
- **"The app calls the sidecar."** In the toy example, yes; in real deployments the sidecar transparently intercepts — that transparency is the value.
- **"A sidecar is a microservice."** A **microservice** serves many clients and owns a **domain** (a business area). A sidecar serves exactly **one** application instance, owns no domain, and its lifecycle is bound to its host.
- **"Sidecars are only for service meshes." Log shippers** (forward logs to a central store), metrics **agents**, config reloaders and **secret-fetchers** (pull keys from a vault) are all sidecars.
- **"It's free because it's just localhost."** One process of CPU and memory **per instance** — the main reason teams hesitate.
- **"You need Kubernetes."** Pods make it easy, but the pattern predates it — anything that co-schedules two processes with shared networking works.

Part 13 — Check your understanding

1. What makes a concern a "cross-cutting" one?
2. Why doesn't a shared library solve cross-cutting concerns in a polyglot organisation?
3. What Kubernetes feature makes the pattern practical, and which two properties of it matter most?
4. Why is "intercepts" a more important word than "calls"?
5. What's the main objection to adopting a service mesh at scale?
6. When is a shared library the better choice?

(Answers: 1 — it's needed by many components but isn't the core purpose of any of them; 2 — you'd need a separate implementation per language, and they inevitably drift; 3 — the Pod, because it co-schedules containers on one machine and gives them a shared network namespace and lifecycle; 4 — interception means the application needs no knowledge or code changes, which is what lets you add mTLS and tracing to services you can't modify; 5 — cumulative resource cost, one extra process per instance across thousands of instances; 6 — a single language, few services, and latency-critical paths.)

Part 14 — Interview soundbite (30 seconds)

"The sidecar pattern attaches a helper process beside each service instance for cross-cutting concerns — logging, metrics, auth, networking — so the service holds only business logic. It's co-located: a separate container in the same Kubernetes Pod, sharing the network namespace and lifecycle, talking over localhost. Its key role is a proxy that transparently intercepts all inbound and outbound traffic, doing load balancing, circuit breaking, mTLS and service discovery without the application knowing it exists — so you can add mTLS and distributed tracing without changing a line of code. The canonical example is Istio, with an Envoy sidecar beside every service and no networking code in the app. Over a shared library it's language-agnostic, so a polyglot estate gets consistent behaviour and you upgrade policy without redeploying every service. The costs: one extra process per instance — 100 MB across a thousand pods is 100 GB of RAM; an extra local hop; versioning the sidecar against the app; and sidecar permissions that widen your attack surface."

Part 15 — Implementation

A. Runnable demo

Full source: **demos/SidecarDemo.java**

```
cd demos && ./gradlew runCh09
```

The whole point in two class definitions. Before:

```
String handleRequest(String payload) {
    requestCount.incrementAndGet();                    // metrics
    System.out.println("[app] logging inbound request"); // logging
    if (!payload.startsWith("signed:")) return "401";    // security
    String result = businessLogic(payload);             // <- the only real work
    // ... emit latency metric
}
```

After:

```
static class CleanApp {
    String handleRequest(String payload) {
        return "order-created:" + payload;              // business logic ONLY
    }
}
```

Everything removed now lives in the sidecar, which **intercepts** rather than being called:

```
String handleInbound(String rawRequest, String clientCert) {
    if (!"valid-cert".equals(clientCert)) {
        return "403 Forbidden";                         // app never invoked
    }
    String response = app.handleRequest(rawRequest);     // plain call, no plumbing
    // ... record latency
}
```

B. What you should see

```
=== 3. mTLS enforced by the sidecar ===
  -> 403 Forbidden
  >> rejected before the application was ever invoked

=== 4. Retry + backoff, also in the sidecar ===
    [sidecar] upstream failed (connection reset), retrying
    [sidecar] upstream ok on attempt 3
  upstream was actually called 3 times; the app asked once
  >> CleanApp contains no retry code whatsoever

=== 5. Telemetry collected without any app instrumentation ===
      requests_ok        1
      requests_rejected  1
      upstream_retries   2

=== 7. The cost: resource multiplier ===
      1,000 pods x 100MB sidecar = 100,000 MB (97.7 GB) of RAM
      5,000 pods x 100MB sidecar = 500,000 MB (488.3 GB) of RAM
```

Section 4 is the pattern in one line: the **upstream** (the called service) was called three times; the application asked once — retry, backoff and failure handling happened outside your code. **Section 7 is the honest counterweight:** ~98 GB of RAM on infrastructure at a thousand pods — the number that comes up when someone asks whether the mesh is worth it.

C. Production — Istio

Injection — label the **namespace** (a named group of Kubernetes resources) and every pod gets an Envoy sidecar automatically (`kubectl` is the Kubernetes CLI):

```
kubectl label namespace production istio-injection=enabled
```

Your **Deployment** (the object describing which containers to run, and how many) is unchanged; the sidecar is added at **admission time** (when Kubernetes accepts the object and may modify it):

```
spec:
  containers:
    - name: order-service          # your app - you wrote this
      image: order-service:1.4.2
    # - name: istio-proxy          # injected automatically, you did NOT write this
```

mTLS across the whole mesh — no application code, no **certificate** (signed identity file) handling:

```yaml
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: production
spec:
  mtls:
    mode: STRICT       # every service-to-service call is encrypted and authenticated
```

Retries, timeouts and circuit breaking as configuration. A `VirtualService` sets routing rules; a `DestinationRule` sets traffic policy. **Outlier detection** temporarily *ejects* (stops routing to) instances that keep returning errors:

```yaml
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: inventory
spec:
  hosts: [inventory]
  http:
    - timeout: 3s
      retries:
        attempts: 3
        perTryTimeout: 1s
        retryOn: 5xx,reset,connect-failure
      route:
        - destination: { host: inventory }
---
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: inventory
spec:
  host: inventory
  trafficPolicy:
    connectionPool:
      tcp:  { maxConnections: 100 }
      http: { http1MaxPendingRequests: 50, maxRequestsPerConnection: 10 }   # a bulkhead
    outlierDetection:                      # this IS a circuit breaker
      consecutive5xxErrors: 5
      interval: 10s
      baseEjectionTime: 30s
      maxEjectionPercent: 50
```

You just configured Bulkhead, Retry and Circuit Breaker in YAML, for every service, in every language, without touching application code — the argument for a service mesh in one **manifest** (a YAML file of desired Kubernetes resources).

Non-mesh sidecars are just as common (**Fluent Bit** is a lightweight log shipper; the Cloud SQL proxy opens a secure **tunnel** — an encrypted pass-through connection — to a Google Cloud database):

```yaml
containers:
  - name: app
    image: my-app:1.0
    volumeMounts: [{ name: logs, mountPath: /var/log/app }]
  - name: log-shipper                      # sidecar: tails and ships logs
    image: fluent-bit:2.2
    volumeMounts: [{ name: logs, mountPath: /var/log/app }]
  - name: cloud-sql-proxy                  # sidecar: secure DB tunnel
    image: gcr.io/cloud-sql-connectors/cloud-sql-proxy:2.8
volumes:
  - name: logs
    emptyDir: {}                           # shared between the two containers
```

Kubernetes native sidecars (1.29+) solve the startup/shutdown ordering problem. An **initContainer** normally runs to completion before the app starts; `restartPolicy: Always` turns it into a long-running sidecar:

```
initContainers:
  - name: istio-proxy
    image: istio/proxyv2
    restartPolicy: Always        # <- makes it a true sidecar: starts first, stops last
```

D. Implementation pitfalls

1. The app starting before the sidecar is ready. The app's first outbound call fails because the proxy isn't up. Use native sidecars (`restartPolicy: Always` on an initContainer) or Istio's `holdApplicationUntilProxyStarts: true`.

2. The sidecar stopping before the app finishes. The mirror image at shutdown: **in-flight** requests fail because the proxy died first. Native sidecars fix this too.

3. No resource limits on the sidecar. An unbounded proxy can starve the app it serves. Always set them (`100m` = a tenth of a CPU core; `128Mi` = 128 mebibytes):

```
annotations:
  sidecar.istio.io/proxyCPU: "100m"
  sidecar.istio.io/proxyMemory: "128Mi"
```

4. A full mesh for three services. The resource multiplier and operational complexity are fixed costs. Below roughly a dozen services in one or two languages, a shared library is cheaper and simpler — be able to justify the mesh.

5. Forgetting the sidecar is in the network path when debugging. A 503 ("service unavailable") may come from Envoy, not your service. Learn to read the proxy's logs and response flags before blaming application code.

6. Duplicating retries in app and mesh. App client 3× and mesh 3× = 9 requests. Pick one layer — usually the mesh — and disable the other (Retry: one layer only).

7. Assuming mTLS means you're done with authorization. mTLS proves *who* is calling, not *what they may do* — **authorization**. You still need `AuthorizationPolicy` resources.

Related patterns

- **Service Discovery** — meshes absorb discovery into the sidecar entirely
- **Circuit Breaker**, **Retry**, **Bulkhead** — all commonly enforced in the sidecar rather than in application code
- **API Gateway** — the gateway handles north-south (external, into and out of the system) traffic; the mesh of sidecars handles east-west (internal, service-to-service)
- **Config Externalization** — a sidecar can fetch config and expose it locally to the app

Saga Pattern

Category: Data consistency / distributed transactions **One-liner:** Break a business transaction that spans several services into a sequence of local transactions, each with a compensating action that undoes it — achieving eventual consistency without **distributed locks** (locks held across several machines at once so no one else can change the data).

> This chapter is the book's home for the **Transactional Outbox** (Part 13D); Event-Driven and others link here.

Part 0 — Concepts you need first

What is a transaction?

A **transaction** is one logical operation of one or more data steps that **succeed or fail together**. A bank transfer — subtract £100 from A, add £100 to B — must be **one indivisible unit**, or £100 vanishes.

What is ACID?

The four guarantees of a traditional database transaction (§ Transactions and ACID):

Property	Meaning
Atomicity	Succeeds completely or fails completely; any failure leaves the database unchanged. *Atomic = indivisible.*
Consistency	Moves the database from one **valid state to another**, respecting all rules and constraints
Isolation	**Concurrent** transactions end in the same state as if run **one after another**; your half-finished work is invisible to others
Durability	Once **committed**, it **stays committed**, even if power fails a millisecond later

Together: a completed transaction **either fully occurred or didn't occur at all**.

What is a "rollback"?

Undoing a transaction's changes back to the pre-transaction state. In one database it's automatic and free — the database keeps a log and reverses it. **That's the capability you lose when a transaction spans services**; replacing it by hand is what Saga is about.

What is a database lock?

While a transaction modifies a row, the database **locks** it so nobody else can — that's how isolation works. **Locks are held until commit**: milliseconds for a short transaction, but seconds if it waits on a network call to another continent — during which nobody else can touch that row.

What is "database per service"?

A core **microservices** principle: **each service owns its database**, and no other service reads or writes it directly — Order Service *asks* Inventory Service rather than querying its tables. This gives independence (Inventory can change its **schema** freely), but it's precisely what rules out one database transaction spanning both.

What is eventual consistency?

The system **may temporarily be inconsistent** but **eventually converges**. Strong consistency: the moment the transfer completes, every observer sees both accounts updated. Eventual: briefly, an observer might see the debit but not yet the credit. Distributed systems accept this because strong consistency across a network needs locks and coordination that cost **availability** and speed (§ Consistency models, § CAP).

Part 1 — The problem

It works fine in a monolith

Checkout = adjust inventory, create order, take payment. In a **monolith** with one database that's one ACID transaction: if payment fails, inventory and order roll back automatically. **Atomicity for free.**

Why it breaks in microservices

Coordination. Inventory, Order and Payment run on different servers, each with its **own database**. Atomicity means undoing earlier steps when any fails — and **coordinating that rollback across independent services is genuinely hard**: no shared transaction log, no single database to tell "undo everything".

Network latency. Services may be geographically distributed, so a transaction can take a long time, and **holding locks that long is impractical**.

> A database transaction normally holds locks for milliseconds; a distributed one across three services might hold them for **seconds**, during which every other customer buying the same product is blocked. At real traffic, the system grinds to a halt.

Conclusion: ACID's model **doesn't translate to services that operate independently and communicate over networks.**

Part 2 — The solution

A saga is a sequence of local transactions. Each one:

1. **Updates its own database** (a normal, fully ACID local transaction)
2. **Publishes an event** (a message announcing what happened) that triggers the next local transaction

If a local transaction **fails because it violates a business rule** (e.g. "stock can't go below zero"), the saga runs **compensating transactions** that undo the preceding local transactions. Rather than enforcing strict consistency, Saga accepts **eventual consistency**.

The guarantee it actually gives

Either **all transactions complete**, or **compensating transactions roll back** the ones already executed — integrity through explicit compensation rather than automatic rollback. Ideal for **long-lived business transactions spanning multiple microservices** and globally distributed, **highly available** databases.

The cost: designing sagas is complex — intricate coordination and **extensive error handling**. You're hand-writing what the database used to do for you. (The course's image: ACID is the internal wiring of each building; a saga is the power line connecting buildings.)

Part 3 — The two types

Choreography — decentralized

Every local transaction **publishes domain events** (business facts, like `OrderCreated`) that trigger local transactions elsewhere. **No central coordination**: each service listens to others' events and decides whether to act (an orchestra with no conductor).

In practice: Order publishes `OrderCreated` → Payment charges and publishes `PaymentCompleted` → Inventory reserves stock and publishes `StockReserved`. Nobody is in charge; the flow emerges from who **subscribes** to what.

Orchestration — centralized

A central **saga orchestrator** executes the steps, telling each participant when to run its local transaction (the conductor, or an air-traffic controller).

In practice: the orchestrator calls Order, waits, calls Payment, waits, calls Inventory; if Inventory fails it calls the compensations in reverse. The whole flow is readable in one place.

Choosing between them

	Choreography	Orchestration
Coordination	None — fully decentralized	Central orchestrator
Service autonomy	Higher	Lower — services do as told
Visibility of the flow	Poor — scattered across services	Good — in one place
Error handling	Harder	Easier
Coupling (dependence between parts)	Services coupled to events	Services coupled to the orchestrator
Best for	**Simple** transactions	**Complex** transactions

Rule of thumb: simple transaction → choreography needs less coordination; as complexity grows, **orchestration gives better visibility and control**.

> Choreography's practical failure mode: with six services reacting to events, **nobody can answer "what happens when a customer places an order?"** without reading six **codebases**. The business process exists but is written nowhere — which is why complex flows migrate to orchestration.

Part 4 — Architecture

Local transactions — the backbone

Each runs **within one service and its database**; strung together they maintain consistency across the system. **Every local transaction has an associated compensating transaction.**

Compensating transactions

When a local transaction fails, the saga runs compensations for the previously successful ones, returning the system to a consistent state — retracing your steps to the wrong turn.

The Saga Log

A **saga log** records **every local and compensating transaction executed**, so if the saga is interrupted (a service crashes mid-flow) it can **resume where it left off** or know exactly which compensations to run — a breadcrumb trail back to a consistent state.

Database per service

Each service **owns its database**, so one failure doesn't directly impact others — no **single point of failure**, more robust overall.

Eventual consistency

The ultimate goal: the system **may pass through inconsistent states** but **eventually reaches a consistent one**.

Part 5 — Worked example: online shopping (orchestration)

Scenario: you check out, creating an order; Inventory checks availability; if available it updates stock and the order completes, otherwise the order is cancelled.

The services — each with its own database

```
public class OrderService {
    private OrderDatabase orderDatabase = new OrderDatabase();

    public void createOrder(Order order) {
        orderDatabase.save(order);
    }

    public void cancelOrder(String orderId) {
        // COMPENSATING TRANSACTION for createOrder
    }
}
```

```
public class InventoryService {
    private InventoryDatabase inventoryDatabase = new InventoryDatabase();

    public boolean checkAndReduceInventory(Order order) {
        // local transaction — returns false if stock is insufficient
    }

    public void addBackInventory(Order order) {
        // COMPENSATING TRANSACTION for checkAndReduceInventory
    }
}
```

The pattern in miniature: each service has its **own database**, and each forward operation is **paired with an inverse**:

Forward action	Compensating action
`createOrder`	`cancelOrder`
`checkAndReduceInventory`	`addBackInventory`

That pairing *is* the Saga pattern. Everything else is plumbing to invoke them in the right order.

The orchestrator

```
public class OrderOrchestrator {
    private OrderService orderService = new OrderService();
    private InventoryService inventoryService = new InventoryService();

    public void processOrder(Order order) {
        orderService.createOrder(order);

        if (!inventoryService.checkAndReduceInventory(order)) {
            orderService.cancelOrder(order.getId());   // compensate
        }
    }
}
```

Create the order; try to reduce inventory; if that returns `false`, **undo the order**. Forward and compensation paths are both visible in one method — orchestration's main selling point.

Handling mid-saga failures

What if a service fails partway? That's what the **Saga Log** is for: record each step, and after a failure consult the log and run the needed compensations.

> This example has **no saga log**. A crash between `createOrder` and `checkAndReduceInventory` leaves an order in limbo forever, with no stock reserved and nothing to fix it. Real implementations persist saga state before every step (Part 13D).

What the example is and isn't

A basic illustration — no retries, no **asynchronous messaging**, no log. But the value holds: **split a large transaction into independent parts that can be executed and compensated separately.**

Part 6 — Issues and considerations

Complexity

Each service must now provide **both** its local transaction **and** a compensation. **Concrete difficulty:** if a **discount was applied** and the order is then cancelled for lack of stock, the Order Service's compensation must unwind the discount too — perhaps returning a single-use voucher. Complexity creeps in fast; careful design avoids a tangled web of services.

Longer latency

Each step is a network call, slower than local calls, so sagas have **longer latency than traditional transactions** — what a monolith does in one quick database transaction now crosses the network.

Application consistency — not just data consistency

The most important point in the chapter. Saga restores **data** consistency by compensating — but what about the **business process**?

> If the inventory check fails the order is cancelled — **but cancelling an order is not the same as never having placed it.**

The customer may have a confirmation email; an **audit record** shows the order existed; **analytics** counted it; a warehouse worker saw it on screen. **Compensation is not time travel**: it makes the data consistent but can't un-happen the visible intermediate state. Design the saga with that in mind.

Idempotency

An operation can **fail after executing but before reporting success**, then be retried — so steps must be **idempotent** (repeating has the same effect as once). If `checkAndReduceInventory` reduced stock but failed before reporting, the retry must **not reduce it again** — typically by recording the **saga ID** (unique ID of one saga run) and refusing to apply the same operation twice. *(Same technique as idempotency keys.)*

Increased load

Maintaining a **saga log** means **extra reads and writes in every service**, raising load and possibly response times — the Order Service now handles orders *and* its saga log.

It's a trade-off

Consistency versus complexity, speed versus distributed transactions — understanding the trade-offs is what lets you decide well.

Part 7 — Use cases

E-commerce systems

Order creates the order, **Payment** charges, **Inventory** checks and reduces stock. **If the inventory check fails**, the order is cancelled **and the payment refunded** — a compensating transaction, not a rollback: the charge really happened and appears on the statement; the refund is a second, separate movement of money.

Travel booking systems

Flight, hotel and car rental must **succeed together or fail as a whole**, but they're **independent services, possibly owned by different companies**.

> The clearest case for Saga: there's no shared database, so one ACID transaction across an airline, a hotel chain and a car rental firm is impossible. Compensation (cancel the flight, cancel the hotel) is the *only* mechanism.

Part 8 — System design examples

Online food delivery

Order created → restaurant confirms → delivery partner assigned → payment processed. **What if the restaurant runs out of an item?** Cancel the order and refund the payment; the saga keeps data consistent and handles the failure gracefully.

Internet banking

Transfer from savings to checking: **debit savings**, **credit checking** — if one fails, the other must not stand.

> Without a saga, a failure in the credit step could leave **money disappearing into thin air** — debited and never arriving.

Part 9 — Saga vs. two-phase commit (2PC)

Not in the course, but you'll be asked. **2PC** makes a transaction atomic across databases: a **coordinator** asks every participant "can you commit?" (phase 1) and, if all say yes, tells them "commit now" (phase 2). Background: § Distributed transactions.

	2PC	Saga
Consistency	Strong (genuinely ACID)	Eventual
Locks	Holds locks across all participants until commit	No distributed locks
Availability	**Blocks** if the coordinator fails mid-protocol	Stays available
Rollback	Automatic	Manual — you write the compensating logic
Suits	Short transactions in one trust domain (systems run by one organisation that trust each other)	Long-lived, cross-service, cross-company

Why microservices generally avoid 2PC: it holds locks for the whole distributed operation (Part 1's problem); if the coordinator dies between phases, participants hold locks with no instruction — a

genuinely stuck system; and every participant must support the protocol, which external companies won't.

Part 10 — Common beginner misconceptions

- **"A compensating transaction is a rollback."** A rollback erases history; a compensation is a **new forward action that offsets the previous one**. The refund appears beside the charge; both are real.
- **"Every step can be compensated."** You can't un-send an email, un-ship a package or un-launch a rocket. Put such steps **last**, or design a real-world compensation ("please ignore that email").
- **"Sagas give you ACID across services."** They explicitly don't: you give up isolation entirely — other transactions **can see** your half-finished saga. That's the price of no distributed locks.
- **"Choreography is simpler, so always start there."** For two or three services. Beyond that the invisible business process becomes a serious maintenance problem.
- **"The orchestrator is a single point of failure, so choreography is safer."** The orchestrator can be made highly available with persisted state. Choreography's risk isn't availability — it's comprehensibility.
- **"Eventual consistency means the data might just be wrong."** It means a *window* during which observers may see intermediate state; it converges. The design question is whether the business tolerates that window.

Part 11 — Check your understanding

1. Why can't you use one ACID transaction across three microservices?
2. Why is holding database locks across a distributed transaction impractical?
3. What is the pairing that defines the Saga pattern?
4. Why is "cancelling an order is not the same as never placing it" an important point?
5. Why must saga steps be idempotent?
6. Why does travel booking make the strongest case for Saga?

*(Answers: 1 — each service owns its own database, so there's no single **transaction manager** (the component that coordinates commit and rollback) or shared log to coordinate commit and rollback; 2 — the transaction spans network calls taking seconds, during which locked rows block every other user; 3 — each forward local transaction paired with a compensating transaction that undoes it; 4 — Saga restores data consistency but cannot undo visible side effects like confirmation emails and audit records, so the business process must account for the intermediate state; 5 — a step can succeed but fail before reporting success, then be retried, and must not apply its effect twice; 6 — the participants are separate companies, so a shared database transaction is not merely difficult but impossible.)*

Part 12 — Interview soundbite (30 seconds)

"Saga handles distributed transactions where ACID can't cross service boundaries. Each service owns its database, so there's no shared transaction manager — and holding locks across network calls for seconds would block everyone anyway. Instead you break the business transaction into local transactions, each paired with a compensating transaction; if step three fails you run the compensations for two and one. You get eventual consistency and give up isolation — others can see your half-finished saga. Two flavours: choreography, where services react to each other's events — fine for simple flows but it scatters the process across codebases so nobody can read it; and orchestration, a central coordinator with much better visibility and error handling for complex flows. A saga log persists progress so you can resume or compensate after a crash, written with a transactional outbox so state and events can't diverge. Classic examples: checkout — order,

payment, inventory, with a refund as compensation — and travel booking across companies, where a shared transaction is impossible. Hard parts: steps must be idempotent, since a step can succeed but fail to report and get retried; and compensation isn't rollback — the refund is a second real money movement, and a cancelled order isn't a never-placed one, because the confirmation email already went out."

Part 13 — Implementation

A. Runnable demo

Full source: `demos/SagaDemo.java`

```
cd demos && ./gradlew runCh10
```

Compensation driven by the log, in reverse order:

```
void compensate(String sagaId) {
    for (String step : log.stepsNeedingCompensation(sagaId)) {   // newest first
        switch (step) {
            case "reserveStock"  -> inventory.release(sagaId);
            case "chargePayment" -> payments.refund(sagaId);
            case "createOrder"   -> orders.cancelOrder(sagaId);
        }
        log.record(sagaId, step, StepState.COMPENSATED);
    }
}
```

Idempotency using the saga ID as the key:

```
void charge(String sagaId, int amount) {
    if (charges.containsKey(sagaId)) {
        return;                       // duplicate - do NOT charge again
    }
    charges.put(sagaId, amount);
    ledger.add("CHARGE  +" + amount);
}
```

B. What you should see

```
=== 2. Step 3 fails -> compensations run in REVERSE ===
      OrderService: order created
      PaymentService: charged 500
      InventoryService: INSUFFICIENT STOCK (have 3, need 99)
    !!! step failed: out of stock -> compensating
      PaymentService: REFUNDED 500 (compensation)
      OrderService: order CANCELLED (compensation)
  order status: CANCELLED | stock: 3

=== 3. Idempotency: the same step retried ===
      PaymentService: charged 150
      PaymentService: duplicate detected, NOT charging again
      PaymentService: duplicate detected, NOT charging again

=== 4. Compensation is NOT rollback ===
  payment ledger: [CHARGE +300, CHARGE +500, REFUND -500, CHARGE +150]
```

Section 4 is the conceptual point made concrete. In the **ledger** (an append-only list of money movements) the £500 charge and £500 refund **both exist**. A rollback would have erased the charge; compensation adds an **offsetting entry**, and the customer sees both lines.

C. Production — orchestration

Hand-rolled orchestration means reinventing **persistence**, **timeouts** and retries. Real options are **workflow engines** that run multi-step processes reliably: **Temporal**, **Camunda/Zeebe**, **AWS Step Functions** (managed), or **Axon Framework** (a Java framework for event-based systems).

Temporal is the most popular code-first choice: your saga is ordinary code and the framework makes it **durable** — if the process crashes mid-saga, it resumes from the last completed step on another **worker**:

```
public class OrderWorkflowImpl implements OrderWorkflow {

    private final OrderActivities activities = Workflow.newActivityStub(
        OrderActivities.class,
        ActivityOptions.newBuilder()
            .setStartToCloseTimeout(Duration.ofSeconds(10))
            .setRetryOptions(RetryOptions.newBuilder()
                .setMaximumAttempts(3)
                .setBackoffCoefficient(2.0)
                .build())
            .build());

    @Override
    public void processOrder(Order order) {
        Saga saga = new Saga(new Saga.Options.Builder().build());
        try {
            activities.createOrder(order);
            saga.addCompensation(activities::cancelOrder, order.id());   // register the undo

            activities.chargePayment(order);
            saga.addCompensation(activities::refundPayment, order.id());

            activities.reserveStock(order);                              // may fail
        } catch (ActivityFailure e) {
            saga.compensate();      // runs registered compensations in reverse
            throw e;
        }
    }
}
```

The pattern to copy even without Temporal: register each compensation **immediately after** its step succeeds, so the undo list is always exactly correct with no bookkeeping.

Choreography with Kafka — no orchestrator (Kafka basics — **topics**, consumers, `groupId` — are in Event-Driven 14C):

```
@KafkaListener(topics = "order-created", groupId = "payment-service")
public void onOrderCreated(OrderCreated event) {
    try {
        paymentService.charge(event.orderId(), event.amount());
        kafka.send("payment-completed", new PaymentCompleted(event.orderId()));
    } catch (PaymentFailedException e) {
        kafka.send("payment-failed", new PaymentFailed(event.orderId()));   // triggers compensation
    }
}
```

Every step publishes success or failure, and compensation is just another consumer reacting to a failure event. Nobody is in charge — exactly why complex flows get hard to follow.

D. Implementing the saga log properly

The demo's log is in memory, so a crash loses everything. Real implementations persist it **in the same transaction as the business change**, using the **Transactional Outbox** pattern: write outgoing events into an "outbox" table **in the same local database transaction** as the data change, and publish them afterwards. (In the SQL, `UUID PRIMARY KEY` is a random unique ID as the row's identifying column; `JSONB` is PostgreSQL's binary JSON type.)

```sql
CREATE TABLE saga_state (
    saga_id       UUID PRIMARY KEY,
    current_step  VARCHAR(50)  NOT NULL,
    state         VARCHAR(20)  NOT NULL,   -- STARTED / COMPLETED / COMPENSATING / DONE
    payload       JSONB        NOT NULL,
    updated_at    TIMESTAMP    NOT NULL
);

CREATE TABLE outbox (
    id          UUID PRIMARY KEY,
    aggregate   VARCHAR(50),
    event_type  VARCHAR(100),
    payload     JSONB,
    published   BOOLEAN DEFAULT FALSE
);
```

```java
@Transactional
public void completeStep(String sagaId, String step, Event next) {
    sagaRepo.markCompleted(sagaId, step);      // both writes
    outboxRepo.save(next);                     // in ONE local transaction
}
// A separate poller (or Debezium CDC) publishes unpublished outbox rows to Kafka
```

`@Transactional` runs the method in one database transaction. The relay is either a **poller** (a process checking the table for unpublished rows) or **Debezium CDC** (change data capture: streaming new rows straight from the database's change log — see Strangler Fig 13D).

Why the outbox exists. Writing to the database and then publishing to Kafka as two separate operations means a crash between them leaves them permanently inconsistent — the step completed but nobody was told (or an event announced a change that rolled back). Writing the event into the same database in the same transaction makes the two atomic; a separate process relays it. Relays can redeliver, so consumers must be idempotent. This is the standard solution — know it by name. It applies to **any** service that changes state and publishes an event, not just sagas.

E. Implementation pitfalls

1. Compensations that aren't idempotent. Compensations get retried too; refunding twice is as bad as charging twice. Guard with the saga ID.

2. Compensations that can fail permanently. If the refund API is down: retries with **backoff**, ultimately a **dead-letter queue** (holding area for messages that keep failing) **plus human intervention**. A saga stuck half-compensated is the worst state — alert on it.

3. An uncompensatable step in the middle. You can't un-send an email or un-ship a parcel. **Irreversible actions come last**, after everything that could fail has succeeded.

4. An in-memory saga log. A crash leaves **orphaned** state (half-finished records nothing will clean up). Persist before each step.

5. Writing to the database and publishing separately. Use the outbox — the single most common correctness bug in event-driven sagas.

6. Assuming isolation. Sagas have **no isolation**: a customer may see an order about to be cancelled. Design the **UI** for it ("processing", not "confirmed").

7. Choreography for a complex flow. Six services reacting to each other's events means nobody can say what happens on order placement. Move to orchestration past three or four steps.

8. No timeouts on saga steps. A step that never responds hangs the saga forever. Give every step a deadline, after which it's failed and compensation begins.

Related patterns

- **Event-Driven Architecture** — the messaging substrate choreography sagas run on
- **CQRS** — frequently paired with sagas in **event-sourced** systems (which store every change as an event rather than just the current state)
- **Retry** — idempotency concerns are shared; retries happen within saga steps
- **Circuit Breaker** — protects individual saga participants

Event-Driven Architecture (EDA) Pattern

Category: Data / messaging **One-liner:** Components communicate by publishing and reacting to events rather than calling each other directly — giving loose coupling, asynchrony, independent scaling, and **real-time** responsiveness (reacting within moments of something happening).

Part 0 — Concepts you need first

What is an "event"?

An **event** is a record that **something happened** — a fact about the past, named in the **past tense**: `OrderPlaced`, `PaymentReceived`, `UserRegistered`. The tense isn't cosmetic: an event is a *notification of something already done*, not a request. Contrast a **command** — an instruction named imperatively (`PlaceOrder`, `ChargeCard`). **A command can be refused; an event cannot** — it already happened.

Synchronous vs. asynchronous

Synchronous — send a request and **wait** for the response (a phone call). **Asynchronous** — send a message and **carry on immediately**, handling any response when it arrives (email). For a server: in a synchronous call the calling **thread** is **blocked and useless** for the whole wait; asynchronously it's free at once. (§ Sync vs async.)

What is a message queue / event bus?

Middleware that accepts messages from senders and delivers them to receivers — **Kafka** (a distributed, durable event log) and **RabbitMQ** (a classic message broker) are common (§ Message queues). Key properties:

- The **sender doesn't need to know who receives** — it publishes to the bus
- Messages are usually **persisted**, so if a receiver is down the message waits
- **Many receivers** can consume the same message independently

What is publish-subscribe?

Publishers send to a named channel ("topic") without knowing who listens; **subscribers** register interest and receive everything published there. **The publisher holds no subscriber list**, so adding a subscriber needs no publisher change — the source of EDA's flexibility.

What is "coupling"?

How much one component depends on knowledge of another.

- **Tight coupling:** A calls `serviceB.doThing()` — A must know B exists, where it lives, its **API** and response format; change B and A may break.
- **Loose coupling:** A publishes `ThingHappened` without knowing who listens — B, five services, or none. **A is unaffected either way.**

Part 1 — The problem

Distributed systems brought **scalability** and **resilience** but new challenges:

- **Managing complex interactions** — many components must communicate effectively, and changes in one part must be reflected in others. (Everyday example: the "your product shipped" or "new blog post" email is a system handling complex interactions efficiently.)
- **The call for asynchronous communication** — synchronous request-and-wait works small but **quickly becomes a bottleneck**: *what happens when thousands or millions of components are all waiting?* Each holds resources doing nothing. Async is far more efficient but **hard to manage well**. (Why can you

keep adding items to a cart while earlier requests are still processing? Asynchronous communication.)

- **The demand for real-time responsiveness** — users expect a status update to reach friends immediately; multiplayer games can't tolerate delay.
- **Data consistency** — with components reading and writing **concurrently**, how do others learn of a change? A banking transaction must update your balance promptly and accurately even while another transaction hits the same account.

Part 2 — The solution

EDA revolves around the **production, detection, consumption of, and reaction to events**. An **event** is a state change produced by an **event producer** and consumed by one or more **event consumers**. (The courier analogy: sender = producer, package = event, recipient = consumer, delivery service = event bus.)

Part 3 — The benefits

Decoupling

Producers and consumers **don't need to know each other exist**, making the system adaptable. One **photo upload** event can independently trigger notifying followers, updating the profile, and alerting moderation if content violates policy.

> The part worth pausing on: **six months later you add "generate a thumbnail"** by writing a new consumer of `PhotoUploaded`. **You never touch the upload code.** With direct calls you'd modify, retest and redeploy the upload service for every new reaction, forever. That difference in change cost is the pattern's real value.

Asynchrony

The producer emits and **carries on without waiting** — **non-blocking**, improving responsiveness and **throughput** (work done per second).

Scalability

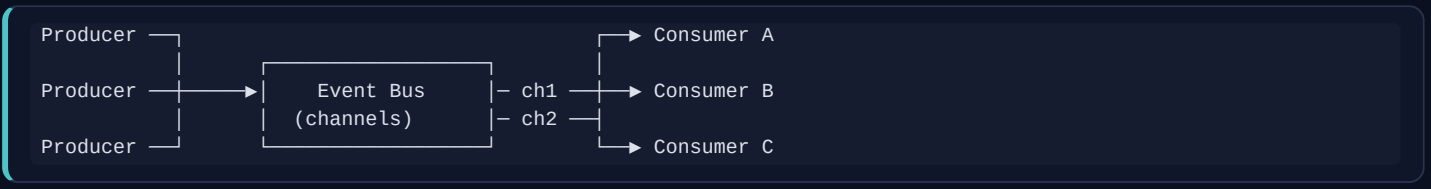
Decoupled producers and consumers **scale independently**: a surge of events → add consumer **instances**, and vice versa.

Real-time processing

Events can be handled **immediately** — essential for **fraud detection in banking** or real-time **analytics**.

Part 4 — The components

Component	Role
Event Producers	Create events. They define what an event is, encapsulate the relevant data, and send it to the Event Bus
Event Bus (message queue)	The backbone. Receives events from producers and transmits them to consumers. **This is what decouples them**
Event Channels	Pathways on the bus. They categorize events by nature, or by the kind of consumers they cater to, so each consumer listens only to the channel relevant to it
Event Consumers	Listen to events via the bus and react. The reaction could be starting a new process, or merely recording the event



Part 5 — Three structural variants

Choose by requirements, complexity and how much control you need.

Variant	How it works	Advantage	Disadvantage
Mediator EDA	Producers send events to a **central mediator** that knows all producers and consumers, decides routing, and performs **aggregation, filtering and transformation** (combining, dropping, reshaping events)	**Control** — complex routing and transformation	The mediator can become a **bottleneck**, limiting scalability
Broker EDA	**No mediator**: producers publish straight to channels; consumers subscribe directly. **The broker *is* the event bus**	**Scalability and simplicity**	Less control over routing and processing
Hybrid EDA	Both — e.g. a mediator for a few complex flows, direct broker delivery for everything else	Mix of **centralized control and high scalability**	Two models to run

---

Part 6 — Inner workings

Event generation and propagation

An event starts with a **change of state** — a user action, a sensor reading, an internal update — packaged as **a bundle of data representing a significant occurrence**, then carried by the **Event Bus** through **Event Channels** that categorize and route by type so the right events reach the right consumers.

Routing

- **Mediator EDA:** a central component routes by event type and known consumers, possibly aggregating, filtering or transforming first.
- **Broker EDA:** no central decision-maker — producers place events in channels, consumers subscribe. Simpler, less controlled.

Event processing and state change

The consumer acts on the event data, **changing system state** — from updating a database value to triggering a physical action in an **IoT** (Internet of Things) device.

How consumers know when to act

Consumers are **always listening** on their subscribed channels and act when an event arrives — usually implemented with the **Observer** (objects register with a subject and are called back on change) or **Publish-Subscribe** patterns.

Real-time illustration — traffic management

Sensors (producers) emit an event per vehicle detected; events flow via the bus to traffic signals (consumers), which **adjust their timing** by vehicle count — changing system state (traffic flow) in real time.

Event Sourcing and CQRS

Event Sourcing stores changes as a **sequence of events**, not just a **snapshot** of current state, giving **full auditability** (proof of what happened when), **versioning**, and **"time-travel"** to any past state. Instead of "balance = £70" you store `Deposited £100`, `Withdrew £30` and **derive** the balance by replaying — how accounting ledgers have always worked.

CQRS separates reads from writes — natural in EDA, where event handlers (writes) are modelled separately from views (reads). Both are covered in depth, including when event sourcing is worth its cost, in CQRS Part 3.

Part 7 — Worked example: traffic management (Java)

The event

```java
public class VehicleEvent {
    private final String vehicleId;
    private final String type;

    public VehicleEvent(String vehicleId, String type) {
        this.vehicleId = vehicleId;
        this.type = type;
    }

    public String getVehicleId() { return vehicleId; }
    public String getType()      { return type; }
}
```

A vehicle 'entering' or 'leaving' a zone. `final` **fields and no setters — the event is immutable**, deliberately: it describes something that **already happened**; letting a consumer modify it is meaningless (you can't change the past) and dangerous (other consumers would see the altered version).

The producer — a sensor

```java
public class Sensor implements EventProducer {
    private EventBus eventBus;

    public Sensor(EventBus eventBus) {
        this.eventBus = eventBus;
    }

    public void vehicleDetected(String vehicleId, String action) {
        VehicleEvent event = new VehicleEvent(vehicleId, action);
        eventBus.publish(event);
    }
}
```

Notice what's absent: the sensor has no reference to any traffic light and doesn't know consumers exist. It publishes and returns — **fire and forget**.

The bus and channels

```
public interface EventBus {
    void publish(VehicleEvent event);
    void addChannel(EventChannel channel);
}

public class SimpleEventBus implements EventBus {
    private List<EventChannel> channels = new ArrayList<>();

    @Override
    public void publish(VehicleEvent event) {
        for (EventChannel channel : channels) {
            channel.add(event);
        }
    }

    @Override
    public void addChannel(EventChannel channel) {
        channels.add(channel);
    }
}

public class SimpleEventChannel implements EventChannel {
    private List<EventConsumer> consumers = new ArrayList<>();

    @Override
    public void add(VehicleEvent event) {
        for (EventConsumer consumer : consumers) {
            consumer.onEvent(event);
        }
    }

    @Override
    public void addConsumer(EventConsumer consumer) {
        consumers.add(consumer);
    }
}
```

Publishing adds the event to every channel; each channel passes it to all its consumers.

The consumer — a traffic light

```
public class TrafficLight implements EventConsumer {
    @Override
    public void onEvent(VehicleEvent event) {
        if ("entering".equals(event.getType())) {
            System.out.println("Vehicle " + event.getVehicleId()
                + " is entering. Adjusting traffic light timings.");
        } else if ("leaving".equals(event.getType())) {
            System.out.println("Vehicle " + event.getVehicleId()
                + " has left. Resetting traffic light timings.");
        }
    }
}
```

What this example simplifies — read before copying it

The roles are right; the mechanics aren't production-shaped:

- **Synchronous and in-process.** `publish` calls consumers **on the caller's thread**, so the sensor waits for every traffic light — destroying the asynchrony that is the main benefit. Real EDA uses Kafka or RabbitMQ with consumers in **separate processes** and the producer returning immediately.
- **No persistence.** If the process dies, **in-flight** events (published, not yet processed) are lost; real buses persist so events survive restarts and can be replayed.

- **No error handling.** One consumer throwing breaks the loop; later consumers never see the event.
- **The bus broadcasts to every channel** instead of routing by type, so channels don't categorize anything here.

Use it to understand **producer, bus, channel, consumer** — not the implementation.

Part 8 — Performance implications and considerations

Scalability

Scale components independently — consumers can't keep up? Add consumers without touching producers. But **keeping parts balanced so none is overwhelmed takes care.**

Processing speed and latency

Async usually boosts performance — producers don't wait. **The flip side is latency** (event-to-handling delay): as volume grows, **the time from production to processing can grow**. The channel choice (synchronous or asynchronous) also affects speed.

Error handling and message durability

When processing fails, you need to **retry, log, or alert an operator**; **message durability** may need **persistent storage** until events are processed. **These mechanisms add complexity and cost performance.**

System complexity

Tracking event flow and debugging are hard in an async, distributed system; you need **special tools and practices**, and every component must agree on event format and meaning.

> ⚠️ **The cost beginners most underestimate.** In a synchronous system a **stack trace** shows what called what. In EDA, causation spans services *and time* — today's bug was caused by an event published twenty minutes ago by a service you weren't watching. You need **correlation IDs** (one identifier threaded through every event in a causal chain) and **distributed tracing** (recording a request's journey across services) from day one, not after your first **incident**. (§ Observability.)

Event schemas and backward compatibility

All components must understand event structure, so events follow a **well-defined schema** (fields, types, which are required). When it changes you need **backward compatibility** so consumers expecting the old schema don't break.

> **Adding an optional field is safe; removing or renaming one is not.** Old consumers ignore unknown fields but break when one they rely on vanishes. **Schema registries** (services storing every schema version and rejecting incompatible changes) enforce this.

Order of events

Events are **asynchronous and may be processed in any order**; dependencies between events need extra machinery to preserve order.

> If `AccountCreated` and `AccountDeleted` arrive swapped, a consumer deletes an account that doesn't exist yet, then creates one that should be gone. Kafka guarantees ordering only *within a partition* (a topic is split into **partitions**, each an ordered log processed in parallel), so choosing the **partition key** (the value deciding an event's partition, e.g. account ID) matters. (§ Event streaming.)

Testing

Unit tests of individual components miss the interactions; you need **integration and end-to-end tests**, which are harder to write and maintain.

None of this should deter you — it's a map of the trade-offs.

Part 9 — Use cases

- **Real-time analytics** — events consumed as generated give instant insight; especially in **finance**, where live market data drives trading decisions.
- **IoT systems** — sensors, smart devices and alarms generate floods of events; EDA's **asynchrony suits them**: a room-temperature rise triggers an event the air conditioning consumes.
- **E-commerce systems** — placing an order triggers events: inventory verifies stock, billing takes payment, shipping handles delivery — **asynchronously and efficiently**.
- **Microservices architecture** — **instead of direct calls (tight coupling), services emit events others consume**, staying loosely coupled and **evolving independently**.

Part 10 — System design example: a ride-sharing app

Uber/Lyft with EDA:

1. **A rider requests a ride** → an event with pickup and destination, **broadcast** to the system
2. **Nearby drivers** consume it
3. **A driver accepts** → an event consumed by the rider's app announcing the match
4. **During the ride** the driver's app streams **current-location** events; the rider's app **updates the map in real time**
5. **The ride ends** → a **ride completion event**
6. **Billing** consumes it, calculates the fare, and **triggers a payment event**

> A ride genuinely *is* a sequence of things that happened. When the domain is already event-shaped, EDA stops being a technical choice and becomes the obvious model.

Part 11 — Common beginner misconceptions

- **"Events are just asynchronous function calls."** A call has one target and expects a return; an event has **zero or many** targets, unknown to the publisher, and expects nothing.
- **"EDA means no more direct calls anywhere."** When you need an immediate answer ("is this user authorized?") a synchronous call is right. EDA is for notification and reaction, not queries.
- **"The event bus makes my system reliable."** It makes it *decoupled*. Reliability needs persistence, retries, **dead-letter queues** (holding areas for messages that keep failing), and **idempotent** consumers on top.
- **"If I publish an event, it definitely gets processed."** Only with **durable** messaging and **acknowledgements** (the consumer confirming it finished). Fire-and-forget over an in-memory bus loses events on restart.
- **"Consumers see each event exactly once."** Most systems guarantee **at-least-once** delivery (every message arrives, possibly more than once), so **duplicates happen** and consumers must be idempotent *(Retry: idempotency)*.
- **"Ordering is guaranteed."** Generally not system-wide — only within a partition or channel, if the **broker** offers even that.

Part 12 — Check your understanding

1. Why are events named in the past tense?
2. What's the change-cost argument for EDA, using the photo upload example?
3. What's the difference between Mediator and Broker EDA, and what does each trade away?
4. Why is debugging harder in EDA, and what two tools address it?
5. Which schema changes are safe and which break consumers?
6. What's wrong with the example's `publish` method, relative to real EDA?

(Answers: 1 — an event records something that already happened and cannot be refused, unlike a command; 2 — adding a new reaction means writing a new consumer without ever modifying the upload service, whereas direct calls require changing and redeploying the producer each time; 3 — Mediator centralizes routing and transformation, gaining control but risking a bottleneck, while Broker publishes straight to channels, gaining scalability but losing routing control; 4 — causation is spread across services and time so there's no single stack trace, addressed by correlation IDs and distributed tracing; 5 — adding optional fields is safe, removing or renaming is not; 6 — it calls consumers synchronously on the producer's thread, so the producer blocks until all consumers finish, eliminating the asynchrony that is the point.)

Part 13 — Interview soundbite (30 seconds)

"In event-driven architecture components publish events rather than calling each other. A producer publishes a state change to a bus, and any number of consumers subscribe to the channels they care about. The producer doesn't know who's listening — that's the point: one photo-upload event triggers follower notifications, a profile update and moderation, and later you add thumbnails by writing a new consumer without touching the upload code. You also get asynchrony and independent scaling of producers and consumers. Three variants: broker, publishing straight to channels — simple and scalable; mediator, a central router that filters and transforms — more control, potential bottleneck; and hybrid. Ride-sharing is a natural fit — request, acceptance, live location, completion are all events. Costs: no system-wide ordering, so pick partition keys; schemas must evolve carefully because removing a field breaks old consumers; delivery is usually at-least-once, so consumers must be idempotent; publish via an outbox so DB and events can't diverge; and debugging is hard because causation spans services and time — so correlation IDs and distributed tracing from the start. It pairs naturally with event sourcing and CQRS."

Part 14 — Implementation

A. Runnable demo

Full source: **demos/EventDrivenDemo.java**

```
cd demos && ./gradlew runCh11
```

Failure isolation — one consumer throwing must not affect the others:

```
for (Subscriber s : subs) {
    Runnable task = () -> {
        try { s.handler().accept(e); }
        catch (Exception ex) {
            System.out.printf("      [%s] FAILED: %s (isolated)%n", s.name(), ex.getMessage());
        }
    };
    if (async) pool.submit(task); else task.run();
}
```

Idempotent consumer, using a set of seen event IDs:

```
void handle(Event e) {
    String eventId = e.payload().get("eventId");
    if (!processed.add(eventId)) return;      // add() is false if already present
    chargesApplied++;
}
```

B. What you should see

```
=== 1. SYNCHRONOUS bus: producer waits for every consumer ===
  producer blocked for 114ms  <- the sum of all consumers

=== 2. ASYNCHRONOUS bus: producer returns immediately ===
  producer returned in 0ms  <- did NOT wait
      [moderation] FAILED: moderation service unavailable (isolated)

=== 4. Add a NEW consumer - producer code untouched ===
    [bus] publishing PhotoUploaded (cid=cid-3) to 4 subscriber(s)
  >> 4 reactions now. The upload service never changed.

=== 5. At-least-once delivery -> idempotent consumers ===
      [billing]    charge applied (total=1)
      [billing]    duplicate evt-abc ignored
      [billing]    duplicate evt-abc ignored
  charges actually applied: 1 (must be 1)
```

114ms → 0ms is the value of asynchrony as a measured number; section 4 is the architectural argument — subscribers went from 3 to 4 and **the producer's code was never edited**.

C. Production — Kafka

`bootstrap-servers` are the first Kafka machines a client contacts; `acks: all` makes the producer wait until every **replica** (copy on another broker) has the message; an **offset** is a consumer's position in a partition, and **committing** it records how far it has read.

```
spring:
  kafka:
    bootstrap-servers: kafka-1:9092,kafka-2:9092,kafka-3:9092
    producer:
      acks: all                          # wait for all replicas - no silent data loss
      retries: 3
      properties:
        enable.idempotence: true         # broker de-duplicates producer retries
        max.in.flight.requests.per.connection: 5
    consumer:
      group-id: notification-service
      auto-offset-reset: earliest
      enable-auto-commit: false          # commit only AFTER successful processing
      isolation-level: read_committed
```

Producer — the partition key is the important decision (`MDC` is the logging library's per-request context, carrying the correlation ID):

```java
@Service
public class PhotoEventPublisher {
    private final KafkaTemplate<String, PhotoUploaded> kafka;

    public void publish(String photoId, String userId) {
        PhotoUploaded event = new PhotoUploaded(
                UUID.randomUUID().toString(),      // eventId, for consumer de-duplication
                MDC.get("correlationId"),          // trace the causal chain
                photoId, userId, Instant.now());

        // Key = userId -> all events for one user land on one partition,
        // and Kafka guarantees ordering WITHIN a partition.
        kafka.send("photo-uploaded", userId, event);
    }
}
```

Consumer — manual acknowledgement plus idempotency (a **consumer group**, named by `groupId`, shares a topic's partitions among its members):

```java
@KafkaListener(topics = "photo-uploaded", groupId = "thumbnail-service")
public void onPhotoUploaded(PhotoUploaded event, Acknowledgment ack) {
    if (processedEvents.existsById(event.eventId())) {   // at-least-once => duplicates happen
        ack.acknowledge();
        return;
    }
    try {
        thumbnailService.generate(event.photoId());
        processedEvents.save(new ProcessedEvent(event.eventId(), Instant.now()));
        ack.acknowledge();                                // commit only on success
    } catch (TransientException e) {
        // Do NOT ack - Kafka redelivers
        throw e;
    }
}
```

Dead-letter topic (DLT) — where messages go after retries are exhausted:

```java
@Bean
public DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
    var recoverer = new DeadLetterPublishingRecoverer(template,
            (record, ex) -> new TopicPartition(record.topic() + ".DLT", record.partition()));
    return new DefaultErrorHandler(recoverer,
            new ExponentialBackOff(1000L, 2.0));   // 1s, 2s, 4s...
}
```

Schema evolution — register schemas so a **breaking change** (one that makes existing consumers fail) is rejected at build time, not discovered in production:

```yaml
spring.kafka.properties:
  schema.registry.url: http://schema-registry:8081
  value.subject.name.strategy: io.confluent.kafka.serializers.subject.TopicNameStrategy
```

Compatibility mode `BACKWARD` means **new consumers can read old events**: adding an optional field passes; removing a field fails the build.

D. Implementation pitfalls

1. Auto-commit enabled. With `enable-auto-commit: true` Kafka commits offsets on a timer — possibly *before* your handler finishes. Crash mid-processing and the message is **silently lost**. Commit manually after success.

2. Consumers that aren't idempotent. At-least-once makes duplicates certain, not hypothetical. Store processed event IDs.

3. No partition key, then expecting ordering. Kafka orders **within a partition only**; without a key, events for one **entity** (e.g. one account) scatter and arrive out of order. Key by entity ID.

4. Publishing before committing the database transaction. Write to the DB, then publish, crash in between: state changed, nobody told. Use the **Transactional Outbox** — explained in Saga 13D.

5. Events carrying too much or too little. Too little and every consumer calls back to the producer, recreating coupling; too much and the event becomes a schema everyone depends on. Carry **identifiers plus the facts that actually changed**.

6. Removing or renaming event fields. Old consumers break. Additive changes only, enforced by a schema registry.

7. No correlation ID. You can't reconstruct what caused what. Generate at the edge, propagate through every event, log on every line.

8. No dead-letter queue. A permanently failing message blocks its partition forever, stalling everything behind it. Always configure a DLT with alerting.

Related patterns

- **CQRS** — separate read/write models, commonly built on EDA
- **Saga** — choreography sagas are EDA applied to distributed transactions
- **Retry** — requeueing and dead-letter queues are retry in an async context
- **Bulkhead** — a queue between components is a natural isolation boundary

CQRS — Command Query Responsibility Segregation

Category: Data / messaging **One-liner:** Split a system's operations into commands (writes) and queries (reads) with separate models, so each can be optimised, scaled, and evolved independently.

Part 0 — Concepts you need first

What is CRUD?

Create, Read, Update, Delete — the four basic data operations. Most applications start here: one `User` class, one `users` table, one set of methods. CRUD is simple and usually right; this chapter is about when it stops being enough.

What is a "model"?

The code and data structures representing part of your **domain** (the business area) — the `User` class, its **validation** rules, its table, the code that loads and saves it. CQRS's central claim: **one model serving both reads and writes ends up serving neither well**, because their requirements are opposed.

Reads vs. writes — why they differ

The intuition the pattern rests on:

	Writes	Reads
Must be	Correct, validated, secure	Fast
Volume	Usually lower	Usually far higher
Complexity	Business rules (domain rules), constraints (database-enforced rules)	Joining (combining tables) and formatting
Cost of error	Severe — corrupt data	Mild — a stale (out-of-date) display
Optimal storage	Normalised (no duplication)	Denormalised (pre-joined)

The last row is the deep conflict. **Normalised** storage keeps each fact once — great for writes (update one row). **Denormalised** storage duplicates data into ready-made shapes — great for reads (no **joins** at query time). **One schema can't be both**; every design compromises — and CQRS says stop compromising, have two.

What is a DTO?

A **Data Transfer Object** — a plain object that just carries data, with no **business logic**. CQRS's read side returns DTOs shaped for the screen rather than domain objects.

What does "side effect" mean?

A state change that persists after a method returns. Reading has none; saving does. CQRS formalises this distinction.

Part 1 — The problem

A **monolith** usually has **one database for everything**, simultaneously serving **complex join queries** and **CUD** (Create, Update, Delete) operations; as the application grows, **handling both becomes really hard to manage.**

The scale mismatch

Complex applications need many **concurrent** operations, diverse data needs and efficient scaling, and plain CRUD struggles. **E-commerce:** thousands of users browsing (**reads**) while inventory updates, orders and new products arrive (**writes**). **Reads are far more numerous; writes carry more complexity** — and with one model you can't scale them independently.

The optimisation conflict — the banking example

- **Transactions** (deposits, withdrawals) — writes that must be **accurate and secure**, since they move money.
- **Balance inquiries** — reads that must be **quick**, in **real time**.

> **The trap:** optimise transactions for speed and security and balance inquiries slow down; optimise for quick balance checks and you compromise transaction efficiency or security. **One model can't be tuned for both** — it becomes hard to maintain, scale, and perform under different kinds of **load**.

Part 2 — The solution

- **Command** — an instruction to *do* something: create, update, delete (a deposit or withdrawal).
- **Query** — a request to *retrieve* data **without changing it** (checking your balance).
- **Responsibility Segregation** — handling commands (writes) separately from queries (reads).

The two sides

The Command Side handles all writes (processing transactions). **The Query Side** handles all reads, optimised for fast retrieval (showing the balance).

The formal rule

Worth memorising — it gives the pattern its rigour. As a **separation of concerns**, **every method is either a query or a command, never both:**

	Returns a value?	Changes state?
Query	✅ Returns a result	❌ No state change, no side effects
Command *(also called modifiers or mutators — methods that change data)*	❌ Only an indication of success or failure	✅ Changes system state

> **Why "never both" matters.** A method that changes state *and* returns data can't be called safely: you can't retry it (it might apply twice), can't **cache** it, can't reason about it. The split makes queries freely repeatable and cacheable, and commands explicitly consequential.

Why this is useful

- **Optimization** — tune each side: command side for security and accuracy, query side for speed.
- **Simplicity** — each side developed and maintained separately.
- **Scalability** — scale reads and writes independently by demand.
- **Flexibility in technology choice** — different **stacks** per side: a **relational database** (tables linked by keys, SQL) with constraints and **transactions** for writes; a **document store** (self-contained JSON-like documents) or **search index** (e.g. Elasticsearch) for reads.

In essence: reading and writing have different needs, and CQRS lets you treat them separately.

Part 3 — Architecture: three levels

CQRS is a spectrum, and the costs differ enormously by level — always say which you mean.

Level 1 — Separate models, one database

The Command Model handles writes and guards the **integrity and consistency** of data: receives commands, validates, applies business rules, changes state. **The Query Model** handles reads, returning current state efficiently. (The course's image: a disciplined librarian shelving every book correctly vs. a friendly one fetching what you need.)

The simplest useful form — two models, one database. **Most teams should stop here:** most of the design clarity, almost none of the operational cost.

Level 2 — Separate read and write databases

Separate databases add flexibility and independent scaling, but

> ⚠️ **significantly increase complexity**: you must propagate changes from the write database to the read database.

The problems it creates:

- **No atomic transactions across the read and write models** — so you rely on **asynchronous messaging and events** for **eventual consistency** (the read side catches up shortly — § Consistency models)
- **Out-of-order event propagation**
- **Event loss**
- **Sync discrepancies**
- **Read Model update failures**
- **The timing of UI updates after writing** — what does the user see right after hitting save, if the read model hasn't caught up?

It demands **a deep understanding of distributed transactions** and suits you **only if your non-functional requirements** (speed, scale, reliability — not features) **genuinely demand it.**

Level 3 — Event Sourcing

Exists to solve one Level 2 problem: **ordering**. With separate databases kept in sync eventually, **the sequence of events from write side to read side is crucial**; rapid successive writes **received out of order leave the read model outdated** — and **many asynchronous message buses prioritise availability and performance and don't guarantee delivery order.**

Event Sourcing stores the write model as an **append-only event store** (events only ever added, never edited or deleted) recording every action; a model's state is rebuilt by **replaying its event history**. Each write-model instance is an **independent event stream**, which enables:

- **Various data views**, by replaying events differently
- **Rebuilding read models** from scratch from the write side's events — the direct fix for sync problems
- **Simplified auditing** — the streams record every change to every model
- **New read models** built by replaying past events — new views of history that never existed before

> **However:** event sourcing **adds complexity and is a significant undertaking**, especially if you're new to CQRS, event sourcing, or distributed systems. Don't adopt Level 3 because you adopted Level 1 — separate decisions with very different costs.

Part 4 — Inner workings

The journey of a command

A command expresses an **intention to change state** — e.g. `UpdateUserAddress`:

1. **Validation** — **data format, permissions, business rules**: is the address valid, may this user change it? Fail → **rejected**.
2. **Handling** — the **command handler**, the command side's workhorse, updates state.
3. **Generating events** — the change is represented as an **event**, `UserAddressUpdated`, stored in the **event store** as a historical record.

The query's quest

`GetUserAddress` is just a data request. **Queries need no validation and generate no events** — the **query handler** fetches and returns data.

> That asymmetry follows directly from the formal rule: a query changes nothing, so there's nothing to validate or record — which is why the read path can be much simpler and faster.

Synchronising the two sides

Event Sourcing keeps them in step: the `UserAddressUpdated` event is **replayed on the query side**, updating its state, so the separately-run command and query sides **stay in sync**.

Part 5 — Worked example: user profile management

Create a user (command), **update their email** (command), **view user details** (query).

The pieces

Side	Piece	Role
Command (write) model	**Command class**	An action to perform, e.g. `CreateUserCommand` (`userId` , `name` , `email`)
	Command handler	Takes the command, runs business logic, persists changes
	Domain model	The state of the data — the `User` class
	Repository	Data **persistence** (saving and loading)
Query (read) model	**DTO**	The data to read, e.g. `UserProfileDTO`
	Query class	A query, e.g. `GetUserProfileQuery` (`userId`)
	Query handler	Returns the data, often from **read-optimised storage**
Infrastructure	**Command bus / Query bus**	Route commands / queries to their handlers
	Event store	In advanced setups, stores the events resulting from commands

Command classes

```
public class CreateUserCommand {
    private String userId;
    private String name;
    private String email;
}

public class UpdateUserEmailCommand {
    private String userId;
    private String newEmail;
}
```

Just data — intent expressed as an object, with no behaviour. Making intent an explicit object is part of the point: you can log it, queue it, or replay it.

Command handlers

```
public class CreateUserCommandHandler {
    private UserRepository userRepository;

    public CreateUserCommandHandler(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public void handle(CreateUserCommand command) {
        User user = new User(command.getUserId(), command.getName(), command.getEmail());
        userRepository.save(user);
    }
}

public class UpdateUserEmailCommandHandler {
    private UserRepository userRepository;

    public UpdateUserEmailCommandHandler(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public void handle(UpdateUserEmailCommand command) {
        if (!isValidEmail(command.getNewEmail())) {
            throw new IllegalArgumentException("Invalid email format");
        }
        User user = userRepository.findById(command.getUserId());
        if (user != null) {
            user.setEmail(command.getNewEmail());
            userRepository.save(user);
        }
    }

    private boolean isValidEmail(String email) {
        return email != null && email.contains("@");
    }
}
```

Notice: `handle` returns `void` — commands don't return data (the formal rule). And **validation lives here**, before any state change — the write side's "accuracy and security" focus made concrete.

Domain model and repository

```
public class User {
    private String id;
    private String name;
    private String email;
}

public interface UserRepository {
    void save(User user);
    User findById(String id);
}
```

The read side

```
public class UserProfileDTO {
    private String id;
    private String name;
    private String email;
}

public class GetUserProfileQuery { private String userId; }
public class ReadUserEmailQuery  { private String userId; }
```

```
public class GetUserProfileQueryHandler {
    private UserRepository userRepository;

    public UserProfileDTO handle(GetUserProfileQuery query) {
        User user = userRepository.findById(query.getUserId());
        return new UserProfileDTO(user.getId(), user.getName(), user.getEmail());
    }
}

public class ReadUserEmailQueryHandler {
    private UserRepository userRepository;

    public String handle(ReadUserEmailQuery query) {
        User user = userRepository.findById(query.getUserId());
        return user != null ? user.getEmail() : null;
    }
}
```

Granularity: `ReadUserEmailQuery` returns just a `String`. Read models can be **as narrow as the screen needs** — each query gets a purpose-built shape instead of one bloated shared object. And handlers return **DTOs, not domain objects**: the read side doesn't expose the write model.

Infrastructure

```
public class SimpleCommandBus implements CommandBus {
    private CreateUserCommandHandler handler;

    @Override
    public void dispatch(CreateUserCommand command) {
        handler.handle(command);
    }
}

public class SimpleQueryBus implements QueryBus {
    private GetUserProfileQueryHandler handler;

    @Override
    public UserProfileDTO dispatch(GetUserProfileQuery query) {
        return handler.handle(query);
    }
}

public class SimpleEventStore implements EventStore {
    private List<UserEvent> events = new ArrayList<>();

    @Override
    public void storeEvent(UserEvent event) {
        events.add(event);
    }
}
```

The buses route to handlers; the event store captures changes that, in a full application, can be **replayed to rebuild an entity's state** (an entity = one business object, e.g. one user).

> **Where this sits: Level 1** — both sides share `UserRepository` and one database; the event store is unused by reads. Not a flaw: Level 1 is where most real systems should live and already delivers CQRS's main benefit, design clarity.

Part 6 — Issues, considerations, and performance

Complexity

Maintaining two models is genuinely complex, a roadblock for developers new to the pattern, with **a steep learning curve**.

Managing data consistency

The scenario: the command model changes before the query model is updated; **a query arriving in that interval returns outdated data.**

> The central trade-off of CQRS — raise it in interviews. In UI terms: a user saves their profile, the page reloads, and the old name is still there. Mitigations: the UI displays **optimistically** (shows the expected result before the server confirms), or reads right after a write go to the write model (Part 12C).

Handling transactions

The scenario: several commands form one logical transaction and one fails — **how do you roll back the earlier commands' changes?** Commands change state, and there's no simple erase. *(Exactly what Saga addresses with compensating transactions.)*

Dealing with asynchronicity

CQRS often brings async operations, making **error handling and debugging harder** — **finding an error's source is challenging**.

The performance upside

Segregating commands and queries lets you optimise each side; if queries far outnumber commands, **give the query side more resources** (more trains on the busiest routes).

Part 7 — Use cases

- **High-performance applications** — the most compelling case: **reads far outweigh writes**, e.g. millions browsing products vs. far fewer purchases; optimise for the read volume.
- **Complex business logic** — separating responsibilities simplifies **state-changing logic** (division of labour on an assembly line): the write model can be rich with rules and **invariants** (conditions that must always hold, e.g. "balance never negative") without slowing or complicating reads.
- **Collaborative domains** — **many users updating the same data** (a shared document). **Treating updates as commands that can fail or be rejected by rules** makes conflicts manageable.

Part 8 — System design examples

Event-driven microservices

Orders move through **placed, paid, shipped, delivered** — each a **command-driven state change** — while other microservices **query** order state (shipment tracking, customer notifications). CQRS gives a cleaner, more efficient design.

Real-time analytics

A social platform where users constantly **post updates (commands)** while the platform serves **real-time engagement analytics (queries)**. Their needs are opposite: **commands must be fast and reliable**; **queries need heavy computation but aren't time-critical**. Segregation **allocates resources to each need** — a textbook fit, since the two **workloads** would harm each other sharing one model.

Part 9 — Common beginner misconceptions

- **"CQRS requires two databases."** That's Level 2. Level 1 — two models, one database — is where most systems should live.
- **"CQRS requires event sourcing."** Frequently paired, but independent choices; CQRS can have no events at all.
- **"CQRS is the same as read replicas."** **Read replicas** are synced copies of the *same* database to spread read **load**; CQRS is a *different model* shaped for reading. You can use both.
- **"Commands are just HTTP POSTs."** A command is a design concept, not a **transport** detail — an explicit object representing intent, which is what lets you queue, log and replay it.
- **"It's always faster."** It adds **indirection**. Faster when read and write workloads genuinely conflict; pure overhead when they don't.
- **"Apply it to the whole application."** Almost always wrong. Apply it where read and write needs genuinely diverge; leave simple CRUD as CRUD.

Part 10 — Check your understanding

1. Why can't one database schema be optimal for both reads and writes?

2. What are the two halves of the formal rule, and why does "never both" matter?
3. Which level of CQRS should most teams adopt, and why?
4. What specific problem does event sourcing solve that Level 2 has?
5. What is the central trade-off, expressed as something a user would notice?
6. Why is the social media analytics example a textbook fit?

(Answers: 1 — writes want normalised storage where each fact lives once, reads want denormalised pre-joined shapes, and these are directly opposed; 2 — queries return data and change nothing, commands change state and return only success or failure; "never both" makes queries safely repeatable and cacheable and makes commands explicitly consequential; 3 — Level 1, two models over one database, because it gives the design clarity with almost none of the operational cost; 4 — out-of-order event delivery leaving the read model stale, solved by replaying an append-only event stream; 5 — the consistency window: a user saves their profile and the page still shows the old value; 6 — posting must be fast and reliable while analytics is computation-heavy but not time-critical, so the two workloads would degrade each other if they shared a model.)

Part 11 — Interview soundbite (30 seconds)

"CQRS separates writes from reads — commands that change state and return nothing, queries that return data and change nothing. Reads and writes usually have opposite needs: in banking, transactions need accuracy and security while balance inquiries need speed, and at the storage level writes want normalised data while reads want denormalised, pre-joined shapes — one model can't be tuned for both. It's a spectrum. Level 1 is two models over one database — where most systems should stop: design clarity, almost no operational cost. Level 2 is separate read and write databases: independent scaling, but no atomic transactions across them, so eventual consistency via async events with out-of-order delivery and lost messages to handle. Level 3 is event sourcing: the write side is an append-only event log and read models are rebuilt by replay — fixing ordering and giving audit history and brand-new read models from past events, but a big undertaking and a separate decision. The main downside throughout is the consistency window — save your profile and still see the old name — plus complexity and harder debugging. It pairs with event-driven architecture, and with sagas when one logical transaction spans several commands."

Part 12 — Implementation

A. Runnable demo

Full source: `demos/CqrsDemo.java`

```
cd demos && ./gradlew runCh12
```

The formal rule made mechanical — command handlers return `void`:

```
void handle(ChangeEmail cmd) {                    // <- returns void
    User u = writeStore.get(cmd.userId());
    if (!cmd.newEmail().contains("@")) throw new IllegalArgumentException("invalid email");
    String old = u.email;
    u.email = cmd.newEmail();
    publish(new UserEvent.EmailChanged(cmd.userId(), old, cmd.newEmail()));
}

UserProfileView get(String id) { return store.get(id); }   // <- returns data, changes nothing
```

Rebuilding a read model from the event log — the Level 3 superpower:

```
fast.clear();                              // wipe the read model entirely
eventStore.all().forEach(fast::apply);     // replay history
```

B. What you should see

```
=== 3. Level 2: separate read store -> CONSISTENCY WINDOW ===
  write store immediately: carol@new.com
  read model immediately:  carol@old.com   <- STALE
  read model after 300ms:  carol@new.com   <- converged

=== 4. Several read models from ONE write ===
  profile view : UserProfileView[id=u4, name=Dave, email=dave@x.com]
  directory view: [UserDirectoryEntry[name=Dave, initial=D], ...]

=== 5. Level 3: rebuild the read model by replaying events ===
  after wiping the read model:  null
  after replaying the log:      UserProfileView[id=u1, name=Alice, ...]
```

Section 3 is the trade-off to describe in an interview, made observable: the write store has the new email while the read model still has the old — not a bug, the deal you accepted. Section 5 is the compensating power: **read models are disposable**, because the event log is the truth.

C. Production — Axon Framework

(**Axon**: a Java framework for CQRS and event sourcing. An **aggregate** is a cluster of objects changed as one unit — here, one user; annotations such as `@CommandHandler` map messages to methods.)

```java
@Aggregate
public class UserAggregate {

    @AggregateIdentifier private String userId;
    private String email;

    @CommandHandler
    public UserAggregate(CreateUserCommand cmd) {
        if (!cmd.email().contains("@")) throw new IllegalArgumentException("invalid email");
        AggregateLifecycle.apply(new UserCreatedEvent(cmd.userId(), cmd.name(), cmd.email()));
    }

    @CommandHandler
    public void handle(ChangeEmailCommand cmd) {
        if (cmd.newEmail().equals(this.email)) return;              // no-op, no event
        AggregateLifecycle.apply(new EmailChangedEvent(cmd.userId(), this.email, cmd.newEmail()));
    }

    // State is derived by applying events - never assigned directly
    @EventSourcingHandler
    public void on(UserCreatedEvent e) { this.userId = e.userId(); this.email = e.email(); }

    @EventSourcingHandler
    public void on(EmailChangedEvent e) { this.email = e.newEmail(); }
}
```

The projection — the read side (a **projection** builds a read model by applying events):

```java
@Component
public class UserProfileProjection {

    private final UserProfileRepository repo;

    @EventHandler
    public void on(UserCreatedEvent e) {
        repo.save(new UserProfileView(e.userId(), e.name(), e.email()));
    }

    @EventHandler
    public void on(EmailChangedEvent e) {
        repo.findById(e.userId()).ifPresent(v -> { v.setEmail(e.newEmail()); repo.save(v); });
    }

    @QueryHandler
    public UserProfileView handle(FindUserProfileQuery q) {
        return repo.findById(q.userId()).orElseThrow();
    }
}
```

Plain Spring, no framework — Level 1, where most systems should live (**JPA**, the Java Persistence API, maps classes to tables; a **native query** is hand-written SQL):

```java
// Write side: JPA against a normalised schema
public interface UserRepository extends JpaRepository<User, String> {}

// Read side: a projection interface or native query against a denormalised view
public interface UserProfileReadRepository extends Repository<UserProfileView, String> {
    @Query(value = """
        SELECT u.id, u.name, u.email, COUNT(o.id) AS order_count
        FROM users u LEFT JOIN orders o ON o.user_id = u.id
        WHERE u.id = :id GROUP BY u.id
        """, nativeQuery = true)
    UserProfileView findProfile(@Param("id") String id);
}
```

> **You don't need Axon, Kafka or event sourcing to do CQRS.** Two repositories and two model classes over one database is legitimate CQRS and delivers most of the benefit.

Handling the consistency window in the UI — three practical options:

```java
// 1. Read-your-own-writes: route reads right after a write to the write model
if (recentlyWrittenBy(userId)) return writeModel.find(userId);
return readModel.find(userId);

// 2. Return the new state from the command endpoint so the UI can render optimistically
// 3. Return a version token; the client polls the read model until it catches up
```

(A **version token** is a number identifying the write; to **poll** is to ask repeatedly until ready.)

D. Implementation pitfalls

1. Jumping straight to Level 3. Event sourcing is a large commitment — event schema evolution, **snapshotting** for long streams, replay tooling. Adopting it *because* you adopted CQRS is the most common way teams get into trouble. Start at Level 1.

2. Commands that return data. Once `handle()` returns the updated entity the rule collapses and you're back to one model. Return void, or at most an ID.

3. Business logic in the read model. Projections only *shape* data; any deciding rule belongs on the write side, or the sides will disagree.

4. Ignoring the consistency window in the UI. Users who save and see stale data file bugs. Design for it with one of the three techniques above.

5. Not being able to rebuild projections. Disposable read models are event sourcing's main benefit — only if replay is *tested*. Make rebuilding routine, not theoretical.

6. Unbounded event streams. An aggregate with 100,000 events takes forever to load by replay. Use **snapshots** — persist state every N events and replay only from there.

7. Applying CQRS everywhere. Simple CRUD entities stay CRUD. Apply it where read/write needs genuinely diverge, and be able to say *why* for each place.

8. Events named as commands. `UpdateEmail` is a command; `EmailChanged` is an event. Wrong tense signals events that are disguised instructions, reintroducing **coupling** (Event-Driven Part 0).

Related patterns

- **Event-Driven Architecture** — the substrate that syncs command and query sides
- **Saga** — for transactions spanning multiple commands, where rollback isn't automatic
- **BFF** — read models shaped per client is the same instinct as BFF applied to data

Configuration Externalization Pattern

Category: Operations **One-liner:** Move configuration out of application code and deployment packages into a centralized store that services fetch at **runtime** (while they are running) — so changing config doesn't require rebuilding and **redeploying** (releasing a new build to the servers).

Part 0 — Concepts you need first

What is "configuration"?

Values that control how an application behaves but **aren't its logic**:

- Database **connection strings** (address and options for connecting) and **credentials** (usernames, passwords)
- **API keys** (secret tokens identifying your program to another service) and **endpoint** URLs of external services
- **Timeout** durations, retry counts, **thread pool** sizes
- Feature flags — switches turning functionality on and off
- Region-specific values: currency, tax rate, language

The test: would you want a different value in development vs production, or one region vs another, without changing code? Then it's configuration.

What does "hardcoded" mean?

Written directly into source:

```
String dbUrl = "jdbc:postgresql://prod-db-01:5432/orders";
```

Changing it means editing source, **recompiling** and redeploying; the value is **frozen into the built artifact**.

What is a "deployment package"?

The bundle you ship — a **JAR** (packaged Java program), a **container image** (an app plus everything it needs, used to start containers), a zip. **Bundled configuration** travels *inside* it: better than hardcoding (no recompile), but **still frozen at build time** — changing it means rebuilding and redeploying.

What is a "deployment cycle"?

Build → test → review → deploy → verify: maybe an hour in a well-run organisation, days in a regulated one. **The chapter's core complaint:** that entire process, for changing one number.

What is an environment?

A separate copy of the system for a purpose — **development**, **staging/testing** (rehearsal), **production** (live users). Same code, different configuration: databases, API keys, scale.

What is a secret?

Configuration that must not be disclosed — database passwords, API keys, **encryption keys**. **Secrets are configuration with much stricter handling requirements** — a recurring theme below.

Part 1 — The framing

Would an architect carve every building's blueprint into its walls, or keep blueprints separate, accessible and modifiable? The same applies to software. In **microservices**, **each service has**

configuration governing its behaviour — network settings, database parameters, feature toggles — traditionally hardcoded or bundled. **So what happens when you need to change one parameter? A whole deployment cycle for a minor tweak?**

Part 2 — The problem

Volume and diversity

Many services, possibly in **different languages and technology stacks**, each with its own conventions for where config lives and how it's read — managing it all quickly becomes a nightmare.

The drawbacks of hardcoding

Hardcoding ties configuration **tightly to code**:

- **Any change, however minor, needs a new application version and a full redeployment**
- It **slows the adaptability** of your services
- **It introduces risk**: a config error in production means **another build-and-deploy round** — wasted resources and, at worst, **significant downtime**

> A typo in a database **hostname** is a five-second fix conceptually, but a full deployment cycle in practice — done while production is broken.

The burden of scale

In a **monolith** configuration is usually **centralized in one place**; in microservices each service maintains its own. That **amplifies the workload** and **the risk of inconsistencies and errors** — imagine changing the blueprint of every building in a city.

> You **rotate** a database password (replace it, as routine security hygiene) and update fourteen services. You miss one. It keeps working until its **connection pool** (already-open, reused connections) recycles — possibly hours later — then fails in a way that looks unrelated to anything you did.

Inflexibility of deployments

With hardcoded or bundled config, **each minor change may force a full redeployment**, which **negates independent deployment** — microservices' main selling point — and breeds **configuration inconsistencies across environments** (dev, test, prod).

Inability to adapt to dynamic environments

Optimal configuration shifts with conditions: **network settings** with **traffic patterns**, **feature toggles** with **user behaviour**. Traditional config makes **real-time** response nearly impossible — services are **stuck with outdated configuration until the next deployment**.

Security implications

Hardcoding risks **exposing or leaking sensitive information** — database credentials, API keys.

> The usual way: a credential hardcoded in source is **committed** to **version control** (e.g. **Git**, which records every change, who made it, and when) and now lives in the **repository**'s history **forever**, readable by everyone with access. Rewriting Git history is painful and rarely complete; **rotating the credential** is the only real remedy.

Part 3 — The solution

Apply **separation of concerns** to configuration: **externalize it from application code** into a **separate, centralized location** (the blueprints kept in a central town hall). Configuration is no longer in the deployment package; it lives in a **configuration server** (a service whose only job is to hand out configuration) or a network-accessible file, and the service **retrieves it at runtime**, always getting current values.

Why this is advantageous

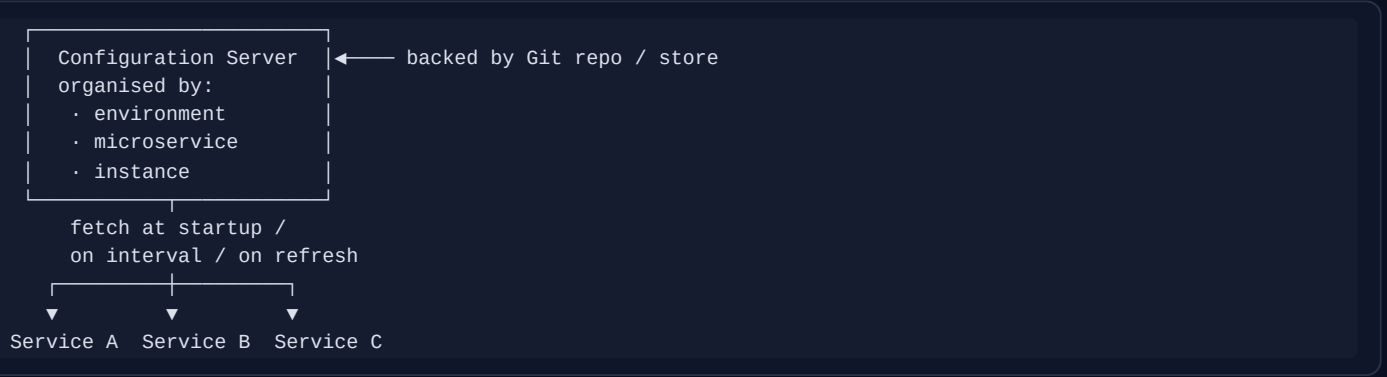
- **One unified location** — solves volume and diversity: **whatever a service's language or stack, configuration comes from the same place**.
- **Support for dynamic environments** — fetched at runtime, so it **updates in real time**; no waiting for the next deployment.
- **Preserved independent deployment** — config changes **no longer need a redeploy**; they're updated centrally and available immediately.
- **Enhanced security** — secrets are **no longer scattered across services**; central storage makes **security policies and controls easier to enforce**.
- **Efficient scalability** — a new service's configuration just joins the central store; no touring every service.
- **Consistency across environments** — all environments' configuration is **centrally managed**, reducing errors when promoting dev → test → prod.

Part 4 — Architecture

Two components: **the microservice** and the **configuration server** (or file).

The interaction

The service, **at startup or at pre-defined intervals**, asks the server for its configuration — **connection settings, API keys, any environment-specific data** — and the server returns what that service needs. This happens **dynamically, at runtime**, so the service adapts **without a restart or redeployment**, reducing downtime and increasing **agility**.



The configuration server

The nerve centre: it stores, retrieves and manages everyone's configuration, organised **by environment** (dev, staging, prod), **by microservice**, and **by instance** (one running copy). That organisation is what fixes traditional config management — it's not just centralized, it's *navigable*.

Dynamic adaptability in practice

Database connection parameters suddenly change: **instead of redeploying**, you update the config server, and the service **receives the new parameters on its next fetch and carries on seamlessly.**

Security — safeguarding sensitive configuration

Not every "book" is open to everyone. The server protects sensitive values through **encryption** and **access controls** (who may read or change what); centralization makes **strict policies easier to enforce**, reducing accidental exposure and attack risk.

Scalability and consistency

New services just add their configuration to the server. And you **switch configuration as you move through environments** — no environment-specific code, no mixed-up configurations.

Part 5 — Worked example: Spring Cloud Config

Spring Cloud Config (part of Spring Cloud, the microservices toolkit on the Spring Java framework) is widely adopted in Java for centralized configuration, for its simplicity.

The configuration server

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServer {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServer.class, args);
    }
}
```

The `@EnableConfigServer` **annotation** turns an application into a configuration server in a few lines.

The configuration store — a Git repository

Tell the server where configuration lives, in a **properties file** (plain `key=value` settings):

```
spring.cloud.config.server.git.uri=https://github.com/yourusername/config-repo
```

> **Using Git is genuinely clever.** A version-controlled config store gives you, free: full history of every change and its author, **diffs** of exactly what changed, **pull requests** (proposed changes reviewed before merging) and **code review** *for configuration*, and **rollback to any previous state** with a standard Git operation. Configuration becomes a first-class engineering artifact, not a value someone typed into a form.

The client setup

Add the Spring Cloud Config Client **dependency** to the service's **build file** (`pom.xml` or `build.gradle`) and annotate the main class with `@EnableConfigClient` — it now consumes configuration from the server.

Pulling configuration — the `@Value` annotation

```
@Value("${my.property}")
private String myProperty;
```

`@Value` fetches `my.property` from the store into the field. **The code knows only a property name** — not a location or mechanism.

Refreshing without restarting

```
POST /actuator/refresh
```

A **POST** to this endpoint, provided by **Spring Boot Actuator** (Spring's management module), makes the client **re-fetch configuration** and apply it **without restarting**. The whole pattern in one endpoint:

change centrally, refresh, done.

Encryption and decryption

```
encrypt.key=MySuperSecretKey
```

Sets the key for encrypting/decrypting properties, via the `/encrypt` and `/decrypt` endpoints.

> ⚠️ **Notice the irony.** `encrypt.key` is itself a secret, sitting in a plain properties file — you've encrypted every secret and left the key in the open. In production the master key belongs in a dedicated **secrets manager** (built to store, issue and rotate secrets) — **HashiCorp Vault, AWS Secrets Manager, or a cloud KMS** (key management service, holding keys in protected hardware) — fetched by the config server at startup. **The pattern relocates the secret problem; it doesn't eliminate it.** For secrets, a purpose-built manager usually beats a general config server.

Part 6 — Considerations and implications

Data security

The pattern aids security but **adds a new attack surface**: sensitive data now stored **remotely**. How secure is the store, and the transfer? Beyond the `MySuperSecretKey` example: **secure, encrypted connections** and **regularly rotated keys**.

The configuration server as a single point of failure

Centralizing **introduces a single point of failure**:

> **If the config server goes down, every microservice is paralysed — unable to fetch or refresh configuration.**

Mitigations:

- **Redundancy** — **multiple instances** in a **high-availability configuration**, with **regular health checks**.
- **Client-side caching of last-known-good configuration** — each service keeps the last values it fetched successfully. This is what makes the trade acceptable: you swapped "config changes need a redeploy" for "a config outage could take everything down", and caching turns the second from fatal to inconvenient because **running services keep their current values**. It's the same trick, explained in full, as in Service Discovery Part 7.3.

Configuration versioning and rollbacks

Faster rollouts raise the question: **can you revert quickly when something goes wrong?** Storing configuration in **Git** means you can **roll back to a previous state** — but only with **good practices**: documented changes, correctly labelled versions. Git provides the capability; discipline makes it usable at 3am during an **incident**.

Service discovery and the config service

Services come and go with **load** — **how does a new service find the config service? Hardcoding its address isn't scalable** — and is ironic: hardcoding a location to avoid hardcoding configuration. Use **service discovery** — **Netflix's Eureka** or **HashiCorp's Consul** find services dynamically, **including the configuration server** *(Service Discovery)*.

Performance implications

Remote fetching adds latency, especially under server load or network issues. **Caching helps** — Spring Cloud Config clients **cache remote properties by default** — but raises its own questions: **how often to**

refresh? (too often hammers the server; too rarely delays changes) and **what memory cost?** A balance between **performance and freshness**.

Part 7 — Real-world use cases

Case	Situation	How the pattern helps
E-commerce platform	Inventory, Order, User Management — **hundreds of services across production, staging and development**, each with databases and APIs	All configuration on a central server; **changes made in one place, reflected across the system**
Financial services firm	Highly regulated; **API keys and database passwords must not be exposed**	Sensitive config stored centrally, **possibly encrypted**, fetched **over a secure (encrypted) channel**; **nothing sensitive in the codebase or containers** — what **auditors** want to see
Global fast-food chain	Outlets in many countries, each with **its own tax laws, currency and language**	At boot each app **fetches its country's configuration** — per-country behaviour from one shared codebase
Tech startup	Streaming service, **agile** (short cycles, small frequent releases) with **multiple deployments a day**, each needing **feature flags or A/B testing parameters**	Services fetch updated config when needed — **faster, smoother deployments**
Telecommunications giant	Many services managing **bandwidth, routing information and security protocols**	All network configuration central; each service fetches what it needs

---

Part 8 — Common beginner misconceptions

- **"Externalizing config means putting it in a file next to the app."** That's still bundled. Externalized means **outside the deployment artifact**, retrieved over the network.
- **"A config server is the right place for secrets."** Better than hardcoding, but a dedicated secrets manager (Vault, AWS Secrets Manager) adds **leasing** (secrets issued for a limited time, then expiring), automatic rotation, fine-grained access policies and **audit logs** that a config server lacks.
- **"Environment variables are the same thing."** An **environment variable** (a named value the OS passes a program at start) is a good, simple form of externalization and often sufficient — but gives **no central management, versioning, or refresh without restart**.
- **"If the config server is down, everything stops."** Only without client-side caching; with it **running services keep running** — you just can't start new ones or push changes.
- **"Config changes are low risk because there's no deploy."** The opposite: no deploy means **no build, no tests, no review** unless you add them — hence Git-backed stores with required pull requests.
- **"One config server for everything."** Separate by **trust boundary** (parts of the system that shouldn't automatically trust each other); not every service should read every other's config.

Part 9 — Check your understanding

1. What's the difference between hardcoded, bundled, and externalized configuration?
2. Why does hardcoded configuration undermine the main benefit of microservices?
3. What does backing the config store with Git give you for free?
4. What's the central irony of the `encrypt.key` property?
5. Why do you need service discovery in order to do configuration externalization well?

6. What makes the config server's single-point-of-failure risk acceptable?

*(Answers: 1 — hardcoded is in source and needs recompiling; bundled is in the deployment package and needs rebuilding and redeploying; externalized lives outside the artifact and is fetched at runtime; 2 — independent deployability, because a one-line config change forces a full redeploy of the service; 3 — change history, authorship, diffs, code review, and rollback to any previous state; 4 — it's a secret protecting all other secrets, stored in **plaintext** (unencrypted), so it belongs in a real secrets manager; 5 — hardcoding the config server's address in every service would reintroduce the exact problem you're solving; 6 — client-side caching of last-known-good values, so running services survive a config server outage.)*

Part 10 — Interview soundbite (30 seconds)

"Configuration externalization moves config out of code and deployment packages into a central config server that services fetch at runtime. Hardcoded config ties every change to a full build-and-deploy cycle, destroying the independent deployability that's the point of microservices, and stops you reacting to conditions until the next release; it also scatters secrets across services and into version-control history. With a config server you organise config by environment, service and instance; services fetch at startup or on an interval, and Spring Cloud Config's actuator refresh endpoint applies changes without a restart. It's typically backed by Git — the clever part — giving versioning, diffs, code review and rollback for free. Trade-offs: the config server is a single point of failure, so it needs HA plus client-side caching of last-known-good values; it's a new attack surface holding your secrets, so encryption in transit and key rotation matter; and services need service discovery to find it, since hardcoding its address reintroduces the problem. For secrets I'd use a dedicated manager like Vault — leasing, rotation and audit logging."

Part 11 — Implementation

A. Runnable demo

Full source: `demos/ConfigExternalizationDemo.java`

```
cd demos && ./gradlew runCh13
```

Layered resolution — later layers override earlier ones:

```
Map<String, String> fetch() {
    Map<String, String> merged = new LinkedHashMap<>();
    layers.values().forEach(merged::putAll);   // default <- environment <- instance
    return merged;
}
```

The cache that survives a server outage:

```
void refresh() {
    try {
        cache = server.fetch();
        cachedVersion = server.version();
    } catch (IllegalStateException e) {
        // server unreachable - KEEP the previous config rather than failing
    }
}
```

And what makes runtime flips work — read config **at use time**, not at startup:

```
String checkout() {
    return config.flagEnabled("feature.newCheckout")   // read on every call
            ? "NEW checkout flow" : "legacy checkout flow";
}
```

B. What you should see

```
=== 3. Change a value + refresh, NO restart ===
  before refresh: ... timeout=1000ms ...
  after refresh:  ... timeout=5000ms ...
  >> same running process, new value

=== 4. Layering: instance overrides environment overrides default ===
  ... timeout=9000ms ...
  >> instance layer won (9000), overriding environment (5000)

=== 5. Feature flag flipped centrally ===
  checkout now: legacy checkout flow
  checkout now: NEW checkout flow
  and svc2 too: NEW checkout flow
  >> every service flipped at once, with no deployment

=== 6. Config server outage ===
    [client] server DOWN - keeping cached config (v8)
  checkout still: NEW checkout flow
  >> running services keep working on cached config
```

Section 5 is the payoff — one central change flipped every running service at once, with no build or deploy. **Section 6 is the safety net** that makes centralizing acceptable.

C. Production — Spring Cloud Config

The server (same `@EnableConfigServer` class as Part 5, here named `ConfigServerApplication`), configured in YAML:

```yaml
server.port: 8888
spring:
  cloud.config.server.git:
    uri: https://github.com/acme/config-repo
    default-label: main
    clone-on-start: true
    search-paths: '{application}'          # per-service folders
  security.user:
    name: configuser
    password: ${CONFIG_SERVER_PASSWORD}    # itself injected, not committed
```

The repository layout — this gives you environment layering:

```
config-repo/
├── application.yml                # defaults for EVERY service
├── application-prod.yml           # defaults for every service in prod
├── order-service.yml              # all environments of one service
├── order-service-dev.yml
└── order-service-prod.yml         # most specific: wins
```

The client:

```yaml
spring:
  application.name: order-service          # picks which files apply
  config.import: "optional:configserver:http://config-server:8888"
  cloud.config:
    fail-fast: true                        # don't start with wrong config
    retry: { initial-interval: 1000, max-attempts: 6, multiplier: 1.5 }
```

Refresh without restart — the crucial bit is `@RefreshScope` (a **bean** is an object Spring creates and manages; `curl` sends HTTP requests from the command line):

```java
@RefreshScope                              // bean is recreated on refresh
@Component
public class CheckoutService {

    @Value("${feature.newCheckout:false}")
    private boolean newCheckoutEnabled;

    public String checkout() {
        return newCheckoutEnabled ? newFlow() : legacyFlow();
    }
}
```

```
curl -X POST http://order-service:8080/actuator/refresh
```

For **fleet**-wide refresh (every running instance), **Spring Cloud Bus** broadcasts over **Kafka** or **RabbitMQ** (message brokers) so one call refreshes all:

```
curl -X POST http://any-instance:8080/actuator/busrefresh
```

Secrets belong in a secrets manager, not here (`KUBERNETES` auth means Vault trusts the identity Kubernetes gives each **pod**):

```yaml
spring.cloud.vault:
  uri: https://vault:8200
  authentication: KUBERNETES            # pod identity, no static credentials
  kv: { enabled: true, backend: secret }
```

Vault gives **leasing, automatic rotation and audit logs**, none of which a general config server provides. Ordinary configuration in Git; secrets in Vault.

Kubernetes-native alternative — often all you need. A **ConfigMap** is Kubernetes' built-in configuration object, which can be **mounted as a volume** (exposed to the container as files):

```yaml
apiVersion: v1
kind: ConfigMap
metadata: { name: order-service-config }
data:
  http.timeoutMs: "3000"
  feature.newCheckout: "true"
---
# Mounted as a volume, changes propagate to the file within ~1 min
volumes:
  - name: config
    configMap: { name: order-service-config }
```

ConfigMaps mounted as **volumes** update live; consumed as **environment variables** they do **not** — the pod must restart. That trips people up constantly.

D. Implementation pitfalls

1. Reading config once into a `static final` field (set once, shared by the class). Refresh then does nothing — the value was captured at **class-load time**. Use `@RefreshScope`, or read through a provider at each use.

2. No client-side caching, or `fail-fast` misconfigured. Without caching, a config server outage takes down every service. With `fail-fast: true` **and** retry, startup waits and retries rather than booting with silently wrong defaults.

3. Secrets in the Git config repo. Once committed, they're in history permanently. Use Vault or **sealed-secrets** (encrypted in Git, decryptable only by the cluster), and treat any leaked credential as needing rotation, not deletion.

4. The encryption key stored beside the encrypted values. Part 5's irony — locking the door and leaving the key in it. The master key belongs in a KMS or Vault, injected at startup.

5. No validation before config propagates. A typo in a Git commit reaches production in seconds with no build or test to catch it. Add a **CI** (continuous integration — automated checks on every change) job validating the repo, and require pull requests — config's speed is a hazard as well as a feature.

6. Changing config in production directly. If the repo is the **source of truth**, an **out-of-band** change (made outside the normal process) is silently reverted by the next refresh — a baffling incident.

7. A single config server instance. A single point of failure for the whole estate. Run at least two **replicas** behind a service, and monitor them.

8. Hardcoding the config server's address. Self-defeating. Use service discovery or a stable Kubernetes Service **DNS** name.

9. Assuming env-var ConfigMaps refresh. They don't; only volume mounts propagate without a restart.

Related patterns

- Service Discovery — how services locate the config server; identical HA and caching concerns
- Sidecar — a sidecar can fetch config and expose it locally to the application
- Strangler Fig — the feature flags driving migration routing live in externalized config
- Circuit Breaker — breaker thresholds are configuration you'll want to tune without redeploying

Chapter 14 — Interview Question Bank

How to use this bank

This chapter collects the microservices questions that come up again and again in interviews: the ones on popular question lists (InterviewBit, GeeksforGeeks, devinterview.io, DataCamp, TechPrep), in LeetCode Discuss interview-experience posts for backend and Java/Spring roles, and in senior "scenario" rounds. They are worded the way interviewers usually ask them.

- ⋆ marks a question that appears on many lists or is reported very often. If you are short on time, drill the ⋆ questions first.
- Every question has a **short answer you can say out loud** in 30–60 seconds. The answers are written for this book and assume the versions used in the rest of it: **Java 25, Spring Boot 4.1, Spring Framework 7, Spring Cloud 2025.1 ("Oakwood"), Kafka, PostgreSQL, Kubernetes**.
- → **More:** links point to the chapter (or the section of Chapter 1) that explains the topic in full. Questions without a link are not covered elsewhere in the book, so the answer here is the complete one.
- New terms are defined the first time they appear. If a word is still unfamiliar, the 300-term glossary at the end of Ch 1 — System Design Fundamentals has it.

How to drill: cover the answer, say your own answer out loud, then uncover and compare. Mark the ones you fumbled and repeat them the next day. Interviewers care less about exact wording than about whether you name the *trade-off* — almost every good answer here ends with "the cost is…" or "use it when…".

---

1. Fundamentals: monolith vs microservices

Q1. ⋆ What are microservices? Microservices is an architectural style in which an application is built as a set of small services, each owning one business capability, running in its own process, owning its own data and deployable independently. Services talk over the network (HTTP, gRPC or messaging). The point is *independent change*: one team can build, test, deploy and scale its service without coordinating a release with everyone else. → *More:* Ch 1 — Architectural styles

Q2. ⋆ What is the difference between a monolithic and a microservices architecture? A monolith is one deployable unit: all features share one codebase, one process and usually one database, and calls between modules are in-memory function calls. Microservices split those features into separate deployables that call each other over the network and keep separate databases. A monolith is simpler to build, test, debug and run; microservices give independent deployment, independent scaling and team autonomy, at the price of network failures, distributed data and much more operational work. → *More:* Ch 1 — Architectural styles · Ch 2 — Strangler Fig, Part 0

Q3. ⋆ What are the advantages of microservices? Independent deployment (ship one service without redeploying everything), independent scaling (scale only the hot service), fault isolation (one crash need not take down the whole product), technology freedom per service, and smaller codebases that a single team can understand and own. The biggest real-world benefit is organisational: many teams can move in parallel with fewer merge conflicts and release trains.

Q4. ⋆ What are the disadvantages or challenges of microservices? Every call is now a network call that can be slow or fail; data is spread across databases so you lose easy joins and ACID transactions; debugging needs distributed tracing; testing needs contract tests; you need service discovery, config management, CI/CD per service, monitoring and on-call for dozens of things. Versioning APIs between teams and keeping data eventually consistent are ongoing costs. This is often called the "microservices tax". → *More:* Ch 1 — Synchronous vs asynchronous communication

Q5. ⋆ When should you NOT use microservices? When the team is small (one or two teams), the domain is still being discovered, the product is an early-stage startup, or you lack automated deployment and monitoring. In those cases a well-structured monolith ships faster. Martin Fowler's "Monolith First" advice says: start with a monolith, find the stable boundaries, and extract services only when a real pain (scaling, team contention, release speed) appears.

Q6. What is a modular monolith? A single deployable whose code is split into strongly separated modules, each with its own public API and private internals, and ideally its own database schema. It gives most of the design benefits of microservices (clear boundaries, ownership) without the network and operational costs, and it is the easiest starting point for later extraction. In Spring, **Spring Modulith** verifies module boundaries in tests and offers in-process events between modules. → *More:* Ch 1 — Architectural styles

Q7. ⋆ What is the difference between SOA and microservices? SOA (Service-Oriented Architecture) also splits a system into services, but typically shares databases, uses heavyweight protocols (SOAP, WS-*), and puts routing and business logic into a central Enterprise Service Bus (ESB). Microservices push the opposite way: "smart endpoints, dumb pipes", a database per service, lightweight protocols and decentralised governance. Microservices are often described as "SOA done with finer grains and without the ESB".

Q8. What are the key characteristics of a microservice? Single responsibility around a business capability; owns its data; independently deployable; communicates through a well-defined API; loosely coupled to other services but highly cohesive inside; owned by one team ("you build it, you run it"); designed for failure; and observable (logs, metrics, traces).

Q9. ⋆ What do you mean by cohesion and coupling? Cohesion is how strongly the code inside one module belongs together — high cohesion means a change to one business rule touches only one service. Coupling is how much one module depends on another — low coupling means you can change or deploy one service without changing the others. Good microservices aim for **high cohesion, low coupling**; a design where every feature needs changes to five services is a "distributed monolith".

Q10. ⋆ What is a distributed monolith and how do you avoid it? A system split into many services that still have to be deployed together because they are tightly coupled — shared database tables, chatty synchronous call chains, shared domain libraries, or lock-step API changes. You get all the costs of microservices with none of the benefits. Avoid it with database per service, boundaries drawn around business capabilities, backward-compatible API changes, and asynchronous events where an immediate answer is not needed.

Q11. How big should a microservice be? There is no line count. A useful test: one team can own it, it maps to one bounded context (Q17), it can be rewritten in a couple of weeks, and most changes touch only this service. Too small ("nanoservices") creates chatty calls and distributed transactions; too big recreates the monolith's coordination problems.

Q12. What is Conway's Law and why does it matter here? Conway's Law says a system's structure mirrors the communication structure of the organisation that builds it. So service boundaries and team boundaries should match: one team per service (or a few services). The "inverse Conway manoeuvre" is reorganising teams first to get the architecture you want.

Q13. What does "stateless" mean for a microservice? Stateless vs stateful services? A stateless service keeps no client-specific data in memory between requests — any instance can serve any request, so you can add, remove or restart instances freely behind a load balancer. State (sessions, carts) lives in a database or cache such as Redis. Stateful services (databases, Kafka brokers, game servers) keep data locally and need stable identity, replication and careful scaling; in Kubernetes they run as StatefulSets. → *More:* Ch 1 — Stateless and stateful services

Q14. What are the main components of a microservices architecture? The services themselves plus the platform around them: an API gateway (single entry point), service discovery/registry, a config server, a message broker (Kafka/RabbitMQ), databases per service, a container platform (Docker + Kubernetes), CI/CD pipelines, observability (logs, metrics, traces) and security infrastructure (identity provider, secrets store). Optionally a service mesh for traffic, mTLS and telemetry.

Q15. Name companies that use microservices and why they adopted them. Netflix (moved from a monolith after a 2008 database corruption outage, to scale streaming globally), Amazon (the "two-pizza team" model with service-owned APIs), Uber (split a monolith to scale teams and cities), Spotify (squads owning services). The common reason is scaling the *organisation* — hundreds of engineers shipping independently — not just scaling traffic.

Q16. What is the twelve-factor app methodology? A set of twelve rules for building cloud-ready services: one codebase per app, explicit dependencies, config in the environment, backing services as attached resources, separate build/release/run stages, stateless processes, port binding, scale out via

processes, disposability (fast start, graceful shutdown), dev/prod parity, logs as event streams, and admin tasks as one-off processes. It is the checklist that makes a service easy to containerise and run on Kubernetes. → *More:* Ch 13 — Configuration Externalization

---

2. Decomposition, DDD and migration

Q17. ⋆ What is a bounded context? A bounded context is a term from Domain-Driven Design: a boundary inside which a domain model and its vocabulary have one precise meaning. "Customer" in Billing (payment methods, invoices) is a different model from "Customer" in Shipping (addresses, delivery preferences). Each bounded context is a natural candidate for a microservice, because inside it the language and rules are consistent and it can own its own data.

Q18. ⋆ What is Domain-Driven Design (DDD) and how does it relate to microservices? DDD is an approach to software design that models the code closely on the business domain, built with domain experts using a shared "ubiquitous language". Its strategic part (bounded contexts, context maps, subdomains) tells you where to draw service boundaries; its tactical part (entities, value objects, aggregates, domain events) tells you how to structure the code inside a service. Microservices without DDD-style boundaries tend to end up chatty and tightly coupled.

Q19. ⋆ How would you decompose a monolith into microservices? Decompose by **business capability** (what the business does: ordering, payments, catalogue) or by **subdomain/bounded context**, not by technical layer. Practical steps: run an event-storming workshop to find domain events and contexts; map which data each capability owns; pick an edge capability with few dependencies and high change rate; extract it behind the existing API using the Strangler Fig pattern; split its data last. Repeat, measuring whether deployments really got more independent. → *More:* Ch 2 — Strangler Fig

Q20. ⋆ What is the Strangler Fig pattern? A way to replace a legacy system gradually: put a facade (proxy or gateway) in front of the monolith, build new functionality as separate services, and route one capability at a time from the old system to the new one. Over time the new system "strangles" the old one until it can be switched off. It avoids a risky big-bang rewrite and lets you roll back per route. → *More:* Ch 2 — Strangler Fig

Q21. What is an aggregate in DDD and why does it matter for microservices? An aggregate is a cluster of objects treated as one unit for changes, with one root entity as its only entry point (for example, `Order` with its `OrderLines`). All invariants inside an aggregate are kept in one local transaction; anything across aggregates is eventually consistent. This maps neatly to microservices: a transaction never spans services, and other services refer to an aggregate only by its ID.

Q22. What are entities, value objects and domain events? An entity has identity that persists over time (an `Order` with an ID). A value object is defined only by its values and is immutable (`Money(100, EUR)`, an `Address`) — a Java `record` is a good fit. A domain event records something that happened that the business cares about (`OrderPlaced`), named in the past tense; it is what other services subscribe to.

Q23. What is a context map? Name some relationships between bounded contexts. A context map is a diagram of the bounded contexts and how they relate. Common relationships: **customer–supplier** (downstream depends on upstream, who considers its needs), **conformist** (downstream simply adopts upstream's model), **anti-corruption layer** (downstream translates upstream's model into its own), **shared kernel** (a small shared model both own), **open host service / published language** (upstream publishes a stable public API or schema).

Q24. ⋆ What is an anti-corruption layer? A translation layer between your service and another system (often a legacy monolith or third-party API) so that the other system's model and quirks do not leak into your clean domain model. It converts their data and calls into your concepts and back. It is standard during Strangler Fig migrations. → *More:* Ch 2 — Strangler Fig

Q25. What is event storming? A workshop technique in which developers and domain experts put domain events (orange sticky notes, past tense: "Order Placed", "Payment Captured") on a timeline, then add commands, actors, policies and aggregates. Clusters that emerge suggest bounded contexts and thus service boundaries. It is a fast way to find boundaries before writing code.

Q26. Decompose by business capability vs by subdomain — what is the difference? A business capability is something the business does to create value (manage inventory, take payments) and is fairly stable

over time. A subdomain is DDD's split of the problem space into **core** (your competitive edge — invest most here), **supporting** (needed but specific to you) and **generic** (commodity, often bought, like identity). Both lead to similar boundaries; subdomains add the guidance about where to invest and what to buy.

Q27. What is the "shared database" anti-pattern? Several services reading and writing the same tables. It looks convenient but couples services at the schema level: one team's column change breaks another service, you cannot scale or choose storage per service, and ownership of data rules is unclear. It is acceptable only as a temporary migration step, with a plan to split. → *More:* Ch 10 — Saga, "What is database per service?"

Q28. How do you split the database when migrating from a monolith? Go step by step: first separate the code module and make it the only writer of its tables; then move those tables into a separate schema; then a separate database. Keep data in sync during the transition with change data capture (CDC — streaming the database's change log, e.g. Debezium reading the write-ahead log into Kafka) or with the service's events, and replace cross-table joins with API calls or local read copies. → *More:* Ch 2 — Strangler Fig, "The part the pattern doesn't solve: data"

Q29. What is CDC (Change Data Capture)? CDC streams every insert, update and delete from a database's transaction log to other systems, usually through Kafka, without changing application code. Uses: syncing data during a monolith migration, feeding search indexes and caches, and as the relay in the transactional outbox pattern (Q107). Debezium is the common open-source tool.

Q30. What is the difference between a microservice and a serverless function? A microservice is a long-running process you deploy and scale (usually as containers); a serverless function (AWS Lambda, Cloud Functions) is code the platform starts per event and bills per invocation, scaling to zero. Functions suit spiky, event-driven, short tasks; they bring cold-start latency, execution time limits and vendor lock-in. A microservice can be implemented with several functions, so they are not exclusive. → *More:* Ch 1 — Architectural styles

---

3. Communication: REST, gRPC and messaging

Q31. ⋆ How do microservices communicate with each other? Two styles. **Synchronous** request/response — REST over HTTP or gRPC — where the caller waits for the answer; simple, but couples availability (if the callee is down, the caller fails). **Asynchronous** messaging — events or commands through a broker such as Kafka or RabbitMQ — where the sender does not wait; decoupled and resilient, but eventually consistent and harder to trace. Most systems use sync for queries that need an answer now and async for propagating state changes. → *More:* Ch 1 — Synchronous vs asynchronous communication · Ch 11 — Event-Driven

Q32. ⋆ What is the difference between synchronous and asynchronous communication? Synchronous: the caller sends a request and blocks (or awaits) until the response arrives — the two services must both be up at the same time. Asynchronous: the caller hands a message to a broker and carries on; the receiver processes it later. Async gives temporal decoupling, load levelling (the queue absorbs bursts) and better fault isolation, but adds a broker to operate, eventual consistency, and duplicate/out-of-order message handling. → *More:* Ch 11 — Event-Driven, Part 0

Q33. ⋆ What is REST? What makes an API RESTful? REST (Representational State Transfer) is an architectural style for HTTP APIs: resources are identified by URLs (`/orders/42`), manipulated with standard HTTP methods (GET, POST, PUT, PATCH, DELETE), represented as JSON or similar, and each request is stateless (carries all needed context). Good REST APIs use correct status codes, are cacheable where possible, and use nouns, not verbs, in paths. → *More:* Ch 1 — API styles · Ch 1 — API design basics

Q34. ⋆ REST vs gRPC vs message queues — when do you use each? REST/JSON: public and browser-facing APIs, simple CRUD, easy debugging. gRPC: internal service-to-service calls that need low latency, strong typed contracts and streaming — it uses HTTP/2 and binary Protocol Buffers. Message queues/streams: when the caller does not need an immediate answer, when many consumers must react to one event, or to absorb load spikes. → *More:* Ch 1 — API styles

Q35. What is gRPC and why is it faster than REST/JSON? gRPC is a remote procedure call framework: you define services and messages in a `.proto` file and generate client and server code in many languages.

It is faster because Protocol Buffers are a compact binary format (smaller and cheaper to parse than JSON) and HTTP/2 multiplexes many calls over one connection with header compression. It supports four call types: unary, server streaming, client streaming and bidirectional streaming. Downsides: not natively callable from browsers (needs gRPC-Web or a gateway), harder to inspect by hand.

Q36. What is GraphQL and when would you use it in a microservices system? GraphQL is a query language for APIs where the client asks for exactly the fields it needs in one request, against a typed schema. It suits a client-facing aggregation layer (often a BFF or gateway) over many services, avoiding over-fetching and multiple round trips. Costs: caching is harder (one POST endpoint), expensive queries need depth/complexity limits, and N+1 calls to backends need batching (DataLoader). → *More:* Ch 4 — BFF

Q37. ⋆ Which HTTP methods are idempotent? What is the difference between PUT and PATCH, and PUT and POST? Idempotent means repeating the request has the same effect as doing it once. GET, HEAD, PUT, DELETE (and OPTIONS) are idempotent; POST and PATCH are not guaranteed to be. PUT replaces the whole resource at a known URL; PATCH applies a partial change; POST usually creates a new resource whose ID the server chooses, so retrying a POST can create duplicates unless you add an idempotency key (Q119). → *More:* Ch 1 — HTTP · Ch 8 — Retry, Part 0

Q38. ⋆ How do you version APIs in microservices? Options: URI version (`/v2/orders`), a header (`API-Version: 2`), a query parameter, or a media type (`application/vnd.shop.v2+json`). More important than the mechanism is the rule: make changes backward compatible (add optional fields, never remove or rename without a new version), support old versions for a deprecation period, and announce sunset dates. Spring Framework 7 adds first-class API versioning to Spring MVC and WebFlux (`@GetMapping(version = "2")` with a configured strategy for path, header, query or media type). → *More:* Ch 3 — API Gateway, "Version management"

Q39. What does "backward compatible" mean for an API or event schema? Old clients keep working against the new version. Safe changes: adding optional fields, adding new endpoints, accepting more input. Breaking changes: removing or renaming fields, changing types or meaning, making optional fields required. Consumers should follow the "tolerant reader" rule — ignore fields they do not understand — so producers can add fields freely.

Q40. ⋆ What is a message broker? Kafka vs RabbitMQ? A message broker is middleware that receives messages from producers and delivers them to consumers, decoupling them in time. **RabbitMQ** is a traditional queue broker: smart routing (exchanges, routing keys), messages are removed once acknowledged, good for task queues and complex routing. **Kafka** is a distributed, partitioned, replicated log: messages are retained for a configured time, consumers track their own offset and can replay, ordering is per partition, and throughput is very high — good for event streaming and event sourcing. → *More:* Ch 1 — Message queues · Ch 1 — Event streaming

Q41. Point-to-point queue vs publish-subscribe? In point-to-point, each message is consumed by exactly one consumer among competing workers — good for distributing tasks. In publish-subscribe, each message is delivered to every subscribed group — good for broadcasting events ("OrderPlaced" goes to billing, shipping and analytics). Kafka does both with consumer groups: within one group partitions are split among consumers (point-to-point), and each group gets every message (pub-sub). → *More:* Ch 11 — Event-Driven, "What is publish-subscribe?"

Q42. ⋆ How does Kafka guarantee message ordering? Only within a partition. Messages with the same key go to the same partition (by hash of the key), so key by the entity ID (for example `orderId`) to keep all events for one order in order. There is no ordering across partitions. Retries with more than one in-flight request could reorder messages, which the idempotent producer (on by default in modern Kafka) prevents. → *More:* Ch 11 — Event-Driven · Ch 1 — Event streaming

Q43. What are Kafka partitions, consumer groups and offsets? A topic is split into partitions, each an ordered append-only log; partitions are the unit of parallelism and are replicated across brokers. A consumer group is a set of consumers sharing the work — each partition is read by at most one consumer in the group, so more consumers than partitions sit idle. An offset is a consumer's position in a partition; committing it after processing gives at-least-once delivery. → *More:* Ch 1 — Event streaming

Q44. Events vs commands vs queries in messaging? A command asks one specific receiver to do something (`ReserveStock`) and may be rejected. An event announces that something already happened (`StockReserved`) to anyone interested; the publisher does not know or care who listens. A query asks for data and

changes nothing. Naming them correctly matters: events are past tense and immutable. → *More:* Ch 11 — Event-Driven, "What is an event?" · Ch 12 — CQRS

Q45. Event notification vs event-carried state transfer? An event notification is thin — "Order 42 changed" — and consumers call back the source to get details (simpler, but adds load and coupling). Event-carried state transfer puts the needed data in the event — "Order 42 placed, items…, total…" — so consumers can keep a local copy and never call back (more decoupled and resilient, but bigger events and schema-evolution concerns).

Q46. ⋆ What is a dead-letter queue (DLQ)? A separate queue or topic where messages go after they fail processing a set number of times (for example a malformed "poison" message). It stops one bad message from blocking the whole stream or retrying forever. You alert on DLQ growth, inspect, fix the bug or data, and replay the messages. Spring Kafka's `DefaultErrorHandler` with a `DeadLetterPublishingRecoverer` does this. → *More:* Ch 11 — Event-Driven, "Error handling and message durability"

Q47. What is backpressure? A way for a slow consumer to signal a fast producer to slow down, so the consumer is not overwhelmed and memory does not grow without limit. Pull-based consumers (Kafka) have it naturally; reactive streams (Project Reactor's `Flux`) have it built into the protocol; for HTTP you use rate limits, bounded queues and 429/503 responses.

Q48. How do you manage schema evolution in event-driven systems? Use a schema format with evolution rules (Avro, Protobuf or JSON Schema) and a schema registry that rejects incompatible changes at publish time. Prefer backward-compatible changes (add optional fields with defaults). For a breaking change, publish a new event type or version alongside the old one, migrate consumers, then retire the old one. Consumers should be tolerant readers.

Q49. When would you use WebSockets or Server-Sent Events instead of REST? When the server must push updates to the client in real time — chat, live dashboards, notifications, collaborative editing. WebSockets give a two-way persistent connection; SSE gives a simpler one-way server-to-client stream over HTTP. Costs: long-lived connections are stateful and harder to load-balance and scale, so a dedicated gateway/connection service is common. → *More:* Ch 1 — Real-time communication

Q50. How do you avoid long synchronous call chains? Each hop multiplies latency and failure probability (five services at 99.9% availability each give about 99.5% together). Fixes: let services keep local copies of the data they need (updated by events), aggregate in a gateway/BFF in parallel instead of chaining, move non-urgent steps to async messaging, and set timeouts at every hop with a total deadline. → *More:* Ch 3 — API Gateway

---

4. API Gateway, BFF, service discovery and service mesh

Q51. ⋆ What is an API Gateway and why do we need it? An API gateway is a single entry point that sits between clients and the services. It routes each request to the right service and handles cross-cutting concerns once: authentication, rate limiting, TLS termination, request/response transformation, response aggregation, caching, logging and metrics. Clients then depend on one stable API instead of knowing the internal topology. Examples: Spring Cloud Gateway, Kong, NGINX, AWS API Gateway, Envoy. → *More:* Ch 3 — API Gateway

Q52. ⋆ What are the drawbacks of an API Gateway and how do you mitigate them? It is a potential single point of failure and bottleneck, adds a network hop of latency, and can grow into a "god service" full of business logic owned by nobody. Mitigate by running several stateless gateway instances behind a load balancer across zones, keeping it thin (routing and cross-cutting only, no business rules), using timeouts and circuit breakers on its upstream calls, and giving it clear team ownership. → *More:* Ch 3 — API Gateway, Lesson 4

Q53. ⋆ What is the Backend for Frontend (BFF) pattern? How is it different from an API Gateway? A BFF is a separate backend per client type (web, iOS/Android, TV, partner API), owned by the frontend team, which aggregates and shapes data exactly for that client. A general API gateway is one shared entry point for all clients. They are often combined: a thin gateway handles auth and routing, and BFFs behind it do client-specific aggregation. The cost of BFFs is some duplicated code across them. → *More:* Ch 4 — BFF · Ch 4, "BFF vs. API Gateway"

Q54. What is API composition / the aggregator pattern? A component (gateway, BFF or dedicated service) calls several services, ideally in parallel, and merges the results into one response — for example a product page combining catalogue, price, stock and reviews. It replaces the cross-service join you lost with database per service. Watch out for the slowest dependency dominating latency; use timeouts and return partial results when an optional part fails. → *More:* Ch 3 — API Gateway, "Aggregation of results"

Q55. ⋆ What is service discovery and why is it needed? In containers and auto-scaling environments, service instances start and stop constantly and their IP addresses change. Service discovery lets a caller find a healthy instance by a logical name (`order-service`) instead of a hard-coded address. It needs a **service registry** — a database of live instances that services register with and that is kept fresh by health checks or heartbeats. → *More:* Ch 5 — Service Discovery

Q56. ⋆ Client-side vs server-side service discovery? Client-side: the caller queries the registry, gets the list of instances and load-balances itself (Eureka + Spring Cloud LoadBalancer). Fewer hops, but every client needs the library. Server-side: the caller sends to a router or load balancer that does the lookup (Kubernetes Service with DNS, AWS ALB). Clients stay simple and language-agnostic, at the cost of an extra hop. On Kubernetes, server-side via Services and DNS is the default and you rarely need Eureka. → *More:* Ch 5 — Client-side vs. server-side discovery

Q57. Self-registration vs third-party registration? With self-registration the service registers itself on startup and sends heartbeats (Eureka client). With third-party registration a separate component — the platform — watches instances and registers them (Kubernetes endpoints controller, Consul with a registrator). Third-party keeps service code free of registry logic. → *More:* Ch 5 — Self-registration vs. third-party registration

Q58. ⋆ What is Eureka? Why is Eureka described as AP? Eureka is Netflix's service registry, integrated through Spring Cloud Netflix. In CAP terms (consistency, availability, partition tolerance) it favours availability: during network problems it keeps serving possibly stale instance lists and enters "self-preservation mode" instead of evicting many instances at once. A stale list is survivable because clients retry another instance; a registry that refuses to answer would stop all traffic. → *More:* Ch 5 — Service Discovery · Ch 1 — CAP theorem

Q59. How does service discovery work in Kubernetes? A Kubernetes **Service** gives a stable virtual IP and DNS name (`orders.shop.svc.cluster.local`) in front of a changing set of Pods selected by labels. The control plane tracks ready Pods (via readiness probes) in EndpointSlices, and kube-proxy (or an eBPF data plane) load-balances connections to them. Callers just use the DNS name. → *More:* Ch 5 — Service Discovery · Ch 1 — Kubernetes basics

Q60. ⋆ What is client-side load balancing? Which algorithms exist? The client picks the target instance itself from the list it got from discovery, avoiding a central load balancer hop. In Spring, a `@LoadBalanced` `RestClient.Builder` or `WebClient.Builder` uses Spring Cloud LoadBalancer (which replaced Netflix Ribbon). Common algorithms: round robin, random, least connections, weighted, and zone- or latency-aware choices. → *More:* Ch 1 — Load balancers · Ch 5 — Load balancing strategies

Q61. ⋆ What is a service mesh? When would you use one? A service mesh is an infrastructure layer that handles service-to-service traffic through a proxy next to each service (a sidecar such as Envoy, or a per-node proxy in Istio's "ambient" mode). A control plane (Istio, Linkerd) configures the proxies. It provides mTLS, retries, timeouts, circuit breaking, traffic splitting for canaries, and uniform metrics and traces — without changing application code and in any language. Use it with many services and polyglot teams; it adds latency, resource cost and operational complexity. → *More:* Ch 9 — Sidecar · Ch 1 — Service discovery, sidecars and service mesh

Q62. ⋆ What is the sidecar pattern? A helper process deployed alongside the main application (in Kubernetes, another container in the same Pod) sharing its lifecycle, network and possibly storage. It takes over cross-cutting concerns — proxying, mTLS, log shipping, config reloading — so the application stays focused on business logic and the concern is solved once for every language. → *More:* Ch 9 — Sidecar

Q63. Library (e.g. Resilience4j) vs service mesh for resilience — which is better? A library runs in-process, knows the business context (fallback logic, which calls are idempotent) and costs no extra hop, but must be implemented per language. A mesh is language-agnostic and centrally configured, but sees only network-level information and can retry non-idempotent calls blindly. A common split: mesh

for mTLS, timeouts and traffic shaping; library for fallbacks and business-aware retries. Avoid configuring retries in both places. → *More:* Ch 9 — Sidecar · Ch 8 — Retry, "Placement in the architecture"

Q64. What is the difference between a reverse proxy, a load balancer and an API gateway? A reverse proxy sits in front of servers and forwards client requests to them (TLS termination, caching, hiding servers). A load balancer is a reverse proxy whose main job is spreading load across instances with health checks. An API gateway is a reverse proxy with API-level features: routing by path/version, auth, rate limiting per client, transformation and aggregation. In practice the same product (NGINX, Envoy) can play all three roles. → *More:* Ch 1 — Proxies and API gateways

Q65. Edge (north-south) vs internal (east-west) traffic? North-south traffic enters or leaves the system (clients to the gateway); east-west traffic flows between services inside the cluster. The API gateway governs north-south; a service mesh or internal policies govern east-west. Security for east-west is often forgotten — zero trust says encrypt and authenticate it too (mTLS).

Q66. Should the API gateway do authentication, authorization, or both? Typically the gateway authenticates (validates the token, rejects anonymous calls) and does coarse checks (scopes, rate limits per client). Fine-grained authorization ("may this user cancel *this* order?") belongs in the owning service, which knows the data. Each service should still validate the token it receives rather than blindly trusting the network. → *More:* Ch 3 — API Gateway, "Security and authentication"

---

5. Resilience and fault tolerance

Q67. ⋆ How do you handle failures in microservices / what is fault tolerance? Assume every remote call can fail or be slow. Combine: timeouts on every call, retries with backoff for transient errors, circuit breakers to stop calling a failing dependency, bulkheads to isolate resources, fallbacks or graceful degradation, rate limiting and load shedding to protect yourself, async messaging to decouple, and redundancy (several instances across zones). Then verify with chaos testing. → *More:* Ch 6, Ch 7, Ch 8 · Ch 1 — Faults, failures and fault tolerance

Q68. ⋆ What is a cascading failure? A failure that spreads: service C becomes slow, so B's threads waiting on C pile up until B runs out of threads and stops responding, then A, which calls B, does the same — and the whole system goes down because of one slow dependency. Timeouts, circuit breakers and bulkheads exist mainly to break this chain. → *More:* Ch 6 — Circuit Breaker, "Cascading failures, step by step"

Q69. ⋆ Explain the circuit breaker pattern and its states. A circuit breaker wraps calls to a dependency and tracks failures. **Closed**: calls flow normally and failures are counted in a sliding window. When the failure (or slow-call) rate passes a threshold it goes **Open**: calls fail immediately (or go to a fallback) without touching the dependency, giving it time to recover. After a wait it goes **Half-Open**: a few trial calls are allowed; if they succeed it closes, otherwise it opens again. → *More:* Ch 6 — Circuit Breaker, "The three states"

Q70. ⋆ Why is a slow service often worse than a dead one? A dead service fails fast (connection refused) so the caller frees its thread at once. A slow service holds the caller's threads and connections until the timeout, so they pile up and exhaust the caller's pools — that is how cascades start. That is why circuit breakers should also count *slow calls* (Resilience4j `slowCallRateThreshold`) and every call needs a timeout. → *More:* Ch 6 — Circuit Breaker

Q71. ⋆ What is the retry pattern? What should and should not be retried? Automatically repeat a failed call in the hope that the failure was transient. Retry timeouts, connection resets, 503/502/504, 429 (honouring `Retry-After`). Do not retry 400, 401, 403, 404, 409 or validation errors — they will fail again. Only retry operations that are idempotent, or make them idempotent with an idempotency key. → *More:* Ch 8 — Retry, "Transient vs. permanent"

Q72. ⋆ What is exponential backoff with jitter and why use it? Exponential backoff waits longer after each failure (for example 100 ms, 200 ms, 400 ms, up to a cap) to give the dependency room to recover. Jitter adds randomness to each wait ("full jitter": random between 0 and the backoff), so that thousands of clients that failed together do not all retry at the same instant — the thundering-herd

problem. → *More:* Ch 8 — Retry, "Backoff strategy" and "Jitter" · Ch 1 — Timeouts, retries, exponential backoff and jitter

Q73. ⋆ What is a retry storm and how do you avoid it? Retries multiplying load on an already struggling service. If three layers each retry three times, one user request can become 27 calls to the bottom service. Avoid it by retrying at one layer only (usually the one closest to the failing call or at the edge), capping attempts, using backoff with jitter, a retry budget (retries limited to, say, 10% of traffic), and a circuit breaker that stops retries when the dependency is clearly down. → *More:* Ch 8 — Retry, "Retry storms and thundering herd"

Q74. ⋆ What is the bulkhead pattern? Partition resources (thread pools, connection pools, semaphores, even whole service instances) per dependency or per type of work, like watertight compartments in a ship's hull. If the recommendation service hangs, it can exhaust only its own pool of, say, 10 threads, while checkout keeps its own pool and continues to work. → *More:* Ch 7 — Bulkhead

Q75. ⋆ Circuit breaker vs bulkhead vs retry — how do they work together? Retry bounds *how many* attempts; circuit breaker bounds *how long* you keep calling a failing dependency; bulkhead bounds *how much* of your resources one dependency can consume. A typical order (outermost first, Resilience4j default aspect order): Retry → CircuitBreaker → RateLimiter → TimeLimiter → Bulkhead → the call. Each retry attempt then goes through the breaker, so an open breaker stops retries quickly. → *More:* Ch 7 — Bulkhead · Ch 8 — Retry, "Circuit breaker integration"

Q76. ⋆ Why is a timeout essential on every remote call? How do you choose its value? Without a timeout a hung dependency holds your thread forever. Choose from the dependency's measured latency: slightly above its p99 (e.g. p99 = 300 ms → timeout 500 ms), never the library's default (often infinite or very long). Set both connect and read timeouts, and propagate a total deadline so downstream calls do not keep working after the upstream caller has already given up. → *More:* Ch 1 — Timeouts, retries, exponential backoff and jitter · Ch 6 — "What is a timeout?"

Q77. What is deadline propagation? The original caller sets a total time budget (say 2 s) and each service passes the *remaining* budget downstream (gRPC does this with deadlines; with HTTP you use a header). A service that receives a request with too little time left fails fast instead of doing work nobody will wait for.

Q78. ⋆ What is a fallback? Give examples. What you return when the real call fails or the breaker is open: cached or last-known-good data, a default value (generic recommendations instead of personalised ones), an empty but valid response, or queuing the action for later ("we will email your receipt"). Fallbacks must be cheap and must not themselves call the same failing dependency. For critical data (payments, balances) the honest fallback is often a clear error. → *More:* Ch 6 — "What is a fallback?" · Ch 1 — Circuit breakers, bulkheads and fallbacks

Q79. ⋆ What is rate limiting and which algorithms are used? Rate limiting caps how many requests a client may make in a time window, protecting services from overload and abuse; excess requests get HTTP 429. Algorithms: **token bucket** (tokens refill at a fixed rate, each request takes one, allows short bursts — the most common), **leaky bucket** (smooth constant output), **fixed window counter** (simple, but bursts at window edges), **sliding window log/counter** (accurate, a bit more costly). → *More:* Ch 1 — Rate limiting and throttling · Ch 3 — API Gateway, "Rate limiting challenges"

Q80. How do you rate-limit across many gateway instances? In-memory counters only limit per instance, so N instances allow N times the limit. Use a shared store — typically Redis with an atomic Lua script implementing the token bucket (Spring Cloud Gateway's `RequestRateLimiter` with `RedisRateLimiter` does this). Alternatively accept approximate limits by dividing the quota by the instance count. → *More:* Ch 3 — API Gateway

Q81. Rate limiting vs throttling vs load shedding? Rate limiting rejects a *client* above its quota (fairness, abuse). Throttling slows down or queues requests rather than rejecting them. Load shedding is the server protecting *itself*: when overall load is above capacity it drops low-priority work (e.g. analytics calls) to keep critical paths (checkout) alive. → *More:* Ch 1 — Load shedding and graceful degradation

Q82. What is graceful degradation? Keeping the core function working with reduced features when parts fail: the shop still sells when the recommendation or review service is down, showing a page without those sections. It requires deciding upfront which dependencies are critical and which are optional, and coding fallbacks for the optional ones. → *More:* Ch 1 — Load shedding and graceful degradation

Q83. ⋆ Show how you would add a circuit breaker and retry to a Spring Boot call. With Resilience4j's Spring Boot integration:

```
@CircuitBreaker(name = "inventory", fallbackMethod = "stockFallback")
@Retry(name = "inventory")
public Stock stock(String sku) {
    return inventoryClient.getStock(sku);   // RestClient / HTTP interface call
}
private Stock stockFallback(String sku, Throwable t) {
    return Stock.unknown(sku);              // degrade, do not fail the page
}
```

Thresholds go in `application.yml` under `resilience4j.circuitbreaker.instances.inventory` (e.g. `failureRateThreshold: 50`, `slidingWindowSize: 20`, `waitDurationInOpenState: 10s`). The fallback must have the same return type plus a `Throwable` parameter. → *More:* Ch 6 — Circuit Breaker · Ch 8 — Retry

Q84. What resilience features are built into Spring Framework 7 itself? Spring Framework 7 adds `@Retryable` (with backoff, jitter and include/exclude exception types) and `@ConcurrencyLimit` (a simple bulkhead) in core, enabled with `@EnableResilientMethods`, plus a programmatic `RetryTemplate` in `org.springframework.core.retry`. By default `@Retryable` retries any exception 3 times with a 1 s delay. They replace the separate Spring Retry project (Spring Boot 4 no longer manages its version) and cover retry and concurrency throttling without an extra dependency; for circuit breakers, rate limiters and time limiters you still use Resilience4j (often via Spring Cloud Circuit Breaker).

Q85. ⋆ What is idempotency and why is it critical in microservices? An operation is idempotent if doing it twice has the same effect as doing it once. Because retries, at-least-once message delivery and user double-clicks all cause duplicates, every handler that changes state must tolerate duplicates — otherwise a retry can charge a customer twice. Techniques: idempotency keys, natural keys with unique constraints, a processed-message table, and conditional updates (`UPDATE … WHERE version = ?`). → *More:* Ch 8 — Retry, "Idempotency and side effects" · Ch 1 — Delivery guarantees, idempotency and retries

Q86. How do you tune circuit breaker parameters? Base them on data: the dependency's normal error rate and latency percentiles. Set the minimum number of calls high enough that one or two failures at low traffic do not trip it; set the failure-rate threshold well above the normal error rate (e.g. 50%); set the slow-call threshold just above normal p99; set the open-state wait to roughly how long the dependency takes to recover (5–30 s). Alert on state transitions to Open — a breaker that hides errors silently is dangerous. → *More:* Ch 6 — Circuit Breaker, "Performance implications"

Q87. What is chaos engineering? Deliberately injecting failures (killing instances, adding latency, cutting networks, failing a zone) in a controlled way, ideally in production with a limited blast radius, to verify that the system's resilience mechanisms actually work. Netflix's Chaos Monkey popularised it; tools include Chaos Mesh, LitmusChaos, Gremlin and AWS Fault Injection Service. Start with a hypothesis ("if one payment instance dies, p99 stays below 500 ms") and measure.

Q88. How do you size a bulkhead or a thread pool for a dependency? Use Little's law: concurrent requests in flight = arrival rate × latency. If you call a dependency 200 times per second at 50 ms each, about 10 calls are in flight; add headroom (e.g. 15–20). Cap it at what the dependency can handle, and set a small queue or none, so excess fails fast rather than waiting. → *More:* Ch 7 — Bulkhead · Ch 1 — Little's law

Q89. Do virtual threads remove the need for bulkheads and timeouts? No. Virtual threads (final since Java 21; enable in Spring Boot with `spring.threads.virtual.enabled=true`) make blocking calls cheap, so you rarely run out of *threads*. But the downstream service, connection pools and database still have limits — without a concurrency cap you can now flood them with thousands of simultaneous calls. Keep timeouts, and use semaphore-style bulkheads (e.g. `@ConcurrencyLimit`) instead of thread-pool bulkheads.

---

6. Data management: database per service, saga, outbox, CQRS, event sourcing

Q90. ⋆ What is the database-per-service pattern? Why should each microservice have its own database? Each service owns its data store privately; other services can reach that data only through the

service's API or events, never by querying its tables. This keeps services loosely coupled (schema changes stay internal), lets each pick the right storage (PostgreSQL, MongoDB, Redis, Elasticsearch — "polyglot persistence"), and allows independent scaling. "Own database" can mean a separate schema on a shared server; what matters is that no other service touches it. → *More:* Ch 10 — Saga, "Database per service"

Q91. ⋆ What problems does database per service create? No cross-service ACID transactions, no SQL joins across services, data duplication, and eventual consistency between copies. Answers: sagas for multi-service business transactions, API composition or CQRS read models for queries across services, events plus the outbox pattern to keep copies in sync. → *More:* Ch 10 — Saga · Ch 12 — CQRS

Q92. ⋆ How do you manage transactions across multiple microservices? Avoid them where possible by drawing boundaries so one business invariant lives in one service. When a business process must span services, use a **saga**: a sequence of local transactions, each publishing an event or message that triggers the next, with compensating transactions to undo completed steps if a later step fails. Use the transactional outbox so each step's state change and its message are atomic. Two-phase commit is generally avoided. → *More:* Ch 10 — Saga · Ch 1 — Distributed transactions

Q93. ⋆ What is the Saga pattern? A saga implements a business transaction that spans services as a series of local transactions. Each step commits in its own service's database. If step *n* fails, the saga runs compensating transactions for steps *n-1* … 1 in reverse (refund the payment, release the stock). It gives eventual consistency, not isolation: other requests can see the intermediate states. → *More:* Ch 10 — Saga

Q94. ⋆ Choreography vs orchestration sagas? In **choreography** each service listens for events and decides what to do next ("OrderCreated" → payment charges → "PaymentCompleted" → inventory reserves); there is no central coordinator, which is simple for 2–4 steps but the flow is hard to see and cyclic dependencies creep in. In **orchestration** a central orchestrator (a service or a workflow engine such as Temporal or Camunda) tells each participant what to do and tracks state; the flow is explicit, testable and easier to monitor, at the cost of a central component. → *More:* Ch 10 — Saga, "The two types"

Q95. ⋆ What is a compensating transaction? Why is it not the same as a rollback? A compensating transaction is a new business action that semantically undoes an earlier committed step: "refund payment" for "charge payment", "release seats" for "reserve seats". It is not a rollback because the original step already committed and others may have seen it; some effects cannot be undone (an email was sent), so compensation may be "send a correction email". Compensations must be idempotent and must eventually succeed (retried until they do). → *More:* Ch 10 — Saga, "Compensating transactions"

Q96. ⋆ What is two-phase commit (2PC) and why is it avoided in microservices? 2PC is a protocol where a coordinator asks all participants to *prepare* (lock resources and vote) and then tells all to *commit* or *abort*. It gives atomicity across databases but holds locks during the protocol, blocks if the coordinator fails at the wrong moment, needs every participant to support XA transactions (Kafka and most NoSQL stores do not), and couples availability — one slow participant stalls everyone. Sagas trade isolation for availability and loose coupling. → *More:* Ch 1 — Distributed transactions · Ch 10 — Saga

Q97. Sagas have no isolation. What anomalies can happen and how do you deal with them? Because each step commits independently, other transactions can see or change intermediate data: lost updates, dirty reads (reading an order that will be cancelled), fuzzy reads. Countermeasures: **semantic locks** (a status like `PENDING` that other operations respect), **commutative updates** (operations that can apply in any order), re-reading values before acting, ordering steps so that risky or irreversible ones come last (pivot transaction), and versioning.

Q98. What are compensatable, pivot and retriable transactions in a saga? Compensatable steps come first and can be undone. The pivot transaction is the go/no-go point (e.g. charging the card): if it succeeds, the saga will complete. Retriable steps come after the pivot and must not fail permanently — they are retried until they succeed (e.g. sending the shipping request). Ordering steps this way minimises the compensation you need.

Q99. ⋆ What is the transactional outbox pattern? It solves the **dual-write problem**: a service must update its database *and* publish an event, but you cannot make a database commit and a Kafka send atomic. With the outbox, the service writes the business change and an `outbox` row containing the event in the same local database transaction. A separate relay — a poller, or CDC like Debezium reading the

database log — publishes outbox rows to the broker and marks them sent. Delivery becomes at-least-once, so consumers must be idempotent. → *More:* Ch 10 — Saga · Ch 11 — Event-Driven

```
BEGIN;
UPDATE orders SET status = 'PAID' WHERE id = 42;
INSERT INTO outbox (id, aggregate_id, type, payload)
VALUES (gen_random_uuid(), 42, 'OrderPaid', '{"orderId":42}');
COMMIT;   -- both or neither; the relay publishes the outbox row later
```

Q100. ⋆ What is the dual-write problem? Writing to two systems (database and message broker, or two databases) without a shared transaction. If the process crashes between the two writes, they disagree: the order is saved but no event is published, or the event is published but the save rolled back. Publishing "after commit" in code does not fix it — a crash still loses the event. Solutions: transactional outbox, CDC, or event sourcing (the event *is* the write).

Q101. What is the inbox pattern / idempotent consumer? The consumer stores the IDs of messages it has processed in a table, in the same transaction as the business change. When a duplicate arrives (at-least-once delivery), it finds the ID already there and skips it. A unique constraint on `message_id` makes this race-safe. Together, outbox (producer side) and inbox (consumer side) give "effectively once" processing. → *More:* Ch 1 — Delivery guarantees, idempotency and retries

Q102. ⋆ What is CQRS? Command Query Responsibility Segregation: use separate models for writes (commands that change state and enforce business rules) and reads (queries that return data shaped for screens). It can be just separate classes over one database, or separate databases where the write side publishes events that update one or more denormalised read models (e.g. in Elasticsearch or Redis). It shines when reads vastly outnumber writes or need very different shapes; the cost is complexity and, with separate stores, eventually consistent reads. → *More:* Ch 12 — CQRS

Q103. ⋆ What is event sourcing? How is it different from traditional CRUD? In CRUD you store the current state and overwrite it on each update. In event sourcing you store the full sequence of events that happened to an entity (`AccountOpened` , `MoneyDeposited` , `MoneyWithdrawn`) in an append-only event store, and rebuild current state by replaying them. Benefits: a complete audit log, time travel and debugging, easy creation of new read models, and no dual-write problem. Costs: querying needs projections (usually CQRS), event schemas live forever, and it is unfamiliar to most teams. → *More:* Ch 12 — CQRS, "Level 3 — Event Sourcing" · Ch 11 — Event-Driven, "Event Sourcing and CQRS"

Q104. ⋆ Does CQRS require event sourcing (or vice versa)? No. CQRS can run on one database with separate read and write models, or two databases synced by CDC or events. Event sourcing almost always needs CQRS, because an event log cannot be queried efficiently and you build projections for reads. Treat them as separate decisions and adopt each only when its problem is real. → *More:* Ch 12 — CQRS, "Architecture: three levels"

Q105. What are snapshots in event sourcing? Periodically saving the computed state of an aggregate (say every 100 events) so that loading it means reading the latest snapshot plus only the events after it, instead of replaying thousands of events. They are an optimisation; the event log stays the source of truth.

Q106. How do you query data that lives in several services? Three options: **API composition** — call each service and join in memory (simple, but latency and partial-failure issues); **CQRS read model / materialized view** — a service subscribes to the others' events and keeps a denormalised local copy built for the query (fast, eventually consistent); **data replication** of just the fields you need into your own database. For reporting, stream everything into a data warehouse instead of querying services. → *More:* Ch 12 — CQRS · Ch 3 — API Gateway

Q107. How do you implement the outbox relay — polling vs CDC? Polling: a scheduled job selects unsent rows (`SELECT … FOR UPDATE SKIP LOCKED` so several instances do not grab the same rows), publishes them in order, and marks them sent; simple but adds database load and latency. CDC: Debezium tails the write-ahead log and publishes outbox inserts with low latency and no polling, but adds Kafka Connect to operate. Either way, clean up old rows and keep per-aggregate ordering.

Q108. Can you use Kafka transactions instead of an outbox? Kafka transactions make a consume-transform-produce loop *within Kafka* atomic (exactly-once processing between topics). They do not include your

PostgreSQL commit, so they do not solve the database-plus-event dual write. For that you still need the outbox, CDC or event sourcing.

Q109. How do you handle shared reference data (e.g. country codes, product catalogue) across services? Make one service the owner and publish changes as events; other services keep a read-only local copy updated from those events or cache it with a TTL. For truly static data (currency codes), a versioned shared library or config is acceptable. Never let several services write the same reference tables.

Q110. What is polyglot persistence? Using different storage technologies for different services according to their access patterns: a relational database for orders (transactions), a document store for the catalogue (flexible schema), a search engine for product search, a key-value store for sessions, a graph database for recommendations. Database per service makes it possible; the cost is more technologies to operate and know. → *More:* Ch 1 — Choosing a database

Q111. How do you generate unique IDs across services? Use IDs that need no central coordination: UUIDs (prefer time-ordered UUIDv7 for better index locality), Snowflake-style 64-bit IDs (timestamp + machine ID + sequence), or ULIDs. Avoid a shared database sequence across services — it couples them and becomes a bottleneck. → *More:* Ch 1 — Unique ID generation

Q112. How should a microservice cache data from another service? Cache-aside with a TTL matched to how stale the data may be, invalidated early by the owner's change events if freshness matters. Cache only data you can tolerate being stale (product names, yes; account balance, usually no), and protect against stampedes when hot keys expire (request coalescing, jittered TTLs). → *More:* Ch 1 — Caching strategies · Ch 1 — Cache invalidation

---

7. Consistency, delivery guarantees and ordering

Q113. ⋆ What is eventual consistency? A guarantee that, if no new updates are made, all copies of the data will become consistent after some (usually short) time. Between update and convergence, different services or replicas may return different values. It is the price of independent services and asynchronous messaging; the design question is how long the window is and whether the business can tolerate it (an order showing "processing" for two seconds is fine; overdrawing an account is not). → *More:* Ch 10 — Saga, "Eventual consistency" · Ch 1 — Consistency models

Q114. ⋆ How do you ensure data consistency across microservices? Keep each invariant inside one service's transaction where possible; use sagas with compensations for multi-service processes; publish events reliably with the outbox; make consumers idempotent; use optimistic locking (version columns) for concurrent updates; and add reconciliation jobs that compare systems and fix drift. Accept and design for eventual consistency in the UI (pending states, "we will notify you"). → *More:* Ch 10 — Saga · Ch 11 — Event-Driven, "Data consistency"

Q115. ⋆ Explain the CAP theorem. How does it apply to microservices? CAP says that during a network partition a distributed data store must choose between consistency (every read sees the latest write) and availability (every request gets a non-error answer). Since partitions happen, the real choice is CP or AP behaviour during them. PACELC extends it: else (no partition) you choose between latency and consistency. In microservices, most cross-service data is AP and eventually consistent; things like payments or inventory reservations may need CP within their owning service. → *More:* Ch 1 — CAP theorem and PACELC

Q116. ACID vs BASE? ACID (Atomicity, Consistency, Isolation, Durability) describes strong guarantees of a single database transaction. BASE (Basically Available, Soft state, Eventually consistent) describes systems that stay available and let state converge over time. Inside a service you usually get ACID from its database; across services you design for BASE. → *More:* Ch 1 — Transactions and ACID · Ch 1 — SQL vs NoSQL and BASE

Q117. ⋆ At-most-once, at-least-once and exactly-once delivery — what is the difference? At-most-once: a message may be lost but never duplicated (commit offset before processing). At-least-once: never lost but may be duplicated (commit after processing) — the common default. Exactly-once delivery is impossible in general across a network; what systems provide is exactly-once *processing* — at-least-once delivery plus idempotent handling or transactional deduplication (Kafka transactions within Kafka, inbox tables with a database). → *More:* Ch 1 — Delivery guarantees, idempotency and retries

Q118. How do you handle out-of-order events? Keep ordering where it matters by partitioning on the entity key (one partition per entity's stream). Include a version or sequence number in each event and have consumers ignore events older than what they have already applied. Design handlers to be commutative where possible, and use timestamps only with care because clocks on different machines drift. → *More:* Ch 1 — Clocks and ordering

Q119. ⋆ How do you make a payment API safe to retry (idempotency key)? The client generates a unique idempotency key (a UUID) once per logical operation and sends it in a header on every retry. The server stores the key with the result in the same transaction as the charge, guarded by a unique constraint. A repeated request with the same key returns the stored result instead of charging again; a concurrent duplicate fails the constraint and waits or gets a 409. Keys expire after a retention period (e.g. 24 h). → *More:* Ch 8 — Retry, "Idempotency revisited"

Q120. What is optimistic locking and why is it useful between services? Each record carries a version number; an update succeeds only if the version is unchanged (`UPDATE … SET …, version = version + 1 WHERE id = ? AND version = ?`). If someone else updated it first, zero rows change and the caller retries with fresh data. In JPA it is the `@Version` field. It avoids holding database locks across slow network calls, and ETags with `If-Match` bring the same idea to REST APIs.

Q121. What is read-your-writes consistency and how do you provide it with CQRS? The guarantee that a user sees their own update immediately even if the system is eventually consistent. With an async read model the user might save and then see old data. Options: return the new state in the command's response and update the UI optimistically; read from the write model for that user for a short time; or have the query wait until the read model has reached the version returned by the command. → *More:* Ch 12 — CQRS, "Managing data consistency"

Q122. What is a reconciliation job and why do you still need one? A periodic process that compares data across systems (orders vs payments vs the payment provider's report) and fixes or flags mismatches. Even with sagas and outboxes, bugs, manual fixes and third-party failures cause drift; reconciliation is the safety net that makes eventual consistency trustworthy, especially for money.

Q123. What is the "two generals" / why can't you be sure a remote call failed? When a call times out you do not know whether the request never arrived, was processed and the reply was lost, or is still running. That is why retries need idempotency and why a timeout must be treated as "unknown", not "failed". For important actions, query the status or rely on reconciliation before compensating.

---

8. Observability: logs, metrics and tracing

Q124. ⋆ How do you monitor microservices? What are the three pillars of observability? Observability is the ability to understand a system's internal state from what it emits. The three pillars: **logs** (timestamped records of discrete events, structured as JSON), **metrics** (numeric time series such as request rate, error rate, latency percentiles, CPU), and **traces** (the path and timing of one request across all services). Typical stack: Micrometer in the app, Prometheus + Grafana for metrics, Loki or ELK for logs, OpenTelemetry + Tempo/Jaeger for traces, plus alerting. → *More:* Ch 1 — Observability: logs, metrics and traces

Q125. ⋆ What is distributed tracing and how does it work? It follows one request through every service it touches. The first service creates a **trace ID**; each unit of work (an HTTP call, a DB query) is a **span** with its own span ID, parent span ID, timing and tags. The trace context is propagated in headers (the W3C `traceparent` header is the standard) and through message headers for async calls. Spans are exported to a backend (Jaeger, Zipkin, Tempo) that draws the request as a waterfall, so you see which hop was slow or failed. → *More:* Ch 1 — Observability · Ch 9 — Sidecar

Q126. ⋆ What is a correlation ID? A unique ID attached to a request at the edge and passed through every service call and message, and written into every log line. It lets you search all logs for one user request across services. Today the trace ID from distributed tracing usually plays this role; Spring Boot with Micrometer Tracing puts `traceId` and `spanId` into the logging MDC (Mapped Diagnostic Context) automatically.

Q127. What is OpenTelemetry? OpenTelemetry (OTel) is the vendor-neutral CNCF standard for producing and exporting traces, metrics and logs: APIs, SDKs, auto-instrumentation agents, the OTLP wire protocol and

the Collector (which receives, processes and forwards telemetry to any backend). Instrument once with OTel and switch backends freely. In Spring Boot, Micrometer Tracing bridges to OpenTelemetry, and Spring Boot 4 also ships a `spring-boot-starter-opentelemetry`. → *More:* Ch 1 — Observability

Q128. ⋆ What is centralised logging and why is it needed? With dozens of services and short-lived containers, you cannot SSH into machines to read log files. Each service writes structured logs to stdout; an agent (Fluent Bit, the OTel Collector) ships them to a central store (Elasticsearch/OpenSearch, Loki, a cloud logging service) where you can search by trace ID, service and time. Spring Boot supports structured JSON logging out of the box (`logging.structured.format.console=ecs` or `logstash`).

Q129. What metrics would you track for a microservice? What are the RED and USE methods? RED for request-driven services: Rate (requests per second), Errors (failed requests per second), Duration (latency distribution, especially p95/p99). **USE** for resources: Utilisation, Saturation (queue lengths, thread pool waits), Errors. Add business metrics (orders per minute), JVM metrics (heap, GC pauses), connection pool usage, consumer lag for Kafka, and circuit breaker state. Google's "four golden signals" are latency, traffic, errors and saturation. → *More:* Ch 1 — Percentiles, tail latency and why averages lie

Q130. ⋆ What is Spring Boot Actuator? Actuator adds production-ready endpoints to a Spring Boot app: `/actuator/health` (with liveness and readiness groups for Kubernetes), `/actuator/metrics` and `/actuator/prometheus` (via Micrometer), `/actuator/info`, `/actuator/env`, `/actuator/loggers` (change log levels at runtime) and more. Expose only what you need over HTTP and secure the rest.

Q131. What are SLI, SLO and SLA? What is an error budget? An SLI (service level indicator) is a measured value, e.g. the percentage of requests faster than 300 ms. An SLO (objective) is the target for it, e.g. 99.9% over 30 days. An SLA (agreement) is a contract with customers with penalties, set looser than the SLO. The error budget is 100% minus the SLO (0.1% ≈ 43 minutes per month): while budget remains you ship features; when it is spent you prioritise reliability. → *More:* Ch 1 — SLA, SLO, SLI and error budgets

Q132. How do you alert well in a microservices system? Alert on user-visible symptoms (SLO burn rate, error rate, latency at the edge) rather than every internal cause (high CPU on one Pod), so alerts are few and actionable. Each alert should link to a runbook and dashboard. Use multi-window burn-rate alerts to catch both fast and slow budget burns, and route alerts to the owning team. → *More:* Ch 1 — Dashboards, alerting, on-call and SLO-based alerts

Q133. What is semantic monitoring / synthetic monitoring? Running scripted fake user journeys (log in, search, add to cart) against production continuously and alerting when they fail or get slow. It checks the system works as a whole from the user's point of view, catching problems that per-service metrics miss.

Q134. What is the difference between liveness and readiness health checks? Liveness answers "is the process stuck?" — if it fails, Kubernetes restarts the container. Readiness answers "can it take traffic now?" — if it fails, the Pod is removed from the Service's endpoints but not restarted (e.g. during warm-up or when a critical dependency is unavailable). Do not put downstream dependencies in liveness checks, or one database blip restarts every Pod. A **startup probe** protects slow-starting apps from liveness kills. → *More:* Ch 5 — Health checks

Q135. How do you trace requests across asynchronous messaging? Put the trace context into message headers when producing and extract it when consuming, creating a consumer span linked to the producer span. Spring Kafka and Spring AMQP do this automatically with observation enabled (`spring.kafka.template.observation-enabled=true` and the listener equivalent). With an outbox, store the trace context in the outbox row so the relay can propagate it. → *More:* Ch 11 — Event-Driven

Q136. What is trace sampling and why is it needed? Recording every span of every request is expensive at high traffic, so you keep only a fraction. **Head sampling** decides at the start (e.g. 10% of traces) — cheap but may miss rare errors. **Tail sampling** (in the OTel Collector) decides after the trace finishes, keeping all errors and slow traces and a small sample of normal ones. Spring Boot's `management.tracing.sampling.probability` sets head sampling (default 0.1).

---

9. Deployment: containers, Kubernetes, CI/CD and release strategies

Q137. ⋆ Why are Docker containers used for microservices? A container packages a service with its runtime and dependencies into an image that runs the same on a laptop, in CI and in production. Containers start in seconds, use fewer resources than virtual machines (they share the host kernel), isolate services from each other, and give a uniform unit for orchestration. Spring Boot can build images without a Dockerfile (`./mvnw spring-boot:build-image` using Cloud Native Buildpacks). → *More:* Ch 1 — Virtual machines, containers and Docker

Q138. ⋆ Why is Kubernetes used for microservices? Name its core objects. Kubernetes is a container orchestrator: it schedules containers on a cluster, restarts failed ones, scales them, does rolling updates and provides service discovery and config. Core objects: **Pod** (one or more containers sharing network), **Deployment** (desired replicas, rolling updates), **Service** (stable name and load balancing), **Ingress/Gateway API** (HTTP routing from outside), **ConfigMap** and **Secret** (configuration), **HorizontalPodAutoscaler**, **StatefulSet** (stateful workloads), **Namespace** (isolation). → *More:* Ch 1 — Kubernetes basics

Q139. ⋆ What deployment strategies do you know? Blue-green vs canary vs rolling? Rolling: replace instances a few at a time — no extra capacity, but old and new versions run together. **Blue-green**: run the new version (green) next to the old (blue) and switch all traffic at once; instant rollback by switching back, but doubles capacity during the switch. **Canary**: send a small share of traffic (1%, 5%, 25%…) to the new version, watch metrics, then expand or roll back; safest for catching real-traffic bugs. Also **recreate** (stop all, start new — downtime) and **shadow/dark launch** (copy traffic to the new version without returning its responses). → *More:* Ch 1 — Deployment strategies and rollback

Q140. How do you do a canary release in Kubernetes? Basic: two Deployments behind one Service with replica counts in the desired ratio (coarse). Better: weighted routing in a service mesh (Istio VirtualService), the Gateway API (`HTTPRoute` weights) or an ingress controller, automated with Argo Rollouts or Flagger, which shift traffic in steps and roll back automatically when metrics such as error rate or p99 degrade. → *More:* Ch 1 — Deployment strategies and rollback

Q141. ⋆ How do you achieve zero-downtime deployment? Run several replicas and use rolling or blue-green updates; set readiness probes so new Pods get traffic only when ready; implement graceful shutdown (Spring Boot's `server.shutdown=graceful` finishes in-flight requests after SIGTERM, with a `preStop` delay so the load balancer stops sending first); keep APIs and database schemas backward compatible across the two versions running side by side; use a PodDisruptionBudget.

Q142. ⋆ How do you handle database schema changes with zero downtime? Use the **expand-and-contract** (parallel change) pattern: (1) expand — add the new column or table in a backward-compatible way; (2) deploy code that writes both old and new and reads new with fallback; (3) backfill data; (4) switch reads fully to new; (5) contract — remove the old column in a later release. Run migrations with Flyway or Liquibase as part of the deployment. Never rename or drop in the same release as the code change.

Q143. What is the difference between deployment and release? What are feature flags? Deployment puts new code into production; release exposes the feature to users. Feature flags (toggles checked at runtime) separate the two: you deploy dark, then turn the feature on for internal users, a percentage, or a region, and turn it off instantly if it misbehaves. Flags need cleanup afterwards, or they become technical debt. → *More:* Ch 2 — Strangler Fig, "What is a feature flag?"

Q144. ⋆ What does CI/CD look like for microservices? Each service has its own pipeline triggered by its own repository or path: build, unit tests, contract tests, static analysis and security scanning, build and sign a container image, deploy to staging, run integration/smoke tests, then progressive delivery to production (canary) with automated rollback. GitOps (Argo CD, Flux) keeps the desired cluster state in Git and syncs it. Independence of pipelines is the point — no shared release train. → *More:* Ch 1 — CI/CD

Q145. Monorepo vs polyrepo for microservices? A monorepo puts all services in one repository: easier atomic cross-service changes, shared tooling and code search, but needs build tooling that only builds what changed (Bazel, Nx, Gradle) and discipline to avoid hidden coupling. Polyrepo gives each service its own repository: clear ownership and independent pipelines, but cross-cutting changes and dependency upgrades need many PRs. Both work; the boundary discipline matters more.

Q146. How does autoscaling work for microservices on Kubernetes? The HorizontalPodAutoscaler adds or removes Pods based on metrics (CPU, memory, or custom ones like requests per second or Kafka consumer lag); KEDA scales on event-source metrics including down to zero. The Cluster Autoscaler or Karpenter adds nodes when Pods cannot be scheduled. Scaling only works if services are stateless and start quickly; set resource requests correctly, or HPA's percentages are meaningless. → *More:* Ch 1 — Autoscaling

Q147. What is Infrastructure as Code (IaC)? Defining infrastructure (networks, clusters, databases, DNS) in version-controlled files — Terraform/OpenTofu, Pulumi, CloudFormation, Helm charts or Kustomize for Kubernetes manifests — and applying them through automation instead of manual console clicks. It makes environments reproducible, reviewable and easy to recreate. → *More:* Ch 1 — Infrastructure as code

Q148. How do you make a Spring Boot service start fast and use less memory in containers? Options: layered JARs and buildpacks for cache-friendly images; Class Data Sharing (CDS) and Project Leyden's AOT cache (ahead-of-time class loading and linking since Java 24, with simpler creation and method profiling added in Java 25) to cut startup; GraalVM native images for millisecond startup and small memory (with build-time restrictions on reflection); virtual threads to serve many requests with few platform threads; and setting `-XX:MaxRAMPercentage` so the JVM respects container memory limits.

Q149. What is a Helm chart? A package of templated Kubernetes manifests with a `values.yaml` file for per-environment settings, installed and upgraded as a versioned "release". It lets you deploy the same service to dev, staging and production with different replica counts, resources and config, and roll back a release.

Q150. What is the Kubernetes Gateway API and how does it relate to Ingress? Ingress is the original, limited API for HTTP routing into a cluster. The Gateway API is its more expressive successor: `Gateway` resources (owned by platform teams) and `HTTPRoute` / `GRPCRoute` resources (owned by app teams) supporting header-based routing, traffic splitting by weight and cross-namespace delegation. Implementations include Envoy Gateway, Istio, NGINX and cloud load balancers.

Q151. How do you roll back a bad deployment? Keep the previous image and config: `kubectl rollout undo`, `helm rollback`, revert the Git commit in GitOps, or switch blue-green back. Automate rollback on canary metric failure. Rollback is only safe if the database migration was backward compatible (expand-and-contract) — which is why you never ship a destructive migration together with the code. → *More:* Ch 1 — Deployment strategies and rollback

---

10. Configuration management

Q152. ⋆ How do you manage configuration across microservices? What is externalised configuration? Keep configuration out of the build artifact so the same image runs in every environment. Sources, by increasing centralisation: environment variables, Kubernetes ConfigMaps and Secrets mounted as files or env vars, a config server (Spring Cloud Config backed by Git), or a store such as Consul, AWS AppConfig or Parameter Store. Secrets go to a dedicated vault. Config should be versioned, auditable and refreshable without redeploying. → *More:* Ch 13 — Configuration Externalization

Q153. ⋆ How does Spring Cloud Config work? A Config Server (`@EnableConfigServer`) serves properties from a backend, usually a Git repository with files like `order-service-prod.yml` . Clients import it on startup with `spring.config.import=optional:configserver:http://config:8888` and receive the properties for their `spring.application.name` and active profiles. Values can be encrypted (`{cipher}…`), and Git gives history and rollback. → *More:* Ch 13 — Worked example: Spring Cloud Config

Q154. ⋆ How do you refresh configuration without restarting a service? Beans annotated `@RefreshScope` (and `@ConfigurationProperties` beans) are rebuilt when a refresh event fires. Trigger it per instance with `POST /actuator/refresh` , or for all instances at once with **Spring Cloud Bus** (`/actuator/busrefresh`) over Kafka or RabbitMQ, often triggered by a Git webhook. On Kubernetes, Spring Cloud Kubernetes can reload on ConfigMap changes, or you roll the Pods. → *More:* Ch 13 — "Refreshing without restarting"

Q155. ⋆ How do you manage secrets in microservices? Never in Git or images. Use a secrets manager — HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager — accessed with short-lived credentials (Kubernetes service account identity), or Kubernetes Secrets encrypted at rest and synced from the vault (External Secrets Operator). Prefer dynamic, short-lived credentials (Vault database

secrets) and rotate regularly. Spring Cloud Vault integrates Vault as a property source. → *More:* Ch 13 — "Security — safeguarding sensitive configuration" · Ch 1 — Encryption, hashing, passwords and secrets

Q156. What if the config server is down? Clients must not fail to start because of it: run the config server with several replicas, cache last-known-good config locally, use `optional:` imports with sensible defaults, or enable fail-fast with retry when config is mandatory. Many Kubernetes shops avoid the extra server entirely by using ConfigMaps, which the platform already keeps highly available. → *More:* Ch 13 — "The configuration server as a single point of failure"

Q157. How do you override Spring Boot properties for different environments? Profiles (`application-prod.yml` activated with `spring.profiles.active=prod`), environment variables (relaxed binding: `SPRING_DATASOURCE_URL` maps to `spring.datasource.url`), command-line arguments, config-tree files mounted from Kubernetes, or the config server. Later sources override earlier ones; command-line arguments and environment variables beat packaged files.

Q158. Config server vs Kubernetes ConfigMaps — which would you choose? ConfigMaps/Secrets: native, no extra server, work for every language, but no built-in history beyond GitOps and refresh needs reload or restart. Spring Cloud Config: Git history, encryption, per-profile layering and bus refresh, but it is another critical service, and it is Spring-specific. On Kubernetes with GitOps, ConfigMaps are usually enough; the config server shines outside Kubernetes or when many Spring apps share layered config. → *More:* Ch 13 — Configuration Externalization

---

11. Security: OAuth2, JWT, mTLS and zero trust

Q159. ⋆ How do you secure microservices? In layers. At the edge: TLS everywhere, an API gateway that authenticates requests (OAuth 2.0 / OpenID Connect tokens), rate limiting and a WAF (web application firewall). Between services: mTLS for encrypted, mutually authenticated traffic and network policies that allow only expected calls. Inside services: validate the token, enforce fine-grained authorization, validate input. Plus secrets in a vault, least-privilege service accounts, image scanning and audit logs. → *More:* Ch 1 — Authentication and authorization · Ch 1 — Least privilege and zero trust

Q160. ⋆ What is OAuth 2.0? What are its roles? OAuth 2.0 is an authorization framework that lets a client obtain a limited-scope access token to call an API on behalf of a user (or itself), without handling the user's password. Roles: **resource owner** (user), **client** (the app), **authorization server** (issues tokens — Keycloak, Okta, Auth0, Spring Authorization Server), **resource server** (the API that validates tokens). OAuth is about authorization; **OpenID Connect** adds an ID token and user info on top for authentication (login). → *More:* Ch 1 — OAuth 2.0, OpenID Connect, SSO and API keys

Q161. ⋆ Which OAuth 2.0 grant types do you use, and when? Authorization code with PKCE for user-facing apps (web, mobile, single-page apps) — PKCE protects the code exchange. **Client credentials** for service-to-service calls with no user. **Refresh token** to get new access tokens without re-login. **Device code** for TVs and CLIs. Do not use the implicit grant (tokens leak through the browser URL) or the resource-owner password grant (the client sees the user's password): the OAuth 2.0 Security Best Current Practice (RFC 9700) says the password grant must not be used and the implicit grant should not be, and the OAuth 2.1 draft drops both.

Q162. ⋆ What is a JWT and how is it validated? A JSON Web Token is a compact, signed token with three Base64URL parts: header (algorithm, key ID), payload (claims such as `sub` , `iss` , `aud` , `exp` , `scope`) and signature. A service validates it locally — without calling the auth server — by checking the signature with the issuer's public key (fetched from its JWKS endpoint and cached, picked by the `kid` header) using only the algorithms you expect (never accept `alg: none`), then `exp` (and `nbf`), `iss` and `aud` . JWTs are signed, not encrypted, so never put secrets in them. In Spring: `spring.security.oauth2.resourceserver.jwt.issuer-uri` . → *More:* Ch 1 — Sessions, cookies, tokens and JWT

Q163. ⋆ How do you revoke a JWT? Opaque tokens vs JWTs? A JWT is valid until it expires, so keep access tokens short-lived (5–15 minutes) and revoke the refresh token; for urgent revocation keep a denylist of token IDs (`jti`) in a shared store such as Redis, checked by the gateway or each resource server, with each entry kept only until that token would have expired anyway. That check brings back a lookup per request, which is the cost of revocability. Opaque tokens are random strings the resource server

must check with the authorization server (token introspection) — instantly revocable but each check costs a call (usually cached). A common mix: opaque at the edge, exchanged by the gateway for a short-lived internal JWT.

Q164. How do you propagate user identity between services? Forward the user's access token on downstream calls (simple, but every service receives a broad token), or use **token exchange** (RFC 8693) to get a narrower token for the downstream audience. For calls with no user, services authenticate as themselves with the client-credentials grant or workload identity. Never trust a plain `X-User-Id` header unless it comes from a component you authenticate (e.g. via mTLS from the gateway).

Q165. ⋆ What is mTLS and why is it used between services? In normal TLS only the server proves its identity with a certificate. In mutual TLS both sides present certificates, so each service knows exactly which service is calling, and traffic is encrypted. Managing certificates for hundreds of services by hand is hard, so service meshes (Istio, Linkerd) issue and rotate short-lived certificates automatically (often with SPIFFE identities) and enforce mTLS in the sidecar without code changes. → *More:* Ch 9 — Sidecar · Ch 1 — HTTPS and TLS

Q166. ⋆ What is zero trust? A security model that assumes the network is already compromised: no request is trusted because it comes from "inside". Every call is authenticated and authorised (mTLS plus tokens), access is least-privilege and short-lived, and traffic is encrypted and logged. It replaces the old "hard shell, soft centre" perimeter model. → *More:* Ch 1 — Least privilege and zero trust

Q167. Authentication vs authorization? Authentication answers "who are you?" (verifying identity with a password, token or certificate). Authorization answers "what are you allowed to do?" (roles, scopes, ownership checks). A request is first authenticated, then authorised for each action. → *More:* Ch 1 — Authentication and authorization

Q168. Where should a single-page app keep its tokens? What is the token-handler (BFF) pattern for security? Tokens in browser storage (`localStorage`) are readable by any XSS script. The recommended pattern is a security BFF: the browser holds only an `HttpOnly`, `Secure`, `SameSite` session cookie, while the BFF performs the OAuth flow, stores tokens server-side and attaches them to API calls. Spring Cloud Gateway with the `TokenRelay` filter is a common way to build it. → *More:* Ch 4 — BFF · Ch 1 — Common web vulnerabilities

Q169. How do you protect services from the OWASP API Top 10 risks? Object-level authorization on every resource (the most common bug: user A reading `/orders/123` of user B), strict input validation, rate limiting, no excessive data in responses, inventory of all API versions, secured management endpoints (Actuator), and protection against server-side request forgery when calling URLs supplied by users. Test these explicitly; gateways alone do not fix object-level authorization.

Q170. What are API keys and when are they acceptable? A static secret string identifying a calling application. They are fine for identifying and rate-limiting partner or server-side clients of low-risk APIs, but they carry no user identity, rarely expire and are easily leaked, so do not use them to authorise user actions. Rotate them, scope them and send them only over TLS. → *More:* Ch 1 — OAuth 2.0, OpenID Connect, SSO and API keys

Q171. How do you secure Kafka or other message brokers? TLS for encryption in transit, authentication of clients (mTLS or SASL/OAUTHBEARER), ACLs so each service can produce or consume only its own topics, encryption at rest, and no sensitive personal data in events unless needed (or encrypt those fields). Remember that events are long-lived copies of data, which matters for privacy rules such as GDPR deletion.

Q172. How do you secure the software supply chain for containers? Use minimal base images, scan images and dependencies for vulnerabilities in CI, generate an SBOM (software bill of materials), sign images (Sigstore cosign) and verify signatures at admission in Kubernetes, pin image digests, run containers as non-root with read-only file systems, and apply Pod Security Standards.

---

12. Testing microservices

Q173. ⋆ How do you test microservices? What types of tests do you use? Follow the test pyramid: many fast **unit tests** of domain logic; **integration tests** of each service with its real infrastructure

(database, broker) using Testcontainers; **component tests** of one service in isolation with its dependencies stubbed (WireMock); **contract tests** between consumers and providers; and a few **end-to-end tests** of critical journeys. Add performance, resilience (chaos) and security tests, and monitor production as a test of last resort.

Q174. ⋆ What is contract testing? What is consumer-driven contract testing? A contract test checks that a provider's API matches what each consumer expects, without deploying both together. In consumer-driven contracts, each consumer writes down the requests it makes and the responses it relies on; the provider's build verifies it satisfies every consumer contract, so a breaking change fails the provider's CI before release. Tools: **Pact** (contracts shared through a Pact Broker, `can-i-deploy` checks) and **Spring Cloud Contract** (contracts generate provider tests and consumer stubs). → *More:* Ch 4 — BFF

Q175. What is Pact? Pact is a consumer-driven contract testing tool available for many languages. The consumer's test runs against a Pact mock server and records a "pact" file of interactions; the pact is published to a broker; the provider replays those interactions against its real API to verify them. The broker's `can-i-deploy` check tells you whether a version is compatible with what is deployed in each environment.

Q176. ⋆ What is the test pyramid (Mike Cohn)? A guideline that a test suite should have many cheap, fast unit tests at the bottom, fewer integration/service tests in the middle, and very few slow, brittle UI/end-to-end tests at the top. In microservices, contract tests replace much of the expensive end-to-end layer.

Q177. Why are end-to-end tests problematic in microservices? They need many services, databases and brokers deployed together in the right versions; they are slow and flaky, failures are hard to pinpoint, and they create a shared environment that becomes a release bottleneck — reintroducing the coupling microservices try to remove. Keep a handful for critical journeys and rely on contracts and component tests for the rest.

Q178. What is Testcontainers and why use it? A library that starts real dependencies — PostgreSQL, Kafka, Redis, even LocalStack for AWS — in throwaway Docker containers during tests. Tests run against the same technology as production instead of in-memory fakes like H2. Spring Boot supports it with `@ServiceConnection`, which wires the container's connection details automatically. Spring Boot 4 uses Testcontainers 2, where each module has its own package and the container classes are no longer generic (`org.testcontainers.postgresql.PostgreSQLContainer` from `testcontainers-postgresql`).

```
@SpringBootTest
@Testcontainers
class OrderRepositoryIT {
  @Container @ServiceConnection
  static PostgreSQLContainer pg = new PostgreSQLContainer("postgres:17");
  // tests now use a real PostgreSQL
}
```

Q179. How do you test a service that calls another service? In unit tests, mock the client interface. In component tests, stub the remote HTTP API with WireMock (or Spring Cloud Contract stub runner) and test timeouts, errors and slow responses too, since those paths are where resilience bugs hide. Verify the stub matches reality with contract tests.

Q180. How do you test asynchronous, event-driven flows? Test producers by asserting the event published (to an embedded or Testcontainers Kafka) for a given command; test consumers by publishing an event and asserting the resulting state, using polling assertions (Awaitility) instead of sleeps. Include tests for duplicate and out-of-order messages, and for poison messages reaching the DLQ. Contract-test event schemas too (Pact supports message pacts).

Q181. What is consumer-side vs provider-side stubbing in Spring Cloud Contract? The provider writes contracts (Groovy/YAML DSL, ideally with consumer input) in its repository; the build generates provider verification tests and publishes WireMock stubs as a JAR. Consumers pull those stubs with the Stub Runner and test against them, so both sides are tested against the same contract.

Q182. What is canary analysis / testing in production? Because staging never matches production exactly, teams verify releases with production traffic under control: canaries with automated metric comparison, feature flags for small user groups, shadow traffic, synthetic monitoring, and chaos

experiments. It complements pre-production tests, it does not replace them. → *More:* Ch 1 — Deployment strategies and rollback

Q183. How do you performance-test a microservice? Load-test with realistic traffic patterns (k6, Gatling, JMeter) against a production-like environment; measure throughput and p95/p99 latency, not averages; find the saturation point and which resource saturates first; test with downstream latency injected; and re-run in CI for key services to catch regressions. → *More:* Ch 1 — Percentiles, tail latency and why averages lie

---

13. Spring Cloud and Spring Boot specifics

Q184. ⋆ What is Spring Cloud? What problems does it solve? Spring Cloud is a set of projects on top of Spring Boot that implement common distributed-system patterns: configuration (Spring Cloud Config, Vault, Kubernetes), service discovery (Eureka, Consul, Kubernetes), client-side load balancing (Spring Cloud LoadBalancer), API gateway (Spring Cloud Gateway), circuit breakers (Spring Cloud Circuit Breaker over Resilience4j), messaging (Spring Cloud Stream, Bus), contract testing (Spring Cloud Contract) and declarative HTTP clients (OpenFeign). It is versioned as a release train: **2025.1 "Oakwood"** is the line for Spring Boot 4.

Q185. ⋆ Spring Boot vs Spring Cloud? Spring Boot builds a single production-ready application quickly: auto-configuration, embedded server, starters, Actuator. Spring Cloud adds the tools for many such applications to work together as a distributed system: discovery, config, gateway, resilience, messaging. Spring Boot is about one service; Spring Cloud is about the system of services.

Q186. ⋆ Why is Spring Boot widely used for microservices? It removes boilerplate with auto-configuration and starters, runs as a self-contained JAR with an embedded server, has production features built in (Actuator health checks, Micrometer metrics and tracing, graceful shutdown, structured logging), integrates natively with Spring Cloud, Kafka, security and Testcontainers, builds container images and native images, and now supports virtual threads with a single property.

Q187. ⋆ What is Spring Cloud Gateway? How do routes, predicates and filters work? Spring Cloud Gateway is Spring's API gateway. A **route** has an ID, a destination URI (which can be `lb://order-service` to load-balance through discovery), **predicates** that decide which requests match (path, host, header, method) and **filters** that change requests or responses (strip prefix, add headers, rate limit, circuit breaker, retry, token relay). In Spring Cloud 2025.x it comes in two flavours — **Server WebFlux** (reactive, Netty) and **Server Web MVC** (servlet, works well with virtual threads) — with the starters `spring-cloud-starter-gateway-server-webflux` and `spring-cloud-starter-gateway-server-webmvc` (the old `spring-cloud-starter-gateway` names were removed in 2025.1). → *More:* Ch 3 — API Gateway

```
@Bean
RouteLocator routes(RouteLocatorBuilder rlb) {
  return rlb.routes()
      .route("orders", r -> r.path("/api/orders/**")
          .filters(f -> f.stripPrefix(1)
              .circuitBreaker(c -> c.setName("orders").setFallbackUri("forward:/fallback")))
          .uri("lb://order-service"))
      .build();
}
```

Q188. What replaced Netflix Zuul, Hystrix and Ribbon? They entered maintenance mode in Spring Cloud Greenwich (2019) and were later removed. Zuul → **Spring Cloud Gateway**; Hystrix → **Resilience4j** (via Spring Cloud Circuit Breaker); Ribbon → **Spring Cloud LoadBalancer**; Hystrix Dashboard/Turbine → **Micrometer** with Prometheus/Grafana. Eureka is still supported. Spring Cloud Sleuth was replaced by **Micrometer Tracing** from Spring Boot 3 onwards.

Q189. ⋆ How do you set up a Eureka server and client? Server: add `spring-cloud-starter-netflix-eureka-server` and annotate the application with `@EnableEurekaServer`. Client: add `spring-cloud-starter-netflix-eureka-client`, set `spring.application.name` and `eureka.client.service-url.defaultZone`; it registers and sends heartbeats automatically (`@EnableDiscoveryClient` is optional). Callers then use the logical name through a `@LoadBalanced` client or `lb://` gateway routes. → *More:* Ch 5 — Service Discovery

Q190. ⋆ What is OpenFeign? What are HTTP interface clients and which should you use now? OpenFeign lets you declare an interface annotated with `@FeignClient("order-service")` and Spring generates the HTTP client, integrated with discovery and load balancing. The Spring Cloud team has declared OpenFeign feature-complete and recommends Spring's own **HTTP interface clients**: an interface with `@GetExchange` / `@PostExchange` methods backed by `RestClient` or `WebClient`. Spring Framework 7 makes them easy to register in groups with `@ImportHttpServices`.

```
public interface InventoryClient {
  @GetExchange("/stock/{sku}")
  Stock stock(@PathVariable String sku);
}
```

Q191. RestTemplate vs RestClient vs WebClient? `RestTemplate` is the old synchronous client: Spring announced its deprecation with Framework 7.0, it is marked deprecated for removal in 7.1, and removal is planned for 8.0. `RestClient` (Spring 6.1+) is the modern synchronous client with a fluent API — the default choice, especially with virtual threads. `WebClient` is the reactive, non-blocking client from WebFlux, for reactive stacks or streaming. All can be `@LoadBalanced` and back HTTP interface clients.

Q192. What is Spring Cloud Circuit Breaker? An abstraction over circuit-breaker libraries (`CircuitBreakerFactory`, `ReactiveCircuitBreakerFactory`) so code is not tied to one implementation; Resilience4j is the main implementation (the Spring Retry implementation is in maintenance-only mode since Spring Cloud 2025.1). You can use it programmatically (`factory.create("inventory").run(supplier, fallback)`) or use Resilience4j's own annotations directly. → *More:* Ch 6 — Circuit Breaker

Q193. ⋆ What is Spring Cloud Stream? A framework for event-driven services that hides the broker behind "binders" (Kafka, RabbitMQ, others). You write plain `java.util.function` beans — `Supplier` (producer), `Function` (processor), `Consumer` (sink) — and bind them to destinations in configuration, e.g. `spring.cloud.function.definition=processOrder` with `spring.cloud.stream.bindings.processOrder-in-0.destination=orders`. It handles serialization, consumer groups, partitioning, retries and DLQs. For Kafka-specific control, plain Spring for Apache Kafka (`@KafkaListener`, `KafkaTemplate`) is also common. → *More:* Ch 11 — Event-Driven

Q194. What is Spring Cloud Bus? It links service instances through a message broker (Kafka or RabbitMQ) so that management events reach all of them, most commonly a configuration refresh: one call to `/actuator/busrefresh` makes every instance reload its `@RefreshScope` beans. → *More:* Ch 13 — Configuration Externalization

Q195. What is Spring Cloud Kubernetes? Integration that lets Spring apps use Kubernetes-native features through Spring abstractions: ConfigMaps and Secrets as property sources (with reload), Kubernetes Services for `DiscoveryClient` and load balancing, so you do not need Eureka or a config server on Kubernetes. Many teams use none of it and rely on plain Kubernetes DNS and mounted config.

Q196. How do you implement a saga or outbox in Spring? Outbox: in a `@Transactional` method, save the entity and an outbox entity; relay with a scheduled poller or Debezium. **Spring Modulith**'s event publication registry gives a ready outbox-like mechanism: `ApplicationEvent`s published in a transaction are stored and re-delivered (and can be externalised to Kafka) if a listener fails. Saga orchestration can be a Spring state machine-style service, or a workflow engine such as Temporal or Camunda with Spring Boot starters; choreography uses Kafka listeners plus compensating handlers. → *More:* Ch 10 — Saga

Q197. ⋆ What is the `@Transactional` pitfall when publishing Kafka messages? Sending to Kafka inside a `@Transactional` database method is a dual write: if the database commit fails after the send, consumers see an event for data that does not exist; if you send after commit and crash, the event is lost. Also, `@Transactional` on a private method or a self-call is ignored because Spring's proxy is bypassed. Use the outbox, or `@TransactionalEventListener(phase = AFTER_COMMIT)` only when losing an occasional event is acceptable. → *More:* Ch 10 — Saga

Q198. How does Spring Boot support Kubernetes probes and graceful shutdown? On Kubernetes, Actuator exposes `/actuator/health/liveness` and `/actuator/health/readiness` automatically, driven by the application's availability state (readiness goes down during shutdown). `server.shutdown=graceful` (the default since Boot 3.4) lets in-flight requests finish within `spring.lifecycle.timeout-per-shutdown-phase` after SIGTERM.

Q199. What is Spring Modulith and when would you use it? A Spring project for building well-structured modular monoliths: each top-level package is an application module; tests verify modules access each other only through public APIs; modules communicate through application events (with a persistent publication registry); and it generates module documentation. It is the recommended way to start before splitting into microservices, and it makes later extraction easier.

Q200. How do you add distributed tracing to a Spring Boot microservice? Add Actuator plus Micrometer Tracing with a bridge (OpenTelemetry) and an exporter (OTLP), or simply Spring Boot 4's `spring-boot-starter-opentelemetry`; set `management.opentelemetry.tracing.export.otlp.endpoint` (Spring Boot 3 called it `management.otlp.tracing.endpoint`) and `management.tracing.sampling.probability`. HTTP server and client calls (`RestClient`, `WebClient`), Kafka, JDBC (with extra instrumentation) and `@Observed` methods produce spans, and trace IDs appear in logs automatically. → *More:* Ch 1 — Observability

---

14. Scenario, troubleshooting and senior-level questions

These are the open-ended questions of senior rounds. Structure every answer the same way: **clarify → find the cause or the constraint → name the pattern(s) → state the trade-off**.

Q201. ⋆ One microservice is slow, but the system is otherwise fine. How do you find the cause? Start from the trace: open a slow request's distributed trace and see which span dominates — its own code, a database query, or a downstream call. Check the service's RED metrics and when the slowdown started (correlate with a deploy, traffic spike or dependency change). Then look at resources: CPU throttling from container limits, GC pauses, thread or connection pool saturation, slow queries (missing index, lock waits), and noisy neighbours. Fix the cause, and add a timeout/bulkhead so this service cannot drag callers down next time. → *More:* Ch 1 — Observability · Ch 1 — Bottlenecks

Q202. ⋆ A downstream service your service depends on is down. What happens and what should you do? Without protection your threads wait on timeouts and you fail too — a cascade. With protection: short timeouts, the circuit breaker opens after the failure threshold and calls fail fast; a fallback serves cached or default data, or the request is queued for later; the bulkhead keeps other features working; alerts fire on the Open state. If the dependency is critical (payments), return a clear error quickly instead of hanging. Long-term, ask whether the call needs to be synchronous at all. → *More:* Ch 6 — Circuit Breaker · Ch 7 — Bulkhead

Q203. ⋆ Design the order placement flow for an e-commerce system with Order, Payment and Inventory services. The Order service saves the order as `PENDING` and an `OrderCreated` event in its outbox, returning 202 Accepted with the order ID. An orchestrator (or choreography) runs the saga: reserve stock in Inventory, then charge the card in Payment (the pivot), then confirm the order. On failure, compensate: release stock, refund, mark the order `CANCELLED`. Every step is idempotent (keyed by order ID), consumers dedupe, the UI shows the pending state and is notified on completion. Add a timeout for stuck sagas and reconciliation with the payment provider. → *More:* Ch 10 — Saga, "Worked example: online shopping"

Q204. ⋆ A customer was charged twice. How could that happen and how do you prevent it? Likely causes: a client or gateway retried a POST after a timeout while the first request had actually succeeded; a message consumer processed a redelivered message twice; or two instances processed the same outbox row. Prevent with an idempotency key on the payment API stored with a unique constraint, idempotent consumers (inbox table), retries in only one layer, `SKIP LOCKED` or a single relay for the outbox, passing the same idempotency key to the payment provider, and a reconciliation job that detects and refunds duplicates. → *More:* Ch 8 — Retry, "Idempotency revisited"

Q205. ⋆ How would you migrate a large monolith to microservices without stopping feature work? Do not rewrite. First modularise inside the monolith and add observability and CI/CD. Put a facade (gateway) in front. Pick a first service with clear boundaries and business value, build it, route its traffic through the facade behind a feature flag, run old and new in parallel and compare results, then switch over and delete the old code. Split data with CDC or events. Repeat capability by capability, measuring deployment frequency and lead time to check it is paying off. → *More:* Ch 2 — Strangler Fig

Q206. ⋆ Kafka consumer lag is growing. What do you check? Lag means consumers process slower than producers write. Check: a sudden traffic spike; slow processing per message (a slow downstream call or

database); a poison message causing endless retries on one partition; frequent consumer-group rebalances (processing taking longer than `max.poll.interval.ms`); too few partitions to add more consumers (parallelism is capped at partitions); or one hot partition from a skewed key. Fixes: speed up or batch the handler, add consumers up to the partition count, increase partitions (which changes key mapping), send poison messages to a DLQ, autoscale on lag (KEDA). → *More:* Ch 1 — Event streaming

Q207. How would you handle a sudden 10× traffic spike (e.g. a flash sale)? Prepare: load-test, pre-scale critical services, warm caches, and set autoscaling on the right metrics. During the spike: rate limit at the gateway, queue orders asynchronously to absorb the burst, shed non-critical work (recommendations, analytics), serve read-heavy pages from CDN and cache, and protect the database (connection pool limits, read replicas). Use a virtual waiting room if even that is not enough. → *More:* Ch 1 — Load shedding and graceful degradation · Ch 7 — Bulkhead

Q208. ⋆ Two services need each other's data and call each other synchronously (a cycle). How do you fix it? A cycle means the boundary is probably wrong or the dependency should be asynchronous. Options: merge the two services if they always change together; move the shared concept into the service that owns it; have one side keep a local copy of the data it needs, updated by events; or introduce a third service for the shared workflow (orchestration). Cyclic synchronous calls also risk deadlocked thread pools during load.

Q209. How do you implement a "get order details" endpoint needing data from five services? Either API composition in a BFF/aggregator — parallel calls with per-call timeouts, returning partial data with flags when optional parts fail — or a CQRS read model (an "order view" service) that subscribes to all five services' events and stores a denormalised document, answering in one fast query at the cost of eventual consistency. Choose composition for low traffic or freshness-critical data; the read model for hot, read-heavy pages. → *More:* Ch 12 — CQRS · Ch 4 — BFF

Q210. A deployment caused errors in production. Walk through your response. Mitigate first, investigate second: roll back (or flip the feature flag off) as soon as the deploy is the likely cause, communicate status, and confirm metrics recover. Then use traces, logs and the diff to find the root cause, write a blameless post-mortem, and add the missing safeguard — a contract test, a canary with automated analysis, an alert. Ask why the canary or tests did not catch it.

Q211. How would you design rate limiting for a public API used by many clients? Identify the client (API key or OAuth client ID); define tiers with limits per second and per day; enforce with a token bucket in Redis at the gateway, using atomic Lua scripts; return 429 with `Retry-After` and rate-limit headers; add per-endpoint limits for expensive operations and a global limit to protect the backend; and consider local in-memory limiters for speed with a shared store for accuracy. → *More:* Ch 1 — Rate limiting and throttling · Ch 3 — API Gateway

Q212. ⋆ A service needs to send an email/notification only after an order is committed. How do you guarantee it is sent exactly once? You cannot guarantee exactly-once delivery to an external email provider, but you can get "at least once, deduplicated". Write an `OrderConfirmed` event to the outbox in the order transaction; a notification service consumes it idempotently (keyed by event ID), sends the email with an idempotency key if the provider supports it, and records it as sent. Retries with backoff handle provider outages; a DLQ catches permanent failures.

Q213. How do you handle a long-running business process (days, e.g. loan approval)? Model it as an orchestrated saga or workflow with durable state: a workflow engine (Temporal, Camunda) or a state machine persisted in the database. Steps are triggered by events or timers, wait for human tasks, have timeouts and escalation, and can resume after crashes. Expose the status through an API and notify users on changes instead of keeping requests open. → *More:* Ch 10 — Saga

Q214. You see intermittent 502/503 errors during deployments. What is the likely cause? Traffic is reaching Pods that are not ready or are shutting down: missing or too-lenient readiness probes, no graceful shutdown, or the load balancer still sending to a Pod after it received SIGTERM. Fix with correct readiness probes, `server.shutdown=graceful`, a short `preStop` sleep so endpoints are removed before the app stops, `maxUnavailable: 0` in the rolling update, and retries for idempotent requests at the gateway. → *More:* Ch 1 — Deployment strategies and rollback

Q215. How do you make a microservice multi-tenant? Choose an isolation model: shared database with a `tenant_id` column on every table (cheapest; enforce with row-level security in PostgreSQL), schema per tenant, or database per tenant (strongest isolation, costliest). Carry the tenant ID in the token,

propagate it in context, apply per-tenant rate limits and quotas so one tenant cannot starve others (a bulkhead per tenant), and include it in logs and metrics.

Q216. ⋆ How do you decide whether a feature should be a new microservice or part of an existing one? Ask: does it belong to an existing bounded context (same language, same data, changes together)? Will a different team own it? Does it need different scaling, availability or technology? Would separating it create chatty calls or a distributed transaction? Default to adding it to the existing service; split when ownership, change rate or scaling needs clearly diverge.

Q217. How would you design a system to guarantee that inventory is never oversold? Make the inventory service the single owner of stock counts and reserve atomically in its database (`UPDATE stock SET available = available - :q WHERE sku = :s AND available >= :q` , checking rows affected). Reserve at checkout with an expiry, confirm after payment, release on cancellation or timeout via the saga. For very hot items, shard counts or use Redis atomic decrements with periodic reconciliation. Don't let the read model (eventually consistent) decide availability. → *More:* Ch 10 — Saga

Q218. How do you reduce the blast radius of failures in a large system? Isolate at every level: bulkheads within services; separate deployments for critical and non-critical paths; per-tenant or per-customer limits; multiple availability zones; cell-based architecture (independent copies of the full stack each serving a subset of users); progressive rollouts per cell or region; and avoiding shared global dependencies that everything calls synchronously. → *More:* Ch 7 — Bulkhead

Q219. What would you look at in a "design a URL shortener/ride-hailing/food delivery with microservices" round? Requirements and scale numbers first; then identify bounded contexts and service list; data ownership and storage per service; synchronous vs event-driven interactions; the critical write path and how it stays consistent (sagas, outbox); read paths and caching; the edge (gateway, auth, rate limits); resilience; observability; deployment. Finish with bottlenecks and trade-offs. Don't create 20 services — justify each split. → *More:* Ch 1 — How to approach a system design interview · Ch 1 — Worked example: URL shortener

Q220. ⋆ What are the most common microservices anti-patterns you have seen? Distributed monolith (lock-step deploys); shared database; too-fine "nanoservices"; chatty synchronous chains; no timeouts; retries at every layer; dual writes without an outbox; business logic in the API gateway; shared domain libraries that force synchronized upgrades; no observability; breaking API changes without versioning; and adopting microservices before having CI/CD and monitoring.

Q221. How do you keep many teams' services consistent without central control? Provide a "paved road" platform: service templates (starter projects with logging, tracing, health checks, security and CI pre-wired), shared base images, an internal developer portal (Backstage) listing owners and APIs, API style guides and linting, and lightweight architecture decision records (ADRs). Standardise interfaces and operations, not internal implementation.

Q222. Your read model (CQRS) is out of sync with the write side. How do you recover? Find why: a consumer bug, lost or skipped events, or out-of-order handling. Fix the bug, then rebuild the projection by replaying events from the beginning (Kafka retention, event store, or a fresh snapshot of the write database via CDC) into a new read store, and switch over when it has caught up. Design projections to be rebuildable from day one and monitor projection lag. → *More:* Ch 12 — CQRS, "Synchronising the two sides"

Q223. How do you deprecate and remove an API endpoint used by other teams? Find the consumers (contract tests, gateway logs, traces); publish a new version and a migration guide; mark the old one deprecated with the `Deprecation` and `Sunset` headers (Spring Framework 7 can add them via its API versioning support); track remaining usage; contact laggards; then remove it once traffic reaches zero.

Q224. When would you choose events over a synchronous call for the same interaction? Events when: the caller does not need the result to answer its own client; several services need to react; the receiver may be down or slow and you do not want to depend on it; or you want to absorb load spikes. Synchronous when: the user needs the answer now (a price check, an authorisation decision), the operation is a query, or strict ordering and immediate consistency outweigh decoupling. → *More:* Ch 11 — Event-Driven · Ch 1 — Synchronous vs asynchronous communication

Q225. ⋆ Tell me about a time a microservices design went wrong. What did you learn? Behavioural, but expect it. Use STAR (Situation, Task, Action, Result) and pick a real example with a technical lesson: a missing timeout that caused a cascade, a dual write that lost events, a split that created a

distributed monolith. Explain the detection, the fix, the safeguard you added (outbox, contract test, alert) and what you would do differently — interviewers want to hear trade-off thinking and ownership.

---

15. Rapid-fire "difference between X and Y" and short-answer round

Phone screens often finish with quick comparisons. Aim for two or three sentences each.

Q226. ⋆ Orchestration vs choreography (in general, not only sagas)? Orchestration: one component directs the others step by step and owns the workflow state — explicit, easy to monitor, but a central dependency. Choreography: each service reacts to events on its own and no one sees the whole flow — loosely coupled, but the end-to-end process is implicit and harder to debug. → *More:* Ch 10 — Saga, "Choosing between them" · Ch 11 — Event-Driven, "Mediator EDA"

Q227. Message queue vs event stream? A queue delivers each message to one consumer and deletes it after acknowledgement — work distribution. A stream (Kafka) keeps an ordered, retained log that many consumer groups read independently at their own offsets and can replay — event history and fan-out. → *More:* Ch 1 — Message queues

Q228. Horizontal vs vertical scaling — which suits microservices? Vertical scaling gives one machine more CPU/RAM; it is simple but has a ceiling and a single point of failure. Horizontal scaling adds more instances behind a load balancer; it suits stateless microservices and gives redundancy, but needs externalised state. → *More:* Ch 1 — Vertical and horizontal scaling

Q229. Container vs virtual machine? A VM virtualises hardware and runs a full guest operating system — strong isolation, heavier, slower to start. A container shares the host kernel and isolates only processes, file system and network with namespaces and cgroups — lightweight and fast, but weaker isolation. → *More:* Ch 1 — Virtual machines, containers and Docker

Q230. Docker vs Kubernetes? Docker builds and runs containers on one machine. Kubernetes orchestrates containers across many machines — scheduling, scaling, self-healing, rolling updates, service discovery. Kubernetes runs images built by Docker (or any OCI tool) using a container runtime such as containerd. → *More:* Ch 1 — Kubernetes basics

Q231. Kubernetes Deployment vs StatefulSet? A Deployment manages interchangeable, stateless Pods with random names that can be replaced freely. A StatefulSet gives each Pod a stable name (`kafka-0` , `kafka-1`), stable storage and ordered start/stop — for databases, brokers and other stateful systems.

Q232. ConfigMap vs Secret? Both inject configuration into Pods as environment variables or files. Secrets are meant for sensitive values: they are kept separately, can be encrypted at rest and access-controlled more tightly — but they are only Base64-encoded by default, so enable encryption and RBAC, or sync them from a vault. → *More:* Ch 13 — Configuration Externalization

Q233. Liveness vs readiness vs startup probe — one line each? Liveness: if this fails, the container is hung, so the kubelet kills and restarts it. Readiness: if this fails, the Pod stays up but is taken out of the Service until it passes again. Startup: holds back the liveness and readiness checks until a slow-booting app has come up. → *More:* Ch 5 — Health checks

Q234. Logs vs metrics vs traces — when do you reach for each? Start with metrics: they are cheap aggregates that drive alerts and dashboards and show that error rate or latency has moved. Then open traces for affected requests to locate the service or call where the time or error appears. Finally read the logs for that trace ID to get the detail — the exception, the input, the decision the code made. → *More:* Ch 1 — Observability

Q235. Circuit breaker vs retry? Retry assumes the failure is brief and tries again; circuit breaker assumes the failure is lasting and stops trying for a while. Used together, the breaker caps the damage retries could do to an unhealthy dependency. → *More:* Ch 6 — Circuit Breaker · Ch 8 — Retry, "Retry vs. fail-fast"

Q236. API Gateway vs service mesh? The gateway handles north-south traffic at the edge — client authentication, public API shape, rate limits per client. The mesh handles east-west traffic between services — mTLS, retries, traffic splitting, telemetry. Many systems use both. → *More:* Ch 3 — API Gateway · Ch 9 — Sidecar

Q237. Event sourcing vs change data capture? Event sourcing makes domain events the source of truth inside the service — intentional, business-meaningful events. CDC derives row-level change events from an ordinary database's log — no code change, but the events mirror table structure, not business intent (so the outbox table is often what CDC reads). → *More:* Ch 12 — CQRS

Q238. Saga vs 2PC — the one-sentence version? 2PC gives atomic, isolated commits across participants at the cost of locks and blocking; a saga gives availability and loose coupling at the cost of isolation and hand-written compensations. → *More:* Ch 10 — Saga

Q239. Stateless JWT vs server-side session? A JWT carries the user's claims and is validated locally by any service — scales easily, but hard to revoke before expiry. A server-side session stores state in a shared store (Redis) referenced by a cookie — easy to revoke, but every service needs access to that store. → *More:* Ch 1 — Sessions, cookies, tokens and JWT

Q240. Strong vs eventual consistency — give a business example for each. Strong: account balances and inventory reservation — a stale read can lose money. Eventual: product reviews, follower counts, search indexes, order history views — a few seconds of staleness is invisible to users. → *More:* Ch 1 — Consistency models

Q241. Spring Cloud Gateway WebFlux vs Web MVC — which one? WebFlux (Netty, reactive) handles very high concurrency with few threads and supports every gateway feature first. Web MVC (servlet) fits teams on the blocking stack; with virtual threads it scales well and is simpler to debug. Pick the one that matches your team's stack. → *More:* Ch 3 — API Gateway

Q242. `@LoadBalanced` — what does it actually do? It marks a `RestClient.Builder`, `RestTemplate` or `WebClient.Builder` bean so Spring Cloud adds an interceptor that treats the host in `http://order-service/...` as a logical service name, resolves it to an instance through the `DiscoveryClient`, and applies the load-balancing algorithm. → *More:* Ch 5 — Service Discovery

Q243. `@RefreshScope` — what is the catch? Refresh-scoped beans are recreated lazily after a refresh event, so references captured elsewhere (e.g. in a constructor of a non-refresh bean) can keep old values, and stateful beans lose their state. Not every property can change at runtime (server port, datasource pool sizes usually need a restart). → *More:* Ch 13 — "Refreshing without restarting"

Q244. Kafka `acks=all` and `min.insync.replicas` — why do they matter? `acks=all` makes the producer wait until all in-sync replicas have the message; `min.insync.replicas=2` (with replication factor 3) means a write fails rather than being accepted by only one broker. Together they prevent losing acknowledged messages when a broker dies — trading a little latency for durability.

Q245. What is a poison message? A message that always fails processing (bad format, unexpected data). Without a limit it blocks its partition forever as the consumer retries it. Detect with a retry count, move it to a DLQ, alert, and keep the partition moving. → *More:* Ch 11 — Event-Driven, "Error handling and message durability"

Q246. Hot partition / hot key — what is it and how do you fix it? One key (a celebrity user, a popular product) receives far more traffic than others, overloading one partition or shard. Fix by salting the key (append a random suffix and aggregate later), splitting the hot entity, caching it, or handling it on a dedicated path. → *More:* Ch 1 — Partitioning and sharding

Q247. Thundering herd vs cache stampede? Both are many clients acting at the same moment. Thundering herd: many waiting clients retry or wake together after an outage. Cache stampede: a hot cache key expires and many requests hit the database at once to rebuild it. Fix with jitter, request coalescing (one rebuild, others wait) and early refresh. → *More:* Ch 8 — Retry, "What is the thundering herd problem?" · Ch 1 — Cache invalidation

Q248. What is the "fallacies of distributed computing" list? Eight false assumptions newcomers make: the network is reliable; latency is zero; bandwidth is infinite; the network is secure; topology doesn't change; there is one administrator; transport cost is zero; the network is homogeneous. Almost every pattern in this book exists because one of them is false. → *More:* Ch 1 — Faults, failures and fault tolerance

Q249. What is service granularity's effect on latency? Quick calculation. If a page needs 6 sequential calls of 20 ms each, that is 120 ms before any work — versus about 20 ms if they run in parallel or the data is local. And availability multiplies: 6 dependencies at 99.9% give about 99.4% combined. Fewer hops and parallel fan-out matter. → *More:* Ch 1 — Availability and the nines

Q250. In one minute: which patterns would you use to build a resilient microservice? Timeouts on every call; retries with exponential backoff and jitter for idempotent operations at one layer; a circuit breaker with fallbacks; bulkheads per dependency; rate limiting and load shedding at the edge; async messaging with outbox, idempotent consumers and a DLQ for state changes; health probes and graceful shutdown; multiple replicas across zones; and observability (RED metrics, traces, SLO alerts) to know when any of it is failing. → *More:* Interview Edition — framework and selection table

---

Sources of the question lists

Question wording and frequency were gathered from these public lists and posts (titles only; all answers above are written for this book):

- InterviewBit — *Microservices Interview Questions* (freshers and experienced lists) — interviewbit.com/microservices-interview-questions
- GeeksforGeeks — *Microservices Interview Questions* — geeksforgeeks.org/java/microservices-interview-questions
- devinterview-io — *microservices-interview-questions* (GitHub) — github.com/devinterview-io/microservices-interview-questions
- DataCamp — *Microservices Interview Questions* — datacamp.com/blog/microservices-interview-questions
- TechPrep — *96 Microservices Interview Questions and Answers (2026)* — techprep.app/blog/microservices-interview-questions
- Adaface — *Microservices interview questions* — adaface.com/blog/microservices-interview-questions
- Coding Shuttle — *Top 20 Microservice interview questions for Advanced Spring Boot Developers* — codingshuttle.com
- Simplilearn — *Microservices Interview Questions* — simplilearn.com/microservices-interview-questions-article
- LambdaTest (DEV Community) — *29 Top Microservice Interview Questions*
- Medium — *Microservices interview questions for Java developers (2026)*; LinkedIn posts — *Microservices scenario-based interview questions* and *Java + Spring Boot + Microservices interview questions (8+ years)*
- LeetCode Discuss — interview-experience posts for backend / Java / Spring roles (topics: saga, circuit breaker, idempotency, Kafka, system design)
- Version facts checked against spring.io release announcements (Spring Cloud 2025.1 "Oakwood", Spring Cloud Greenwich maintenance-mode notice) and Spring Framework 7 / Spring Boot 4 release notes